Operaciones más frecuentes

1. Cargar, mirar y seleccionar

Ejemplos de estas tres páginas: tabla de dos clientes. Cada receta parte de la tabla inicial Ana=20, Luis=30; no acumules las modificaciones entre recetas.

Importar y crear

```
import pandas as pd
v = pd.DataFrame({"cliente":["Ana","Luis"],
                  "total":[20.,30.]})
```

Resultado y contrato. DataFrame de 2 filas y 2 columnas; no altera otra tabla.

Cargar CSV

```
from io import StringIO
v = pd.read_csv(StringIO("cliente,total\nAna,20\nLuis,30"))
```

Resultado y contrato. DataFrame nuevo; usa dtype, usecols y parse_dates para controlar la carga.

Inspeccionar

```
v.shape                 # (2, 2)
v.head(2)               # dos primeras filas
v.dtypes                # tipo por columna
```

Resultado y contrato. Atributos sin (); head devuelve un DataFrame. No modifican v.

Filtrar por valores

```
v.loc[v["total"] >= 30, ["cliente","total"]]
```

Resultado y contrato. DataFrame con Luis, 30. La máscara usa & y | con paréntesis, no and/or.

Seleccionar por posición

```
v.iloc[:1, :2]          # Ana, 20
```

Resultado y contrato. DataFrame con primera fila; el límite final se excluye.

Elegir un escalar

```
v.at[0,"total"]         # 20
v.iat[0,1]              # 20
```

Resultado y contrato. Escalar por etiqueta / posición. Lectura sin modificar v.

Operaciones más frecuentes · 2

2. Limpiar y calcular

Ejemplos de estas tres páginas: tabla de dos clientes. Cada receta parte de la tabla inicial Ana=20, Luis=30; no acumules las modificaciones entre recetas.

Añadir una columna

```
v = v.assign(iva=lambda d: d["total"] * .21)
```

Resultado y contrato. Nueva tabla asignada a v; iva contiene 4.2 y 6.3 en el ejemplo de 2 filas.

Editar con una única selección

```
v.loc[v["cliente"].eq("Ana"), "total"] = 25
```

Resultado y contrato. Modifica v. Evita v[mascara]["total"] = 25: asignación encadenada no actualiza v.

Ausencias

```
v["total"].isna()
v = v.fillna({"total": 0})
```

Resultado y contrato. Máscara booleana y tabla rellenada. La reasignación conserva el resultado.

Convertir con errores visibles

```
pd.to_numeric(pd.Series(["20","x"]), errors="coerce")
```

Resultado y contrato. Series [20.0, NaN]. Revisa cuántos valores se han perdido al convertir.

Quitar duplicados

```
v.drop_duplicates(subset="cliente", keep="last")
```

Resultado y contrato. DataFrame con la última aparición de cada cliente, según el orden actual.

Ordenar y sacar un top

```
v.sort_values("total", ascending=False)
v.nlargest(1,"total")
```

Resultado y contrato. DataFrame ordenado / fila de Luis. No modifican v si no se reasigna.

Operaciones más frecuentes · 3

3. Resumir, combinar y guardar

Ejemplos de estas tres páginas: tabla de dos clientes. Cada receta parte de la tabla inicial Ana=20, Luis=30; no acumules las modificaciones entre recetas.

Facturación por cliente

```
v.groupby("cliente", dropna=False)["total"].sum()
```

Resultado y contrato. Series por cliente. Sobre la tabla inicial de esta sección: Ana=20, Luis=30.

Total de grupo en cada fila

```
v.groupby("cliente")["total"].transform("sum")
```

Resultado y contrato. Series de la misma longitud; permite calcular cuotas sin una unión adicional.

Añadir información del cliente

```
c = pd.DataFrame({"cliente":["Ana","Luis"],
                  "segmento":["A","B"]})
v.merge(c, on="cliente", how="left", validate="many_to_one")
```

Resultado y contrato. DataFrame con segmento. validate exige una fila de dimensión por cliente.

Apilar lotes

```
pd.concat([v, v], ignore_index=True)
```

Resultado y contrato. DataFrame de 4 filas; axis=1 añade columnas alineadas por índice.

Cambiar a formato largo

```
v.melt(id_vars="cliente", value_vars="total",
       var_name="medida", value_name="valor")
```

Resultado y contrato. DataFrame con cliente, medida y valor; una fila por medida seleccionada.

Exportar sin índice accidental

```
texto = v.to_csv(index=False)
# Con ruta: v.to_csv("ventas.csv", index=False)
```

Resultado y contrato. str con CSV. Con ruta escribe el archivo y devuelve None; no cambia v.

Guía de uso e índice

Una chuleta para encontrar la operación y comprobar su contrato. Las fichas prácticas están ordenadas por tareas; el anexo localiza los demás elementos documentados de la API pública.

Alcance, versión y fuentes · 5
Datos comunes y preparación · 6
Importación y creación · 8
Lectura y escritura · 10
Inspección y exploración · 15
Selección, filtrado e indexación · 22
Crear, modificar y eliminar · 26
Tipos y conversiones · 29
Nulos y duplicados · 32
Ordenación y clasificación · 35
Estadística y matemáticas · 37
Agrupaciones y agregaciones · 48
Combinar tablas · 54
Transformar estructuras · 57
Texto con .str · 60
Fechas y series temporales · 66
Ventanas · 75
Datos categóricos · 78
Índices y MultiIndex · 82
Funciones y vectorización · 88
Visualización básica · 91
Rendimiento y buenas prácticas · 93
Comparativas rápidas · 95
Compatibilidad y obsolescencias · 96
Anexo de API pública · 97
Índice alfabético · 200

Alcance, versión y fuentes

Versión estable: pandas 3.0.6, publicada el 17-09-2026. **Consulta:** 11-10-2026 (Europe/Madrid). La portada del sitio oficial y las notas de versión confirman esta versión.

Cobertura práctica: 310 fichas con llamada, uso, salida calculada, tipo de retorno, efecto y advertencias. Cubren DataFrame, Series, Index, MultiIndex, GroupBy, texto, fechas, categorías, ventanas y los formatos más habituales. Cada ejemplo es independiente tras ejecutar la preparación común.

Cobertura del anexo: 2111 entradas únicas enlazadas desde las tablas de las 17 familias de la referencia oficial descargada. Incluye API especializada, tipos, estilos, offsets, extensiones y pruebas. Es un catálogo de nombres y firmas oficiales, no una ficha de siete campos para cada entrada. Las entradas sin ficha práctica remiten a sus ejemplos y contrato completos en la documentación oficial.

Exclusiones explícitas: símbolos privados, miembros con nombre iniciado por _, métodos especiales de Python, detalles internos de pandas.core/pandas.compat, API no enumerada en esas tablas, backends externos y funciones retiradas de la versión actual. Las clases de resultados de GroupBy y ventanas se acceden a través del objeto devuelto; no se construyen desde módulos internos. No se afirma exhaustividad sobre cada símbolo importable de pandas.

Cómo leer las firmas: las fichas muestran una llamada habitual que puede copiarse; no una lista completa de parámetros. El anexo reproduce firmas de la documentación. Los parámetros omitidos conservan su valor por defecto. En atributos e indexadores se usa acceso sin (). Los retornos corresponden al ejemplo concreto; opciones como inplace, chunksize, sheet_name y el backend pueden cambiar el contrato.

Verificación: todos los ejemplos de las fichas se ejecutaron con Python 3.12 y pandas 3.0.6. Los formatos CSV, JSON, Excel, Parquet, SQL, XML y pickle se probaron con datos locales o memoria; read_html usa lxml. Los gráficos se ejecutaron con matplotlib sin abrir ventanas. El catálogo se coteja con HTML oficial, no se ejecuta íntegro. Las salidas largas se abrevian expresamente.

Documentación estable

Referencia pública

Cambios de pandas 3

Notas de 3.0.6

Guía del usuario

Fuentes: documentación de pandas, © pandas / NumFOCUS. Los nombres y firmas del anexo proceden de la referencia oficial; las explicaciones prácticas en español y la selección son una elaboración para esta chuleta.

Datos comunes y preparación

Ejecuta este bloque una vez. Cada ficha parte de este estado inicial; si modifica v, vuelve a ejecutar el bloque antes de la siguiente. Las fichas no dependen de ejecutar las anteriores.

```
import pandas as pd
import numpy as np
v = pd.DataFrame({
    'cliente': ['Ana', 'Luis', 'Ana', 'Eva'],
    'ciudad': ['Madrid', 'Sevilla', 'Madrid', None],
    'unidades': [2, 1, 3, 2],
    'precio': [10., 20., 10., 15.],
    'fecha': pd.to_datetime(['2026-01-01', '2026-01-02',
                             '2026-01-03', '2026-01-04'])})
v['total'] = v['unidades'] * v['precio']
c = pd.DataFrame({'cliente': ['Ana', 'Luis', 'Eva'],
                  'segmento': ['A', 'B', 'A']})
s = v['total']
i = pd.Index(['Ana', 'Luis', 'Eva'], name='cliente')
g = v.groupby('cliente', sort=True)
t = v.set_index('fecha')['total']
cat = v['cliente'].astype('category')
m = v.set_index(['cliente', 'fecha'])
```

Ventas: total = [20.0, 20.0, 30.0, 30.0]. Ana suma 50.0; Luis 20.0; Eva 30.0. La ciudad de Eva está ausente. s es total, t es la serie con índice fecha, cat es cliente categórico y m usa MultiIndex.

Dependencias: pandas==3.0.6. Para las fichas de archivos y gráficos: openpyxl, pyarrow, lxml, matplotlib y scipy. NumPy es dependencia de pandas; sqlite3 e io pertenecen a Python.

Preparación para entradas y salidas

Bloque adicional solo para las fichas de lectura/escritura. Escribe archivos temporales dentro de un directorio temporal y crea una base SQLite en memoria. Ejecuta tras los datos comunes.

```
from io import StringIO, BytesIO
from pathlib import Path
from tempfile import TemporaryDirectory
import sqlite3

tmp = TemporaryDirectory()
excel_path = Path(tmp.name) / 'ventas.xlsx'
pickle_path = Path(tmp.name) / 'ventas.pkl'
v.to_excel(excel_path, sheet_name='Ventas', index=False)
excel_bytes = excel_path.read_bytes()
parquet_bytes = v.to_parquet(index=False)
v.to_pickle(pickle_path)
con = sqlite3.connect(':memory:')
v.to_sql('ventas', con, index=False)
info_buf = StringIO()

def options_demo():
    with pd.option_context('display.max_rows', 8):
        return pd.get_option('display.max_rows')

# Al terminar todas las pruebas de E/S:
# con.close()
# tmp.cleanup()
```

Para mostrar gráficos en un script, añade `import matplotlib.pyplot as plt` y `plt.show()`. El código de las fichas crea los objetos de gráfico; su visualización depende del entorno.

Importación y creación

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame

Crea una tabla de columnas; úsalo para registros de ventas.

```
pd.DataFrame({'cliente': ['Ana'], 'total': [20.]})
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Las columnas deben tener longitudes compatibles.

Documentación oficial: parámetros y variantes

pd.Series

Crea un vector etiquetado; úsalo para una medida.

```
pd.Series([20., 30.], name='total')
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El índice alinea operaciones, no solo la posición.

Documentación oficial: parámetros y variantes

pd.Index

Crea etiquetas de un eje; úsalo para identificar clientes.

```
pd.Index(['Ana', 'Luis'], name='cliente')
```

Resultado esperado (abreviado si se indica):

```
Index(['Ana', 'Luis'], dtype='str', name='cliente')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: Los valores del Index son inmutables.

Documentación oficial: parámetros y variantes

pd.array

Crea un array con tipo de extensión; úsalo para enteros con nulos.

```
pd.array([1, None], dtype='Int64')
```

Resultado esperado (abreviado si se indica):

```
<IntegerArray>
[1, <NA>]
Length: 2, dtype: Int64
```

Devuelve: IntegerArray. **Efecto:** No modifica los datos de entrada.

Atención: Int64 es distinto de int64 de NumPy.

Documentación oficial: parámetros y variantes

pd.Categorical

Crea categorías; úsalo para niveles de prioridad.

```
pd.Categorical(['A', 'B', 'A'], categories=['B', 'A'], ordered=True)
```

Resultado esperado (abreviado si se indica):

```
['A', 'B', 'A']
Categories (2, str): ['B' < 'A']
```

Devuelve: Categorical. **Efecto:** No modifica los datos de entrada.

Atención: El orden lo define categories, no el alfabeto.

Documentación oficial: parámetros y variantes

pd.DataFrame.from_records

Convierte registros en tabla; úsalo con tuplas de una consulta.

```
pd.DataFrame.from_records([('Ana',20)], columns=['cliente','total'])
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana     20
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: columns debe corresponder al número de campos.

Documentación oficial: parámetros y variantes

pd.DataFrame.from_dict

Convierte un diccionario; úsalo si sus claves son filas.

```
pd.DataFrame.from_dict({'Ana':[20,30]}, orient='index')
```

Resultado esperado (abreviado si se indica):

```
      0   1
Ana  20  30
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: orient cambia qué representan las claves.

Documentación oficial: parámetros y variantes

Lectura y escritura

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.read_csv

Lee un CSV; úsalo con exportaciones de ventas.

```
pd.read_csv(StringIO('cliente,total\nAna,20\nLuis,30'), dtype={'cliente':'str'})
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana     20
1    Luis     30
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: sep, usecols, dtype, na_values y parse_dates controlan la lectura; chunksize devuelve un lector.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_csv

Serializa como CSV; úsalo para intercambiar tablas.

```
v.to_csv(index=False)
```

Resultado esperado (abreviado si se indica):

```
'cliente,ciudad,unidades,precio,fecha,total\r\nAna,Madrid,2,10.0,2026-01-01,20.0\r\nLuis
    ,Sevilla,1,20.0,2026-01-02,20.0\r\nAna,Madrid,3,10.0,2026-01-03,30.0\r\nEva,,2,15.0,
    2026-01-04,30.0\r\n'
```

Devuelve: str. **Efecto:** No modifica los datos de entrada.

Atención: Sin ruta devuelve str; con ruta escribe y devuelve None. index=False evita una columna de índice.

Documentación oficial: parámetros y variantes

pd.read_excel

Lee Excel; úsalo con hojas de negocio.

```
pd.read_excel(BytesIO(excel_bytes), sheet_name='Ventas', usecols=['cliente','total'])
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana     20
1    Luis     20
2     Ana     30
3     Eva     30
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Requiere un motor como openpyxl; sheet_name=None devuelve un dict de tablas.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_excel

Escribe Excel; úsalo para compartir ventas.

```
v.to_excel(excel_path, sheet_name='Ventas', index=False)
```

Resultado esperado (abreviado si se indica):

```
None; operación realizada sin error.
```

Devuelve: NoneType. **Efecto:** Escribe un archivo; no modifica v.

Atención: Devuelve None; sobrescribe el archivo al escribir una ruta.

Documentación oficial: parámetros y variantes

pd.read_json

Lee JSON; úsalo con registros de una API.

```
pd.read_json(StringIO(v.to_json(orient='records', date_format='iso')), orient='records')
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio                 fecha  total
0     Ana   Madrid         2      10  2026-01-01T00:00:0...     20
1    Luis  Sevilla         1      20  2026-01-02T00:00:0...     20
2     Ana   Madrid         3      10  2026-01-03T00:00:0...     30
3     Eva      NaN         2      15  2026-01-04T00:00:0...     30
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Para texto JSON literal utiliza StringIO. Fechas dependen de orient y convert_dates; evita el formato epoch obsoleto al escribir.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_json

Serializa JSON; úsalo con servicios web.

```
v.to_json(orient='records', date_format='iso')
```

Resultado esperado (abreviado si se indica):

```
[{"cliente":"Ana","ciudad":"Madrid","unidades":2,"precio":10.0,"fecha":"2026-01-01T00:00
    :00.000","total":20.0},{"cliente":"Luis","ciudad":"Sevilla","unidades":1,"precio":20
    .0,"fecha":"2026-01-02T00:00:00.0 ... [texto abreviado]
```

Devuelve: str. **Efecto:** No modifica los datos de entrada.

Atención: Sin ruta devuelve str; NaN y None se representan como null.

Documentación oficial: parámetros y variantes

pd.read_parquet

Lee Parquet; úsalo para datos tipados y selección de columnas.

```
pd.read_parquet(BytesIO(parquet_bytes), columns=['cliente','total'])
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana   20.0
1    Luis   20.0
2     Ana   30.0
3     Eva   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Necesita pyarrow o fastparquet; filters depende del motor.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_parquet

Escribe Parquet; úsalo para almacenar tipos y comprimir.

```
v.to_parquet(index=False)
```

Resultado esperado (abreviado si se indica):

```
bytes: 3970 bytes de Parquet (tamaño dependiente del motor).
```

Devuelve: bytes. **Efecto:** No modifica los datos de entrada.

Atención: Sin ruta devuelve bytes; con ruta devuelve None.

Documentación oficial: parámetros y variantes

pd.read_sql

Lee consulta o tabla SQL; úsalo con una base de datos.

```
pd.read_sql('SELECT cliente,total FROM ventas', con)
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana   20.0
1    Luis   20.0
2     Ana   30.0
3     Eva   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Usa params para valores; nunca interpoles texto no confiable en SQL.

Documentación oficial: parámetros y variantes

pd.read_sql_query

Lee una consulta parametrizada; úsalo para filtrar en la base.

```
pd.read_sql_query('SELECT * FROM ventas WHERE total > ?', con, params=[25])
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio               fecha  total
0     Ana  Madrid         3    10.0  2026-01-03 00:00:00   30.0
1     Eva     NaN         2    15.0  2026-01-04 00:00:00   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: El marcador depende del controlador; ? funciona con sqlite3.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_sql

Escribe una tabla SQL; úsalo para cargar ventas.

```
v.to_sql('copia', con, if_exists='replace', index=False)
```

Resultado esperado (abreviado si se indica):

```
4
```

Devuelve: int. **Efecto:** Modifica la base de datos; no v.

Atención: if_exists='replace' elimina y recrea la tabla. Devuelve filas afectadas o None.

Documentación oficial: parámetros y variantes

pd.read_html

Extrae tablas HTML; úsalo con informes tabulares.

```
pd.read_html(StringIO('<table><tr><th>total</th></tr><tr><td>20</td></tr></table>'))
```

Resultado esperado (abreviado si se indica):

```
[   total
0     20]
```

Devuelve: list. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve una lista de DataFrame; necesita lxml o bs4/html5lib.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_html

Genera una tabla HTML; úsalo en un informe.

```
v.head(1).to_html(index=False)
```

Resultado esperado (abreviado si se indica):

```
<table border="1" class="dataframe">   <thead>     <tr style="text-align: right;">
    <th>cliente</th>       <th>ciudad</th>       <th>unidades</th>       <th>precio</th>
    <th>fecha</th>       <th> ... [texto abreviado]
```

Devuelve: str. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve str sin buf; escape=True protege caracteres HTML.

Documentación oficial: parámetros y variantes

pd.read_xml

Lee XML tabular; úsalo con intercambios estructurados.

```
pd.read_xml(StringIO('<data><row><cliente>Ana</cliente><total>20</total></row></data>'),
    parser='etree')
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana     20
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: xpath, namespaces y parser controlan nodos y motor.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_xml

Genera XML; úsalo cuando el receptor exige etiquetas.

```
v.head(1).to_xml(index=False, parser='etree')
```

Resultado esperado (abreviado si se indica):

```
<?xml version="1.0" encoding="utf-8"?> <data>   <row>     <cliente>Ana</cliente>
    <ciudad>Madrid</ciudad>     <unidades>2</unidades>     <precio>10.0</precio>
    <fecha>2026-01-01 00:00:00</fecha>      ... [texto abreviado]
```

Devuelve: str. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve str si no hay ruta; no conserva automáticamente todos los tipos.

Documentación oficial: parámetros y variantes

pd.read_pickle

Recupera un objeto serializado; úsalo en un entorno controlado.

```
pd.read_pickle(pickle_path)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
2     Ana   Madrid         3    10.0 2026-01-03   30.0
3     Eva      NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No leas pickle de fuentes no confiables: puede ejecutar código.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_pickle

Serializa un objeto Python; úsalo para guardar un estado local.

```
v.to_pickle(pickle_path)
```

Resultado esperado (abreviado si se indica):

```
None; operación realizada sin error.
```

Devuelve: NoneType. **Efecto:** Escribe un archivo; no modifica v.

Atención: Devuelve None; no es un formato de intercambio universal.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_dict

Convierte a diccionarios; úsalo para crear registros.

```
v.head(1).to_dict(orient='records')
```

Resultado esperado (abreviado si se indica):

```
[{'cliente': 'Ana', 'ciudad': 'Madrid', 'unidades': 2, 'precio': 10.0, 'fecha':
    Timestamp('2026-01-01 00:00:00'), 'total': 20.0}]
```

Devuelve: list. **Efecto:** No modifica los datos de entrada.

Atención: orient='list', 'dict', 'index' o 'tight' cambia la estructura.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_records

Convierte a registros NumPy; úsalo con interfaces de arrays.

```
v[['total']].to_records(index=False)
```

Resultado esperado (abreviado si se indica):

```
rec.array([(20.,), (20.,), (30.,), (30.,)],
          dtype=[('total', '<f8')])
```

Devuelve: recarray. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve recarray; puede perder tipos de extensión.

Documentación oficial: parámetros y variantes

Inspección y exploración

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.head

Muestra las primeras filas; úsalo para comprobar una carga.

```
v.head(2)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No garantiza una muestra representativa.

Documentación oficial: parámetros y variantes

pd.DataFrame.tail

Muestra las últimas filas; úsalo para comprobar el final.

```
v.tail(2)
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio      fecha  total
2     Ana  Madrid         3    10.0 2026-01-03   30.0
3     Eva     NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: n negativo excluye las primeras filas.

Documentación oficial: parámetros y variantes

pd.DataFrame.sample

Obtiene una muestra; úsalo para inspección aleatoria.

```
v.sample(n=2, random_state=0)
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio      fecha  total
2     Ana  Madrid         3    10.0 2026-01-03   30.0
3     Eva     NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Fija random_state para reproducibilidad; replace=True permite repetir.

Documentación oficial: parámetros y variantes

pd.DataFrame.info

Resume tipos y nulos; úsalo para revisar el esquema.

```
v.info(buf=info_buf)
```

Resultado esperado (abreviado si se indica):

```
<class 'pandas.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   cliente   4 non-null      str
 1   ciudad    3 non-null      str
 2   unidades  4 non-null      int64
 3   precio    4 non-null      float64
 4   fecha     4 non-null      datetime64[us]
 5   total     4 non-null      float64
dtypes: datetime64[us](1), float64(2), int64(1), str(2)
memory usage: 357.0 bytes
```

Devuelve: NoneType. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve None; escribe el resumen en buf o en la salida estándar.

Documentación oficial: parámetros y variantes

pd.DataFrame.describe

Resume distribución numérica; úsalo para detectar escalas y extremos.

```
v.describe()
```

Resultado esperado (abreviado si se indica):

```
       unidades     precio                fecha      total
count  4.000000   4.000000                    4   4.000000
mean   2.000000  13.750000  2026-01-02 12:00:00  25.000000
min    1.000000  10.000000  2026-01-01 00:00:00  20.000000
...         ...        ...                  ...        ...
75%    2.250000  16.250000  2026-01-03 06:00:00  30.000000
... [salida abreviada; 10 líneas]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: include='all' añade columnas no numéricas; no calcula todos los cuantiles posibles.

Documentación oficial: parámetros y variantes

pd.DataFrame.shape

Da filas y columnas; úsalo para comprobar dimensiones.

```
v.shape
```

Resultado esperado (abreviado si se indica):

```
(4, 6)
```

Devuelve: tuple. **Efecto:** No modifica los datos de entrada.

Atención: Atributo sin paréntesis; devuelve una tupla.

Documentación oficial: parámetros y variantes

pd.DataFrame.size

Cuenta celdas; úsalo para dimensionar la tabla.

```
v.size
```

Resultado esperado (abreviado si se indica):

```
24
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: Incluye celdas nulas; no equivale al número de filas.

Documentación oficial: parámetros y variantes

pd.DataFrame.ndim

Da el número de ejes; úsalo para distinguir tabla y vector.

```
v.ndim
```

Resultado esperado (abreviado si se indica):

```
2
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: DataFrame tiene 2; Series tiene 1.

Documentación oficial: parámetros y variantes

pd.DataFrame.columns

Da las etiquetas de columnas; úsalo para revisar nombres.

```
v.columns
```

Resultado esperado (abreviado si se indica):

```
Index(['cliente', 'ciudad', 'unidades', 'precio', 'fecha', 'total'], dtype='str')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: Atributo Index; asignarle etiquetas modifica el eje.

Documentación oficial: parámetros y variantes

pd.DataFrame.index

Da las etiquetas de filas; úsalo para revisar claves.

```
v.index
```

Resultado esperado (abreviado si se indica):

```
RangeIndex(start=0, stop=4, step=1)
```

Devuelve: RangeIndex. **Efecto:** No modifica los datos de entrada.

Atención: No tiene por qué ser consecutivo ni único.

Documentación oficial: parámetros y variantes

pd.DataFrame.dtypes

Da el tipo de cada columna; úsalo antes de convertir.

```
v.dtypes
```

Resultado esperado (abreviado si se indica):

```
cliente               str
ciudad                str
unidades            int64
precio            float64
fecha       datetime64[us]
total             float64
dtype: object
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Atributo Series; Series usa dtype en singular.

Documentación oficial: parámetros y variantes

pd.Series.dtype

Da el tipo del vector; úsalo para validar una medida.

```
s.dtype
```

Resultado esperado (abreviado si se indica):

```
dtype('float64')
```

Devuelve: Float64DType. **Efecto:** No modifica los datos de entrada.

Atención: Atributo sin paréntesis.

Documentación oficial: parámetros y variantes

pd.DataFrame.empty

Indica si algún eje está vacío; úsalo para comprobar una consulta.

```
v.empty
```

Resultado esperado (abreviado si se indica):

```
False
```

Devuelve: bool. **Efecto:** No modifica los datos de entrada.

Atención: Una tabla llena de NaN no se considera vacía.

Documentación oficial: parámetros y variantes

pd.DataFrame.memory_usage

Estima bytes por columna; úsalo para buscar columnas costosas.

```
v.memory_usage(index=True, deep=True)
```

Resultado esperado (abreviado si se indica):

```
Index      132
cliente     45
ciudad      52
           ...
precio      32
fecha       32
total       32
Length: 7, dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: deep=True inspecciona objetos; no mide toda la memoria del proceso.

Documentación oficial: parámetros y variantes

pd.Series.value_counts

Cuenta frecuencias; úsalo para reparto de clientes.

```
v['cliente'].value_counts(dropna=False)
```

Resultado esperado (abreviado si se indica):

```
cliente
Ana     2
Luis    1
Eva     1
Name: count, dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Por defecto excluye nulos; normalize=True devuelve proporciones.

Documentación oficial: parámetros y variantes

pd.Series.unique

Obtiene valores distintos en orden de aparición; úsalo para explorar etiquetas.

```
v['cliente'].unique()
```

Resultado esperado (abreviado si se indica):

```
<ArrowStringArray>
['Ana', 'Luis', 'Eva']
Length: 3, dtype: str
```

Devuelve: ArrowStringArray. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve ndarray o ExtensionArray; no ordena.

Documentación oficial: parámetros y variantes

pd.Series.nunique

Cuenta valores distintos; úsalo para cardinalidad.

```
v['ciudad'].nunique(dropna=False)
```

Resultado esperado (abreviado si se indica):

```
3
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: dropna=False cuenta también el nulo.

Documentación oficial: parámetros y variantes

pd.Series.name

Da el nombre del vector; úsalo para identificar una medida.

```
s.name
```

Resultado esperado (abreviado si se indica):

```
'total'
```

Devuelve: str. **Efecto:** No modifica los datos de entrada.

Atención: Atributo asignable; name no es el índice.

Documentación oficial: parámetros y variantes

pd.Series.index

Da etiquetas del vector; úsalo para comprobar alineación.

```
s.index
```

Resultado esperado (abreviado si se indica):

```
RangeIndex(start=0, stop=4, step=1)
```

Devuelve: RangeIndex. **Efecto:** No modifica los datos de entrada.

Atención: Atributo Index; las etiquetas no son siempre posiciones.

Documentación oficial: parámetros y variantes

pd.Series.shape

Da la longitud como tupla; úsalo para validar dimensión.

```
s.shape
```

Resultado esperado (abreviado si se indica):

```
(4,)
```

Devuelve: tuple. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve (4,), no un entero.

Documentación oficial: parámetros y variantes

pd.DataFrame.attrs

Da el diccionario de metadatos; úsalo para anotar el origen.

```
v.attrs
```

Resultado esperado (abreviado si se indica):

```
{}
```

Devuelve: dict. **Efecto:** No modifica los datos de entrada.

Atención: API experimental; no confíes en su propagación universal.

Documentación oficial: parámetros y variantes

pd.DataFrame.T

Transpone filas y columnas; úsalo para cambiar orientación.

```
v[['unidades','total']].T
```

Resultado esperado (abreviado si se indica):

```
             0     1     2     3
unidades   2.0   1.0   3.0   2.0
total     20.0  20.0  30.0  30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Atributo sin paréntesis; tipos mixtos pueden homogeneizarse.

Documentación oficial: parámetros y variantes

pd.DataFrame.select_dtypes

Selecciona por tipos; úsalo antes de cálculos numéricos.

```
v.select_dtypes(include='number')
```

Resultado esperado (abreviado si se indica):

```
   unidades  precio  total
0         2    10.0   20.0
1         1    20.0   20.0
2         3    10.0   30.0
3         2    15.0   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Para texto en pandas 3 revisa dtype str; no supongas object.

Documentación oficial: parámetros y variantes

pd.DataFrame.keys

Obtiene las etiquetas de columnas; úsalo en código de estilo mapping.

```
v.keys()
```

Resultado esperado (abreviado si se indica):

```
Index(['cliente', 'ciudad', 'unidades', 'precio', 'fecha', 'total'], dtype='str')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: Equivale a columns para DataFrame; Series.keys devuelve el índice.

Documentación oficial: parámetros y variantes

Selección, filtrado e indexación

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.loc

Selecciona por etiquetas o máscara; úsalo para ventas grandes.

```
v.loc[v['total'] >= 30, ['cliente','total']]
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
2     Ana   30.0
3     Eva   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Los cortes por etiquetas incluyen ambos extremos; una etiqueta ausente puede dar KeyError.

Documentación oficial: parámetros y variantes

pd.DataFrame.iloc

Selecciona por posiciones; úsalo para las dos primeras filas.

```
v.iloc[0:2, [0,5]]
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana   20.0
1    Luis   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: El límite final del corte se excluye; posiciones fuera de rango fallan.

Documentación oficial: parámetros y variantes

pd.DataFrame.at

Accede a un escalar por etiqueta; úsalo para una celda conocida.

```
v.at[0, 'total']
```

Resultado esperado (abreviado si se indica):

```
np.float64(20.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: También permite asignar; no acepta una selección múltiple.

Documentación oficial: parámetros y variantes

pd.DataFrame.iat

Accede a un escalar por posición; úsalo para una celda conocida.

```
v.iat[0, 5]
```

Resultado esperado (abreviado si se indica):

```
np.float64(20.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: La posición de columnas puede cambiar al modificar el esquema.

Documentación oficial: parámetros y variantes

pd.DataFrame.query

Filtra con una expresión; úsalo para condiciones legibles.

```
v.query('total >= 30 and unidades > 1')
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio      fecha  total
2     Ana  Madrid         3    10.0 2026-01-03   30.0
3     Eva     NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Usa @variable para variables externas; no evalúes expresiones no confiables.

Documentación oficial: parámetros y variantes

pd.Series.isin

Construye una máscara de pertenencia; úsalo con una lista de clientes.

```
v['cliente'].isin(['Ana','Eva'])
```

Resultado esperado (abreviado si se indica):

```
0     True
1    False
2     True
3     True
Name: cliente, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Para negar utiliza ~; un string suelto no sustituye a una lista.

Documentación oficial: parámetros y variantes

pd.Series.between

Prueba un intervalo; úsalo para un rango de importes.

```
s.between(20,30, inclusive='both')
```

Resultado esperado (abreviado si se indica):

```
0    True
1    True
2    True
3    True
Name: total, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: inclusive admite both, neither, left y right.

Documentación oficial: parámetros y variantes

pd.DataFrame.filter

Selecciona etiquetas; úsalo para elegir columnas por nombre.

```
v.filter(items=['cliente','total'])
```

Resultado esperado (abreviado si se indica):

```
  cliente  total
0     Ana   20.0
1    Luis   20.0
2     Ana   30.0
3     Eva   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: like y regex filtran nombres, no contenido de celdas.

Documentación oficial: parámetros y variantes

pd.DataFrame.xs

Extrae una sección de un MultiIndex; úsalo para un cliente.

```
m.xs('Ana', level='cliente')
```

Resultado esperado (abreviado si se indica):

```
            ciudad  unidades  precio  total
fecha
2026-01-01  Madrid         2    10.0   20.0
2026-01-03  Madrid         3    10.0   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No uses xs para asignar; utiliza loc.

Documentación oficial: parámetros y variantes

pd.DataFrame.take

Selecciona posiciones en el orden indicado; úsalo para reordenar filas.

```
v.take([3,0])
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio      fecha  total
3     Eva     NaN         2    15.0 2026-01-04   30.0
0     Ana  Madrid         2    10.0 2026-01-01   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: axis=1 actúa sobre columnas.

Documentación oficial: parámetros y variantes

pd.DataFrame.where

Conserva donde la condición es verdadera; úsalo para reemplazar valores no válidos.

```
v[['total']].where(v[['total']] > 20, other=0)
```

Resultado esperado (abreviado si se indica):

```
   total
0    0.0
1    0.0
2   30.0
3   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Mantiene la forma; una máscara de filtrado elimina filas.

Documentación oficial: parámetros y variantes

pd.DataFrame.mask

Reemplaza donde la condición es verdadera; úsalo para ocultar extremos.

```
v[['total']].mask(v[['total']] > 20, other=0)
```

Resultado esperado (abreviado si se indica):

```
   total
0   20.0
1   20.0
2    0.0
3    0.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Es el complemento lógico de where.

Documentación oficial: parámetros y variantes

pd.DataFrame.get

Obtiene columna o valor por defecto; úsalo con esquemas opcionales.

```
v.get('descuento', pd.Series(0, index=v.index))
```

Resultado esperado (abreviado si se indica):

```
0    0
1    0
2    0
3    0
dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No convierte errores arbitrarios; el default solo cubre claves ausentes.

Documentación oficial: parámetros y variantes

Crear, modificar y eliminar

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.assign

Añade o reemplaza columnas en una tabla nueva; úsalo para encadenar cálculos.

```
v.assign(iva=lambda x: x['total'] * .21)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  ...      fecha total  iva
0     Ana   Madrid         2  ... 2026-01-01  20.0  4.2
1    Luis  Sevilla         1  ... 2026-01-02  20.0  4.2
2     Ana   Madrid         3  ... 2026-01-03  30.0  6.3
3     Eva      NaN         2  ... 2026-01-04  30.0  6.3

[4 rows x 7 columns]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Las expresiones posteriores pueden usar columnas creadas antes.

Documentación oficial: parámetros y variantes

pd.DataFrame.insert

Inserta una columna en una posición; úsalo para un esquema ordenado.

```
v.insert(1, 'canal', 'web')
```

Resultado esperado (abreviado si se indica):

```
None; v ahora tiene columnas: cliente, canal, ciudad, unidades, precio, fecha, total
```

Devuelve: NoneType. **Efecto:** Modifica v en el sitio.

Atención: Devuelve None; un nombre existente falla salvo allow_duplicates=True.

Documentación oficial: parámetros y variantes

pd.DataFrame.drop

Elimina etiquetas; úsalo para quitar campos.

```
v.drop(columns=['precio'])
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades      fecha  total
0     Ana   Madrid         2 2026-01-01   20.0
1    Luis  Sevilla         1 2026-01-02   20.0
2     Ana   Madrid         3 2026-01-03   30.0
3     Eva      NaN         2 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: axis=0 elimina filas; errors='ignore' oculta etiquetas ausentes. inplace=True modifica y devuelve None.

Documentación oficial: parámetros y variantes

pd.DataFrame.pop

Extrae y elimina una columna; úsalo para mover un campo.

```
v.pop('ciudad')
```

Resultado esperado (abreviado si se indica):

```
0     Madrid
1    Sevilla
2     Madrid
3        NaN
Name: ciudad, dtype: str
```

Devuelve: Series. **Efecto:** Modifica v en el sitio.

Atención: Devuelve la Series eliminada; KeyError si falta.

Documentación oficial: parámetros y variantes

pd.DataFrame.rename

Cambia nombres de etiquetas; úsalo para un esquema de salida.

```
v.rename(columns={'total':'importe'})
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  importe
0     Ana   Madrid         2    10.0 2026-01-01     20.0
1    Luis  Sevilla         1    20.0 2026-01-02     20.0
2     Ana   Madrid         3    10.0 2026-01-03     30.0
3     Eva      NaN         2    15.0 2026-01-04     30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No cambia valores; errors='raise' permite detectar claves inexistentes.

Documentación oficial: parámetros y variantes

pd.DataFrame.set_axis

Sustituye las etiquetas de un eje; úsalo si conoces todos los nombres.

```
v[['cliente','total']].set_axis(['nombre','importe'], axis=1)
```

Resultado esperado (abreviado si se indica):

```
  nombre  importe
0    Ana     20.0
1   Luis     20.0
2    Ana     30.0
3    Eva     30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: La longitud debe coincidir con el eje.

Documentación oficial: parámetros y variantes

pd.DataFrame.update

Actualiza valores alineando etiquetas; úsalo para correcciones parciales.

```
v.update(pd.DataFrame({'total':[99.]}, index=[0]))
```

Resultado esperado (abreviado si se indica):

```
None; v ahora tiene columnas: cliente, ciudad, unidades, precio, fecha, total
```

Devuelve: NoneType. **Efecto:** Modifica v en el sitio.

Atención: Devuelve None; no añade filas o columnas.

Documentación oficial: parámetros y variantes

pd.DataFrame.reindex

Alinea a nuevas etiquetas; úsalo para completar un esquema.

```
v[['total']].reindex(index=[0,4], fill_value=0)
```

Resultado esperado (abreviado si se indica):

```
   total
0   20.0
4    0.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: fill_value solo afecta huecos nuevos, no nulos existentes; debe ser compatible con el tipo de cada columna.

Documentación oficial: parámetros y variantes

pd.DataFrame.copy

Crea una copia; úsalo para separar una transformación.

```
v.copy(deep=True)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
2     Ana   Madrid         3    10.0 2026-01-03   30.0
3     Eva      NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: deep=True no clona recursivamente objetos Python contenidos en celdas.

Documentación oficial: parámetros y variantes

pd.DataFrame.eval

Evalúa una expresión sobre columnas; úsalo para cálculos legibles.

```
v.eval('neto = total * 0.9')
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  ...      fecha total  neto
0     Ana   Madrid         2  ... 2026-01-01  20.0  18.0
1    Luis  Sevilla         1  ... 2026-01-02  20.0  18.0
2     Ana   Madrid         3  ... 2026-01-03  30.0  27.0
3     Eva      NaN         2  ... 2026-01-04  30.0  27.0

[4 rows x 7 columns]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: inplace=True modifica; no evalúes texto no confiable.

Documentación oficial: parámetros y variantes

Tipos y conversiones

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.astype

Convierte tipos explícitamente; úsalo para fijar el esquema.

```
v.astype({'unidades':'Int64'})
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
2     Ana   Madrid         3    10.0 2026-01-03   30.0
3     Eva      NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Una conversión inválida falla; errors='ignore' puede ocultar problemas.

Documentación oficial: parámetros y variantes

pd.DataFrame.convert_dtypes

Busca tipos de extensión adecuados; úsalo tras una carga.

```
v.convert_dtypes()
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2      10 2026-01-01     20
1    Luis  Sevilla         1      20 2026-01-02     20
2     Ana   Madrid         3      10 2026-01-03     30
3     Eva     <NA>         2      15 2026-01-04     30
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: dtype_backend='pyarrow' usa Arrow; no sustituye una validación del esquema.

Documentación oficial: parámetros y variantes

pd.DataFrame.infer_objects

Infiere tipos en columnas object; úsalo tras construcción genérica.

```
pd.DataFrame({'n':pd.Series([1,2], dtype=object)}).infer_objects()
```

Resultado esperado (abreviado si se indica):

```
   n
0  1
1  2
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No analiza números almacenados como cadenas.

Documentación oficial: parámetros y variantes

pd.to_numeric

Analiza números; úsalo con importes textuales.

```
pd.to_numeric(pd.Series(['20','x']), errors='coerce')
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1     NaN
dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: errors='coerce' produce NaN; downcast puede reducir precisión.

Documentación oficial: parámetros y variantes

pd.to_datetime

Analiza fechas; úsalo con datos de texto.

```
pd.to_datetime(pd.Series(['2026-01-01','mal']), errors='coerce', format='%Y-%m-%d')
```

Resultado esperado (abreviado si se indica):

```
0   2026-01-01
1          NaT
dtype: datetime64[us]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: errors='coerce' produce NaT; utc=True facilita zonas mixtas.

Documentación oficial: parámetros y variantes

pd.to_timedelta

Analiza duraciones; úsalo con tiempos transcurridos.

```
pd.to_timedelta(['1 day','2 hours'])
```

Resultado esperado (abreviado si se indica):

```
TimedeltaIndex(['1 days 00:00:00', '0 days 02:00:00'], dtype='timedelta64[us]',
    freq=None)
```

Devuelve: TimedeltaIndex. **Efecto:** No modifica los datos de entrada.

Atención: Una duración no representa una fecha del calendario.

Documentación oficial: parámetros y variantes

pd.DataFrame.to_numpy

Convierte a matriz NumPy; úsalo con otras librerías.

```
v[['unidades','total']].to_numpy(dtype=float)
```

Resultado esperado (abreviado si se indica):

```
array([[ 2., 20.],
       [ 1., 20.],
       [ 3., 30.],
       [ 2., 30.]])
```

Devuelve: ndarray. **Efecto:** No modifica los datos de entrada.

Atención: copy=False no garantiza ausencia de copia; el array puede ser de solo lectura.

Documentación oficial: parámetros y variantes

pd.Series.array

Expone el array de extensión; úsalo para conservar tipos pandas.

```
s.array
```

Resultado esperado (abreviado si se indica):

```
<NumpyExtensionArray>
[20.0, 20.0, 30.0, 30.0]
Length: 4, dtype: float64
```

Devuelve: NumpyExtensionArray. **Efecto:** No modifica los datos de entrada.

Atención: Sin paréntesis; evita asumir que siempre es un ndarray.

Documentación oficial: parámetros y variantes

pd.Series.tolist

Convierte a lista; úsalo con una API Python.

```
s.tolist()
```

Resultado esperado (abreviado si se indica):

```
[20.0, 20.0, 30.0, 30.0]
```

Devuelve: list. **Efecto:** No modifica los datos de entrada.

Atención: Pierde etiquetas y metadatos del tipo.

Documentación oficial: parámetros y variantes

pd.Series.item

Extrae un escalar de un vector unitario; úsalo al validar una única coincidencia.

```
pd.Series([20]).item()
```

Resultado esperado (abreviado si se indica):

```
20
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: Falla si hay cero o varios elementos.

Documentación oficial: parámetros y variantes

Nulos y duplicados

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.isna

Detecta ausencias; úsalo antes de limpiar.

```
pd.isna(pd.Series([1,None]))
```

Resultado esperado (abreviado si se indica):

```
0    False
1     True
dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No compares con == np.nan ni == pd.NA.

Documentación oficial: parámetros y variantes

pd.notna

Detecta valores presentes; úsalo para filtrar registros completos.

```
pd.notna(v['ciudad'])
```

Resultado esperado (abreviado si se indica):

```
0     True
1     True
2     True
3    False
Name: ciudad, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Cadena vacía e infinito no son ausencias por defecto.

Documentación oficial: parámetros y variantes

pd.DataFrame.isna

Devuelve una máscara por celda; úsalo para localizar huecos.

```
v.isna()
```

Resultado esperado (abreviado si se indica):

```
   cliente  ciudad  unidades  precio  fecha  total
0    False   False     False   False  False  False
1    False   False     False   False  False  False
2    False   False     False   False  False  False
3    False    True     False   False  False  False
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: isnull es un alias; elige un nombre coherente.

Documentación oficial: parámetros y variantes

pd.DataFrame.dropna

Elimina filas con ausencias; úsalo si ciudad es obligatoria.

```
v.dropna(subset=['ciudad'])
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
2     Ana   Madrid         3    10.0 2026-01-03   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: how='all' exige todos nulos; thresh cuenta valores presentes.

Documentación oficial: parámetros y variantes

pd.DataFrame.fillna

Rellena ausencias; úsalo con reglas por columna.

```
v.fillna({'ciudad':'Desconocida'})
```

Resultado esperado (abreviado si se indica):

```
  cliente       ciudad  unidades  precio      fecha  total
0     Ana       Madrid         2    10.0 2026-01-01   20.0
1    Luis      Sevilla         1    20.0 2026-01-02   20.0
2     Ana       Madrid         3    10.0 2026-01-03   30.0
3     Eva  Desconocida         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Usa ffill o bfill para propagación; fillna(method=...) ya no es la API actual.

Documentación oficial: parámetros y variantes

pd.DataFrame.ffill

Propaga el último valor válido; úsalo en datos ordenados.

```
v.ffill(limit=1)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
2     Ana   Madrid         3    10.0 2026-01-03   30.0
3     Eva   Madrid         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Ordena antes; propagar entre clientes puede ser incorrecto.

Documentación oficial: parámetros y variantes

pd.DataFrame.bfill

Rellena desde el siguiente valor válido; úsalo con una regla retrospectiva.

```
v.bfill()
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
2     Ana   Madrid         3    10.0 2026-01-03   30.0
3     Eva      NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Puede introducir información futura en modelos predictivos.

Documentación oficial: parámetros y variantes

pd.Series.interpolate

Estima valores intermedios; úsalo con medidas continuas.

```
pd.Series([20.,None,30.]).interpolate()
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1    25.0
2    30.0
dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: method='time' necesita índice temporal; categorías no se interpolan.

Documentación oficial: parámetros y variantes

pd.DataFrame.duplicated

Marca duplicados; úsalo para revisar clientes repetidos.

```
v.duplicated(subset=['cliente'], keep=False)
```

Resultado esperado (abreviado si se indica):

```
0     True
1    False
2     True
3    False
dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: keep=False marca todas las apariciones.

Documentación oficial: parámetros y variantes

pd.DataFrame.drop_duplicates

Conserva una aparición por clave; úsalo para deduplicar.

```
v.drop_duplicates(subset=['cliente'], keep='last')
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
2     Ana   Madrid         3    10.0 2026-01-03   30.0
3     Eva      NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No determina cuál es más reciente: ordena antes.

Documentación oficial: parámetros y variantes

pd.Series.hasnans

Indica si hay alguna ausencia; úsalo como comprobación rápida.

```
v['ciudad'].hasnans
```

Resultado esperado (abreviado si se indica):

```
True
```

Devuelve: bool. **Efecto:** No modifica los datos de entrada.

Atención: Atributo bool, sin paréntesis.

Documentación oficial: parámetros y variantes

Ordenación y clasificación

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.sort_values

Ordena por valores; úsalo para clasificar ventas.

```
v.sort_values(['total','cliente'], ascending=[False,True])
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
2     Ana   Madrid         3    10.0 2026-01-03   30.0
3     Eva      NaN         2    15.0 2026-01-04   30.0
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: na_position controla nulos; kind='stable' es útil con una única clave.

Documentación oficial: parámetros y variantes

pd.DataFrame.sort_index

Ordena las etiquetas; úsalo antes de cortes por índice.

```
v.sort_index(ascending=False)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
3     Eva      NaN         2    15.0 2026-01-04   30.0
2     Ana   Madrid         3    10.0 2026-01-03   30.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
0     Ana   Madrid         2    10.0 2026-01-01   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No ordena por el contenido de columnas.

Documentación oficial: parámetros y variantes

pd.DataFrame.nlargest

Obtiene los mayores valores; úsalo para un top de ventas.

```
v.nlargest(2, 'total')
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio      fecha  total
2     Ana  Madrid         3    10.0 2026-01-03   30.0
3     Eva     NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: keep='all' puede devolver más de n filas si hay empates.

Documentación oficial: parámetros y variantes

pd.DataFrame.nsmallest

Obtiene los menores valores; úsalo para ventas pequeñas.

```
v.nsmallest(2, 'total')
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Requiere columnas con tipos compatibles.

Documentación oficial: parámetros y variantes

pd.Series.rank

Asigna rangos; úsalo para clasificaciones con empates.

```
s.rank(method='dense', ascending=False)
```

Resultado esperado (abreviado si se indica):

```
0    2.0
1    2.0
2    1.0
3    1.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: method controla empates; pct=True expresa porcentajes.

Documentación oficial: parámetros y variantes

pd.cut

Clasifica en intervalos fijados; úsalo para tramos comerciales.

```
pd.cut(s, bins=[0,25,50], labels=['bajo','alto'])
```

Resultado esperado (abreviado si se indica):

```
0    bajo
1    bajo
2    alto
3    alto
Name: total, dtype: category
Categories (2, str): ['bajo' < 'alto']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Por defecto intervalos abiertos a la izquierda y cerrados a la derecha.

Documentación oficial: parámetros y variantes

pd.qcut

Clasifica por cuantiles; úsalo para grupos de tamaño parecido.

```
pd.qcut(s, q=2, duplicates='drop')
```

Resultado esperado (abreviado si se indica):

```
0    (19.999, 25.0]
1    (19.999, 25.0]
2      (25.0, 30.0]
3      (25.0, 30.0]
Name: total, dtype: category
Categories (2, interval[float64, right]): [(19.999, 25.0] < (25.0, 30.0]]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Empates pueden reducir o impedir los intervalos.

Documentación oficial: parámetros y variantes

Estadística y matemáticas

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.Series.sum

Suma valores; úsalo para facturación.

```
s.sum(min_count=1)
```

Resultado esperado (abreviado si se indica):

```
np.float64(100.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: min_count=1 evita obtener cero si todos los valores son nulos.

Documentación oficial: parámetros y variantes

pd.Series.mean

Calcula la media; úsalo para ticket medio.

```
s.mean()
```

Resultado esperado (abreviado si se indica):

```
np.float64(25.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: Omite nulos por defecto; revisa si necesitas una media ponderada.

Documentación oficial: parámetros y variantes

pd.Series.median

Calcula la mediana; úsalo con importes extremos.

```
s.median()
```

Resultado esperado (abreviado si se indica):

```
np.float64(25.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: No equivale a la media.

Documentación oficial: parámetros y variantes

pd.Series.min

Obtiene el mínimo; úsalo para límites observados.

```
s.min()
```

Resultado esperado (abreviado si se indica):

```
np.float64(20.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: idxmin devuelve la etiqueta del mínimo.

Documentación oficial: parámetros y variantes

pd.Series.max

Obtiene el máximo; úsalo para el mayor importe.

```
s.max()
```

Resultado esperado (abreviado si se indica):

```
np.float64(30.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: idxmax devuelve la etiqueta del máximo.

Documentación oficial: parámetros y variantes

pd.Series.std

Calcula desviación típica muestral; úsalo para dispersión.

```
s.std(ddof=1)
```

Resultado esperado (abreviado si se indica):

```
np.float64(5.773502691896258)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: NumPy usa ddof=0 por defecto; pandas usa 1.

Documentación oficial: parámetros y variantes

pd.Series.var

Calcula varianza; úsalo para dispersión cuadrática.

```
s.var(ddof=1)
```

Resultado esperado (abreviado si se indica):

```
np.float64(33.333333333333336)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: Revisa ddof al comparar con otras librerías.

Documentación oficial: parámetros y variantes

pd.Series.quantile

Calcula cuantiles; úsalo para rangos robustos.

```
s.quantile([.25,.75])
```

Resultado esperado (abreviado si se indica):

```
0.25    20.0
0.75    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: interpolation determina valores entre observaciones.

Documentación oficial: parámetros y variantes

pd.Series.count

Cuenta valores presentes; úsalo para completitud.

```
v['ciudad'].count()
```

Resultado esperado (abreviado si se indica):

```
np.int64(3)
```

Devuelve: int64. **Efecto:** No modifica los datos de entrada.

Atención: size cuenta también ausencias.

Documentación oficial: parámetros y variantes

pd.Series.idxmax

Devuelve la etiqueta del máximo; úsalo para localizar una venta.

```
s.idxmax()
```

Resultado esperado (abreviado si se indica):

```
2
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: En empates devuelve la primera etiqueta; todos nulos puede dar error.

Documentación oficial: parámetros y variantes

pd.Series.idxmin

Devuelve la etiqueta del mínimo; úsalo para localizar una venta.

```
s.idxmin()
```

Resultado esperado (abreviado si se indica):

```
0
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: No devuelve la posición; para posición usa argmin.

Documentación oficial: parámetros y variantes

pd.Series.cumsum

Suma acumulativamente; úsalo para facturación acumulada.

```
s.cumsum()
```

Resultado esperado (abreviado si se indica):

```
0     20.0
1     40.0
2     70.0
3    100.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El orden de las filas determina el resultado.

Documentación oficial: parámetros y variantes

pd.Series.cumprod

Multiplica acumulativamente; úsalo para factores compuestos.

```
pd.Series([1.,1.1,1.2]).cumprod()
```

Resultado esperado (abreviado si se indica):

```
0    1.00
1    1.10
2    1.32
dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No equivale a sumar tasas.

Documentación oficial: parámetros y variantes

pd.Series.cummax

Calcula máximo hasta cada fila; úsalo para récords.

```
s.cummax()
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1    20.0
2    30.0
3    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Ordena antes si la secuencia es temporal.

Documentación oficial: parámetros y variantes

pd.Series.diff

Resta el valor desplazado; úsalo para cambios absolutos.

```
s.diff(periods=1)
```

Resultado esperado (abreviado si se indica):

```
0     NaN
1     0.0
2    10.0
3     0.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: La primera fila queda nula.

Documentación oficial: parámetros y variantes

pd.Series.pct_change

Calcula cambio relativo; úsalo para variación de ventas.

```
s.pct_change()
```

Resultado esperado (abreviado si se indica):

```
0    NaN
1    0.0
2    0.5
3    0.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve fracciones: 0.5 es 50%; rellena nulos explícitamente antes.

Documentación oficial: parámetros y variantes

pd.Series.round

Redondea decimales; úsalo para presentación de importes.

```
pd.Series([20.126]).round(2)
```

Resultado esperado (abreviado si se indica):

```
0    20.13
dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El redondeo de mitades puede ir al par; float no representa decimales exactamente.

Documentación oficial: parámetros y variantes

pd.Series.clip

Acota valores; úsalo para limitar extremos.

```
s.clip(lower=22, upper=28)
```

Resultado esperado (abreviado si se indica):

```
0    22.0
1    22.0
2    28.0
3    28.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No elimina registros; sustituye por los límites.

Documentación oficial: parámetros y variantes

pd.Series.abs

Obtiene valores absolutos; úsalo con diferencias.

```
pd.Series([-20,30]).abs()
```

Resultado esperado (abreviado si se indica):

```
0    20
1    30
dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Elimina el signo, no informa si era una devolución.

Documentación oficial: parámetros y variantes

pd.Series.add

Suma alineando etiquetas; úsalo con ajustes parciales.

```
s.add(pd.Series([5.], index=[0]), fill_value=0)
```

Resultado esperado (abreviado si se indica):

```
0    25.0
1    20.0
2    30.0
3    30.0
dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Los operadores también alinean; fill_value cubre valores ausentes.

Documentación oficial: parámetros y variantes

pd.Series.div

Divide alineando etiquetas; úsalo para precio unitario.

```
s.div(v['unidades'])
```

Resultado esperado (abreviado si se indica):

```
0    10.0
1    20.0
2    10.0
3    15.0
dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Dividir entre cero puede producir infinito.

Documentación oficial: parámetros y variantes

pd.DataFrame.corr

Calcula correlaciones; úsalo para explorar asociaciones.

```
v[['unidades','total']].corr(method='pearson')
```

Resultado esperado (abreviado si se indica):

```
          unidades     total
unidades  1.000000  0.707107
total     0.707107  1.000000
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No demuestra causalidad; se usan pares con datos válidos.

Documentación oficial: parámetros y variantes

pd.DataFrame.cov

Calcula covarianzas; úsalo para relaciones entre medidas.

```
v[['unidades','total']].cov()
```

Resultado esperado (abreviado si se indica):

```
          unidades      total
unidades  0.666667   3.333333
total     3.333333  33.333333
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: La escala de cada variable afecta el resultado.

Documentación oficial: parámetros y variantes

pd.Series.any

Indica si alguna condición es verdadera; úsalo para una alerta.

```
s.gt(25).any()
```

Resultado esperado (abreviado si se indica):

```
np.True_
```

Devuelve: bool. **Efecto:** No modifica los datos de entrada.

Atención: No uses if s: el valor lógico de Series es ambiguo.

Documentación oficial: parámetros y variantes

pd.Series.all

Indica si todas las condiciones son verdaderas; úsalo para validar positivos.

```
s.gt(0).all()
```

Resultado esperado (abreviado si se indica):

```
np.True_
```

Devuelve: bool. **Efecto:** No modifica los datos de entrada.

Atención: Un vector vacío da True; valida también su tamaño.

Documentación oficial: parámetros y variantes

pd.Series.prod

Multiplica valores; úsalo para factores compuestos.

```
pd.Series([2,3,4]).prod(min_count=1)
```

Resultado esperado (abreviado si se indica):

```
np.int64(24)
```

Devuelve: int64. **Efecto:** No modifica los datos de entrada.

Atención: El producto vacío puede ser 1; min_count exige observaciones válidas.

Documentación oficial: parámetros y variantes

pd.Series.sem

Calcula error estándar de la media; úsalo como resumen de precisión.

```
s.sem()
```

Resultado esperado (abreviado si se indica):

```
np.float64(2.886751345948129)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: No es la desviación típica; supone una interpretación muestral apropiada.

Documentación oficial: parámetros y variantes

pd.Series.skew

Calcula asimetría; úsalo para describir la distribución.

```
s.skew()
```

Resultado esperado (abreviado si se indica):

```
np.float64(0.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: Con pocos valores su interpretación es inestable.

Documentación oficial: parámetros y variantes

pd.Series.kurt

Calcula curtosis no sesgada; úsalo para estudiar colas.

```
s.kurt()
```

Resultado esperado (abreviado si se indica):

```
np.float64(-6.0)
```

Devuelve: float64. **Efecto:** No modifica los datos de entrada.

Atención: La definición de Fisher da 0 a una distribución normal.

Documentación oficial: parámetros y variantes

pd.Series.mode

Devuelve valores más frecuentes; úsalo para la moda.

```
s.mode()
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Puede devolver varias modas; siempre es Series.

Documentación oficial: parámetros y variantes

pd.Series.cummin

Calcula mínimo acumulado; úsalo para mínimos hasta la fecha.

```
s.cummin()
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1    20.0
2    20.0
3    20.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El orden importa; no agrupa por cliente automáticamente.

Documentación oficial: parámetros y variantes

pd.Series.sub

Resta valores; úsalo para descuentos fijos.

```
s.sub(5)
```

Resultado esperado (abreviado si se indica):

```
0    15.0
1    15.0
2    25.0
3    25.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Con Series alinea etiquetas; fill_value controla ausencias.

Documentación oficial: parámetros y variantes

pd.Series.mul

Multiplica valores; úsalo para aplicar un impuesto.

```
s.mul(1.21)
```

Resultado esperado (abreviado si se indica):

```
0    24.2
1    24.2
2    36.3
3    36.3
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Con Series alinea etiquetas y puede introducir nulos.

Documentación oficial: parámetros y variantes

pd.Series.floordiv

Hace división entera hacia abajo; úsalo para paquetes completos.

```
s.floordiv(7)
```

Resultado esperado (abreviado si se indica):

```
0    2.0
1    2.0
2    4.0
3    4.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Con negativos redondea hacia menos infinito.

Documentación oficial: parámetros y variantes

pd.Series.mod

Calcula el resto; úsalo para unidades sobrantes.

```
s.mod(7)
```

Resultado esperado (abreviado si se indica):

```
0    6.0
1    6.0
2    2.0
3    2.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El comportamiento de signos sigue las reglas de división de Python.

Documentación oficial: parámetros y variantes

pd.Series.pow

Eleva a una potencia; úsalo para términos cuadráticos.

```
s.pow(2)
```

Resultado esperado (abreviado si se indica):

```
0    400.0
1    400.0
2    900.0
3    900.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Los tipos enteros pueden desbordar con valores grandes.

Documentación oficial: parámetros y variantes

pd.Series.eq

Compara igualdad por elemento; úsalo para una máscara.

```
v['cliente'].eq('Ana')
```

Resultado esperado (abreviado si se indica):

```
0     True
1    False
2     True
3    False
Name: cliente, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No es equals, que devuelve un único bool para el objeto completo.

Documentación oficial: parámetros y variantes

pd.Series.ne

Compara desigualdad; úsalo para excluir un cliente.

```
v['cliente'].ne('Ana')
```

Resultado esperado (abreviado si se indica):

```
0    False
1     True
2    False
3     True
Name: cliente, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Los nulos dependen del tipo; fija una política explícita si es necesario.

Documentación oficial: parámetros y variantes

pd.Series.gt

Compara mayor que; úsalo para umbrales.

```
s.gt(25)
```

Resultado esperado (abreviado si se indica):

```
0    False
1    False
2     True
3     True
Name: total, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Para combinar máscaras usa & o con paréntesis, no and.

Documentación oficial: parámetros y variantes

pd.Series.ge

Compara mayor o igual; úsalo para incluir el umbral.

```
s.ge(30)
```

Resultado esperado (abreviado si se indica):

```
0    False
1    False
2     True
3     True
Name: total, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve Series booleana alineada.

Documentación oficial: parámetros y variantes

pd.Series.lt

Compara menor que; úsalo para un límite superior abierto.

```
s.lt(25)
```

Resultado esperado (abreviado si se indica):

```
0     True
1     True
2    False
3    False
Name: total, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El valor del límite queda excluido.

Documentación oficial: parámetros y variantes

pd.Series.le

Compara menor o igual; úsalo para un límite superior cerrado.

```
s.le(30)
```

Resultado esperado (abreviado si se indica):

```
0    True
1    True
2    True
3    True
Name: total, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: La comparación escalar produce una máscara por fila.

Documentación oficial: parámetros y variantes

pd.Series.argmax

Devuelve posición del máximo; úsalo para selección con iloc.

```
s.argmax()
```

Resultado esperado (abreviado si se indica):

```
np.int64(2)
```

Devuelve: int64. **Efecto:** No modifica los datos de entrada.

Atención: idxmax devuelve etiqueta; ambas coinciden solo con ciertos índices.

Documentación oficial: parámetros y variantes

pd.Series.argmin

Devuelve posición del mínimo; úsalo para selección con iloc.

```
s.argmin()
```

Resultado esperado (abreviado si se indica):

```
np.int64(0)
```

Devuelve: int64. **Efecto:** No modifica los datos de entrada.

Atención: En empates elige la primera posición.

Documentación oficial: parámetros y variantes

pd.Series.searchsorted

Busca posición de inserción; úsalo con medidas ordenadas.

```
s.sort_values().searchsorted(25)
```

Resultado esperado (abreviado si se indica):

```
np.int64(2)
```

Devuelve: int64. **Efecto:** No modifica los datos de entrada.

Atención: No comprueba que la entrada esté ordenada.

Documentación oficial: parámetros y variantes

Agrupaciones y agregaciones

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.groupby

Forma grupos por clave; úsalo para resumir clientes.

```
v.groupby('cliente', dropna=False, observed=True).size()
```

Resultado esperado (abreviado si se indica):

```
cliente
Ana     2
Eva     1
Luis    1
dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: groupby devuelve GroupBy; aquí size lo materializa. observed=True limita categorías a las observadas.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.agg

Reduce cada grupo; úsalo para una tabla por cliente.

```
g.agg(ventas=('total','sum'), n=('total','size'))
```

Resultado esperado (abreviado si se indica):

```
         ventas  n
cliente
Ana        50.0  2
Eva        30.0  1
Luis       20.0  1
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: La agregación nombrada define columnas; múltiples funciones cambian el esquema.

Documentación oficial: parámetros y variantes

SeriesGroupBy.transform

Devuelve resultados alineados con cada fila; úsalo para cuota por cliente.

```
g['total'].transform('sum')
```

Resultado esperado (abreviado si se indica):

```
0    50.0
1    20.0
2    50.0
3    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Conserva el índice y longitud de la entrada; agg reduce.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.apply

Aplica una función por grupo; úsalo si no basta una operación integrada.

```
g[['unidades','total']].apply(lambda x: x.head(1))
```

Resultado esperado (abreviado si se indica):

```
          unidades  total
cliente
Ana     0         2   20.0
Eva     3         2   30.0
Luis    1         1   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Suele ser más lento; pandas 3 excluye columnas de agrupación del bloque aplicado.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.filter

Conserva grupos que cumplen una regla; úsalo para clientes relevantes.

```
g.filter(lambda x: x['total'].sum() >= 40)
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio      fecha  total
0     Ana  Madrid         2    10.0 2026-01-01   20.0
2     Ana  Madrid         3    10.0 2026-01-03   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: La función debe devolver un bool escalar por grupo.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.size

Cuenta filas por grupo; úsalo para número de ventas.

```
g.size()
```

Resultado esperado (abreviado si se indica):

```
cliente
Ana     2
Eva     1
Luis    1
dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: count cuenta valores no nulos por columna.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.count

Cuenta valores válidos por grupo y columna; úsalo para completitud.

```
g.count()
```

Resultado esperado (abreviado si se indica):

```
         ciudad  unidades  precio  fecha  total
cliente
Ana           2         2       2      2      2
Eva           0         1       1      1      1
Luis          1         1       1      1      1
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No cuenta nulos; el resultado puede variar entre columnas.

Documentación oficial: parámetros y variantes

SeriesGroupBy.sum

Suma por grupo; úsalo para facturación por cliente.

```
g['total'].sum(min_count=1)
```

Resultado esperado (abreviado si se indica):

```
cliente
Ana     50.0
Eva     30.0
Luis    20.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Selecciona columnas numéricas para evitar sumar texto sin querer.

Documentación oficial: parámetros y variantes

SeriesGroupBy.mean

Promedia por grupo; úsalo para ticket medio por cliente.

```
g['total'].mean()
```

Resultado esperado (abreviado si se indica):

```
cliente
Ana     25.0
Eva     30.0
Luis    20.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: La media de medias no suele ser la media global.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.get_group

Recupera un grupo; úsalo para inspección de un cliente.

```
g.get_group('Ana')
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio      fecha  total
0     Ana  Madrid         2    10.0 2026-01-01   20.0
2     Ana  Madrid         3    10.0 2026-01-03   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Una clave ausente da KeyError.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.cumcount

Numera filas dentro del grupo; úsalo para secuencia de compras.

```
g.cumcount()
```

Resultado esperado (abreviado si se indica):

```
0    0
1    0
2    1
3    0
dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Empieza en cero y sigue el orden actual.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.ngroup

Codifica cada grupo como número; úsalo para etiquetas temporales.

```
g.ngroup()
```

Resultado esperado (abreviado si se indica):

```
0    0
1    2
2    0
3    1
dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Los números dependen del orden de grupos; no son identificadores persistentes.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.head

Toma primeras filas por grupo; úsalo para primera venta tras ordenar.

```
g.head(1)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
3     Eva      NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No es lo mismo que first: first puede saltar nulos por columna.

Documentación oficial: parámetros y variantes

SeriesGroupBy.shift

Desplaza dentro del grupo; úsalo para la venta anterior del cliente.

```
g['total'].shift(1)
```

Resultado esperado (abreviado si se indica):

```
0     NaN
1     NaN
2    20.0
3     NaN
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Ordena por fecha antes; no cruza límites de grupo.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.groups

Da etiquetas de filas por grupo; úsalo para inspeccionar pertenencia.

```
g.groups
```

Resultado esperado (abreviado si se indica):

```
{'Ana': [0, 2], 'Eva': [3], 'Luis': [1]}
```

Devuelve: PrettyDict. **Efecto:** No modifica los datos de entrada.

Atención: Atributo dict; indices proporciona posiciones.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.first

Obtiene primer valor válido de cada columna por grupo; úsalo para una descripción inicial.

```
g.first()
```

Resultado esperado (abreviado si se indica):

```
          ciudad  unidades  precio      fecha  total
cliente
Ana       Madrid         2    10.0 2026-01-01   20.0
Eva          NaN         2    15.0 2026-01-04   30.0
Luis     Sevilla         1    20.0 2026-01-02   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Puede combinar columnas de filas distintas; head(1) conserva una fila real.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.last

Obtiene último valor válido de cada columna por grupo; úsalo para un estado final.

```
g.last()
```

Resultado esperado (abreviado si se indica):

```
          ciudad  unidades  precio      fecha  total
cliente
Ana       Madrid         3    10.0 2026-01-03   30.0
Eva          NaN         2    15.0 2026-01-04   30.0
Luis     Sevilla         1    20.0 2026-01-02   20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Ordena por fecha antes; los nulos pueden hacer que no corresponda a una única fila.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.nth

Toma la fila en posición n por grupo; úsalo para una venta concreta.

```
g.nth(0)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  precio      fecha  total
0     Ana   Madrid         2    10.0 2026-01-01   20.0
1    Luis  Sevilla         1    20.0 2026-01-02   20.0
3     Eva      NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Respeta filas; no equivale a first con nulos.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.nunique

Cuenta distintos por grupo; úsalo para variedad de valores.

```
g.nunique(dropna=False)
```

Resultado esperado (abreviado si se indica):

```
         ciudad  unidades  precio  fecha  total
cliente
Ana           1         2       1      2      2
Eva           1         1       1      1      1
Luis          1         1       1      1      1
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: dropna=False incluye ausencias.

Documentación oficial: parámetros y variantes

SeriesGroupBy.rolling

Aplica ventanas dentro del grupo; úsalo para ventas recientes por cliente.

```
g['total'].rolling(2, min_periods=1).sum()
```

Resultado esperado (abreviado si se indica):

```
cliente
Ana      0    20.0
         2    50.0
Eva      3    30.0
Luis     1    20.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El resultado suele tener MultiIndex; ordena previamente por cliente y fecha.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.indices

Da posiciones de filas por grupo; úsalo para selección posicional.

```
g.indices
```

Resultado esperado (abreviado si se indica):

```
{'Ana': array([0, 2]), 'Eva': array([3]), 'Luis': array([1])}
```

Devuelve: dict. **Efecto:** No modifica los datos de entrada.

Atención: Atributo dict; groups da etiquetas de índice.

Documentación oficial: parámetros y variantes

DataFrameGroupBy.ngroups

Da número de grupos; úsalo para comprobar cardinalidad.

```
g.ngroups
```

Resultado esperado (abreviado si se indica):

```
3
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: Atributo entero; dropna y observed pueden cambiar el recuento.

Documentación oficial: parámetros y variantes

Combinar tablas

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.merge

Combina por claves; úsalo para añadir el segmento a cada venta.

```
pd.merge(v, c, on='cliente', how='left', validate='many_to_one')
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  ...      fecha total  segmento
0     Ana   Madrid         2  ... 2026-01-01  20.0         A
1    Luis  Sevilla         1  ... 2026-01-02  20.0         B
2     Ana   Madrid         3  ... 2026-01-03  30.0         A
3     Eva      NaN         2  ... 2026-01-04  30.0         A

[4 rows x 7 columns]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Los nulos de claves pueden coincidir; validate detecta multiplicaciones inesperadas.

Documentación oficial: parámetros y variantes

pd.DataFrame.merge

Combina desde una tabla; úsalo para auditar coincidencias.

```
v.merge(c, on='cliente', how='left', indicator=True)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  ...  total segmento  _merge
0     Ana   Madrid         2  ...   20.0        A    both
1    Luis  Sevilla         1  ...   20.0        B    both
2     Ana   Madrid         3  ...   30.0        A    both
3     Eva      NaN         2  ...   30.0        A    both

[4 rows x 8 columns]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: suffixes resuelve nombres repetidos; indicator añade _merge.

Documentación oficial: parámetros y variantes

pd.DataFrame.join

Añade columnas desde un índice; úsalo con una dimensión indexada.

```
v.join(c.set_index('cliente'), on='cliente', validate='many_to_one')
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  ...      fecha total  segmento
0     Ana   Madrid         2  ... 2026-01-01  20.0         A
1    Luis  Sevilla         1  ... 2026-01-02  20.0         B
2     Ana   Madrid         3  ... 2026-01-03  30.0         A
3     Eva      NaN         2  ... 2026-01-04  30.0         A

[4 rows x 7 columns]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Por defecto une por índices y hace left join; merge usa inner por defecto.

Documentación oficial: parámetros y variantes

pd.concat

Apila o alinea objetos; úsalo para juntar lotes.

```
pd.concat([v.head(1), v.tail(1)], ignore_index=True)
```

Resultado esperado (abreviado si se indica):

```
  cliente  ciudad  unidades  precio      fecha  total
0     Ana  Madrid         2    10.0 2026-01-01   20.0
1     Eva     NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: axis=0 apila filas; axis=1 alinea índices y añade columnas. No lo repitas en un bucle.

Documentación oficial: parámetros y variantes

pd.merge_asof

Une por la clave ordenada más cercana; úsalo para cotizaciones vigentes.

```
pd.merge_asof(v.sort_values('fecha'), v[['fecha','precio']].sort_values('fecha'),
    on='fecha', direction='backward')
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  ...      fecha total  precio_y
0     Ana   Madrid         2  ... 2026-01-01  20.0      10.0
1    Luis  Sevilla         1  ... 2026-01-02  20.0      20.0
2     Ana   Madrid         3  ... 2026-01-03  30.0      10.0
3     Eva      NaN         2  ... 2026-01-04  30.0      15.0

[4 rows x 7 columns]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Ambas claves deben estar ordenadas; tolerance limita distancia.

Documentación oficial: parámetros y variantes

pd.merge_ordered

Une secuencias ordenadas; úsalo para series con fechas diferentes.

```
pd.merge_ordered(v[['fecha','total']], v[['fecha','precio']], on='fecha',
    fill_method='ffill')
```

Resultado esperado (abreviado si se indica):

```
       fecha  total  precio
0 2026-01-01   20.0    10.0
1 2026-01-02   20.0    20.0
2 2026-01-03   30.0    10.0
3 2026-01-04   30.0    15.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No equivale a una coincidencia de vecino más cercano.

Documentación oficial: parámetros y variantes

pd.DataFrame.combine_first

Rellena nulos desde otra tabla alineada; úsalo para una fuente de respaldo.

```
v[['ciudad']].combine_first(pd.DataFrame({'ciudad':['X']*4}))
```

Resultado esperado (abreviado si se indica):

```
    ciudad
0   Madrid
1  Sevilla
2   Madrid
3        X
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Puede ampliar índices y columnas; update no amplía.

Documentación oficial: parámetros y variantes

Transformar estructuras

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.pivot

Pasa valores a una matriz sin agregar; úsalo con pares únicos.

```
v.pivot(index='fecha', columns='cliente', values='total')
```

Resultado esperado (abreviado si se indica):

```
cliente      Ana   Eva  Luis
fecha
2026-01-01  20.0   NaN   NaN
2026-01-02   NaN   NaN  20.0
2026-01-03  30.0   NaN   NaN
2026-01-04   NaN  30.0   NaN
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Falla si se repite el par índice-columna; usa pivot_table para agregar.

Documentación oficial: parámetros y variantes

pd.DataFrame.pivot_table

Crea una tabla dinámica agregada; úsalo con claves repetidas.

```
v.pivot_table(index='cliente', columns='ciudad', values='total', aggfunc='sum',
    fill_value=0, observed=True)
```

Resultado esperado (abreviado si se indica):

```
ciudad   Madrid  Sevilla
cliente
Ana        50.0      0.0
Luis        0.0     20.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: aggfunc por defecto es mean; dropna puede excluir claves nulas.

Documentación oficial: parámetros y variantes

pd.DataFrame.melt

Pasa columnas a filas; úsalo para formato largo.

```
v.melt(id_vars=['cliente'], value_vars=['unidades','total'], var_name='medida',
    value_name='valor')
```

Resultado esperado (abreviado si se indica):

```
   cliente    medida  valor
0      Ana  unidades    2.0
1     Luis  unidades    1.0
2      Ana  unidades    3.0
..     ...       ...    ...
5     Luis     total   20.0
... [salida abreviada; 10 líneas]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Los identificadores se repiten; ignore_index=False conserva índice.

Documentación oficial: parámetros y variantes

pd.DataFrame.stack

Lleva columnas al índice; úsalo para un formato jerárquico.

```
v.set_index('fecha')[['unidades','total']].stack()
```

Resultado esperado (abreviado si se indica):

```
fecha
2026-01-01  unidades     2.0
            total       20.0
2026-01-02  unidades     1.0
                        ...
2026-01-03  total       30.0
... [salida abreviada; 9 líneas]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: En pandas 3 usa el nuevo comportamiento; no combines future_stack=True con dropna o sort.

Documentación oficial: parámetros y variantes

pd.DataFrame.unstack

Lleva un nivel del índice a columnas; úsalo para comparar clientes.

```
m[['total']].unstack('cliente')
```

Resultado esperado (abreviado si se indica):

```
           total
cliente      Ana   Eva  Luis
fecha
2026-01-01  20.0   NaN   NaN
2026-01-02   NaN   NaN  20.0
2026-01-03  30.0   NaN   NaN
2026-01-04   NaN  30.0   NaN
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Pares repetidos requieren agregar antes; pueden aparecer nulos.

Documentación oficial: parámetros y variantes

pd.DataFrame.explode

Expande listas en filas; úsalo para líneas por producto.

```
pd.DataFrame({'cliente':['Ana'], 'productos':[['A','B']]}).explode('productos',
    ignore_index=True)
```

Resultado esperado (abreviado si se indica):

```
  cliente productos
0     Ana         A
1     Ana         B
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Listas vacías producen un nulo; varias columnas deben tener longitudes coincidentes.

Documentación oficial: parámetros y variantes

pd.crosstab

Cuenta cruces de categorías; úsalo para contingencias.

```
pd.crosstab(v['cliente'], v['ciudad'])
```

Resultado esperado (abreviado si se indica):

```
ciudad   Madrid  Sevilla
cliente
Ana           2        0
Luis          0        1
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: normalize permite proporciones; values requiere aggfunc.

Documentación oficial: parámetros y variantes

pd.get_dummies

Codifica categorías en columnas; úsalo para modelos.

```
pd.get_dummies(v['cliente'], prefix='cliente', dtype=int)
```

Resultado esperado (abreviado si se indica):

```
   cliente_Ana  cliente_Eva  cliente_Luis
0            1            0             0
1            0            0             1
2            1            0             0
3            0            1             0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: drop_first elimina una categoría; dummy_na representa ausencias.

Documentación oficial: parámetros y variantes

pd.from_dummies

Reconstruye etiquetas desde indicadores; úsalo para deshacer una codificación.

```
pd.from_dummies(pd.get_dummies(v['cliente'], dtype=int))
```

Resultado esperado (abreviado si se indica):

```
0   Ana
1  Luis
2   Ana
3   Eva
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Varias marcas por fila o ninguna pueden requerir tratamiento previo.

Documentación oficial: parámetros y variantes

pd.wide_to_long

Despliega columnas con sufijos; úsalo para medidas por año.

```
pd.wide_to_long(pd.DataFrame({'id':[1], 'total2025':[20], 'total2026':[30]}),
    stubnames='total', i='id', j='anio')
```

Resultado esperado (abreviado si se indica):

```
         total
id anio
1  2025     20
   2026     30
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: El identificador i debe identificar cada fila.

Documentación oficial: parámetros y variantes

Texto con .str

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.Series.str.lower

Pasa texto a minúsculas; úsalo para normalizar claves.

```
v['cliente'].str.lower()
```

Resultado esperado (abreviado si se indica):

```
0     ana
1    luis
2     ana
3     eva
Name: cliente, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El accesor propaga nulos; no garantiza una normalización lingüística completa.

Documentación oficial: parámetros y variantes

pd.Series.str.upper

Pasa texto a mayúsculas; úsalo para códigos.

```
v['cliente'].str.upper()
```

Resultado esperado (abreviado si se indica):

```
0     ANA
1    LUIS
2     ANA
3     EVA
Name: cliente, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No convierte automáticamente datos no textuales.

Documentación oficial: parámetros y variantes

pd.Series.str.strip

Quita espacios de extremos; úsalo tras importar texto.

```
pd.Series([' Ana ',None]).str.strip()
```

Resultado esperado (abreviado si se indica):

```
0    Ana
1    NaN
dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No elimina espacios interiores; chars es un conjunto de caracteres.

Documentación oficial: parámetros y variantes

pd.Series.str.contains

Busca un patrón; úsalo para filtrar nombres.

```
v['cliente'].str.contains('a', case=False, na=False, regex=False)
```

Resultado esperado (abreviado si se indica):

```
0     True
1    False
2     True
3     True
Name: cliente, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: regex=True por defecto; usa regex=False para texto literal.

Documentación oficial: parámetros y variantes

pd.Series.str.startswith

Prueba un prefijo; úsalo para códigos iniciales.

```
v['cliente'].str.startswith('A', na=False)
```

Resultado esperado (abreviado si se indica):

```
0     True
1    False
2     True
3    False
Name: cliente, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No interpreta expresiones regulares.

Documentación oficial: parámetros y variantes

pd.Series.str.endswith

Prueba un sufijo; úsalo para terminaciones.

```
v['cliente'].str.endswith('a', na=False)
```

Resultado esperado (abreviado si se indica):

```
0     True
1    False
2     True
3     True
Name: cliente, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Acepta una tupla de sufijos.

Documentación oficial: parámetros y variantes

pd.Series.str.replace

Sustituye texto; úsalo para limpieza literal.

```
v['cliente'].str.replace('a','@', regex=False)
```

Resultado esperado (abreviado si se indica):

```
0     An@
1    Luis
2     An@
3     Ev@
Name: cliente, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: regex=False por defecto; regex=True activa patrones.

Documentación oficial: parámetros y variantes

pd.Series.str.split

Divide texto en campos; úsalo para un delimitador.

```
pd.Series(['Ana Madrid']).str.split(' ', expand=True)
```

Resultado esperado (abreviado si se indica):

```
     0       1
0  Ana  Madrid
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: expand=True devuelve DataFrame; sin expand devuelve listas.

Documentación oficial: parámetros y variantes

pd.Series.str.extract

Extrae grupos de una expresión regular; úsalo para códigos.

```
pd.Series(['V-20','V-30']).str.extract(r'V-(\d+)', expand=False)
```

Resultado esperado (abreviado si se indica):

```
0    20
1    30
dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El patrón necesita grupos de captura; solo extrae la primera coincidencia.

Documentación oficial: parámetros y variantes

pd.Series.str.extractall

Extrae todas las coincidencias; úsalo con varios códigos por celda.

```
pd.Series(['A1 B2']).str.extractall(r'([A-Z])(\d)')
```

Resultado esperado (abreviado si se indica):

```
         0  1
  match
0 0      A  1
  1      B  2
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve DataFrame con nivel match en MultiIndex.

Documentación oficial: parámetros y variantes

pd.Series.str.len

Mide longitud de texto; úsalo para validar nombres.

```
v['cliente'].str.len()
```

Resultado esperado (abreviado si se indica):

```
0    3
1    4
2    3
3    3
Name: cliente, dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Cuenta caracteres, no bytes codificados.

Documentación oficial: parámetros y variantes

pd.Series.str.cat

Concatena texto alineado; úsalo para etiquetas legibles.

```
v['cliente'].str.cat(v['ciudad'], sep=' / ', na_rep='?')
```

Resultado esperado (abreviado si se indica):

```
0      Ana / Madrid
1    Luis / Sevilla
2      Ana / Madrid
3           Eva / ?
Name: cliente, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Sin others concatena toda la Series a un único str.

Documentación oficial: parámetros y variantes

pd.Series.str.slice

Extrae un segmento; úsalo para prefijos.

```
v['cliente'].str.slice(0,2)
```

Resultado esperado (abreviado si se indica):

```
0    An
1    Lu
2    An
3    Ev
Name: cliente, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El final se excluye igual que en los cortes de Python.

Documentación oficial: parámetros y variantes

pd.Series.str.fullmatch

Valida el texto completo; úsalo para una regla de formato.

```
v['cliente'].str.fullmatch(r'[A-Za-z]+', na=False)
```

Resultado esperado (abreviado si se indica):

```
0    True
1    True
2    True
3    True
Name: cliente, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: match solo exige coincidencia al inicio.

Documentación oficial: parámetros y variantes

pd.Series.str.normalize

Normaliza Unicode; úsalo con textos visualmente iguales.

```
pd.Series(['café']).str.normalize('NFC')
```

Resultado esperado (abreviado si se indica):

```
0    café
dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No elimina acentos; NFKD descompone caracteres.

Documentación oficial: parámetros y variantes

pd.Series.str.title

Pasa a estilo de título; úsalo para presentación sencilla.

```
v['cliente'].str.title()
```

Resultado esperado (abreviado si se indica):

```
0     Ana
1    Luis
2     Ana
3     Eva
Name: cliente, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Puede producir capitalización incorrecta en nombres propios complejos.

Documentación oficial: parámetros y variantes

pd.Series.str.casefold

Normaliza mayúsculas Unicode; úsalo para comparaciones sin distinción de caja.

```
pd.Series(['Straße']).str.casefold()
```

Resultado esperado (abreviado si se indica):

```
0    strasse
dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Es más agresivo que lower; no elimina acentos.

Documentación oficial: parámetros y variantes

pd.Series.str.count

Cuenta coincidencias de regex; úsalo para patrones repetidos.

```
v['cliente'].str.count('a', flags=0)
```

Resultado esperado (abreviado si se indica):

```
0    1
1    0
2    1
3    1
Name: cliente, dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Distingue mayúsculas por defecto; no cuenta solo palabras completas.

Documentación oficial: parámetros y variantes

pd.Series.str.find

Encuentra la primera posición literal; úsalo para delimitadores.

```
v['cliente'].str.find('a')
```

Resultado esperado (abreviado si se indica):

```
0    2
1   -1
2    2
3    2
Name: cliente, dtype: int64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: -1 significa ausente; no es una posición válida encontrada.

Documentación oficial: parámetros y variantes

pd.Series.str.get

Obtiene un carácter o elemento; úsalo para iniciales.

```
v['cliente'].str.get(0)
```

Resultado esperado (abreviado si se indica):

```
0    A
1    L
2    A
3    E
Name: cliente, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No es Series.get, que busca una etiqueta del índice.

Documentación oficial: parámetros y variantes

pd.Series.str.zfill

Rellena con ceros a la izquierda; úsalo para códigos.

```
pd.Series(['7','20']).str.zfill(3)
```

Resultado esperado (abreviado si se indica):

```
0    007
1    020
dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve texto; no convierte el código a número.

Documentación oficial: parámetros y variantes

pd.Series.str.join

Une elementos de una lista textual; úsalo para etiquetas compuestas.

```
pd.Series([['Ana','Madrid']]).str.join(' / ')
```

Resultado esperado (abreviado si se indica):

```
0    Ana / Madrid
dtype: object
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: En una cadena une sus caracteres; comprueba el tipo de las celdas.

Documentación oficial: parámetros y variantes

pd.Series.str.get_dummies

Codifica etiquetas múltiples; úsalo para categorías por venta.

```
pd.Series(['A;B','B']).str.get_dummies(sep=';')
```

Resultado esperado (abreviado si se indica):

```
   A  B
0  1  1
1  0  1
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: El separador divide texto; pd.get_dummies codifica valores completos.

Documentación oficial: parámetros y variantes

Fechas y series temporales

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.date_range

Crea un índice temporal regular; úsalo para un calendario.

```
pd.date_range('2026-01-01', periods=3, freq='D')
```

Resultado esperado (abreviado si se indica):

```
DatetimeIndex(['2026-01-01', '2026-01-02', '2026-01-03'], dtype='datetime64[us]',
    freq='D')
```

Devuelve: DatetimeIndex. **Efecto:** No modifica los datos de entrada.

Atención: Usa ME para fin de mes, h para horas; evita alias antiguos como M o H.

Documentación oficial: parámetros y variantes

pd.bdate_range

Crea días laborables; úsalo para planificación semanal.

```
pd.bdate_range('2026-01-02', periods=3)
```

Resultado esperado (abreviado si se indica):

```
DatetimeIndex(['2026-01-02', '2026-01-05', '2026-01-06'], dtype='datetime64[us]',
    freq='B')
```

Devuelve: DatetimeIndex. **Efecto:** No modifica los datos de entrada.

Atención: Por defecto excluye sábados y domingos, no festivos locales.

Documentación oficial: parámetros y variantes

pd.period_range

Crea períodos de calendario; úsalo para meses contables.

```
pd.period_range('2026-01', periods=3, freq='M')
```

Resultado esperado (abreviado si se indica):

```
PeriodIndex(['2026-01', '2026-02', '2026-03'], dtype='period[M]')
```

Devuelve: PeriodIndex. **Efecto:** No modifica los datos de entrada.

Atención: M sigue siendo frecuencia de Period; no lo confundas con offsets ME.

Documentación oficial: parámetros y variantes

pd.timedelta_range

Crea duraciones regulares; úsalo para tiempos desde un origen.

```
pd.timedelta_range('0 days', periods=3, freq='h')
```

Resultado esperado (abreviado si se indica):

```
TimedeltaIndex(['0 days 00:00:00', '0 days 01:00:00', '0 days 02:00:00'],
    dtype='timedelta64[us]', freq='h')
```

Devuelve: TimedeltaIndex. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve TimedeltaIndex, no fechas.

Documentación oficial: parámetros y variantes

pd.Timestamp

Crea una fecha escalar; úsalo para límites temporales.

```
pd.Timestamp('2026-01-01', tz='Europe/Madrid')
```

Resultado esperado (abreviado si se indica):

```
Timestamp('2026-01-01 00:00:00+0100', tz='Europe/Madrid')
```

Devuelve: Timestamp. **Efecto:** No modifica los datos de entrada.

Atención: Una fecha con zona no se compara libremente con una sin zona.

Documentación oficial: parámetros y variantes

pd.Timedelta

Crea una duración escalar; úsalo para plazos.

```
pd.Timedelta(days=2, hours=3)
```

Resultado esperado (abreviado si se indica):

```
Timedelta('2 days 03:00:00')
```

Devuelve: Timedelta. **Efecto:** No modifica los datos de entrada.

Atención: Un día representa 24 horas; DateOffset expresa calendario.

Documentación oficial: parámetros y variantes

pd.DateOffset

Desplaza por calendario; úsalo para meses contractuales.

```
pd.Timestamp('2026-01-31') + pd.DateOffset(months=1)
```

Resultado esperado (abreviado si se indica):

```
Timestamp('2026-02-28 00:00:00')
```

Devuelve: Timestamp. **Efecto:** No modifica los datos de entrada.

Atención: No equivale a sumar 30 días; n y months tienen significados distintos.

Documentación oficial: parámetros y variantes

pd.Series.dt.year

Extrae el año; úsalo para agrupar por ejercicio.

```
v['fecha'].dt.year
```

Resultado esperado (abreviado si se indica):

```
0    2026
1    2026
2    2026
3    2026
Name: fecha, dtype: int32
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Atributo, sin paréntesis; requiere dtype temporal.

Documentación oficial: parámetros y variantes

pd.Series.dt.month

Extrae el mes numérico; úsalo para estacionalidad.

```
v['fecha'].dt.month
```

Resultado esperado (abreviado si se indica):

```
0    1
1    1
2    1
3    1
Name: fecha, dtype: int32
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Mezclar años por month puede juntar períodos diferentes.

Documentación oficial: parámetros y variantes

pd.Series.dt.day

Extrae día del mes; úsalo para calendarios.

```
v['fecha'].dt.day
```

Resultado esperado (abreviado si se indica):

```
0    1
1    2
2    3
3    4
Name: fecha, dtype: int32
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No equivale al día de la semana.

Documentación oficial: parámetros y variantes

pd.Series.dt.dayofweek

Extrae día semanal; úsalo para patrones laborales.

```
v['fecha'].dt.dayofweek
```

Resultado esperado (abreviado si se indica):

```
0    3
1    4
2    5
3    6
Name: fecha, dtype: int32
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Lunes=0, domingo=6.

Documentación oficial: parámetros y variantes

pd.Series.dt.strftime

Formatea fechas como texto; úsalo para presentación.

```
v['fecha'].dt.strftime('%d/%m/%Y')
```

Resultado esperado (abreviado si se indica):

```
0    01/01/2026
1    02/01/2026
2    03/01/2026
3    04/01/2026
Name: fecha, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve texto; conserva fechas tipadas para cálculos.

Documentación oficial: parámetros y variantes

pd.Series.dt.floor

Redondea hacia abajo; úsalo para agrupar por día.

```
v['fecha'].dt.floor('D')
```

Resultado esperado (abreviado si se indica):

```
0   2026-01-01
1   2026-01-02
2   2026-01-03
3   2026-01-04
Name: fecha, dtype: datetime64[us]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Exige frecuencias fijas; para meses utiliza períodos.

Documentación oficial: parámetros y variantes

pd.Series.dt.normalize

Lleva horas a medianoche; úsalo para fechas diarias.

```
v['fecha'].dt.normalize()
```

Resultado esperado (abreviado si se indica):

```
0   2026-01-01
1   2026-01-02
2   2026-01-03
3   2026-01-04
Name: fecha, dtype: datetime64[us]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Conserva la zona horaria.

Documentación oficial: parámetros y variantes

pd.Series.dt.to_period

Convierte a períodos mensuales; úsalo para agregación contable.

```
v['fecha'].dt.to_period('M')
```

Resultado esperado (abreviado si se indica):

```
0    2026-01
1    2026-01
2    2026-01
3    2026-01
Name: fecha, dtype: period[M]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Al convertir fechas con zona se pierde la zona.

Documentación oficial: parámetros y variantes

pd.Series.dt.tz_localize

Asigna zona a horas locales; úsalo con fechas sin zona.

```
v['fecha'].dt.tz_localize('Europe/Madrid')
```

Resultado esperado (abreviado si se indica):

```
0   2026-01-01 00:00:00...
1   2026-01-02 00:00:00...
2   2026-01-03 00:00:00...
3   2026-01-04 00:00:00...
Name: fecha, dtype: datetime64[us, Europe/Madrid]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: ambiguous y nonexistent resuelven cambios de horario.

Documentación oficial: parámetros y variantes

pd.Series.dt.tz_convert

Cambia la representación de zona; úsalo para mostrar un instante local.

```
v['fecha'].dt.tz_localize('UTC').dt.tz_convert('Europe/Madrid')
```

Resultado esperado (abreviado si se indica):

```
0   2026-01-01 01:00:00...
1   2026-01-02 01:00:00...
2   2026-01-03 01:00:00...
3   2026-01-04 01:00:00...
Name: fecha, dtype: datetime64[us, Europe/Madrid]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Necesita fechas ya conscientes de zona.

Documentación oficial: parámetros y variantes

pd.Series.dt.total_seconds

Convierte duración a segundos totales; úsalo para tiempos de servicio.

```
pd.Series(pd.to_timedelta(['1 day','2 hours'])).dt.total_seconds()
```

Resultado esperado (abreviado si se indica):

```
0    86400.0
1     7200.0
dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: dt.seconds solo da el componente dentro del día.

Documentación oficial: parámetros y variantes

pd.Series.dt.isocalendar

Extrae año, semana y día ISO; úsalo para informes semanales.

```
v['fecha'].dt.isocalendar()
```

Resultado esperado (abreviado si se indica):

```
   year  week  day
0  2026     1    4
1  2026     1    5
2  2026     1    6
3  2026     1    7
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: El año ISO puede diferir del año de calendario.

Documentación oficial: parámetros y variantes

pd.Series.resample

Agrupa por intervalos temporales; úsalo para ventas cada dos días.

```
t.resample('2D').sum()
```

Resultado esperado (abreviado si se indica):

```
fecha
2026-01-01    40.0
2026-01-03    60.0
Freq: 2D, Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: resample devuelve Resampler; necesita índice temporal o on en DataFrame.

Documentación oficial: parámetros y variantes

pd.Series.asfreq

Reindexa a una frecuencia; úsalo para explicitar huecos.

```
t.asfreq('12h')
```

Resultado esperado (abreviado si se indica):

```
fecha
2026-01-01 00:00:00    20.0
2026-01-01 12:00:00     NaN
2026-01-02 00:00:00    20.0
                       ...
2026-01-03 00:00:00    30.0
... [salida abreviada; 9 líneas]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No agrega; resample sí permite agregaciones.

Documentación oficial: parámetros y variantes

pd.Series.shift

Desplaza valores; úsalo para comparar filas consecutivas.

```
s.shift(1)
```

Resultado esperado (abreviado si se indica):

```
0     NaN
1    20.0
2    20.0
3    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: freq desplaza etiquetas temporales, no valores respecto al índice.

Documentación oficial: parámetros y variantes

pd.DataFrame.between_time

Filtra por hora diaria; úsalo para una franja horaria.

```
v.set_index('fecha').between_time('00:00','01:00')
```

Resultado esperado (abreviado si se indica):

```
           cliente   ciudad  unidades  precio  total
fecha
2026-01-01     Ana   Madrid         2    10.0   20.0
2026-01-02    Luis  Sevilla         1    20.0   20.0
2026-01-03     Ana   Madrid         3    10.0   30.0
2026-01-04     Eva      NaN         2    15.0   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Necesita DatetimeIndex; no filtra por fecha del calendario.

Documentación oficial: parámetros y variantes

pd.Series.dt.quarter

Extrae trimestre; úsalo para informes trimestrales.

```
v['fecha'].dt.quarter
```

Resultado esperado (abreviado si se indica):

```
0    1
1    1
2    1
3    1
Name: fecha, dtype: int32
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Atributo entero del 1 al 4.

Documentación oficial: parámetros y variantes

pd.Series.dt.days_in_month

Da días del mes; úsalo para prorratear importes.

```
v['fecha'].dt.days_in_month
```

Resultado esperado (abreviado si se indica):

```
0    31
1    31
2    31
3    31
Name: fecha, dtype: int32
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Atributo; contempla años bisiestos.

Documentación oficial: parámetros y variantes

pd.Series.dt.is_month_end

Indica fin de mes; úsalo para cortes contables.

```
v['fecha'].dt.is_month_end
```

Resultado esperado (abreviado si se indica):

```
0    False
1    False
2    False
3    False
Name: fecha, dtype: bool
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No indica el último día laborable.

Documentación oficial: parámetros y variantes

pd.Series.dt.day_name

Devuelve nombre del día; úsalo para etiquetas legibles.

```
v['fecha'].dt.day_name()
```

Resultado esperado (abreviado si se indica):

```
0    Thursday
1      Friday
2    Saturday
3      Sunday
Name: fecha, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El idioma depende de locale disponible; no es un identificador estable.

Documentación oficial: parámetros y variantes

pd.Series.dt.round

Redondea al intervalo más próximo; úsalo para medidas temporales.

```
v['fecha'].dt.round('D')
```

Resultado esperado (abreviado si se indica):

```
0   2026-01-01
1   2026-01-02
2   2026-01-03
3   2026-01-04
Name: fecha, dtype: datetime64[us]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Frecuencias fijas; ambiguous y nonexistent importan con zonas.

Documentación oficial: parámetros y variantes

pd.Series.dt.ceil

Redondea hacia arriba; úsalo para el siguiente corte diario.

```
v['fecha'].dt.ceil('D')
```

Resultado esperado (abreviado si se indica):

```
0   2026-01-01
1   2026-01-02
2   2026-01-03
3   2026-01-04
Name: fecha, dtype: datetime64[us]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Una fecha ya en el corte se conserva.

Documentación oficial: parámetros y variantes

pd.Series.dt.components

Descompone duraciones; úsalo para informes de tiempos.

```
pd.Series(pd.to_timedelta(['1 day 2 hours'])).dt.components
```

Resultado esperado (abreviado si se indica):

```
   days  hours  minutes  ...  milliseconds  microseconds  nanoseconds
0     1      2        0  ...             0             0            0

[1 rows x 7 columns]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Atributo DataFrame, no método; incluye días, horas, minutos y más.

Documentación oficial: parámetros y variantes

Resampler.agg

Calcula varios resúmenes temporales; úsalo para informes por intervalo.

```
t.resample('2D').agg(['sum','mean'])
```

Resultado esperado (abreviado si se indica):

```
             sum  mean
fecha
2026-01-01  40.0  20.0
2026-01-03  60.0  30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: closed y label se fijan en resample y controlan bordes y etiquetas.

Documentación oficial: parámetros y variantes

Resampler.ffill

Propaga al aumentar frecuencia; úsalo para estados vigentes.

```
t.resample('12h').ffill(limit=1)
```

Resultado esperado (abreviado si se indica):

```
fecha
2026-01-01 00:00:00    20.0
2026-01-01 12:00:00    20.0
2026-01-02 00:00:00    20.0
                       ...
2026-01-03 00:00:00    30.0
... [salida abreviada; 9 líneas]
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Los nulos originales y los huecos de reindexación no son siempre equivalentes.

Documentación oficial: parámetros y variantes

Ventanas

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.Series.rolling

Crea una ventana móvil y la promedia; úsalo para suavizar ventas.

```
s.rolling(window=2, min_periods=1).mean()
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1    20.0
2    25.0
3    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: rolling devuelve Rolling; center cambia etiquetas, no añade observaciones.

Documentación oficial: parámetros y variantes

Rolling.sum

Suma cada ventana; úsalo para facturación de dos observaciones.

```
s.rolling(2, min_periods=1).sum()
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1    40.0
2    50.0
3    60.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Una ventana entera cuenta filas; una temporal cuenta un intervalo.

Documentación oficial: parámetros y variantes

Rolling.mean

Promedia una ventana; úsalo para suavizado temporal.

```
t.rolling('2D', min_periods=1).mean()
```

Resultado esperado (abreviado si se indica):

```
fecha
2026-01-01    20.0
2026-01-02    20.0
2026-01-03    25.0
2026-01-04    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: El índice debe ser temporal y ordenado con ventana de duración.

Documentación oficial: parámetros y variantes

Rolling.apply

Calcula una función por ventana; úsalo para una medida propia.

```
s.rolling(2).apply(lambda a: a.max()-a.min(), raw=True)
```

Resultado esperado (abreviado si se indica):

```
0     NaN
1     0.0
2    10.0
3     0.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: raw=True pasa ndarray; la función debe devolver un escalar.

Documentación oficial: parámetros y variantes

pd.Series.expanding

Crea ventana acumulada; úsalo para media desde el inicio.

```
s.expanding(min_periods=1).mean()
```

Resultado esperado (abreviado si se indica):

```
0    20.000000
1    20.000000
2    23.333333
3    25.000000
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: expanding devuelve Expanding; no limita a las últimas n filas.

Documentación oficial: parámetros y variantes

pd.Series.ewm

Pondera exponencialmente el pasado; úsalo para suavizado reciente.

```
s.ewm(span=2, adjust=False).mean()
```

Resultado esperado (abreviado si se indica):

```
0    20.000000
1    20.000000
2    26.666667
3    28.888889
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: span, alpha, com y halflife parametrizan el decaimiento; elige uno.

Documentación oficial: parámetros y variantes

ExponentialMovingWindow.mean

Calcula la media exponencial; úsalo para una señal suavizada.

```
s.ewm(alpha=.5, adjust=False).mean()
```

Resultado esperado (abreviado si se indica):

```
0    20.0
1    20.0
2    25.0
3    27.5
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: adjust=False aplica una recurrencia; adjust=True normaliza pesos históricos.

Documentación oficial: parámetros y variantes

Window.mean

Promedia con pesos de una ventana; úsalo para suavizado gaussiano.

```
s.rolling(3, win_type='gaussian').mean(std=1)
```

Resultado esperado (abreviado si se indica):

```
0          NaN
1          NaN
2    22.740686
3    27.259314
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: win_type requiere SciPy; std aquí pertenece a la ventana gaussiana.

Documentación oficial: parámetros y variantes

pd.api.indexers.FixedForwardWindowIndexer

Construye una ventana hacia delante; úsalo para objetivos futuros.

```
s.rolling(pd.api.indexers.FixedForwardWindowIndexer(window_size=2), min_periods=1).sum()
```

Resultado esperado (abreviado si se indica):

```
0    40.0
1    50.0
2    60.0
3    30.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Introduce información futura: no la uses como predictor del mismo instante.

Documentación oficial: parámetros y variantes

Datos categóricos

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.CategoricalDtype

Define categorías y orden; úsalo para un esquema compartido.

```
v['cliente'].astype(pd.CategoricalDtype(['Eva','Luis','Ana'], ordered=True))
```

Resultado esperado (abreviado si se indica):

```
0     Ana
1    Luis
2     Ana
3     Eva
Name: cliente, dtype: category
Categories (3, str): ['Eva' < 'Luis' < 'Ana']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Valores fuera de las categorías se convierten en nulos.

Documentación oficial: parámetros y variantes

pd.Series.cat.categories

Da etiquetas de categorías; úsalo para revisar niveles.

```
cat.cat.categories
```

Resultado esperado (abreviado si se indica):

```
Index(['Ana', 'Eva', 'Luis'], dtype='str')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: Atributo Index; usa rename_categories para renombrar.

Documentación oficial: parámetros y variantes

pd.Series.cat.codes

Da códigos enteros; úsalo para inspeccionar la representación.

```
cat.cat.codes
```

Resultado esperado (abreviado si se indica):

```
0    0
1    2
2    0
3    1
dtype: int8
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: -1 significa nulo; los códigos no son una magnitud numérica.

Documentación oficial: parámetros y variantes

pd.Series.cat.ordered

Indica si hay orden semántico; úsalo antes de comparar.

```
cat.cat.ordered
```

Resultado esperado (abreviado si se indica):

```
False
```

Devuelve: bool. **Efecto:** No modifica los datos de entrada.

Atención: Atributo bool, sin paréntesis.

Documentación oficial: parámetros y variantes

pd.Series.cat.add_categories

Añade niveles permitidos; úsalo antes de asignar una etiqueta nueva.

```
cat.cat.add_categories(['Zoe'])
```

Resultado esperado (abreviado si se indica):

```
0     Ana
1    Luis
2     Ana
3     Eva
Name: cliente, dtype: category
Categories (4, str): ['Ana', 'Eva', 'Luis', 'Zoe']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: No añade filas ni cambia valores existentes.

Documentación oficial: parámetros y variantes

pd.Series.cat.remove_unused_categories

Elimina niveles sin uso; úsalo tras filtrar.

```
cat.iloc[:1].cat.remove_unused_categories()
```

Resultado esperado (abreviado si se indica):

```
0    Ana
Name: cliente, dtype: category
Categories (1, str): ['Ana']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Puede cambiar códigos internos.

Documentación oficial: parámetros y variantes

pd.Series.cat.rename_categories

Renombra niveles; úsalo para etiquetas de presentación.

```
cat.cat.rename_categories({'Ana':'ANA'})
```

Resultado esperado (abreviado si se indica):

```
0     ANA
1    Luis
2     ANA
3     Eva
Name: cliente, dtype: category
Categories (3, str): ['ANA', 'Eva', 'Luis']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Los nombres deben seguir siendo únicos y no nulos.

Documentación oficial: parámetros y variantes

pd.Series.cat.reorder_categories

Reordena niveles existentes; úsalo para clasificación personalizada.

```
cat.cat.reorder_categories(['Luis','Eva','Ana'], ordered=True)
```

Resultado esperado (abreviado si se indica):

```
0     Ana
1    Luis
2     Ana
3     Eva
Name: cliente, dtype: category
Categories (3, str): ['Luis' < 'Eva' < 'Ana']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Debe contener exactamente las categorías existentes.

Documentación oficial: parámetros y variantes

pd.Series.cat.set_categories

Sustituye niveles permitidos; úsalo para fijar un dominio.

```
cat.cat.set_categories(['Ana','Luis'], ordered=True)
```

Resultado esperado (abreviado si se indica):

```
0     Ana
1    Luis
2     Ana
3     NaN
Name: cliente, dtype: category
Categories (2, str): ['Ana' < 'Luis']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Las etiquetas eliminadas pasan a ser nulas.

Documentación oficial: parámetros y variantes

pd.Series.cat.as_ordered

Declara un orden semántico; úsalo para comparar categorías.

```
cat.cat.as_ordered()
```

Resultado esperado (abreviado si se indica):

```
0     Ana
1    Luis
2     Ana
3     Eva
Name: cliente, dtype: category
Categories (3, str): ['Ana' < 'Eva' < 'Luis']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Utiliza el orden actual de categories, no el de aparición.

Documentación oficial: parámetros y variantes

pd.Series.cat.remove_categories

Elimina niveles permitidos; úsalo para retirar una categoría.

```
cat.cat.remove_categories(['Eva'])
```

Resultado esperado (abreviado si se indica):

```
0     Ana
1    Luis
2     Ana
3     NaN
Name: cliente, dtype: category
Categories (2, str): ['Ana', 'Luis']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Los valores de niveles eliminados pasan a ser nulos.

Documentación oficial: parámetros y variantes

pd.Series.cat.as_unordered

Quita el orden semántico; úsalo para etiquetas nominales.

```
cat.cat.as_unordered()
```

Resultado esperado (abreviado si se indica):

```
0     Ana
1    Luis
2     Ana
3     Eva
Name: cliente, dtype: category
Categories (3, str): ['Ana', 'Eva', 'Luis']
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Las categorías permanecen; comparaciones de orden dejan de ser válidas.

Documentación oficial: parámetros y variantes

Índices y MultiIndex

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.set_index

Convierte columnas en índice; úsalo para selección por clave.

```
v.set_index('cliente')
```

Resultado esperado (abreviado si se indica):

```
          ciudad  unidades  precio      fecha  total
cliente
Ana       Madrid         2    10.0 2026-01-01   20.0
Luis     Sevilla         1    20.0 2026-01-02   20.0
Ana       Madrid         3    10.0 2026-01-03   30.0
Eva          NaN         2    15.0 2026-01-04   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: El parámetro verify_integrity está OBSOLETO en pandas 3.0.6: comprueba resultado.index.is_unique directamente.

Documentación oficial: parámetros y variantes

pd.DataFrame.reset_index

Lleva niveles del índice a columnas; úsalo para salida tabular.

```
m.reset_index()
```

Resultado esperado (abreviado si se indica):

```
  cliente      fecha   ciudad  unidades  precio  total
0     Ana 2026-01-01   Madrid         2    10.0   20.0
1    Luis 2026-01-02  Sevilla         1    20.0   20.0
2     Ana 2026-01-03   Madrid         3    10.0   30.0
3     Eva 2026-01-04      NaN         2    15.0   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: drop=True elimina etiquetas; los nombres pueden chocar con columnas existentes.

Documentación oficial: parámetros y variantes

pd.Index.is_unique

Indica unicidad de etiquetas; úsalo antes de una unión.

```
i.is_unique
```

Resultado esperado (abreviado si se indica):

```
True
```

Devuelve: bool. **Efecto:** No modifica los datos de entrada.

Atención: Atributo bool; un índice puede admitir duplicados.

Documentación oficial: parámetros y variantes

pd.Index.name

Da el nombre del índice; úsalo para interpretar un eje.

```
i.name
```

Resultado esperado (abreviado si se indica):

```
'cliente'
```

Devuelve: str. **Efecto:** No modifica los datos de entrada.

Atención: No equivale a sus valores; MultiIndex usa names.

Documentación oficial: parámetros y variantes

pd.Index.names

Da nombres de niveles; úsalo para un índice jerárquico.

```
m.index.names
```

Resultado esperado (abreviado si se indica):

```
FrozenList(['cliente', 'fecha'])
```

Devuelve: FrozenList. **Efecto:** No modifica los datos de entrada.

Atención: Atributo de tipo lista inmutable.

Documentación oficial: parámetros y variantes

pd.Index.get_loc

Localiza una etiqueta; úsalo para convertirla a selección posicional.

```
i.get_loc('Luis')
```

Resultado esperado (abreviado si se indica):

```
1
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: Con duplicados puede devolver slice o máscara en vez de int.

Documentación oficial: parámetros y variantes

pd.Index.get_indexer

Obtiene posiciones para etiquetas destino; úsalo para alineación.

```
i.get_indexer(['Eva','X'])
```

Resultado esperado (abreviado si se indica):

```
array([ 2, -1])
```

Devuelve: ndarray. **Efecto:** No modifica los datos de entrada.

Atención: -1 indica ausente; no lo pases sin tratar a iloc porque seleccionaría la última fila.

Documentación oficial: parámetros y variantes

pd.Index.union

Une conjuntos de etiquetas; úsalo para un dominio combinado.

```
i.union(pd.Index(['Zoe','Ana']), sort=False)
```

Resultado esperado (abreviado si se indica):

```
Index(['Ana', 'Luis', 'Eva', 'Zoe'], dtype='str')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: sort=False conserva el orden de la operación.

Documentación oficial: parámetros y variantes

pd.Index.intersection

Obtiene etiquetas comunes; úsalo para comparar claves.

```
i.intersection(pd.Index(['Ana','Zoe']), sort=False)
```

Resultado esperado (abreviado si se indica):

```
Index(['Ana'], dtype='str')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: No es una unión de filas de tablas.

Documentación oficial: parámetros y variantes

pd.Index.difference

Obtiene etiquetas ausentes en otro índice; úsalo para detectar claves nuevas.

```
i.difference(pd.Index(['Ana']), sort=False)
```

Resultado esperado (abreviado si se indica):

```
Index(['Luis', 'Eva'], dtype='str')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: El resultado elimina duplicados.

Documentación oficial: parámetros y variantes

pd.Index.isin

Prueba pertenencia de etiquetas; úsalo para máscaras de índice.

```
i.isin(['Ana','Eva'])
```

Resultado esperado (abreviado si se indica):

```
array([ True, False,  True])
```

Devuelve: ndarray. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve ndarray booleano.

Documentación oficial: parámetros y variantes

pd.Index.rename

Cambia el nombre del eje; úsalo para metadatos claros.

```
i.rename('nombre')
```

Resultado esperado (abreviado si se indica):

```
Index(['Ana', 'Luis', 'Eva'], dtype='str', name='nombre')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: No renombra las etiquetas individuales.

Documentación oficial: parámetros y variantes

pd.MultiIndex.from_product

Crea todas las combinaciones de niveles; úsalo para una rejilla completa.

```
pd.MultiIndex.from_product([['Ana','Luis'], [1,2]], names=['cliente','mes'])
```

Resultado esperado (abreviado si se indica):

```
MultiIndex([( 'Ana', 1),
            ( 'Ana', 2),
            ('Luis', 1),
            ('Luis', 2)],
           names=['cliente', 'mes'])
```

Devuelve: MultiIndex. **Efecto:** No modifica los datos de entrada.

Atención: Puede crecer rápidamente con muchas categorías.

Documentación oficial: parámetros y variantes

pd.MultiIndex.from_tuples

Crea un índice con pares observados; úsalo para claves compuestas.

```
pd.MultiIndex.from_tuples([('Ana',1),('Luis',2)], names=['cliente','mes'])
```

Resultado esperado (abreviado si se indica):

```
MultiIndex([( 'Ana', 1),
            ('Luis', 2)],
           names=['cliente', 'mes'])
```

Devuelve: MultiIndex. **Efecto:** No modifica los datos de entrada.

Atención: Las tuplas deben tener una estructura coherente.

Documentación oficial: parámetros y variantes

pd.MultiIndex.get_level_values

Extrae un nivel; úsalo para filtrar claves compuestas.

```
m.index.get_level_values('cliente')
```

Resultado esperado (abreviado si se indica):

```
Index(['Ana', 'Luis', 'Ana', 'Eva'], dtype='str', name='cliente')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve Index con una etiqueta por fila, no solo los niveles únicos.

Documentación oficial: parámetros y variantes

pd.DataFrame.swaplevel

Intercambia niveles; úsalo para cambiar prioridad de acceso.

```
m.swaplevel('cliente','fecha')
```

Resultado esperado (abreviado si se indica):

```
                    ciudad  unidades  precio  total
fecha      cliente
2026-01-01 Ana       Madrid         2    10.0   20.0
2026-01-02 Luis     Sevilla         1    20.0   20.0
2026-01-03 Ana       Madrid         3    10.0   30.0
2026-01-04 Eva          NaN         2    15.0   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No ordena automáticamente; añade sort_index cuando sea necesario.

Documentación oficial: parámetros y variantes

pd.DataFrame.reorder_levels

Permuta niveles del índice; úsalo para un esquema jerárquico.

```
m.reorder_levels(['fecha','cliente'])
```

Resultado esperado (abreviado si se indica):

```
                    ciudad  unidades  precio  total
fecha      cliente
2026-01-01 Ana       Madrid         2    10.0   20.0
2026-01-02 Luis     Sevilla         1    20.0   20.0
2026-01-03 Ana       Madrid         3    10.0   30.0
2026-01-04 Eva          NaN         2    15.0   30.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Debe incluir todos los niveles exactamente una vez.

Documentación oficial: parámetros y variantes

pd.MultiIndex.droplevel

Elimina un nivel; úsalo para simplificar claves.

```
m.index.droplevel('fecha')
```

Resultado esperado (abreviado si se indica):

```
Index(['Ana', 'Luis', 'Ana', 'Eva'], dtype='str', name='cliente')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: El índice resultante puede dejar de ser único.

Documentación oficial: parámetros y variantes

pd.Index.to_frame

Convierte etiquetas a una tabla; úsalo para materializar claves.

```
i.to_frame(index=False)
```

Resultado esperado (abreviado si se indica):

```
  cliente
0     Ana
1    Luis
2     Eva
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: En MultiIndex crea una columna por nivel.

Documentación oficial: parámetros y variantes

pd.Index.duplicated

Marca etiquetas repetidas; úsalo para validar claves.

```
pd.Index(['Ana','Ana','Eva']).duplicated(keep=False)
```

Resultado esperado (abreviado si se indica):

```
array([ True,  True, False])
```

Devuelve: ndarray. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve ndarray booleano; keep=False marca todas las apariciones.

Documentación oficial: parámetros y variantes

pd.Index.drop_duplicates

Quita etiquetas repetidas; úsalo para un dominio de claves.

```
pd.Index(['Ana','Ana','Eva']).drop_duplicates()
```

Resultado esperado (abreviado si se indica):

```
Index(['Ana', 'Eva'], dtype='str')
```

Devuelve: Index. **Efecto:** No modifica los datos de entrada.

Atención: No elimina las filas de una tabla asociada.

Documentación oficial: parámetros y variantes

pd.Index.is_monotonic_increasing

Indica orden no decreciente; úsalo antes de búsquedas y cortes.

```
i.is_monotonic_increasing
```

Resultado esperado (abreviado si se indica):

```
False
```

Devuelve: bool. **Efecto:** No modifica los datos de entrada.

Atención: Permite valores iguales; no garantiza unicidad.

Documentación oficial: parámetros y variantes

pd.MultiIndex.levels

Da dominios de cada nivel; úsalo para inspección jerárquica.

```
m.index.levels
```

Resultado esperado (abreviado si se indica):

```
FrozenList([['Ana', 'Eva', 'Luis'], [2026-01-01 00:00:00, 2026-01-02 00:00:00,
    2026-01-03 00:00:00, 2026-01-04 00:00:00]])
```

Devuelve: FrozenList. **Efecto:** No modifica los datos de entrada.

Atención: Pueden incluir etiquetas ya no presentes tras filtrar.

Documentación oficial: parámetros y variantes

pd.MultiIndex.codes

Da códigos por nivel; úsalo para inspección de representación.

```
m.index.codes
```

Resultado esperado (abreviado si se indica):

```
FrozenList([[0, 2, 0, 1], [0, 1, 2, 3]])
```

Devuelve: FrozenList. **Efecto:** No modifica los datos de entrada.

Atención: -1 señala ausencia; no manipules los arrays para editar el índice.

Documentación oficial: parámetros y variantes

pd.MultiIndex.remove_unused_levels

Elimina etiquetas no usadas de niveles; úsalo tras selección.

```
m.iloc[:1].index.remove_unused_levels()
```

Resultado esperado (abreviado si se indica):

```
MultiIndex([('Ana', '2026-01-01')],
           names=['cliente', 'fecha'])
```

Devuelve: MultiIndex. **Efecto:** No modifica los datos de entrada.

Atención: No cambia las tuplas visibles del índice.

Documentación oficial: parámetros y variantes

Funciones y vectorización

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.Series.map

Transforma valores por correspondencia; úsalo para una tabla de códigos.

```
v['cliente'].map({'Ana':'A','Luis':'B'})
```

Resultado esperado (abreviado si se indica):

```
0      A
1      B
2      A
3    NaN
Name: cliente, dtype: str
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Claves no presentes producen nulos; merge permite más columnas y validación.

Documentación oficial: parámetros y variantes

pd.DataFrame.map

Aplica una función a cada celda; úsalo para una transformación escalar.

```
v[['total']].map(lambda x: x / 10)
```

Resultado esperado (abreviado si se indica):

```
   total
0    2.0
1    2.0
2    3.0
3    3.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: Prefiere operaciones vectorizadas; applymap fue eliminado, usa map.

Documentación oficial: parámetros y variantes

pd.DataFrame.apply

Aplica una función por eje; úsalo para un resumen personalizado.

```
v[['unidades','total']].apply(lambda col: col.max()-col.min())
```

Resultado esperado (abreviado si se indica):

```
unidades     2.0
total       10.0
dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: axis=0 opera por columnas; axis=1 por filas. El retorno depende de la función.

Documentación oficial: parámetros y variantes

pd.Series.apply

Aplica una función al vector; úsalo si no hay operación integrada.

```
s.apply(lambda x: x * 2)
```

Resultado esperado (abreviado si se indica):

```
0    40.0
1    40.0
2    60.0
3    60.0
Name: total, dtype: float64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Para este ejemplo s*2 es más directo y suele ser más rápido.

Documentación oficial: parámetros y variantes

pd.DataFrame.agg

Agrega con una o varias funciones; úsalo para un resumen compacto.

```
v[['unidades','total']].agg(['sum','mean'])
```

Resultado esperado (abreviado si se indica):

```
      unidades  total
sum        8.0  100.0
mean       2.0   25.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: aggregate es alias; una lista produce otra forma que una sola función.

Documentación oficial: parámetros y variantes

pd.DataFrame.transform

Transforma manteniendo longitud; úsalo para resultados alineados.

```
v[['total']].transform(lambda x: x * 2)
```

Resultado esperado (abreviado si se indica):

```
   total
0   40.0
1   40.0
2   60.0
3   60.0
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: No acepta una reducción como sum que cambia la longitud.

Documentación oficial: parámetros y variantes

pd.DataFrame.pipe

Pasa la tabla a una función; úsalo para pipelines legibles.

```
v.pipe(lambda df, tasa: df.assign(iva=df['total']*tasa), tasa=.21)
```

Resultado esperado (abreviado si se indica):

```
  cliente   ciudad  unidades  ...      fecha total  iva
0     Ana   Madrid         2  ... 2026-01-01  20.0  4.2
1    Luis  Sevilla         1  ... 2026-01-02  20.0  4.2
2     Ana   Madrid         3  ... 2026-01-03  30.0  6.3
3     Eva      NaN         2  ... 2026-01-04  30.0  6.3

[4 rows x 7 columns]
```

Devuelve: DataFrame. **Efecto:** No modifica los datos de entrada.

Atención: La función decide el retorno y si muta; revisa su contrato.

Documentación oficial: parámetros y variantes

pd.DataFrame.itertuples

Itera como tuplas; úsalo si necesitas interactuar fila a fila.

```
list(v[['cliente','total']].itertuples(index=False, name=None))
```

Resultado esperado (abreviado si se indica):

```
[('Ana', 20.0), ('Luis', 20.0), ('Ana', 30.0), ('Eva', 30.0)]
```

Devuelve: list. **Efecto:** No modifica los datos de entrada.

Atención: Suele ser preferible a iterrows; vectoriza cálculos siempre que puedas.

Documentación oficial: parámetros y variantes

pd.DataFrame.iterrows

Itera filas como Series; úsalo para inspección ocasional.

```
[(idx, row['cliente']) for idx, row in v.head(2).iterrows()]
```

Resultado esperado (abreviado si se indica):

```
[(0, 'Ana'), (1, 'Luis')]
```

Devuelve: list. **Efecto:** No modifica los datos de entrada.

Atención: Puede cambiar tipos; no modifiques row esperando actualizar v.

Documentación oficial: parámetros y variantes

pd.DataFrame.items

Itera columnas como pares; úsalo para inspección por campo.

```
[(nombre, len(col)) for nombre,col in v.items()]
```

Resultado esperado (abreviado si se indica):

```
[('cliente', 4), ('ciudad', 4), ('unidades', 4), ('precio', 4), ('fecha', 4), ('total',
    4)]
```

Devuelve: list. **Efecto:** No modifica los datos de entrada.

Atención: Cada col es una Series; no es iteración de filas.

Documentación oficial: parámetros y variantes

pd.factorize

Codifica valores y devuelve únicos; úsalo para códigos temporales.

```
pd.factorize(v['cliente'], sort=True)
```

Resultado esperado (abreviado si se indica):

```
(array([0, 2, 0, 1]), Index(['Ana', 'Eva', 'Luis'], dtype='str'))
```

Devuelve: tuple. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve (códigos, únicos); -1 representa ausencias por defecto.

Documentación oficial: parámetros y variantes

pd.DataFrame.equals

Compara igualdad de datos y estructura; úsalo para comprobar una transformación.

```
v.equals(v.copy())
```

Resultado esperado (abreviado si se indica):

```
True
```

Devuelve: bool. **Efecto:** No modifica los datos de entrada.

Atención: No equivale a ==, que produce una tabla booleana; considera tipos y nulos.

Documentación oficial: parámetros y variantes

Visualización básica

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.DataFrame.plot

Dibuja una serie; úsalo para tendencia temporal.

```
v.plot(x='fecha', y='total', kind='line')
```

Resultado esperado (abreviado si se indica):

```
Objeto(s) Axes de matplotlib con el gráfico solicitado.
```

Devuelve: Axes. **Efecto:** No modifica los datos de entrada.

Atención: Requiere un backend, normalmente matplotlib; devuelve Axes con el backend por defecto.

Documentación oficial: parámetros y variantes

pd.DataFrame.plot.bar

Dibuja barras; úsalo para comparar importes.

```
v.plot.bar(x='cliente', y='total')
```

Resultado esperado (abreviado si se indica):

```
Objeto(s) Axes de matplotlib con el gráfico solicitado.
```

Devuelve: Axes. **Efecto:** No modifica los datos de entrada.

Atención: Cada fila es una barra; agrega antes si quieres una barra por cliente.

Documentación oficial: parámetros y variantes

pd.DataFrame.plot.scatter

Dibuja dispersión; úsalo para explorar una relación.

```
v.plot.scatter(x='unidades', y='total')
```

Resultado esperado (abreviado si se indica):

```
Objeto(s) Axes de matplotlib con el gráfico solicitado.
```

Devuelve: Axes. **Efecto:** No modifica los datos de entrada.

Atención: La forma no demuestra causalidad.

Documentación oficial: parámetros y variantes

pd.DataFrame.hist

Dibuja histogramas; úsalo para distribuciones.

```
v[['unidades','total']].hist(bins=3)
```

Resultado esperado (abreviado si se indica):

```
Objeto(s) Axes de matplotlib con el gráfico solicitado.
```

Devuelve: ndarray. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve array de Axes; bins afecta la interpretación.

Documentación oficial: parámetros y variantes

pd.DataFrame.boxplot

Dibuja cajas por grupo; úsalo para comparar dispersión.

```
v.boxplot(column='total', by='cliente')
```

Resultado esperado (abreviado si se indica):

```
Objeto(s) Axes de matplotlib con el gráfico solicitado.
```

Devuelve: Axes. **Efecto:** No modifica los datos de entrada.

Atención: Con muy pocas observaciones la distribución se interpreta mal.

Documentación oficial: parámetros y variantes

pd.plotting.scatter_matrix

Dibuja pares de variables; úsalo para exploración multivariante.

```
pd.plotting.scatter_matrix(v[['unidades','total']])
```

Resultado esperado (abreviado si se indica):

```
Objeto(s) Axes de matplotlib con el gráfico solicitado.
```

Devuelve: ndarray. **Efecto:** No modifica los datos de entrada.

Atención: Crece cuadráticamente con el número de columnas.

Documentación oficial: parámetros y variantes

Rendimiento y buenas prácticas

Llamadas independientes con los datos comunes. Salidas obtenidas con pandas 3.0.6.

pd.option_context

Cambia opciones temporalmente; úsalo para inspección sin cambiar configuración global.

```
with pd.option_context('display.max_rows', 8):
    resultado = pd.get_option('display.max_rows')
resultado  # 8
```

Resultado esperado (abreviado si se indica):

```
8
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: Utiliza with pd.option_context(...); las opciones se restauran al salir.

Documentación oficial: parámetros y variantes

pd.get_option

Consulta una opción; úsalo para diagnosticar presentación.

```
pd.get_option('display.max_rows')
```

Resultado esperado (abreviado si se indica):

```
60
```

Devuelve: int. **Efecto:** No modifica los datos de entrada.

Atención: La opción cambia cómo se muestra, no los datos.

Documentación oficial: parámetros y variantes

pd.set_option

Cambia una opción global; úsalo en una sesión interactiva.

```
pd.set_option('display.max_rows', 20)
```

Resultado esperado (abreviado si se indica):

```
None; operación realizada sin error.
```

Devuelve: NoneType. **Efecto:** Modifica configuración global; no v.

Atención: Devuelve None; evita cambios globales inesperados en librerías.

Documentación oficial: parámetros y variantes

pd.reset_option

Restaura una opción por defecto; úsalo para limpiar configuración.

```
pd.reset_option('display.max_rows')
```

Resultado esperado (abreviado si se indica):

```
None; operación realizada sin error.
```

Devuelve: NoneType. **Efecto:** Modifica configuración global; no v.

Atención: Devuelve None; reset_option('all') afecta a todas las opciones.

Documentación oficial: parámetros y variantes

pd.testing.assert_frame_equal

Comprueba igualdad con diagnóstico; úsalo en validaciones.

```
pd.testing.assert_frame_equal(v, v.copy())
```

Resultado esperado (abreviado si se indica):

```
None; operación realizada sin error.
```

Devuelve: NoneType. **Efecto:** No modifica los datos de entrada.

Atención: Devuelve None si pasa; AssertionError si no. check_dtype controla tipos.

Documentación oficial: parámetros y variantes

pd.util.hash_pandas_object

Calcula hashes por fila; úsalo para comprobaciones rápidas.

```
pd.util.hash_pandas_object(v, index=True)
```

Resultado esperado (abreviado si se indica):

```
0    14185775940920414849
1    12946545718994452517
2     4294131261203008949
3    16119414848305871216
dtype: uint64
```

Devuelve: Series. **Efecto:** No modifica los datos de entrada.

Atención: Un hash no es prueba criptográfica ni garantiza igualdad sin colisiones.

Documentación oficial: parámetros y variantes

Comparativas rápidas

Selección	Referencia	Regla
loc	Etiquetas / máscara	v.loc[0:2, "total"] incluye etiqueta 2.
iloc	Posiciones enteras	v.iloc[0:2, 5] excluye posición 2.
at / iat	Un escalar	at por etiqueta; iat por posición.
[]	Columnas o máscara	v["total"] es Series; v[["total"]] es DataFrame.

Combinación	Caso habitual	Clave y precaución
merge	Relacionar ventas y clientes	Columnas; how="inner" por defecto. validate evita duplicaciones inesperadas.
join	Añadir dimensión indexada	Índices; how="left" por defecto. on usa columna de la izquierda contra índice derecho.
concat	Apilar lotes o añadir columnas	axis=0 apila; axis=1 alinea etiquetas. No hace coincidencia por columnas clave.

GroupBy	Forma de salida	Cuándo elegirlo
agg	Reduce a una fila por grupo, en el caso habitual	Facturación y número de ventas por cliente.
transform	Conserva longitud y alineación	Total del cliente en cada venta; cuotas y normalizaciones.
apply	Forma definida por la función	Solo cuando una agregación o transformación integrada no expresa la operación.

Alternativas	Diferencia útil
pivot / pivot_table	pivot exige pares únicos; pivot_table agrega repetidos.
asfreq / resample	asfreq reindexa; resample agrupa intervalos para agregar.
size / count / nunique	Filas o celdas totales / valores presentes / valores distintos.
where / mask / filtro	Conservar si True / reemplazar si True / eliminar filas si False.
map / apply / vectorizar	Celda o valor / eje o grupo / operación sobre arrays completos.
tz_localize / tz_convert	Interpretar horas locales / mostrar los mismos instantes en otra zona.

Compatibilidad y obsolescencias

Pandas 3 cambia comportamientos que pueden invalidar recetas antiguas. Comprueba tipos, índices y ausencias al migrar; no basta con que el nombre del método siga existiendo.

Receta antigua o riesgo	Alternativa actual / motivo
DataFrame.append / Series.append (retirados)	pd.concat([a, b]); acumula objetos en una lista y concatena una vez.
DataFrame.applymap (retirado)	DataFrame.map para una función por celda; prefiere vectorizar si es posible.
fillna(method="ffill") / method="bfill" (retirado)	ffill() / bfill().
pd.to_numeric(..., errors="ignore") (retirado)	errors="raise" o "coerce"; captura la excepción explícitamente si procede.
df[col][mascara] = valor	df.loc[mascara, col] = valor. Copy-on-Write es obligatorio en pandas 3; la asignación encadenada no actualiza df.
df[col].fillna(0, inplace=True)	df[col] = df[col].fillna(0), o df.fillna({col:0}, inplace=True).
Asumir object para todas las cadenas	Pandas 3 infiere dtype str. Usa is_string_dtype y conversiones explícitas.
GroupBy.apply(..., include_groups=True)	Las columnas de grupo quedan excluidas en pandas 3; selecciona columnas necesarias y no solicites include_groups=True.
groupby con categorías no observadas	observed=True es el valor predeterminado de pandas 3; fija observed=False si necesitas combinaciones no observadas.
Alias temporales antiguos	Offsets de fin de mes ME, trimestre QE y año YE; horas h. Period sigue usando M/Q/Y para períodos.
Argumento copy en APIs con Copy-on-Write	Algunas firmas conservan copy obsoleto e ignorado; usa copy() explícito cuando necesites una copia y consulta cada ficha oficial.

Los elementos retirados se muestran únicamente para migración y no se incluyen como llamadas ejecutables del catálogo actual. Las entradas obsoletas detectadas en el HTML oficial llevan una marca en el anexo.

Rendimiento: selecciona columnas al leer; fija tipos; usa Parquet para persistencia tipada; procesa CSV por chunks si no cabe; agrega antes de unir cuando sea correcto. Evita apply(axis=1) para cálculos que pueden escribirse con columnas. No asumas que .str es siempre más rápido: mide con datos representativos.

Integridad: valida claves con is_unique o validate; recuerda la alineación por índice en aritmética y asignación. Usa &, | y ~ para máscaras, con paréntesis. Antes de convertir nulos a cero, comprueba su significado. inplace=True no garantiza ahorro de memoria.

Anexo de API pública

Catálogo organizado del resto de la referencia pública. Los nombres se mantienen como aparecen en la documentación; cada uno enlaza a su página. Las entradas con ficha práctica indican su página. Una firma es una referencia de parámetros, no código de ejemplo completo. Los atributos se distinguen por carecer de llamada en la firma.

No importes pandas.core aunque un identificador histórico aparezca así en la documentación. GroupBy, Rolling, Resampler y Styler se obtienen con groupby(), rolling(), resample() y .style. Las anotaciones de tipos pueden usar pandas.api.typing.

Lectura y escritura

57 entradas documentadas.

pandas.read_pickle · ficha p. 13

pd.read_pickle(filepath_or_buffer,compression='infer',storage_options=None)

pandas.DataFrame.to_pickle · ficha p. 14

DataFrame.to_pickle(path,*,compression='infer',protocol=5,storage_options=None)

pandas.read_table

pd.read_table(filepath_or_buffer,*,sep=<no_default>,delimiter=None,header='infer',names=<no_default>,index_col=None,usecols=None,dtype=None,engine=None,converters=None,true_values=None,false_values=None,skipinitialspace=False,skiprows=None,skipfooter=0,nrows=None,na_values=None,keep_default_na=True,na_filter=True,skip_blank_lines=True,parse_dates=None,date_format=None,dayfirst=False,cache_dates=True,iterator=False,chunksize=None,compression='infer',thousands=None,decimal='.',lineterminator=None,quotechar='"',quoting=0,doublequote=True,escapechar=None,comment=None,encoding=None,encoding_errors='strict',dialect=None,on_bad_lines='error',low_memory=True,memory_map=False,float_precision=None,storage_options=None,dtype_backend=<no_default>)

pandas.read_csv · ficha p. 10

pd.read_csv(filepath_or_buffer,*,sep=<no_default>,delimiter=None,header='infer',names=<no_default>,index_col=None,usecols=None,dtype=None,engine=None,converters=None,true_values=None,false_values=None,skipinitialspace=False,skiprows=None,skipfooter=0,nrows=None,na_values=None,keep_default_na=True,na_filter=True,skip_blank_lines=True,parse_dates=None,date_format=None,dayfirst=False,cache_dates=True,iterator=False,chunksize=None,compression='infer',thousands=None,decimal='.',lineterminator=None,quotechar='"',quoting=0,doublequote=True,escapechar=None,comment=None,encoding=None,encoding_errors='strict',dialect=None,on_bad_lines='error',low_memory=True,memory_map=False,float_precision=None,storage_options=None,dtype_backend=<no_default>)

pandas.DataFrame.to_csv · ficha p. 10

DataFrame.to_csv(path_or_buf=None,*,sep=',',na_rep='',float_format=None,columns=None,header=True,index=True,index_label=None,mode='w',encoding=None,compression='infer',quoting=None,quotechar='"',lineterminator=None,chunksize=None,date_format=None,doublequote=True,escapechar=None,decimal='.',errors='strict',storage_options=None)

pandas.read_fwf

pd.read_fwf(filepath_or_buffer,*,colspecs='infer',widths=None,infer_nrows=100,iterator=False,chunksize=None,**kwds)

pandas.read_clipboard

pd.read_clipboard(sep='\\s+',dtype_backend=<no_default>,**kwargs)

pandas.DataFrame.to_clipboard

DataFrame.to_clipboard(*,excel=True,sep=None,**kwargs)

pandas.read_excel · ficha p. 10

pd.read_excel(io,sheet_name=0,*,header=0,names=None,index_col=None,usecols=None,dtype=None,engine=None,converters=None,true_values=None,false_values=None,skiprows=None,nrows=None,na_values=None,keep_default_na=True,na_filter=True,verbose=False,parse_dates=False,date_format=None,thousands=None,decimal='.',comment=None,skipfooter=0,storage_options=None,dtype_backend=<no_default>,engine_kwargs=None)

pandas.DataFrame.to_excel · ficha p. 10

DataFrame.to_excel(excel_writer,*,sheet_name='Sheet1',na_rep='',float_format=None,columns=None,header=True,index=True,index_label=None,startrow=0,startcol=0,engine=None,merge_cells=True,inf_rep='inf',freeze_panes=None,storage_options=None,engine_kwargs=None,autofilter=False)

pandas.ExcelFile

pd.ExcelFile(path_or_buffer,engine=None,storage_options=None,engine_kwargs=None)

pandas.ExcelFile.book

ExcelFile.book

pandas.ExcelFile.sheet_names

ExcelFile.sheet_names

pandas.ExcelFile.parse

ExcelFile.parse(sheet_name=0,header=0,names=None,index_col=None,usecols=None,converters=None,true_values=None,false_values=None,skiprows=None,nrows=None,na_values=None,parse_dates=False,date_format=None,thousands=None,comment=None,skipfooter=0,dtype_backend=<no_default>,**kwds)

pandas.io.formats.style.Styler.to_excel

Styler.to_excel(excel_writer,sheet_name='Sheet1',na_rep='',float_format=None,columns=None,header=True,index=True,index_label=None,startrow=0,startcol=0,engine=None,merge_cells=True,encoding=None,inf_rep='inf',verbose=True,freeze_panes=None,storage_options=None,autofilter=False)

pandas.ExcelWriter

pd.ExcelWriter(path,engine=None,date_format=None,datetime_format=None,mode='w',storage_options=None,if_sheet_exists=None,engine_kwargs=None)

pandas.read_json · ficha p. 11

pd.read_json(path_or_buf,*,orient=None,typ='frame',dtype=None,convert_axes=None,convert_dates=True,keep_default_dates=True,precise_float=False,date_unit=None,encoding=None,encoding_errors='strict',lines=False,chunksize=None,compression='infer',nrows=None,storage_options=None,dtype_backend=<no_default>,engine='ujson')

pandas.json_normalize

pd.json_normalize(data,record_path=None,meta=None,meta_prefix=None,record_prefix=None,errors='raise',sep='.',max_level=None)

pandas.DataFrame.to_json · ficha p. 11

DataFrame.to_json(path_or_buf=None,*,orient=None,date_format=None,double_precision=10,force_ascii=True,date_unit='ms',default_handler=None,lines=False,compression='infer',index=None,indent=None,storage_options=None,mode='w')

AVISO DE OBSOLESCENCIA (API o parámetro). El formato de fecha epoch está obsoleto. Usa date_format="iso"; el método sigue vigente.

pandas.io.json.build_table_schema

pd.io.json.build_table_schema(data,index=True,primary_key=None,version=True)

pandas.read_html · ficha p. 12

pd.read_html(io,*,match='.+',flavor=None,header=None,index_col=None,skiprows=None,attrs=None,parse_dates=False,thousands=',',encoding=None,decimal='.',converters=None,na_values=None,keep_default_na=True,displayed_only=True,extract_links=None,dtype_backend=<no_default>,storage_options=None)

pandas.DataFrame.to_html · ficha p. 13

DataFrame.to_html(buf=None,*,columns=None,col_space=None,header=True,index=True,na_rep='NaN',formatters=None,float_format=None,sparsify=None,index_names=True,justify=None,max_rows=None,max_cols=None,show_dimensions=False,decimal='.',bold_rows=True,classes=None,escape=True,notebook=False,border=None,table_id=None,render_links=False,encoding=None)

pandas.io.formats.style.Styler.to_html

Styler.to_html(buf=None,*,table_uuid=None,table_attributes=None,sparse_index=None,sparse_columns=None,bold_headers=False,caption=None,max_rows=None,max_columns=None,encoding=None,doctype_html=False,exclude_styles=False,**kwargs)

pandas.read_xml · ficha p. 13

pd.read_xml(path_or_buffer,*,xpath='./*',namespaces=None,elems_only=False,attrs_only=False,names=None,dtype=None,converters=None,parse_dates=None,encoding='utf-8',parser='lxml',stylesheet=None,iterparse=None,compression='infer',storage_options=None,dtype_backend=<no_default>)

pandas.DataFrame · ficha p. 8

pd.DataFrame(data=None,index=None,columns=None,dtype=None,copy=None)

pandas.DataFrame.to_xml · ficha p. 13

DataFrame.to_xml(path_or_buffer=None,*,index=True,root_name='data',row_name='row',na_rep=None,attr_cols=None,elem_cols=None,namespaces=None,prefix=None,encoding='utf-8',xml_declaration=True,pretty_print=True,parser='lxml',stylesheet=None,compression='infer',storage_options=None)

pandas.DataFrame.to_latex

DataFrame.to_latex(buf=None,*,columns=None,header=True,index=True,na_rep='NaN',formatters=None,float_format=None,sparsify=None,index_names=True,bold_rows=False,column_format=None,longtable=None,escape=None,encoding=None,decimal='.',multicolumn=None,multicolumn_format=None,multirow=None,caption=None,label=None,position=None)

pandas.io.formats.style.Styler.to_latex

Styler.to_latex(buf=None,*,column_format=None,position=None,position_float=None,hrules=None,clines=None,label=None,caption=None,sparse_index=None,sparse_columns=None,multirow_align=None,multicol_align=None,siunitx=False,environment=None,encoding=None,convert_css=False)

pandas.read_hdf

pd.read_hdf(path_or_buf,key=None,mode='r',errors='strict',where=None,start=None,stop=None,columns=None,iterator=False,chunksize=None,**kwargs)

pandas.HDFStore.put

HDFStore.put(key,value,format=None,index=True,append=False,complib=None,complevel=None,min_itemsize=None,nan_rep=None,data_columns=None,encoding=None,errors='strict',track_times=True,dropna=False)

pandas.HDFStore.append

HDFStore.append(key,value,format=None,axes=None,index=True,append=True,complib=None,complevel=None,columns=None,min_itemsize=None,nan_rep=None,chunksize=None,expectedrows=None,dropna=None,data_columns=None,encoding=None,errors='strict')

pandas.HDFStore.get

HDFStore.get(key)

pandas.HDFStore.select

HDFStore.select(key,where=None,start=None,stop=None,columns=None,iterator=False,chunksize=None,auto_close=False)

pandas.HDFStore.info

HDFStore.info()

pandas.HDFStore.keys

HDFStore.keys(include='pandas')

pandas.HDFStore.groups

HDFStore.groups()

pandas.HDFStore.walk

HDFStore.walk(where='/')

pandas.read_feather

pd.read_feather(path,columns=None,use_threads=True,storage_options=None,dtype_backend=<no_default>)

pandas.DataFrame.to_feather

DataFrame.to_feather(path,**kwargs)

pandas.read_parquet · ficha p. 11

pd.read_parquet(path,engine='auto',columns=None,storage_options=None,dtype_backend=<no_default>,filesystem=None,filters=None,to_pandas_kwargs=None,**kwargs)

pandas.DataFrame.to_parquet · ficha p. 11

DataFrame.to_parquet(path=None,*,engine='auto',compression='snappy',index=None,partition_cols=None,storage_options=None,filesystem=None,**kwargs)

pandas.read_iceberg

pd.read_iceberg(table_identifier,catalog_name=None,*,catalog_properties=None,columns=None,row_filter=None,case_sensitive=True,snapshot_id=None,limit=None,scan_properties=None)

pandas.DataFrame.to_iceberg

DataFrame.to_iceberg(table_identifier,catalog_name=None,*,catalog_properties=None,location=None,append=False,snapshot_properties=None)

pandas.read_orc

pd.read_orc(path,columns=None,dtype_backend=<no_default>,filesystem=None,**kwargs)

pandas.DataFrame.to_orc

DataFrame.to_orc(path=None,*,engine='pyarrow',index=None,engine_kwargs=None)

pandas.read_sas

pd.read_sas(filepath_or_buffer,*,format=None,index=None,encoding=None,chunksize=None,iterator=False,compression='infer')

pandas.read_spss

pd.read_spss(path,usecols=None,convert_categoricals=True,dtype_backend=<no_default>,**kwargs)

pandas.read_sql_table

pd.read_sql_table(table_name,con,schema=None,index_col=None,coerce_float=True,parse_dates=None,columns=None,chunksize=None,dtype_backend=<no_default>)

pandas.read_sql_query · ficha p. 12

pd.read_sql_query(sql,con,index_col=None,coerce_float=True,params=None,parse_dates=None,chunksize=None,dtype=None,dtype_backend=<no_default>)

pandas.read_sql · ficha p. 12

pd.read_sql(sql,con,index_col=None,coerce_float=True,params=None,parse_dates=None,columns=None,chunksize=None,dtype_backend=<no_default>,dtype=None)

pandas.DataFrame.to_sql · ficha p. 12

DataFrame.to_sql(name,con,*,schema=None,if_exists='fail',index=True,index_label=None,chunksize=None,dtype=None,method=None)

pandas.read_stata

pd.read_stata(filepath_or_buffer,*,convert_dates=True,convert_categoricals=True,index_col=None,convert_missing=False,preserve_dtypes=True,columns=None,order_categoricals=True,chunksize=None,iterator=False,compression='infer',storage_options=None)

pandas.DataFrame.to_stata

DataFrame.to_stata(path,*,convert_dates=None,write_index=True,byteorder=None,time_stamp=None,data_label=None,variable_labels=None,version=114,convert_strl=None,compression='infer',storage_options=None,value_labels=None)

pandas.io.stata.StataReader.data_label

StataReader.data_label

pandas.io.stata.StataReader.value_labels

StataReader.value_labels()

pandas.io.stata.StataReader.variable_labels

StataReader.variable_labels()

pandas.io.stata.StataWriter.write_file

StataWriter.write_file()

Funciones generales

35 entradas documentadas.

pandas.melt

pd.melt(frame,id_vars=None,value_vars=None,var_name=None,value_name='value',col_level=None,ignore_index=True)

pandas.pivot

pd.pivot(data,*,columns,index=<no_default>,values=<no_default>)

pandas.pivot_table

pd.pivot_table(data,values=None,index=None,columns=None,aggfunc='mean',fill_value=None,margins=False,dropna=True,margins_name='All',observed=True,sort=True,**kwargs)

pandas.crosstab · ficha p. 59

pd.crosstab(index,columns,values=None,rownames=None,colnames=None,aggfunc=None,margins=False,margins_name='All',dropna=True,normalize=False)

pandas.cut · ficha p. 36

pd.cut(x,bins,right=True,labels=None,retbins=False,precision=3,include_lowest=False,duplicates='raise',ordered=True)

pandas.qcut · ficha p. 36

pd.qcut(x,q,labels=None,retbins=False,precision=3,duplicates='raise')

pandas.merge · ficha p. 54

pd.merge(left,right,how='inner',on=None,left_on=None,right_on=None,left_index=False,right_index=False,sort=False,suffixes=('_x','_y'),copy=<no_default>,indicator=False,validate=None)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.merge_ordered · ficha p. 55

pd.merge_ordered(left,right,on=None,left_on=None,right_on=None,left_by=None,right_by=None,fill_method=None,suffixes=('_x','_y'),how='outer')

pandas.merge_asof · ficha p. 55

pd.merge_asof(left,right,on=None,left_on=None,right_on=None,left_index=False,right_index=False,by=None,left_by=None,right_by=None,suffixes=('_x','_y'),tolerance=None,allow_exact_matches=True,direction='backward')

pandas.concat · ficha p. 55

pd.concat(objs,*,axis=0,join='outer',ignore_index=False,keys=None,levels=None,names=None,verify_integrity=False,sort=<no_default>,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.get_dummies · ficha p. 59

pd.get_dummies(data,prefix=None,prefix_sep='_',dummy_na=False,columns=None,sparse=False,drop_first=False,dtype=None)

pandas.from_dummies · ficha p. 59

pd.from_dummies(data,sep=None,default_category=None)

pandas.factorize · ficha p. 90

pd.factorize(values,sort=False,use_na_sentinel=True,size_hint=None)

pandas.unique

pd.unique(values)

pandas.lreshape

pd.lreshape(data,groups,dropna=True)

pandas.wide_to_long · ficha p. 59

pd.wide_to_long(df,stubnames,i,j,sep='',suffix='\\d+')

pandas.isna · ficha p. 32

pd.isna(obj)

pandas.isnull

pd.isnull(obj)

pandas.notna · ficha p. 32

pd.notna(obj)

pandas.notnull

pd.notnull(obj)

pandas.to_numeric · ficha p. 30

pd.to_numeric(arg,errors='raise',downcast=None,dtype_backend=<no_default>)

pandas.to_datetime · ficha p. 30

pd.to_datetime(arg,errors='raise',dayfirst=False,yearfirst=False,utc=False,format=None,exact=<no_default>,unit=None,origin='unix',cache=True)

pandas.to_timedelta · ficha p. 30

pd.to_timedelta(arg,unit=None,errors='raise')

pandas.date_range · ficha p. 66

pd.date_range(start=None,end=None,periods=None,freq=None,tz=None,normalize=False,name=None,inclusive='both',*,unit=None,**kwargs)

pandas.bdate_range · ficha p. 66

pd.bdate_range(start=None,end=None,periods=None,freq='B',tz=None,normalize=True,name=None,weekmask=None,holidays=None,inclusive='both',**kwargs)

pandas.period_range · ficha p. 66

pd.period_range(start=None,end=None,periods=None,freq=None,name=None)

pandas.timedelta_range · ficha p. 66

pd.timedelta_range(start=None,end=None,periods=None,freq=None,name=None,closed=None,*,unit=None)

pandas.infer_freq

pd.infer_freq(index)

pandas.interval_range

pd.interval_range(start=None,end=None,periods=None,freq=None,name=None,closed='right')

pandas.col

pd.col(col_name)

pandas.eval

pd.eval(expr,parser='pandas',engine=None,local_dict=None,global_dict=None,resolvers=(),level=0,target=None,inplace=False)

pandas.tseries.api.guess_datetime_format

pd.tseries.api.guess_datetime_format(dt_str,dayfirst=False)

pandas.util.hash_array

pd.util.hash_array(vals,encoding='utf8',hash_key='0123456789123456',categorize=True)

pandas.util.hash_pandas_object · ficha p. 94

pd.util.hash_pandas_object(obj,index=True,encoding='utf8',hash_key='0123456789123456',categorize=True)

pandas.api.interchange.from_dataframe

pd.api.interchange.from_dataframe(df,allow_copy=True)

DataFrame

201 entradas documentadas.

pandas.DataFrame.index · ficha p. 17

DataFrame.index

pandas.DataFrame.columns · ficha p. 17

DataFrame.columns

pandas.DataFrame.dtypes · ficha p. 18

DataFrame.dtypes

pandas.DataFrame.info · ficha p. 16

DataFrame.info(verbose=None,buf=None,max_cols=None,memory_usage=None,show_counts=None)

pandas.DataFrame.select_dtypes · ficha p. 21

DataFrame.select_dtypes(include=None,exclude=None)

pandas.DataFrame.values

DataFrame.values

pandas.DataFrame.axes

DataFrame.axes

pandas.DataFrame.ndim · ficha p. 17

DataFrame.ndim

pandas.DataFrame.size · ficha p. 17

DataFrame.size

pandas.DataFrame.shape · ficha p. 16

DataFrame.shape

pandas.DataFrame.memory_usage · ficha p. 18

DataFrame.memory_usage(index=True,deep=False)

pandas.DataFrame.empty · ficha p. 18

DataFrame.empty

pandas.DataFrame.set_flags

DataFrame.set_flags(*,copy=<no_default>,allows_duplicate_labels=None)
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.astype · ficha p. 29

DataFrame.astype(dtype,copy=<no_default>,errors='raise')
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.convert_dtypes · ficha p. 29

DataFrame.convert_dtypes(infer_objects=True,convert_string=True,convert_integer=True,convert_boolean=True,convert_floating=True,dtype_backend='numpy_nullable')

pandas.DataFrame.infer_objects · ficha p. 29

DataFrame.infer_objects(copy=<no_default>)
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.copy · ficha p. 28

DataFrame.copy(deep=True)

pandas.DataFrame.to_numpy · ficha p. 30

DataFrame.to_numpy(dtype=None,copy=False,na_value=<no_default>)

pandas.DataFrame.head · ficha p. 15

DataFrame.head(n=5)

pandas.DataFrame.at · ficha p. 22

DataFrame.at

pandas.DataFrame.iat · ficha p. 22

DataFrame.iat

pandas.DataFrame.loc · ficha p. 22

DataFrame.loc

pandas.DataFrame.iloc · ficha p. 22

DataFrame.iloc

pandas.DataFrame.insert · ficha p. 26

DataFrame.insert(loc,column,value,allow_duplicates=<no_default>)

pandas.DataFrame.items · ficha p. 90

DataFrame.items()

pandas.DataFrame.keys · ficha p. 21

DataFrame.keys()

pandas.DataFrame.iterrows · ficha p. 90

DataFrame.iterrows()

pandas.DataFrame.itertuples · ficha p. 90

DataFrame.itertuples(index=True,name='Pandas')

pandas.DataFrame.pop · ficha p. 27

DataFrame.pop(item)

pandas.DataFrame.tail · ficha p. 15

DataFrame.tail(n=5)

pandas.DataFrame.xs · ficha p. 24

DataFrame.xs(key,axis=0,level=None,drop_level=True)

pandas.DataFrame.get · ficha p. 25

DataFrame.get(key,default=None)

pandas.DataFrame.isin

DataFrame.isin(values)

pandas.DataFrame.where · ficha p. 24

DataFrame.where(cond,other=<no_default>,*,inplace=False,axis=None,level=None)

pandas.DataFrame.mask · ficha p. 24

DataFrame.mask(cond,other=<no_default>,*,inplace=False,axis=None,level=None)

pandas.DataFrame.query · ficha p. 23

DataFrame.query(expr,*,parser='pandas',engine=None,local_dict=None,global_dict=None,resolvers=None,level=0,inplace=False)

pandas.DataFrame.isetitem

DataFrame.isetitem(loc,value)

pandas.DataFrame.add

DataFrame.add(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.sub

DataFrame.sub(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.mul

DataFrame.mul(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.div

DataFrame.div(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.truediv

DataFrame.truediv(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.floordiv

DataFrame.floordiv(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.mod

DataFrame.mod(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.pow

DataFrame.pow(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.dot

DataFrame.dot(other)

pandas.DataFrame.radd

DataFrame.radd(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.rsub

DataFrame.rsub(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.rmul

DataFrame.rmul(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.rdiv

DataFrame.rdiv(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.rtruediv

DataFrame.rtruediv(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.rfloordiv

DataFrame.rfloordiv(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.rmod

DataFrame.rmod(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.rpow

DataFrame.rpow(other,axis='columns',level=None,fill_value=None)

pandas.DataFrame.lt

DataFrame.lt(other,axis='columns',level=None)

pandas.DataFrame.gt

DataFrame.gt(other,axis='columns',level=None)

pandas.DataFrame.le

DataFrame.le(other,axis='columns',level=None)

pandas.DataFrame.ge

DataFrame.ge(other,axis='columns',level=None)

pandas.DataFrame.ne

DataFrame.ne(other,axis='columns',level=None)

pandas.DataFrame.eq

DataFrame.eq(other,axis='columns',level=None)

pandas.DataFrame.combine

DataFrame.combine(other,func,fill_value=None,overwrite=True)

pandas.DataFrame.combine_first · ficha p. 56

DataFrame.combine_first(other)

pandas.DataFrame.apply · ficha p. 88

DataFrame.apply(func,axis=0,raw=False,result_type=None,args=(),by_row='compat',engine=None,engine_kwargs=None,**kwargs)

pandas.DataFrame.map · ficha p. 88

DataFrame.map(func,na_action=None,**kwargs)

pandas.DataFrame.pipe · ficha p. 89

DataFrame.pipe(func,*args,**kwargs)

pandas.DataFrame.agg · ficha p. 89

DataFrame.agg(func=None,axis=0,*args,**kwargs)

pandas.DataFrame.aggregate

DataFrame.aggregate(func=None,axis=0,*args,**kwargs)

pandas.DataFrame.transform · ficha p. 89

DataFrame.transform(func,axis=0,*args,**kwargs)

pandas.DataFrame.groupby · ficha p. 48

DataFrame.groupby(by=None,level=None,*,as_index=True,sort=True,group_keys=True,observed=True,dropna=True)

pandas.DataFrame.rolling

DataFrame.rolling(window,min_periods=None,center=False,win_type=None,on=None,closed=None,step=None,method='single')

pandas.DataFrame.expanding

DataFrame.expanding(min_periods=1,method='single')

pandas.DataFrame.ewm

DataFrame.ewm(com=None,span=None,halflife=None,alpha=None,min_periods=0,adjust=True,ignore_na=False,times=None,method='single')

pandas.DataFrame.abs

DataFrame.abs()

pandas.DataFrame.all

DataFrame.all(*,axis=0,bool_only=False,skipna=True,**kwargs)

pandas.DataFrame.any

DataFrame.any(*,axis=0,bool_only=False,skipna=True,**kwargs)

pandas.DataFrame.clip

DataFrame.clip(lower=None,upper=None,*,axis=None,inplace=False,**kwargs)

pandas.DataFrame.corr · ficha p. 42

DataFrame.corr(method='pearson',min_periods=1,numeric_only=False)

pandas.DataFrame.corrwith

DataFrame.corrwith(other,axis=0,drop=False,method='pearson',numeric_only=False,min_periods=None)

pandas.DataFrame.count

DataFrame.count(axis=0,numeric_only=False)

pandas.DataFrame.cov · ficha p. 42

DataFrame.cov(min_periods=None,ddof=1,numeric_only=False)

pandas.DataFrame.cummax

DataFrame.cummax(axis=0,skipna=True,numeric_only=False,*args,**kwargs)

pandas.DataFrame.cummin

DataFrame.cummin(axis=0,skipna=True,numeric_only=False,*args,**kwargs)

pandas.DataFrame.cumprod

DataFrame.cumprod(axis=0,skipna=True,numeric_only=False,*args,**kwargs)

pandas.DataFrame.cumsum

DataFrame.cumsum(axis=0,skipna=True,numeric_only=False,*args,**kwargs)

pandas.DataFrame.describe · ficha p. 16

DataFrame.describe(percentiles=None,include=None,exclude=None)

pandas.DataFrame.diff

DataFrame.diff(periods=1,axis=0)

pandas.DataFrame.eval · ficha p. 28

DataFrame.eval(expr,*,inplace=False,**kwargs)

pandas.DataFrame.kurt

DataFrame.kurt(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.DataFrame.kurtosis

DataFrame.kurtosis(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.DataFrame.max

DataFrame.max(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.DataFrame.mean

DataFrame.mean(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.DataFrame.median

DataFrame.median(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.DataFrame.min

DataFrame.min(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.DataFrame.mode

DataFrame.mode(axis=0,numeric_only=False,dropna=True)

pandas.DataFrame.pct_change

DataFrame.pct_change(periods=1,fill_method=None,freq=None,**kwargs)

pandas.DataFrame.prod

DataFrame.prod(*,axis=0,skipna=True,numeric_only=False,min_count=0,**kwargs)

pandas.DataFrame.product

DataFrame.product(*,axis=0,skipna=True,numeric_only=False,min_count=0,**kwargs)

pandas.DataFrame.quantile

DataFrame.quantile(q=0.5,axis=0,numeric_only=False,interpolation='linear',method='single')

pandas.DataFrame.rank

DataFrame.rank(axis=0,method='average',numeric_only=False,na_option='keep',ascending=True,pct=False)

pandas.DataFrame.round

DataFrame.round(decimals=0,*args,**kwargs)

pandas.DataFrame.sem

DataFrame.sem(*,axis=0,skipna=True,ddof=1,numeric_only=False,**kwargs)

pandas.DataFrame.skew

DataFrame.skew(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.DataFrame.sum

DataFrame.sum(*,axis=0,skipna=True,numeric_only=False,min_count=0,**kwargs)

pandas.DataFrame.std

DataFrame.std(*,axis=0,skipna=True,ddof=1,numeric_only=False,**kwargs)

pandas.DataFrame.var

DataFrame.var(*,axis=0,skipna=True,ddof=1,numeric_only=False,**kwargs)

pandas.DataFrame.nunique

DataFrame.nunique(axis=0,dropna=True)

pandas.DataFrame.value_counts

DataFrame.value_counts(subset=None,normalize=False,sort=True,ascending=False,dropna=True)

pandas.DataFrame.add_prefix

DataFrame.add_prefix(prefix,axis=None)

pandas.DataFrame.add_suffix

DataFrame.add_suffix(suffix,axis=None)

pandas.DataFrame.align

DataFrame.align(other,join='outer',axis=None,level=None,copy=<no_default>,fill_value=None)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.at_time

DataFrame.at_time(time,asof=False,axis=None)

pandas.DataFrame.between_time · ficha p. 71

DataFrame.between_time(start_time,end_time,inclusive='both',axis=None)

pandas.DataFrame.drop · ficha p. 26

DataFrame.drop(labels=None,*,axis=0,index=None,columns=None,level=None,inplace=False,errors='raise')

pandas.DataFrame.drop_duplicates · ficha p. 34

DataFrame.drop_duplicates(subset=None,*,keep='first',inplace=False,ignore_index=False)

pandas.DataFrame.duplicated · ficha p. 34

DataFrame.duplicated(subset=None,keep='first')

pandas.DataFrame.equals · ficha p. 90

DataFrame.equals(other)

pandas.DataFrame.filter · ficha p. 23

DataFrame.filter(items=None,like=None,regex=None,axis=None)

pandas.DataFrame.idxmax

DataFrame.idxmax(axis=0,skipna=True,numeric_only=False)

pandas.DataFrame.idxmin

DataFrame.idxmin(axis=0,skipna=True,numeric_only=False)

pandas.DataFrame.reindex · ficha p. 28

DataFrame.reindex(labels=None,*,index=None,columns=None,axis=None,method=None,copy=<no_default>,level=None,fill_value=nan,limit=None,tolerance=None)
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.reindex_like

DataFrame.reindex_like(other,method=None,copy=<no_default>,limit=None,tolerance=None)
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro method está obsoleto. Usa reindex sobre los ejes del otro objeto y especifica allí el método; copy también se ignora bajo Copy-on-Write.

pandas.DataFrame.rename · ficha p. 27

DataFrame.rename(mapper=None,*,index=None,columns=None,axis=None,copy=<no_default>,inplace=False,level=None,errors='ignore')
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.rename_axis

DataFrame.rename_axis(mapper=<no_default>,*,index=<no_default>,columns=<no_default>,axis=0,copy=<no_default>,inplace=False)
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.reset_index · ficha p. 82

DataFrame.reset_index(level=None,*,drop=False,inplace=False,col_level=0,col_fill='',allow_duplicates=<no_default>,names=None)

pandas.DataFrame.sample · ficha p. 15

DataFrame.sample(n=None,frac=None,replace=False,weights=None,random_state=None,axis=None,ignore_index=False)

pandas.DataFrame.set_axis · ficha p. 27

DataFrame.set_axis(labels,*,axis=0,copy=<no_default>)
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.set_index · ficha p. 82

DataFrame.set_index(keys,*,drop=True,append=False,inplace=False,verify_integrity=<no_default>)
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro verify_integrity está obsoleto. Comprueba resultado.index.is_unique directamente; el método sigue vigente.

pandas.DataFrame.take · ficha p. 24

DataFrame.take(indices,axis=0,**kwargs)

pandas.DataFrame.truncate

DataFrame.truncate(before=None,after=None,axis=None,copy=<no_default>)
AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.bfill · ficha p. 33

DataFrame.bfill(*,axis=None,inplace=False,limit=None,limit_area=None)

pandas.DataFrame.dropna · ficha p. 33

DataFrame.dropna(*,axis=0,how=<no_default>,thresh=<no_default>,subset=None,inplace=False,ignore_index=False)

pandas.DataFrame.ffill · ficha p. 33

DataFrame.ffill(*,axis=None,inplace=False,limit=None,limit_area=None)

pandas.DataFrame.fillna · ficha p. 33

DataFrame.fillna(value,*,axis=None,inplace=False,limit=None)

pandas.DataFrame.interpolate

DataFrame.interpolate(method='linear',*,axis=0,limit=None,inplace=False,limit_direction=None,limit_area=None,**kwargs)

pandas.DataFrame.isna · ficha p. 32

DataFrame.isna()

pandas.DataFrame.isnull

DataFrame.isnull()

pandas.DataFrame.notna

DataFrame.notna()

pandas.DataFrame.notnull

DataFrame.notnull()

pandas.DataFrame.replace

DataFrame.replace(to_replace=None,value=<no_default>,*,inplace=False,regex=False)

pandas.DataFrame.droplevel

DataFrame.droplevel(level,axis=0)

pandas.DataFrame.pivot · ficha p. 57

DataFrame.pivot(*,columns,index=<no_default>,values=<no_default>)

pandas.DataFrame.pivot_table · ficha p. 57

DataFrame.pivot_table(values=None,index=None,columns=None,aggfunc='mean',fill_value=None,margins=False,dropna=True,margins_name='All',observed=True,sort=True,**kwargs)

pandas.DataFrame.reorder_levels · ficha p. 85

DataFrame.reorder_levels(order,axis=0)

pandas.DataFrame.sort_values · ficha p. 35

DataFrame.sort_values(by,*,axis=0,ascending=True,inplace=False,kind='quicksort',na_position='last',ignore_index=False,key=None)

pandas.DataFrame.sort_index · ficha p. 35

DataFrame.sort_index(*,axis=0,level=None,ascending=True,inplace=False,kind='quicksort',na_position='last',sort_remaining=True,ignore_index=False,key=None)

pandas.DataFrame.nlargest · ficha p. 35

DataFrame.nlargest(n,columns,keep='first')

pandas.DataFrame.nsmallest · ficha p. 36

DataFrame.nsmallest(n,columns,keep='first')

pandas.DataFrame.swaplevel · ficha p. 85

DataFrame.swaplevel(i=-2,j=-1,axis=0)

pandas.MultiIndex

pd.MultiIndex(levels=None,codes=None,sortorder=None,names=None,copy=False,name=None,verify_integrity=True)

pandas.DataFrame.stack · ficha p. 58

DataFrame.stack(level=-1,dropna=<no_default>,sort=<no_default>,future_stack=True)

pandas.DataFrame.unstack · ficha p. 58

DataFrame.unstack(level=-1,fill_value=None,sort=True)

pandas.DataFrame.melt · ficha p. 57

DataFrame.melt(id_vars=None,value_vars=None,var_name=None,value_name='value',col_level=None,ignore_index=True)

pandas.DataFrame.explode · ficha p. 58

DataFrame.explode(column,ignore_index=False)

pandas.DataFrame.squeeze

DataFrame.squeeze(axis=None)

pandas.DataFrame.to_xarray

DataFrame.to_xarray()

pandas.DataFrame.T · ficha p. 20

DataFrame.T

pandas.DataFrame.transpose

DataFrame.transpose(*args,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.assign · ficha p. 26

DataFrame.assign(**kwargs)

pandas.DataFrame.compare

DataFrame.compare(other,align_axis=1,keep_shape=False,keep_equal=False,result_names=('self','other'))

pandas.DataFrame.join · ficha p. 54

DataFrame.join(other,on=None,how='left',lsuffix='',rsuffix='',sort=False,validate=None)

pandas.DataFrame.merge · ficha p. 54

DataFrame.merge(right,how='inner',on=None,left_on=None,right_on=None,left_index=False,right_index=False,sort=False,suffixes=('_x','_y'),copy=<no_default>,indicator=False,validate=None)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.update · ficha p. 27

DataFrame.update(other,join='left',overwrite=True,filter_func=None,errors='ignore')

pandas.DataFrame.asfreq

DataFrame.asfreq(freq,method=None,how=None,normalize=False,fill_value=None)

pandas.DataFrame.asof

DataFrame.asof(where,subset=None)

pandas.DataFrame.shift

DataFrame.shift(periods=1,freq=None,axis=0,fill_value=<no_default>,suffix=None)

pandas.DataFrame.first_valid_index

DataFrame.first_valid_index()

pandas.DataFrame.last_valid_index

DataFrame.last_valid_index()

pandas.DataFrame.resample

DataFrame.resample(rule,closed=None,label=None,convention='start',on=None,level=None,origin='start_day',offset=None,group_keys=False)

pandas.DataFrame.to_period

DataFrame.to_period(freq=None,axis=0,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.to_timestamp

DataFrame.to_timestamp(freq=None,how='start',axis=0,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.tz_convert

DataFrame.tz_convert(tz,axis=0,level=None,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.DataFrame.tz_localize

DataFrame.tz_localize(tz,axis=0,level=None,copy=<no_default>,ambiguous='raise',nonexistent='raise')

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Flags

pd.Flags(obj,*,allows_duplicate_labels)

pandas.DataFrame.attrs · ficha p. 20

DataFrame.attrs

pandas.DataFrame.plot · ficha p. 91

DataFrame.plot(*args,**kwargs)

pandas.DataFrame.plot.area

DataFrame.plot.area(x=None,y=None,stacked=True,**kwargs)

pandas.DataFrame.plot.bar · ficha p. 91

DataFrame.plot.bar(x=None,y=None,color=None,**kwargs)

pandas.DataFrame.plot.barh

DataFrame.plot.barh(x=None,y=None,color=None,**kwargs)

pandas.DataFrame.plot.box

DataFrame.plot.box(by=None,**kwargs)

pandas.DataFrame.plot.density

DataFrame.plot.density(bw_method=None,ind=None,weights=None,**kwargs)

pandas.DataFrame.plot.hexbin

DataFrame.plot.hexbin(x,y,C=None,reduce_C_function=None,gridsize=None,**kwargs)

pandas.DataFrame.plot.hist

DataFrame.plot.hist(by=None,bins=10,**kwargs)

pandas.DataFrame.plot.kde

DataFrame.plot.kde(bw_method=None,ind=None,weights=None,**kwargs)

pandas.DataFrame.plot.line

DataFrame.plot.line(x=None,y=None,color=None,**kwargs)

pandas.DataFrame.plot.pie

DataFrame.plot.pie(y=None,**kwargs)

pandas.DataFrame.plot.scatter · ficha p. 91

DataFrame.plot.scatter(x,y,s=None,c=None,**kwargs)

pandas.DataFrame.boxplot · ficha p. 92

DataFrame.boxplot(column=None,by=None,ax=None,fontsize=None,rot=0,grid=True,figsize=None,layout=None,return_type=None,backend=None,**kwargs)

pandas.DataFrame.hist · ficha p. 91

DataFrame.hist(column=None,by=None,grid=True,xlabelsize=None,xrot=None,ylabelsize=None,yrot=None,ax=None,sharex=False,sharey=False,figsize=None,layout=None,bins=10,backend=None,legend=False,**kwargs)

pandas.DataFrame.sparse.density

DataFrame.sparse.density

pandas.DataFrame.sparse.from_spmatrix

methodDataFrame.sparse.from_spmatrix(data,index=None,columns=None)

pandas.DataFrame.sparse.to_coo

DataFrame.sparse.to_coo()

pandas.DataFrame.sparse.to_dense

DataFrame.sparse.to_dense()

pandas.DataFrame.from_arrow

methodDataFrame.from_arrow(data)

pandas.DataFrame.from_dict · ficha p. 9

methodDataFrame.from_dict(data,orient='columns',dtype=None,columns=None)

pandas.DataFrame.from_records · ficha p. 9

methodDataFrame.from_records(data,index=None,exclude=None,columns=None,coerce_float=False,nrows=None)

pandas.DataFrame.to_hdf

DataFrame.to_hdf(path_or_buf,*,key,mode='a',complevel=None,complib=None,append=False,format=None,index=True,min_itemsize=None,nan_rep=None,dropna=None,data_columns=None,errors='strict',encoding='UTF-8')

pandas.DataFrame.to_dict · ficha p. 14

DataFrame.to_dict(orient='dict',*,into=<class'dict'>,index=True)

pandas.DataFrame.to_records · ficha p. 14

DataFrame.to_records(index=True,column_dtypes=None,index_dtypes=None)

pandas.DataFrame.to_string

DataFrame.to_string(buf=None,*,columns=None,col_space=None,header=True,index=True,na_rep='NaN',formatters=None,float_format=None,sparsify=None,index_names=True,justify=None,max_rows=None,max_cols=None,show_dimensions=False,decimal='.',line_width=None,min_rows=None,max_colwidth=None,encoding=None)

pandas.DataFrame.to_markdown

DataFrame.to_markdown(buf=None,*,mode='wt',index=True,storage_options=None,**kwargs)

pandas.DataFrame.style

DataFrame.style

Series y accesorios .str, .dt, .cat, .sparse

336 entradas documentadas.

pandas.Series · ficha p. 8

pd.Series(data=None,index=None,dtype=None,name=None,copy=None)

pandas.Series.index · ficha p. 20

Series.index

pandas.Series.array · ficha p. 31

Series.array

pandas.Series.values

Series.values

pandas.Series.dtype · ficha p. 18

Series.dtype

pandas.Series.info

Series.info(verbose=None,buf=None,max_cols=None,memory_usage=None,show_counts=True)

pandas.Series.shape · ficha p. 20

Series.shape

pandas.Series.nbytes

Series.nbytes

pandas.Series.ndim

Series.ndim

pandas.Series.size

Series.size

pandas.Series.T

Series.T

pandas.Series.memory_usage

Series.memory_usage(index=True,deep=False)

pandas.Series.hasnans · ficha p. 34

Series.hasnans

pandas.Series.empty

Series.empty

pandas.Series.dtypes

Series.dtypes

pandas.Series.name · ficha p. 19

Series.name

pandas.Series.flags

Series.flags

pandas.Series.set_flags

Series.set_flags(*,copy=<no_default>,allows_duplicate_labels=None)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.astype

Series.astype(dtype,copy=<no_default>,errors='raise')

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.convert_dtypes

Series.convert_dtypes(infer_objects=True,convert_string=True,convert_integer=True,convert_boolean=True,convert_floating=True,dtype_backend='numpy_nullable')

pandas.Series.infer_objects

Series.infer_objects(copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.copy

Series.copy(deep=True)

pandas.Series.to_numpy

Series.to_numpy(dtype=None,copy=False,na_value=<no_default>,**kwargs)

pandas.Series.to_period

Series.to_period(freq=None,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.to_timestamp

Series.to_timestamp(freq=None,how='start',copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.to_list

Series.to_list()

pandas.Series.get

Series.get(key,default=None)

pandas.Series.at

Series.at

pandas.Series.iat

Series.iat

pandas.Series.loc

Series.loc

pandas.Series.iloc

Series.iloc

pandas.Series.items

Series.items()

pandas.Series.keys

Series.keys()

pandas.Series.pop

Series.pop(item)

pandas.Series.item · ficha p. 31

Series.item()

pandas.Series.xs

Series.xs(key,axis=0,level=None,drop_level=True)

pandas.Series.add · ficha p. 41

Series.add(other,level=None,fill_value=None,axis=0)

pandas.Series.sub · ficha p. 44

Series.sub(other,level=None,fill_value=None,axis=0)

pandas.Series.mul · ficha p. 44

Series.mul(other,level=None,fill_value=None,axis=0)

pandas.Series.div · ficha p. 41

Series.div(other,level=None,fill_value=None,axis=0)

pandas.Series.truediv

Series.truediv(other,level=None,fill_value=None,axis=0)

pandas.Series.floordiv · ficha p. 44
Series.floordiv(other,level=None,fill_value=None,axis=0)

pandas.Series.mod · ficha p. 45
Series.mod(other,level=None,fill_value=None,axis=0)

pandas.Series.pow · ficha p. 45
Series.pow(other,level=None,fill_value=None,axis=0)

pandas.Series.radd
Series.radd(other,level=None,fill_value=None,axis=0)

pandas.Series.rsub
Series.rsub(other,level=None,fill_value=None,axis=0)

pandas.Series.rmul
Series.rmul(other,level=None,fill_value=None,axis=0)

pandas.Series.rdiv
Series.rdiv(other,level=None,fill_value=None,axis=0)

pandas.Series.rtruediv
Series.rtruediv(other,level=None,fill_value=None,axis=0)

pandas.Series.rfloordiv
Series.rfloordiv(other,level=None,fill_value=None,axis=0)

pandas.Series.rmod
Series.rmod(other,level=None,fill_value=None,axis=0)

pandas.Series.rpow
Series.rpow(other,level=None,fill_value=None,axis=0)

pandas.Series.combine
Series.combine(other,func,fill_value=None)

pandas.Series.combine_first
Series.combine_first(other)

pandas.Series.round · ficha p. 40
Series.round(decimals=0,*args,**kwargs)

pandas.Series.lt · ficha p. 46
Series.lt(other,level=None,fill_value=None,axis=0)

pandas.Series.gt · ficha p. 46
Series.gt(other,level=None,fill_value=None,axis=0)

pandas.Series.le · ficha p. 46
Series.le(other,level=None,fill_value=None,axis=0)

pandas.Series.ge · ficha p. 46
Series.ge(other,level=None,fill_value=None,axis=0)

pandas.Series.ne · ficha p. 45
Series.ne(other,level=None,fill_value=None,axis=0)

pandas.Series.eq · ficha p. 45
Series.eq(other,level=None,fill_value=None,axis=0)

pandas.Series.product
Series.product(*,axis=None,skipna=True,numeric_only=False,min_count=0,**kwargs)

pandas.Series.dot
Series.dot(other)

pandas.Series.apply · ficha p. 89
Series.apply(func,args=(),*,by_row='compat',**kwargs)

pandas.Series.agg

Series.agg(func=None,axis=0,*args,**kwargs)

pandas.Series.aggregate

Series.aggregate(func=None,axis=0,*args,**kwargs)

pandas.Series.transform

Series.transform(func,axis=0,*args,**kwargs)

pandas.Series.map · ficha p. 88

Series.map(func=None,na_action=None,engine=None,**kwargs)

pandas.Series.groupby

Series.groupby(by=None,level=None,*,as_index=True,sort=True,group_keys=True,observed=True,dropna=True)

pandas.Series.rolling · ficha p. 75

Series.rolling(window,min_periods=None,center=False,win_type=None,on=None,closed=None,step=None,method='single')

pandas.Series.expanding · ficha p. 76

Series.expanding(min_periods=1,method='single')

pandas.Series.ewm · ficha p. 76

Series.ewm(com=None,span=None,halflife=None,alpha=None,min_periods=0,adjust=True,ignore_na=False,times=None,method='single')

pandas.Series.pipe

Series.pipe(func,*args,**kwargs)

pandas.Series.abs · ficha p. 41

Series.abs()

pandas.Series.all · ficha p. 42

Series.all(*,axis=0,bool_only=False,skipna=True,**kwargs)

pandas.Series.any · ficha p. 42

Series.any(*,axis=0,bool_only=False,skipna=True,**kwargs)

pandas.Series.autocorr

Series.autocorr(lag=1)

pandas.Series.between · ficha p. 23

Series.between(left,right,inclusive='both')

pandas.Series.clip · ficha p. 41

Series.clip(lower=None,upper=None,*,axis=None,inplace=False,**kwargs)

pandas.Series.corr

Series.corr(other,method='pearson',min_periods=None)

pandas.Series.count · ficha p. 38

Series.count()

pandas.Series.cov

Series.cov(other,min_periods=None,ddof=1)

pandas.Series.cummax · ficha p. 40

Series.cummax(axis=0,skipna=True,*args,**kwargs)

pandas.Series.cummin · ficha p. 44

Series.cummin(axis=0,skipna=True,*args,**kwargs)

pandas.Series.cumprod · ficha p. 39

Series.cumprod(axis=0,skipna=True,*args,**kwargs)

pandas.Series.cumsum · ficha p. 39

Series.cumsum(axis=0,skipna=True,*args,**kwargs)

pandas.Series.describe

Series.describe(percentiles=None,include=None,exclude=None)

pandas.Series.diff · ficha p. 40

Series.diff(periods=1)

pandas.Series.factorize

Series.factorize(sort=False,use_na_sentinel=True)

pandas.Series.kurt · ficha p. 43

Series.kurt(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.Series.max · ficha p. 38

Series.max(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.Series.mean · ficha p. 37

Series.mean(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.Series.median · ficha p. 37

Series.median(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.Series.min · ficha p. 37

Series.min(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.Series.mode · ficha p. 43

Series.mode(dropna=True)

pandas.Series.nlargest

Series.nlargest(n=5,keep='first')

pandas.Series.nsmallest

Series.nsmallest(n=5,keep='first')

pandas.Series.pct_change · ficha p. 40

Series.pct_change(periods=1,fill_method=None,freq=None,**kwargs)

pandas.Series.prod · ficha p. 42

Series.prod(*,axis=None,skipna=True,numeric_only=False,min_count=0,**kwargs)

pandas.Series.quantile · ficha p. 38

Series.quantile(q=0.5,interpolation='linear')

pandas.Series.rank · ficha p. 36

Series.rank(axis=0,method='average',numeric_only=False,na_option='keep',ascending=True,pct=False)

pandas.Series.sem · ficha p. 43

Series.sem(*,axis=None,skipna=True,ddof=1,numeric_only=False,**kwargs)

pandas.Series.skew · ficha p. 43

Series.skew(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.Series.std · ficha p. 38

Series.std(*,axis=None,skipna=True,ddof=1,numeric_only=False,**kwargs)

pandas.Series.sum · ficha p. 37

Series.sum(*,axis=None,skipna=True,numeric_only=False,min_count=0,**kwargs)

pandas.Series.var · ficha p. 38

Series.var(*,axis=None,skipna=True,ddof=1,numeric_only=False,**kwargs)

pandas.Series.kurtosis

Series.kurtosis(*,axis=0,skipna=True,numeric_only=False,**kwargs)

pandas.Series.unique · ficha p. 19

Series.unique()

pandas.Series.nunique · ficha p. 19

Series.nunique(dropna=True)

pandas.Series.is_unique

Series.is_unique

pandas.Series.is_monotonic_increasing

Series.is_monotonic_increasing

pandas.Series.is_monotonic_decreasing

Series.is_monotonic_decreasing

pandas.Series.value_counts · ficha p. 19

Series.value_counts(normalize=False,sort=True,ascending=False,bins=None,dropna=True)

pandas.Series.align

Series.align(other,join='outer',axis=None,level=None,copy=<no_default>,fill_value=None)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.case_when

Series.case_when(caselist)

pandas.Series.drop

Series.drop(labels=None,*,axis=0,index=None,columns=None,level=None,inplace=False,errors='raise')

pandas.Series.droplevel

Series.droplevel(level,axis=0)

pandas.Series.drop_duplicates

Series.drop_duplicates(*,keep='first',inplace=False,ignore_index=False)

pandas.Series.duplicated

Series.duplicated(keep='first')

pandas.Series.equals

Series.equals(other)

pandas.Series.head

Series.head(n=5)

pandas.Series.idxmax · ficha p. 39

Series.idxmax(axis=0,skipna=True,*args,**kwargs)

pandas.Series.idxmin · ficha p. 39

Series.idxmin(axis=0,skipna=True,*args,**kwargs)

pandas.Series.isin · ficha p. 23

Series.isin(values)

pandas.Series.reindex

Series.reindex(index=None,*,axis=None,method=None,copy=<no_default>,level=None,fill_value=None,limit=None,tolerance=None)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.reindex_like

Series.reindex_like(other,method=None,copy=<no_default>,limit=None,tolerance=None)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro method está obsoleto. Usa reindex sobre los ejes del otro objeto y especifica allí el método; copy también se ignora bajo Copy-on-Write.

pandas.Series.rename

Series.rename(index=None,*,axis=None,copy=<no_default>,inplace=False,level=None,errors='ignore')

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.rename_axis

Series.rename_axis(mapper=<no_default>,*,index=<no_default>,axis=0,copy=<no_default>,inplace=False)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.reset_index

Series.reset_index(level=None,*,drop=False,name=<no_default>,inplace=False,allow_duplicates=False)

pandas.Series.sample

Series.sample(n=None,frac=None,replace=False,weights=None,random_state=None,axis=None,ignore_index=False)

pandas.Series.set_axis

Series.set_axis(labels,*,axis=0,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.take

Series.take(indices,axis=0,**kwargs)

pandas.Series.tail

Series.tail(n=5)

pandas.Series.truncate

Series.truncate(before=None,after=None,axis=None,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.where

Series.where(cond,other=<no_default>,*,inplace=False,axis=None,level=None)

pandas.Series.mask

Series.mask(cond,other=<no_default>,*,inplace=False,axis=None,level=None)

pandas.Series.add_prefix

Series.add_prefix(prefix,axis=None)

pandas.Series.add_suffix

Series.add_suffix(suffix,axis=None)

pandas.Series.filter

Series.filter(items=None,like=None,regex=None,axis=None)

pandas.Series.bfill

Series.bfill(*,axis=None,inplace=False,limit=None,limit_area=None)

pandas.Series.dropna

Series.dropna(*,axis=0,inplace=False,how=None,ignore_index=False)

pandas.Series.ffill

Series.ffill(*,axis=None,inplace=False,limit=None,limit_area=None)

pandas.Series.fillna

Series.fillna(value,*,axis=None,inplace=False,limit=None)

pandas.Series.interpolate · ficha p. 34

Series.interpolate(method='linear',*,axis=0,limit=None,inplace=False,limit_direction=None,limit_area=None,**kwargs)

pandas.Series.isna

Series.isna()

pandas.Series.isnull

Series.isnull()

pandas.Series.notna

Series.notna()

pandas.Series.notnull

Series.notnull()

pandas.Series.replace

Series.replace(to_replace=None,value=<no_default>,*,inplace=False,regex=False)

pandas.Series.argsort

Series.argsort(axis=0,kind='quicksort',order=None,stable=None)

pandas.Series.argmin · ficha p. 47

Series.argmin(axis=None,skipna=True,*args,**kwargs)

pandas.Series.argmax · ficha p. 47

Series.argmax(axis=None,skipna=True,*args,**kwargs)

pandas.Series.reorder_levels

Series.reorder_levels(order)

pandas.Series.sort_values

Series.sort_values(*,axis=0,ascending=True,inplace=False,kind='quicksort',na_position='last',ignore_index=False,key=None)

pandas.Series.sort_index

Series.sort_index(*,axis=0,level=None,ascending=True,inplace=False,kind='quicksort',na_position='last',sort_remaining=True,ignore_index=False,key=None)

pandas.Series.swaplevel

Series.swaplevel(i=-2,j=-1,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.unstack

Series.unstack(level=-1,fill_value=None,sort=True)

pandas.Series.explode

Series.explode(ignore_index=False)

pandas.Series.searchsorted · ficha p. 47

Series.searchsorted(value,side='left',sorter=None)

pandas.Series.repeat

Series.repeat(repeats,axis=None)

pandas.Series.squeeze

Series.squeeze(axis=None)

pandas.Series.compare

Series.compare(other,align_axis=1,keep_shape=False,keep_equal=False,result_names=('self','other'))

pandas.Series.update

Series.update(other)

pandas.Series.asfreq · ficha p. 71

Series.asfreq(freq,method=None,how=None,normalize=False,fill_value=None)

pandas.Series.asof

Series.asof(where,subset=None)

pandas.Series.shift · ficha p. 71

Series.shift(periods=1,freq=None,axis=0,fill_value=<no_default>,suffix=None)

pandas.Series.first_valid_index

Series.first_valid_index()

pandas.Series.last_valid_index

Series.last_valid_index()

pandas.Series.resample · ficha p. 70

Series.resample(rule,closed=None,label=None,convention='start',on=None,level=None,origin='start_day',offset=None,group_keys=False)

pandas.Series.tz_convert

Series.tz_convert(tz,axis=0,level=None,copy=<no_default>)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.tz_localize

Series.tz_localize(tz,axis=0,level=None,copy=<no_default>,ambiguous='raise',nonexistent='raise')

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro copy está obsoleto o se ignora bajo Copy-on-Write. Para una copia explícita usa copy(); consulta el contrato de esta operación.

pandas.Series.at_time

Series.at_time(time,asof=False,axis=None)

pandas.Series.between_time

Series.between_time(start_time,end_time,inclusive='both',axis=None)

pandas.Series.str
Series.str()

pandas.Series.cat
Series.cat()

pandas.Series.dt
Series.dt()

pandas.Series.sparse
Series.sparse()

pandas.DataFrame.sparse
DataFrame.sparse()

pandas.Index.str
Index.str()

pandas.Series.dt.date
Series.dt.date

pandas.Series.dt.time
Series.dt.time

pandas.Series.dt.timetz
Series.dt.timetz

pandas.Series.dt.year · ficha p. 67
Series.dt.year

pandas.Series.dt.month · ficha p. 68
Series.dt.month

pandas.Series.dt.day · ficha p. 68
Series.dt.day

pandas.Series.dt.hour
Series.dt.hour

pandas.Series.dt.minute
Series.dt.minute

pandas.Series.dt.second
Series.dt.second

pandas.Series.dt.microsecond
Series.dt.microsecond

pandas.Series.dt.nanosecond
Series.dt.nanosecond

pandas.Series.dt.dayofweek · ficha p. 68
Series.dt.dayofweek

pandas.Series.dt.day_of_week
Series.dt.day_of_week

pandas.Series.dt.weekday
Series.dt.weekday

pandas.Series.dt.dayofyear
Series.dt.dayofyear

pandas.Series.dt.day_of_year
Series.dt.day_of_year

pandas.Series.dt.days_in_month · ficha p. 72
Series.dt.days_in_month

pandas.Series.dt.quarter · ficha p. 72
Series.dt.quarter

pandas.Series.dt.is_month_start
Series.dt.is_month_start

pandas.Series.dt.is_month_end · ficha p. 72
Series.dt.is_month_end

pandas.Series.dt.is_quarter_start
Series.dt.is_quarter_start

pandas.Series.dt.is_quarter_end
Series.dt.is_quarter_end

pandas.Series.dt.is_year_start
Series.dt.is_year_start

pandas.Series.dt.is_year_end
Series.dt.is_year_end

pandas.Series.dt.is_leap_year
Series.dt.is_leap_year

pandas.Series.dt.daysinmonth
Series.dt.daysinmonth

pandas.Series.dt.tz
Series.dt.tz

pandas.Series.dt.freq
Series.dt.freq

pandas.Series.dt.unit
Series.dt.unit

pandas.Series.dt.isocalendar · ficha p. 70
Series.dt.isocalendar()

pandas.Series.dt.to_period · ficha p. 69
Series.dt.to_period(freq=None)

pandas.Series.dt.to_pydatetime
Series.dt.to_pydatetime()

pandas.Series.dt.tz_localize · ficha p. 69
Series.dt.tz_localize(tz,ambiguous='raise',nonexistent='raise')

pandas.Series.dt.tz_convert · ficha p. 70
Series.dt.tz_convert(tz)

pandas.Series.dt.normalize · ficha p. 69
Series.dt.normalize()

pandas.Series.dt.strftime · ficha p. 68
Series.dt.strftime(date_format)

pandas.Series.dt.round · ficha p. 73
Series.dt.round(freq,ambiguous='raise',nonexistent='raise')

pandas.Series.dt.floor · ficha p. 69
Series.dt.floor(freq,ambiguous='raise',nonexistent='raise')

pandas.Series.dt.ceil · ficha p. 73
Series.dt.ceil(freq,ambiguous='raise',nonexistent='raise')

pandas.Series.dt.month_name
Series.dt.month_name(locale=None)

pandas.Series.dt.day_name · ficha p. 72
Series.dt.day_name(locale=None)

pandas.Series.dt.as_unit
Series.dt.as_unit(unit,round_ok=True)

pandas.Series.dt.qyear
Series.dt.qyear

pandas.Series.dt.start_time
Series.dt.start_time

pandas.Series.dt.end_time
Series.dt.end_time

pandas.Series.dt.days
Series.dt.days

pandas.Series.dt.seconds
Series.dt.seconds

pandas.Series.dt.microseconds
Series.dt.microseconds

pandas.Series.dt.nanoseconds
Series.dt.nanoseconds

pandas.Series.dt.components · ficha p. 73
Series.dt.components

pandas.Series.dt.to_pytimedelta
Series.dt.to_pytimedelta()

pandas.Series.dt.total_seconds · ficha p. 70
Series.dt.total_seconds()

pandas.Series.str.capitalize
Series.str.capitalize()

pandas.Series.str.casefold · ficha p. 64
Series.str.casefold()

pandas.Series.str.cat · ficha p. 63
Series.str.cat(others=None,sep=None,na_rep=None,join='left')

pandas.Series.str.center
Series.str.center(width,fillchar='')

pandas.Series.str.contains · ficha p. 61
Series.str.contains(pat,case=True,flags=0,na=<no_default>,regex=True)

pandas.Series.str.count · ficha p. 64
Series.str.count(pat,flags=0)

pandas.Series.str.decode
Series.str.decode(encoding,errors='strict',dtype=None)

pandas.Series.str.encode
Series.str.encode(encoding,errors='strict')

pandas.Series.str.endswith · ficha p. 61
Series.str.endswith(pat,na=<no_default>)

pandas.Series.str.extract · ficha p. 62
Series.str.extract(pat,flags=0,expand=True)

pandas.Series.str.extractall · ficha p. 62
Series.str.extractall(pat,flags=0)

pandas.Series.str.find · ficha p. 64
Series.str.find(sub,start=0,end=None)

pandas.Series.str.findall
Series.str.findall(pat,flags=0)

pandas.Series.str.fullmatch · ficha p. 63
Series.str.fullmatch(pat,case=True,flags=0,na=<no_default>)

pandas.Series.str.get · ficha p. 65
Series.str.get(i)

pandas.Series.str.index
Series.str.index(sub,start=0,end=None)

pandas.Series.str.isascii
Series.str.isascii()

pandas.Series.str.join · ficha p. 65
Series.str.join(sep)

pandas.Series.str.len · ficha p. 62
Series.str.len()

pandas.Series.str.ljust
Series.str.ljust(width,fillchar=')

pandas.Series.str.lower · ficha p. 60
Series.str.lower()

pandas.Series.str.lstrip
Series.str.lstrip(to_strip=None)

pandas.Series.str.match
Series.str.match(pat,case=<no_default>,flags=<no_default>,na=<no_default>)

pandas.Series.str.normalize · ficha p. 63
Series.str.normalize(form)

pandas.Series.str.pad
Series.str.pad(width,side='left',fillchar=')

pandas.Series.str.partition
Series.str.partition(sep=',expand=True)

pandas.Series.str.removeprefix
Series.str.removeprefix(prefix)

pandas.Series.str.removesuffix
Series.str.removesuffix(suffix)

pandas.Series.str.repeat
Series.str.repeat(repeats)

pandas.Series.str.replace · ficha p. 61
Series.str.replace(pat,repl=None,n=-1,case=None,flags=0,regex=False)

pandas.Series.str.rfind
Series.str.rfind(sub,start=0,end=None)

pandas.Series.str.rindex
Series.str.rindex(sub,start=0,end=None)

pandas.Series.str.rjust
Series.str.rjust(width,fillchar=')

pandas.Series.str.rpartition
Series.str.rpartition(sep=',expand=True)

pandas.Series.str.rstrip

Series.str.rstrip(to_strip=None)

pandas.Series.str.slice · ficha p. 63

Series.str.slice(start=None,stop=None,step=None)

pandas.Series.str.slice_replace

Series.str.slice_replace(start=None,stop=None,repl=None)

pandas.Series.str.split · ficha p. 62

Series.str.split(pat=None,*,n=-1,expand=False,regex=None)

pandas.Series.str.rsplit

Series.str.rsplit(pat=None,*,n=-1,expand=False)

pandas.Series.str.startswith · ficha p. 61

Series.str.startswith(pat,na=<no_default>)

pandas.Series.str.strip · ficha p. 60

Series.str.strip(to_strip=None)

pandas.Series.str.swapcase

Series.str.swapcase()

pandas.Series.str.title · ficha p. 64

Series.str.title()

pandas.Series.str.translate

Series.str.translate(table)

pandas.Series.str.upper · ficha p. 60

Series.str.upper()

pandas.Series.str.wrap

Series.str.wrap(width,expand_tabs=True,tabsize=8,replace_whitespace=True,drop_whitespace=True,initial_indent='',subsequent_indent=''
,fix_sentence_endings=False,break_long_words=True,break_on_hyphens=True,max_lines=None,placeholder='[...]')

pandas.Series.str.zfill · ficha p. 65

Series.str.zfill(width)

pandas.Series.str.isalnum

Series.str.isalnum()

pandas.Series.str.isalpha

Series.str.isalpha()

pandas.Series.str.isdigit

Series.str.isdigit()

pandas.Series.str.isspace

Series.str.isspace()

pandas.Series.str.islower

Series.str.islower()

pandas.Series.str.isupper

Series.str.isupper()

pandas.Series.str.istitle

Series.str.istitle()

pandas.Series.str.isnumeric

Series.str.isnumeric()

pandas.Series.str.isdecimal

Series.str.isdecimal()

pandas.Series.str.get_dummies · ficha p. 65

Series.str.get_dummies(sep='|',dtype=None)

pandas.Series.cat.categories · ficha p. 78

Series.cat.categories

pandas.Series.cat.ordered · ficha p. 78

Series.cat.ordered

pandas.Series.cat.codes · ficha p. 78

Series.cat.codes

pandas.Series.cat.rename_categories · ficha p. 79

Series.cat.rename_categories(new_categories)

pandas.Series.cat.reorder_categories · ficha p. 80

Series.cat.reorder_categories(new_categories,ordered=None)

pandas.Series.cat.add_categories · ficha p. 79

Series.cat.add_categories(new_categories)

pandas.Series.cat.remove_categories · ficha p. 81

Series.cat.remove_categories(removals)

pandas.Series.cat.remove_unused_categories · ficha p. 79

Series.cat.remove_unused_categories()

pandas.Series.cat.set_categories · ficha p. 80

Series.cat.set_categories(new_categories,ordered=None,rename=False)

pandas.Series.cat.as_ordered · ficha p. 80

Series.cat.as_ordered()

pandas.Series.cat.as_unordered · ficha p. 81

Series.cat.as_unordered()

pandas.Series.sparse.npoints

Series.sparse.npoints

pandas.Series.sparse.density

Series.sparse.density

pandas.Series.sparse.fill_value

Series.sparse.fill_value

pandas.Series.sparse.sp_values

Series.sparse.sp_values

pandas.Series.sparse.from_coo

methodSeries.sparse.from_coo(A,dense_index=False)

pandas.Series.sparse.to_coo

Series.sparse.to_coo(row_levels=(0,),column_levels=(1,),sort_labels=False)

pandas.Series.list.flatten

Series.list.flatten()

pandas.Series.list.len

Series.list.len()

pandas.Series.struct.dtypes

Series.struct.dtypes

pandas.Series.struct.field

Series.struct.field(name_or_index)

pandas.Series.struct.explode

Series.struct.explode()

pandas.Series.attrs

Series.attrs

pandas.Series.plot

Series.plot(*args,**kwargs)

pandas.Series.plot.area

Series.plot.area(x=None,y=None,stacked=True,**kwargs)

pandas.Series.plot.bar

Series.plot.bar(x=None,y=None,color=None,**kwargs)

pandas.Series.plot.barh

Series.plot.barh(x=None,y=None,color=None,**kwargs)

pandas.Series.plot.box

Series.plot.box(by=None,**kwargs)

pandas.Series.plot.density

Series.plot.density(bw_method=None,ind=None,weights=None,**kwargs)

pandas.Series.plot.hist

Series.plot.hist(by=None,bins=10,**kwargs)

pandas.Series.plot.kde

Series.plot.kde(bw_method=None,ind=None,weights=None,**kwargs)

pandas.Series.plot.line

Series.plot.line(x=None,y=None,color=None,**kwargs)

pandas.Series.plot.pie

Series.plot.pie(y=None,**kwargs)

pandas.Series.hist

Series.hist(by=None,ax=None,grid=True,xlabelsize=None,xrot=None,ylabelsize=None,yrot=None,figsize=None,bins=10,backend=None,legend=False,**kwargs)

pandas.Series.from_arrow

methodSeries.from_arrow(data)

pandas.Series.to_pickle

Series.to_pickle(path,*,compression='infer',protocol=5,storage_options=None)

pandas.Series.to_csv

Series.to_csv(path_or_buf=None,*,sep=',',na_rep='',float_format=None,columns=None,header=True,index=True,index_label=None,mode='w',encoding=None,compression='infer',quoting=None,quotechar='"',lineterminator=None,chunksize=None,date_format=None,doublequote=True,escapechar=None,decimal='.',errors='strict',storage_options=None)

pandas.Series.to_dict

Series.to_dict(*,into=<class'dict'>)

pandas.Series.to_excel

Series.to_excel(excel_writer,*,sheet_name='Sheet1',na_rep='',float_format=None,columns=None,header=True,index=True,index_label=None,startrow=0,startcol=0,engine=None,merge_cells=True,inf_rep='inf',freeze_panes=None,storage_options=None,engine_kwargs=None,autofilter=False)

pandas.Series.to_frame

Series.to_frame(name=<no_default>)

pandas.Series.to_xarray

Series.to_xarray()

pandas.Series.to_hdf

Series.to_hdf(path_or_buf,*,key,mode='a',complevel=None,complib=None,append=False,format=None,index=True,min_itemsize=None,nan_rep=None,dropna=None,data_columns=None,errors='strict',encoding='UTF-8')

pandas.Series.to_sql

Series.to_sql(name,con,*,schema=None,if_exists='fail',index=True,index_label=None,chunksize=None,dtype=None,method=None)

pandas.Series.to_json

Series.to_json(path_or_buf=None,*,orient=None,date_format=None,double_precision=10,force_ascii=True,date_unit='ms',default_handler=None,lines=False,compression='infer',index=None,indent=None,storage_options=None,mode='w')

AVISO DE OBSOLESCENCIA (API o parámetro). El formato de fecha epoch está obsoleto. Usa date_format="iso"; el método sigue vigente.

pandas.Series.to_string

Series.to_string(buf=None,*,na_rep='NaN',float_format=None,header=True,index=True,length=False,dtype=False,name=False,max_rows=None,min_rows=None)

pandas.Series.to_clipboard

Series.to_clipboard(*,excel=True,sep=None,**kwargs)

pandas.Series.to_latex

Series.to_latex(buf=None,*,columns=None,header=True,index=True,na_rep='NaN',formatters=None,float_format=None,sparsify=None,index_names=True,bold_rows=False,column_format=None,longtable=None,escape=None,encoding=None,decimal='.',multicolumn=None,multicolumn_format=None,multirow=None,caption=None,label=None,position=None)

pandas.Series.to_markdown

Series.to_markdown(buf=None,*,mode='wt',index=True,storage_options=None,**kwargs)

Index y MultiIndex

242 entradas documentadas.

pandas.Index · ficha p. 8

pd.Index(data=None,dtype=None,copy=None,name=None,tupleize_cols=True)

pandas.Index.values

Index.values

pandas.Index.is_monotonic_increasing · ficha p. 86

Index.is_monotonic_increasing

pandas.Index.is_monotonic_decreasing

Index.is_monotonic_decreasing

pandas.Index.is_unique · ficha p. 82

Index.is_unique

pandas.Index.has_duplicates

Index.has_duplicates

pandas.Index.hasnans

Index.hasnans

pandas.Index.dtype

Index.dtype

pandas.Index.inferred_type

Index.inferred_type

pandas.Index.shape

Index.shape

pandas.Index.name · ficha p. 82

Index.name

pandas.Index.names · ficha p. 83

Index.names

pandas.Index.nbytes

Index.nbytes

pandas.Index.ndim

Index.ndim

pandas.Index.size

Index.size

pandas.Index.empty

Index.empty

pandas.Index.T

Index.T

pandas.Index.memory_usage

Index.memory_usage(deep=False)

pandas.Index.array

Index.array

pandas.Index.all

Index.all(*args,**kwargs)

pandas.Index.any

Index.any(*args,**kwargs)

pandas.Index.argmin

Index.argmin(axis=None,skipna=True,*args,**kwargs)

pandas.Index.argmax

Index.argmax(axis=None,skipna=True,*args,**kwargs)

pandas.Index.copy

Index.copy(name=None,deep=False)

pandas.Index.delete

Index.delete(loc)

pandas.Index.drop

Index.drop(labels,errors='raise')

pandas.Index.drop_duplicates · ficha p. 86

Index.drop_duplicates(*,keep='first')

pandas.Index.duplicated · ficha p. 86

Index.duplicated(keep='first')

pandas.Index.equals

Index.equals(other)

pandas.Index.factorize

Index.factorize(sort=False,use_na_sentinel=True)

pandas.Index.identical

finalIndex.identical(other)

pandas.Index.insert

Index.insert(loc,item)

pandas.Index.is_

finalIndex.is_(other)

pandas.Index.min

Index.min(axis=None,skipna=True,*args,**kwargs)

pandas.Index.max

Index.max(axis=None,skipna=True,*args,**kwargs)

pandas.Index.reindex

Index.reindex(target,method=None,level=None,limit=None,tolerance=None)

pandas.Index.rename · ficha p. 84

Index.rename(name,*,inplace=False)

pandas.Index.repeat

Index.repeat(repeats,axis=None)

pandas.Index.where

finalIndex.where(cond,other=None)

pandas.Index.take

Index.take(indices,axis=0,allow_fill=True,fill_value=None,**kwargs)

pandas.Index.putmask

Index.putmask(mask,value)

pandas.Index.unique

Index.unique(level=None)

pandas.Index.nunique

Index.nunique(dropna=True)

pandas.Index.value_counts

Index.value_counts(normalize=False,sort=True,ascending=False,bins=None,dropna=True)

pandas.Index.set_names

Index.set_names(names,*,level=None,inplace=False)

pandas.Index.droplevel

finalIndex.droplevel(level=0)

pandas.Index.fillna

Index.fillna(value)

pandas.Index.dropna

Index.dropna(how='any')

pandas.Index.isna

finalIndex.isna()

pandas.Index.notna

finalIndex.notna()

pandas.Index.astype

Index.astype(dtype,copy=True)

pandas.Index.infer_objects

finalIndex.infer_objects(copy=True)

pandas.Index.item

Index.item()

pandas.Index.map

Index.map(mapper,na_action=None)

pandas.Index.ravel

finalIndex.ravel(order='C')

pandas.Index.to_list

Index.to_list()

pandas.Index.to_series

finalIndex.to_series(index=None,name=None)

pandas.Index.to_frame · ficha p. 86

Index.to_frame(index=True,name=<no_default>)

pandas.Index.to_numpy

Index.to_numpy(dtype=None,copy=False,na_value=<no_default>,**kwargs)

pandas.Index.view

Index.view(cls=None)

pandas.Index.argsort

Index.argsort(*args,**kwargs)

pandas.Index.searchsorted

Index.searchsorted(value,side='left',sorter=None)

pandas.Index.sort_values

Index.sort_values(*,return_indexer=False,ascending=True,na_position='last',key=None)

pandas.Index.shift

Index.shift(periods=1,freq=None)

pandas.Index.append

Index.append(other)

pandas.Index.join

finalIndex.join(other,*,how='left',level=None,return_indexers=False,sort=False)

pandas.Index.intersection · ficha p. 83

finalIndex.intersection(other,sort=False)

pandas.Index.union · ficha p. 83

finalIndex.union(other,sort=None)

pandas.Index.difference · ficha p. 84

finalIndex.difference(other,sort=None)

pandas.Index.symmetric_difference

Index.symmetric_difference(other,result_name=None,sort=None)

pandas.Index.asof

finalIndex.asof(label)

pandas.Index.asof_locs

Index.asof_locs(where,mask)

pandas.Index.get_indexer · ficha p. 83

finalIndex.get_indexer(target,method=None,limit=None,tolerance=None)

pandas.Index.get_indexer_for

finalIndex.get_indexer_for(target)

pandas.Index.get_indexer_non_unique

Index.get_indexer_non_unique(target)

pandas.Index.get_level_values

Index.get_level_values(level)

pandas.Index.get_loc · ficha p. 83

Index.get_loc(key)

pandas.Index.get_slice_bound

Index.get_slice_bound(label,side)

pandas.Index.isin · ficha p. 84

Index.isin(values,level=None)

pandas.Index.slice_indexer

Index.slice_indexer(start=None,end=None,step=None)

pandas.Index.slice_locs

Index.slice_locs(start=None,end=None,step=None)

pandas.RangeIndex

pd.RangeIndex(start=None,stop=None,step=None,dtype=None,copy=False,name=None)

pandas.RangeIndex.start

RangeIndex.start

pandas.RangeIndex.stop

RangeIndex.stop

pandas.RangeIndex.step

RangeIndex.step

pandas.RangeIndex.from_range

methodRangeIndex.from_range(data,name=None,dtype=None)

pandas.CategoricalIndex

pd.CategoricalIndex(data=None,categories=None,ordered=None,dtype=None,copy=False,name=None)

pandas.Categorical · ficha p. 9

pd.Categorical(values,categories=None,ordered=None,dtype=None,copy=True)

pandas.CategoricalIndex.append

CategoricalIndex.append(other)

pandas.CategoricalIndex.codes

CategoricalIndex.codes

pandas.CategoricalIndex.categories

CategoricalIndex.categories

pandas.CategoricalIndex.ordered

CategoricalIndex.ordered

pandas.CategoricalIndex.rename_categories

CategoricalIndex.rename_categories(new_categories)

pandas.CategoricalIndex.reorder_categories

CategoricalIndex.reorder_categories(new_categories,ordered=None)

pandas.CategoricalIndex.add_categories

CategoricalIndex.add_categories(new_categories)

pandas.CategoricalIndex.remove_categories

CategoricalIndex.remove_categories(removals)

pandas.CategoricalIndex.remove_unused_categories

CategoricalIndex.remove_unused_categories()

pandas.CategoricalIndex.set_categories

CategoricalIndex.set_categories(new_categories,ordered=None,rename=False)

pandas.CategoricalIndex.as_ordered

CategoricalIndex.as_ordered()

pandas.CategoricalIndex.as_unordered

CategoricalIndex.as_unordered()

pandas.CategoricalIndex.map

CategoricalIndex.map(mapper,na_action=None)

pandas.CategoricalIndex.equals

CategoricalIndex.equals(other)

pandas.IntervalIndex

pd.IntervalIndex(data,closed=None,dtype=None,copy=None,name=None,verify_integrity=True)

pandas.IntervalIndex.from_arrays

methodIntervalIndex.from_arrays(left,right,closed='right',name=None,copy=False,dtype=None)

pandas.IntervalIndex.from_tuples

methodIntervalIndex.from_tuples(data,closed='right',name=None,copy=False,dtype=None)

pandas.IntervalIndex.from_breaks

methodIntervalIndex.from_breaks(breaks,closed='right',name=None,copy=False,dtype=None)

pandas.IntervalIndex.left

IntervalIndex.left

pandas.IntervalIndex.right

IntervalIndex.right

pandas.IntervalIndex.mid

IntervalIndex.mid

pandas.IntervalIndex.closed

IntervalIndex.closed

pandas.IntervalIndex.length

IntervalIndex.length

pandas.IntervalIndex.values

IntervalIndex.values

pandas.IntervalIndex.is_empty

IntervalIndex.is_empty

pandas.IntervalIndex.is_non_overlapping_monotonic

IntervalIndex.is_non_overlapping_monotonic

pandas.IntervalIndex.is_overlapping

IntervalIndex.is_overlapping

pandas.IntervalIndex.get_loc

IntervalIndex.get_loc(key)

pandas.IntervalIndex.get_indexer

IntervalIndex.get_indexer(target,method=None,limit=None,tolerance=None)

pandas.IntervalIndex.set_closed

IntervalIndex.set_closed(closed)

pandas.IntervalIndex.contains

IntervalIndex.contains(other)

pandas.IntervalIndex.overlaps

IntervalIndex.overlaps(other)

pandas.IntervalIndex.to_tuples

IntervalIndex.to_tuples(na_tuple=True)

pandas.MultiIndex.from_arrays

methodMultiIndex.from_arrays(arrays,sortorder=None,names=<no_default>)

pandas.MultiIndex.from_tuples · ficha p. 85

methodMultiIndex.from_tuples(tuples,sortorder=None,names=None)

pandas.MultiIndex.from_product · ficha p. 84

methodMultiIndex.from_product(iterables,sortorder=None,names=<no_default>)

pandas.MultiIndex.from_frame

methodMultiIndex.from_frame(df,sortorder=None,names=None)

pandas.MultiIndex.names

MultiIndex.names

pandas.MultiIndex.levels · ficha p. 87

MultiIndex.levels

pandas.MultiIndex.codes · ficha p. 87

MultiIndex.codes

pandas.MultiIndex.nlevels

MultiIndex.nlevels

pandas.MultiIndex.levshape

MultiIndex.levshape

pandas.MultiIndex.dtypes

MultiIndex.dtypes

pandas.MultiIndex.set_levels

MultiIndex.set_levels(levels,*,level=None,verify_integrity=True)

pandas.MultiIndex.set_codes

MultiIndex.set_codes(codes,*,level=None,verify_integrity=True)

pandas.MultiIndex.to_flat_index

MultiIndex.to_flat_index()

pandas.MultiIndex.to_frame

MultiIndex.to_frame(index=True,name=<no_default>,allow_duplicates=False)

pandas.MultiIndex.sortlevel

MultiIndex.sortlevel(level=0,ascending=True,sort_remaining=True,na_position='first')

pandas.MultiIndex.droplevel · ficha p. 86

MultiIndex.droplevel(level=0)

pandas.MultiIndex.swaplevel

MultiIndex.swaplevel(i=-2,j=-1)

pandas.MultiIndex.reorder_levels

MultiIndex.reorder_levels(order)

pandas.MultiIndex.remove_unused_levels · ficha p. 87

MultiIndex.remove_unused_levels()

pandas.MultiIndex.drop

MultiIndex.drop(codes,level=None,errors='raise')

pandas.MultiIndex.copy

MultiIndex.copy(names=None,deep=False,name=None)

pandas.MultiIndex.append

MultiIndex.append(other)

pandas.MultiIndex.truncate

MultiIndex.truncate(before=None,after=None)

pandas.MultiIndex.get_loc

MultiIndex.get_loc(key)

pandas.MultiIndex.get_locs

MultiIndex.get_locs(seq)

pandas.MultiIndex.get_loc_level

MultiIndex.get_loc_level(key,level=0,drop_level=True)

pandas.MultiIndex.get_indexer

MultiIndex.get_indexer(target,method=None,limit=None,tolerance=None)

pandas.MultiIndex.get_level_values · ficha p. 85

MultiIndex.get_level_values(level)

pandas.IndexSlice

pd.IndexSlice=<pd.core.indexing._IndexSliceobject>

pandas.DatetimeIndex

pd.DatetimeIndex(data=None,freq=<no_default>,tz=<no_default>,ambiguous='raise',dayfirst=False,yearfirst=False,dtype=None,copy=None,name=None)

pandas.DatetimeIndex.year

DatetimeIndex.year

pandas.DatetimeIndex.month

DatetimeIndex.month

pandas.DatetimeIndex.day

DatetimeIndex.day

pandas.DatetimeIndex.hour

DatetimeIndex.hour

pandas.DatetimeIndex.minute

DatetimeIndex.minute

pandas.DatetimeIndex.second

DatetimeIndex.second

pandas.DatetimeIndex.microsecond

DatetimeIndex.microsecond

pandas.DatetimeIndex.nanosecond

DatetimeIndex.nanosecond

pandas.DatetimeIndex.date

DatetimeIndex.date

pandas.DatetimeIndex.time

DatetimeIndex.time

pandas.DatetimeIndex.timetz

DatetimeIndex.timetz

pandas.DatetimeIndex.dayofyear

DatetimeIndex.dayofyear

pandas.DatetimeIndex.day_of_year

DatetimeIndex.day_of_year

pandas.DatetimeIndex.dayofweek

DatetimeIndex.dayofweek

pandas.DatetimeIndex.day_of_week

DatetimeIndex.day_of_week

pandas.DatetimeIndex.weekday

DatetimeIndex.weekday

pandas.DatetimeIndex.quarter

DatetimeIndex.quarter

pandas.DatetimeIndex.tz

DatetimeIndex.tz

pandas.DatetimeIndex.freq

DatetimeIndex.freq

pandas.DatetimeIndex.freqstr

DatetimeIndex.freqstr

pandas.DatetimeIndex.is_month_start

DatetimeIndex.is_month_start

pandas.DatetimeIndex.is_month_end

DatetimeIndex.is_month_end

pandas.DatetimeIndex.is_quarter_start

DatetimeIndex.is_quarter_start

pandas.DatetimeIndex.is_quarter_end

DatetimeIndex.is_quarter_end

pandas.DatetimeIndex.is_year_start

DatetimeIndex.is_year_start

pandas.DatetimeIndex.is_year_end

DatetimeIndex.is_year_end

pandas.DatetimeIndex.is_leap_year

DatetimeIndex.is_leap_year

pandas.DatetimeIndex.inferred_freq

DatetimeIndex.inferred_freq

pandas.DatetimeIndex.indexer_at_time

DatetimeIndex.indexer_at_time(time,asof=False)

pandas.DatetimeIndex.indexer_between_time

DatetimeIndex.indexer_between_time(start_time,end_time,include_start=True,include_end=True)

pandas.DatetimeIndex.normalize

DatetimeIndex.normalize()

pandas.DatetimeIndex.strftime

DatetimeIndex.strftime(date_format)

pandas.DatetimeIndex.snap

DatetimeIndex.snap(freq='S')

pandas.DatetimeIndex.tz_convert

DatetimeIndex.tz_convert(tz)

pandas.DatetimeIndex.tz_localize

DatetimeIndex.tz_localize(tz,ambiguous='raise',nonexistent='raise')

pandas.DatetimeIndex.round

DatetimeIndex.round(freq,ambiguous='raise',nonexistent='raise')

pandas.DatetimeIndex.floor

DatetimeIndex.floor(freq,ambiguous='raise',nonexistent='raise')

pandas.DatetimeIndex.ceil

DatetimeIndex.ceil(freq,ambiguous='raise',nonexistent='raise')

pandas.DatetimeIndex.month_name

DatetimeIndex.month_name(locale=None)

pandas.DatetimeIndex.day_name

DatetimeIndex.day_name(locale=None)

pandas.DatetimeIndex.as_unit

DatetimeIndex.as_unit(unit,round_ok=True)

pandas.DatetimeIndex.to_period

DatetimeIndex.to_period(freq=None)

pandas.DatetimeIndex.to_pydatetime

DatetimeIndex.to_pydatetime()

pandas.DatetimeIndex.to_series

DatetimeIndex.to_series(index=None,name=None)

pandas.DatetimeIndex.to_frame

DatetimeIndex.to_frame(index=True,name=<no_default>)

pandas.DatetimeIndex.to_julian_date

DatetimeIndex.to_julian_date()

pandas.DatetimeIndex.mean

DatetimeIndex.mean(*,skipna=True,axis=0)

pandas.DatetimeIndex.std

DatetimeIndex.std(axis=None,dtype=None,out=None,ddof=1,keepdims=False,skipna=True)

pandas.TimedeltaIndex

pd.TimedeltaIndex(data=None,freq=<no_default>,dtype=None,copy=None,name=None)

pandas.TimedeltaIndex.days

TimedeltaIndex.days

pandas.TimedeltaIndex.seconds

TimedeltaIndex.seconds

pandas.TimedeltaIndex.microseconds

TimedeltaIndex.microseconds

pandas.TimedeltaIndex.nanoseconds

TimedeltaIndex.nanoseconds

pandas.TimedeltaIndex.components

TimedeltaIndex.components

pandas.TimedeltaIndex.inferred_freq

TimedeltaIndex.inferred_freq

pandas.TimedeltaIndex.as_unit

TimedeltaIndex.as_unit(unit)

pandas.TimedeltaIndex.to_pytimedelta

TimedeltaIndex.to_pytimedelta()

pandas.TimedeltaIndex.to_series

TimedeltaIndex.to_series(index=None,name=None)

pandas.TimedeltaIndex.round

TimedeltaIndex.round(freq,ambiguous='raise',nonexistent='raise')

pandas.TimedeltaIndex.floor

TimedeltaIndex.floor(freq,ambiguous='raise',nonexistent='raise')

pandas.TimedeltaIndex.ceil

TimedeltaIndex.ceil(freq,ambiguous='raise',nonexistent='raise')

pandas.TimedeltaIndex.to_frame

TimedeltaIndex.to_frame(index=True,name=<no_default>)

pandas.TimedeltaIndex.mean

TimedeltaIndex.mean(*,skipna=True,axis=0)

pandas.PeriodIndex

pd.PeriodIndex(data=None,freq=None,dtype=None,copy=None,name=None)

pandas.PeriodIndex.day

PeriodIndex.day

pandas.PeriodIndex.dayofweek

PeriodIndex.dayofweek

pandas.PeriodIndex.day_of_week

PeriodIndex.day_of_week

pandas.PeriodIndex.dayofyear

PeriodIndex.dayofyear

pandas.PeriodIndex.day_of_year

PeriodIndex.day_of_year

pandas.PeriodIndex.days_in_month

PeriodIndex.days_in_month

pandas.PeriodIndex.daysinmonth

PeriodIndex.daysinmonth

pandas.PeriodIndex.end_time

PeriodIndex.end_time

pandas.PeriodIndex.freq

PeriodIndex.freq

pandas.PeriodIndex.freqstr

PeriodIndex.freqstr

pandas.PeriodIndex.hour

PeriodIndex.hour

pandas.PeriodIndex.is_leap_year

PeriodIndex.is_leap_year

pandas.PeriodIndex.minute

PeriodIndex.minute

pandas.PeriodIndex.month

PeriodIndex.month

pandas.PeriodIndex.quarter

PeriodIndex.quarter

pandas.PeriodIndex.qyear

PeriodIndex.qyear

pandas.PeriodIndex.second

PeriodIndex.second

pandas.PeriodIndex.start_time

PeriodIndex.start_time

pandas.PeriodIndex.week

PeriodIndex.week

pandas.PeriodIndex.weekday

PeriodIndex.weekday

pandas.PeriodIndex.weekofyear

PeriodIndex.weekofyear

pandas.PeriodIndex.year

PeriodIndex.year

pandas.PeriodIndex.asfreq

PeriodIndex.asfreq(freq=None,how='E')

pandas.PeriodIndex.strftime

PeriodIndex.strftime(date_format)

pandas.PeriodIndex.to_timestamp

PeriodIndex.to_timestamp(freq=None,how='start')

pandas.PeriodIndex.from_fields

methodPeriodIndex.from_fields(*,year=None,quarter=None,month=None,day=None,hour=None,minute=None,second=None,freq=None)

pandas.PeriodIndex.from_ordinals

methodPeriodIndex.from_ordinals(ordinals,*,freq,name=None)

GroupBy

125 entradas documentadas.

pandas.api.typing.DataFrameGroupBy.groups · ficha p. 51

DataFrameGroupBy.groups

pandas.api.typing.SeriesGroupBy.groups

SeriesGroupBy.groups

pandas.api.typing.DataFrameGroupBy.indices · ficha p. 53

DataFrameGroupBy.indices

pandas.api.typing.SeriesGroupBy.indices

SeriesGroupBy.indices

pandas.api.typing.DataFrameGroupBy.get_group · ficha p. 50

DataFrameGroupBy.get_group(name)

pandas.api.typing.SeriesGroupBy.get_group

SeriesGroupBy.get_group(name)

pandas.Grouper

pd.Grouper(*args,**kwargs)

pandas.NamedAgg

pd.NamedAgg(column,aggfunc,*args,**kwargs)

pandas.api.typing.SeriesGroupBy.apply

SeriesGroupBy.apply(func,*args,**kwargs)

pandas.api.typing.DataFrameGroupBy.apply · ficha p. 49

DataFrameGroupBy.apply(func,*args,include_groups=False,**kwargs)

pandas.api.typing.SeriesGroupBy.agg

SeriesGroupBy.agg(func=None,*args,engine=None,engine_kwargs=None,**kwargs)

pandas.api.typing.DataFrameGroupBy.agg · ficha p. 48

DataFrameGroupBy.agg(func=None,*args,engine=None,engine_kwargs=None,**kwargs)

pandas.api.typing.SeriesGroupBy.aggregate

SeriesGroupBy.aggregate(func=None,*args,engine=None,engine_kwargs=None,**kwargs)

pandas.api.typing.DataFrameGroupBy.aggregate

DataFrameGroupBy.aggregate(func=None,*args,engine=None,engine_kwargs=None,**kwargs)

pandas.api.typing.SeriesGroupBy.transform · ficha p. 48

SeriesGroupBy.transform(func,*args,engine=None,engine_kwargs=None,**kwargs)

pandas.api.typing.DataFrameGroupBy.transform

DataFrameGroupBy.transform(func,*args,engine=None,engine_kwargs=None,**kwargs)

pandas.api.typing.SeriesGroupBy.pipe

SeriesGroupBy.pipe(func,*args,**kwargs)

pandas.api.typing.DataFrameGroupBy.pipe

DataFrameGroupBy.pipe(func,*args,**kwargs)

pandas.api.typing.DataFrameGroupBy.filter · ficha p. 49

DataFrameGroupBy.filter(func,dropna=True,*args,**kwargs)

pandas.api.typing.SeriesGroupBy.filter

SeriesGroupBy.filter(func,dropna=True,*args,**kwargs)

pandas.api.typing.DataFrameGroupBy.all

DataFrameGroupBy.all(skipna=True)

pandas.api.typing.DataFrameGroupBy.any

DataFrameGroupBy.any(skipna=True)

pandas.api.typing.DataFrameGroupBy.bfill

DataFrameGroupBy.bfill(limit=None)

pandas.api.typing.DataFrameGroupBy.corr

DataFrameGroupBy.corr(method='pearson',min_periods=1,numeric_only=False)

pandas.api.typing.DataFrameGroupBy.corrwith

DataFrameGroupBy.corrwith(other,drop=False,method='pearson',numeric_only=False)

AVISO DE OBSOLESCENCIA (API o parámetro). El método está obsoleto. Alternativa: GroupBy.apply con DataFrame.corrwith sobre cada grupo, seleccionando columnas y alineando índices explícitamente.

pandas.api.typing.DataFrameGroupBy.count · ficha p. 49

DataFrameGroupBy.count()

pandas.api.typing.DataFrameGroupBy.cov

DataFrameGroupBy.cov(min_periods=None,ddof=1,numeric_only=False)

pandas.api.typing.DataFrameGroupBy.cumcount · ficha p. 50

DataFrameGroupBy.cumcount(ascending=True)

pandas.api.typing.DataFrameGroupBy.cummax

DataFrameGroupBy.cummax(numeric_only=False,**kwargs)

pandas.api.typing.DataFrameGroupBy.cummin

DataFrameGroupBy.cummin(numeric_only=False,**kwargs)

pandas.api.typing.DataFrameGroupBy.cumprod

DataFrameGroupBy.cumprod(numeric_only=False,*args,**kwargs)

pandas.api.typing.DataFrameGroupBy.cumsum

DataFrameGroupBy.cumsum(numeric_only=False,*args,**kwargs)

pandas.api.typing.DataFrameGroupBy.describe

DataFrameGroupBy.describe(percentiles=None,include=None,exclude=None)

pandas.api.typing.DataFrameGroupBy.diff

DataFrameGroupBy.diff(periods=1)

pandas.api.typing.DataFrameGroupBy.ewm

DataFrameGroupBy.ewm(com=None,span=None,halflife=None,alpha=None,min_periods=0,adjust=True,ignore_na=False,times=None,method='single')

pandas.api.typing.DataFrameGroupBy.expanding

DataFrameGroupBy.expanding(min_periods=1,method='single')

pandas.api.typing.DataFrameGroupBy.ffill

DataFrameGroupBy.ffill(limit=None)

pandas.api.typing.DataFrameGroupBy.first · ficha p. 52

DataFrameGroupBy.first(numeric_only=False,min_count=-1,skipna=True)

pandas.api.typing.DataFrameGroupBy.head · ficha p. 51

DataFrameGroupBy.head(n=5)

pandas.api.typing.DataFrameGroupBy.idxmax

DataFrameGroupBy.idxmax(skipna=True,numeric_only=False)

pandas.api.typing.DataFrameGroupBy.idxmin

DataFrameGroupBy.idxmin(skipna=True,numeric_only=False)

pandas.api.typing.DataFrameGroupBy.last · ficha p. 52

DataFrameGroupBy.last(numeric_only=False,min_count=-1,skipna=True)

pandas.api.typing.DataFrameGroupBy.max

DataFrameGroupBy.max(numeric_only=False,min_count=-1,skipna=True,engine=None,engine_kwargs=None)

pandas.api.typing.DataFrameGroupBy.mean

DataFrameGroupBy.mean(numeric_only=False,skipna=True,engine=None,engine_kwargs=None)

pandas.api.typing.DataFrameGroupBy.median

DataFrameGroupBy.median(numeric_only=False,skipna=True)

pandas.api.typing.DataFrameGroupBy.min

DataFrameGroupBy.min(numeric_only=False,min_count=-1,skipna=True,engine=None,engine_kwargs=None)

pandas.api.typing.DataFrameGroupBy.ngroup · ficha p. 51

DataFrameGroupBy.ngroup(ascending=True)

pandas.api.typing.DataFrameGroupBy.nth · ficha p. 52

DataFrameGroupBy.nth

pandas.api.typing.DataFrameGroupBy.nunique · ficha p. 52

DataFrameGroupBy.nunique(dropna=True)

pandas.api.typing.DataFrameGroupBy.ohlc

DataFrameGroupBy.ohlc()

pandas.api.typing.DataFrameGroupBy.pct_change

DataFrameGroupBy.pct_change(periods=1,fill_method=None,freq=None)

pandas.api.typing.DataFrameGroupBy.prod

DataFrameGroupBy.prod(numeric_only=False,min_count=0,skipna=True)

pandas.api.typing.DataFrameGroupBy.quantile

DataFrameGroupBy.quantile(q=0.5,interpolation='linear',numeric_only=False)

pandas.api.typing.DataFrameGroupBy.rank

DataFrameGroupBy.rank(method='average',ascending=True,na_option='keep',pct=False)

pandas.api.typing.DataFrameGroupBy.resample

DataFrameGroupBy.resample(rule,*args,include_groups=False,**kwargs)

pandas.api.typing.DataFrameGroupBy.rolling

DataFrameGroupBy.rolling(window,min_periods=None,center=False,win_type=None,on=None,closed=None,method='single')

pandas.api.typing.DataFrameGroupBy.sample

DataFrameGroupBy.sample(n=None,frac=None,replace=False,weights=None,random_state=None)

pandas.api.typing.DataFrameGroupBy.sem

DataFrameGroupBy.sem(ddof=1,numeric_only=False,skipna=True)

pandas.api.typing.DataFrameGroupBy.shift

DataFrameGroupBy.shift(periods=1,freq=None,fill_value=<no_default>,suffix=None)

pandas.api.typing.DataFrameGroupBy.size · ficha p. 49

DataFrameGroupBy.size()

pandas.api.typing.DataFrameGroupBy.skew

DataFrameGroupBy.skew(skipna=True,numeric_only=False,**kwargs)

pandas.api.typing.DataFrameGroupBy.kurt

DataFrameGroupBy.kurt(skipna=True,numeric_only=False,**kwargs)

pandas.api.typing.DataFrameGroupBy.std

DataFrameGroupBy.std(ddof=1,engine=None,engine_kwargs=None,numeric_only=False,skipna=True)

pandas.api.typing.DataFrameGroupBy.sum

DataFrameGroupBy.sum(numeric_only=False,min_count=0,skipna=True,engine=None,engine_kwargs=None)

pandas.api.typing.DataFrameGroupBy.var

DataFrameGroupBy.var(ddof=1,engine=None,engine_kwargs=None,numeric_only=False,skipna=True)

pandas.api.typing.DataFrameGroupBy.tail

DataFrameGroupBy.tail(n=5)

pandas.api.typing.DataFrameGroupBy.take

DataFrameGroupBy.take(indices,**kwargs)

pandas.api.typing.DataFrameGroupBy.value_counts

DataFrameGroupBy.value_counts(subset=None,normalize=False,sort=True,ascending=False,dropna=True)

pandas.api.typing.SeriesGroupBy.all

SeriesGroupBy.all(skipna=True)

pandas.api.typing.SeriesGroupBy.any

SeriesGroupBy.any(skipna=True)

pandas.api.typing.SeriesGroupBy.bfill

SeriesGroupBy.bfill(limit=None)

pandas.api.typing.SeriesGroupBy.corr

SeriesGroupBy.corr(other,method='pearson',min_periods=None)

pandas.api.typing.SeriesGroupBy.count

SeriesGroupBy.count()

pandas.api.typing.SeriesGroupBy.cov

SeriesGroupBy.cov(other,min_periods=None,ddof=1)

pandas.api.typing.SeriesGroupBy.cumcount

SeriesGroupBy.cumcount(ascending=True)

pandas.api.typing.SeriesGroupBy.cummax

SeriesGroupBy.cummax(numeric_only=False,**kwargs)

pandas.api.typing.SeriesGroupBy.cummin

SeriesGroupBy.cummin(numeric_only=False,**kwargs)

pandas.api.typing.SeriesGroupBy.cumprod

SeriesGroupBy.cumprod(numeric_only=False,*args,**kwargs)

pandas.api.typing.SeriesGroupBy.cumsum

SeriesGroupBy.cumsum(numeric_only=False,*args,**kwargs)

pandas.api.typing.SeriesGroupBy.describe

SeriesGroupBy.describe(percentiles=None,include=None,exclude=None)

pandas.api.typing.SeriesGroupBy.diff

SeriesGroupBy.diff(periods=1)

pandas.api.typing.SeriesGroupBy.ewm

SeriesGroupBy.ewm(com=None,span=None,halflife=None,alpha=None,min_periods=0,adjust=True,ignore_na=False,times=None,method='single')

pandas.api.typing.SeriesGroupBy.expanding

SeriesGroupBy.expanding(min_periods=1,method='single')

pandas.api.typing.SeriesGroupBy.ffill

SeriesGroupBy.ffill(limit=None)

pandas.api.typing.SeriesGroupBy.first

SeriesGroupBy.first(numeric_only=False,min_count=-1,skipna=True)

pandas.api.typing.SeriesGroupBy.head

SeriesGroupBy.head(n=5)

pandas.api.typing.SeriesGroupBy.last

SeriesGroupBy.last(numeric_only=False,min_count=-1,skipna=True)

pandas.api.typing.SeriesGroupBy.idxmax

SeriesGroupBy.idxmax(skipna=True)

pandas.api.typing.SeriesGroupBy.idxmin

SeriesGroupBy.idxmin(skipna=True)

pandas.api.typing.SeriesGroupBy.is_monotonic_increasing

SeriesGroupBy.is_monotonic_increasing

pandas.api.typing.SeriesGroupBy.is_monotonic_decreasing

SeriesGroupBy.is_monotonic_decreasing

pandas.api.typing.SeriesGroupBy.max

SeriesGroupBy.max(numeric_only=False,min_count=-1,skipna=True,engine=None,engine_kwargs=None)

pandas.api.typing.SeriesGroupBy.mean · ficha p. 50

SeriesGroupBy.mean(numeric_only=False,skipna=True,engine=None,engine_kwargs=None)

pandas.api.typing.SeriesGroupBy.median

SeriesGroupBy.median(numeric_only=False,skipna=True)

pandas.api.typing.SeriesGroupBy.min

SeriesGroupBy.min(numeric_only=False,min_count=-1,skipna=True,engine=None,engine_kwargs=None)

pandas.api.typing.SeriesGroupBy.ngroup

SeriesGroupBy.ngroup(ascending=True)

pandas.api.typing.SeriesGroupBy.nlargest

SeriesGroupBy.nlargest(n=5,keep='first')

pandas.api.typing.SeriesGroupBy.nsmallest

SeriesGroupBy.nsmallest(n=5,keep='first')

pandas.api.typing.SeriesGroupBy.nth

SeriesGroupBy.nth

pandas.api.typing.SeriesGroupBy.nunique

SeriesGroupBy.nunique(dropna=True)

pandas.api.typing.SeriesGroupBy.unique

SeriesGroupBy.unique()

pandas.api.typing.SeriesGroupBy.ohlc

SeriesGroupBy.ohlc()

pandas.api.typing.SeriesGroupBy.pct_change

SeriesGroupBy.pct_change(periods=1,fill_method=None,freq=None)

pandas.api.typing.SeriesGroupBy.prod

SeriesGroupBy.prod(numeric_only=False,min_count=0,skipna=True)

pandas.api.typing.SeriesGroupBy.quantile

SeriesGroupBy.quantile(q=0.5,interpolation='linear',numeric_only=False)

pandas.api.typing.SeriesGroupBy.rank

SeriesGroupBy.rank(method='average',ascending=True,na_option='keep',pct=False)

pandas.api.typing.SeriesGroupBy.resample

SeriesGroupBy.resample(rule,*args,include_groups=False,**kwargs)

pandas.api.typing.SeriesGroupBy.rolling · ficha p. 53

SeriesGroupBy.rolling(window,min_periods=None,center=False,win_type=None,on=None,closed=None,method='single')

pandas.api.typing.SeriesGroupBy.sample

SeriesGroupBy.sample(n=None,frac=None,replace=False,weights=None,random_state=None)

pandas.api.typing.SeriesGroupBy.sem

SeriesGroupBy.sem(ddof=1,numeric_only=False,skipna=True)

pandas.api.typing.SeriesGroupBy.shift · ficha p. 51

SeriesGroupBy.shift(periods=1,freq=None,fill_value=<no_default>,suffix=None)

pandas.api.typing.SeriesGroupBy.size

SeriesGroupBy.size()

pandas.api.typing.SeriesGroupBy.skew

SeriesGroupBy.skew(skipna=True,numeric_only=False,**kwargs)

pandas.api.typing.SeriesGroupBy.kurt

SeriesGroupBy.kurt(skipna=True,numeric_only=False,**kwargs)

pandas.api.typing.SeriesGroupBy.std

SeriesGroupBy.std(ddof=1,engine=None,engine_kwargs=None,numeric_only=False,skipna=True)

pandas.api.typing.SeriesGroupBy.sum · ficha p. 50

SeriesGroupBy.sum(numeric_only=False,min_count=0,skipna=True,engine=None,engine_kwargs=None)

pandas.api.typing.SeriesGroupBy.var

SeriesGroupBy.var(ddof=1,engine=None,engine_kwargs=None,numeric_only=False,skipna=True)

pandas.api.typing.SeriesGroupBy.tail

SeriesGroupBy.tail(n=5)

pandas.api.typing.SeriesGroupBy.take

SeriesGroupBy.take(indices,**kwargs)

pandas.api.typing.SeriesGroupBy.value_counts

SeriesGroupBy.value_counts(normalize=False,sort=True,ascending=False,bins=None,dropna=True)

pandas.api.typing.DataFrameGroupBy.boxplot

DataFrameGroupBy.boxplot(subplots=True,column=None,fontsize=None,rot=0,grid=True,ax=None,figsize=None,layout=None,sharex=False,sharey=True,backend=None,**kwargs)

pandas.api.typing.DataFrameGroupBy.hist

DataFrameGroupBy.hist(column=None,by=None,grid=True,xlabelsize=None,xrot=None,ylabelsize=None,yrot=None,ax=None,sharex=False,sharey=False,figsize=None,layout=None,bins=10,backend=None,legend=False,**kwargs)

pandas.api.typing.SeriesGroupBy.hist

SeriesGroupBy.hist(by=None,ax=None,grid=True,xlabelsize=None,xrot=None,ylabelsize=None,yrot=None,figsize=None,bins=10,backend=None,legend=False,**kwargs)

pandas.api.typing.DataFrameGroupBy.plot

DataFrameGroupBy.plot

pandas.api.typing.SeriesGroupBy.plot

SeriesGroupBy.plot

Ventanas

55 entradas documentadas.

pandas.api.typing.Rolling.count

Rolling.count(numeric_only=False)

pandas.api.typing.Rolling.sum · ficha p. 75

Rolling.sum(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Rolling.mean · ficha p. 75

Rolling.mean(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Rolling.median

Rolling.median(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Rolling.var

Rolling.var(ddof=1,numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Rolling.std

Rolling.std(ddof=1,numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Rolling.min

Rolling.min(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Rolling.max

Rolling.max(numeric_only=False,*args,engine=None,engine_kwargs=None,**kwargs)

pandas.api.typing.Rolling.first

Rolling.first(numeric_only=False)

pandas.api.typing.Rolling.last

Rolling.last(numeric_only=False)

pandas.api.typing.Rolling.corr

Rolling.corr(other=None,pairwise=None,ddof=1,numeric_only=False)

pandas.api.typing.Rolling.cov

Rolling.cov(other=None,pairwise=None,ddof=1,numeric_only=False)

pandas.api.typing.Rolling.skew

Rolling.skew(numeric_only=False)

pandas.api.typing.Rolling.kurt

Rolling.kurt(numeric_only=False)

pandas.api.typing.Rolling.apply · ficha p. 76

Rolling.apply(func,raw=False,engine=None,engine_kwargs=None,args=None,kwargs=None)

pandas.api.typing.Rolling.pipe

finalRolling.pipe(func,*args,**kwargs)

pandas.api.typing.Rolling.aggregate

Rolling.aggregate(func=None,*args,**kwargs)

pandas.api.typing.Rolling.quantile

Rolling.quantile(q,interpolation='linear',numeric_only=False)

pandas.api.typing.Rolling.sem

Rolling.sem(ddof=1,numeric_only=False)

pandas.api.typing.Rolling.rank

Rolling.rank(method='average',ascending=True,pct=False,numeric_only=False)

pandas.api.typing.Rolling.nunique

Rolling.nunique(numeric_only=False)

pandas.api.typing.Window.mean · ficha p. 77

Window.mean(numeric_only=False,**kwargs)

pandas.api.typing.Window.sum

Window.sum(numeric_only=False,**kwargs)

pandas.api.typing.Window.var

Window.var(ddof=1,numeric_only=False,**kwargs)

pandas.api.typing.Window.std

Window.std(ddof=1,numeric_only=False,**kwargs)

pandas.api.typing.Expanding.count

Expanding.count(numeric_only=False)

pandas.api.typing.Expanding.sum

Expanding.sum(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Expanding.mean

Expanding.mean(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Expanding.median

Expanding.median(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Expanding.var

Expanding.var(ddof=1,numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Expanding.std

Expanding.std(ddof=1,numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Expanding.min

Expanding.min(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Expanding.max

Expanding.max(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.Expanding.first

Expanding.first(numeric_only=False)

pandas.api.typing.Expanding.last

Expanding.last(numeric_only=False)

pandas.api.typing.Expanding.corr

Expanding.corr(other=None,pairwise=None,ddof=1,numeric_only=False)

pandas.api.typing.Expanding.cov

Expanding.cov(other=None,pairwise=None,ddof=1,numeric_only=False)

pandas.api.typing.Expanding.skew

Expanding.skew(numeric_only=False)

pandas.api.typing.Expanding.kurt

Expanding.kurt(numeric_only=False)

pandas.api.typing.Expanding.apply

Expanding.apply(func,raw=False,engine=None,engine_kwargs=None,args=None,kwargs=None)

pandas.api.typing.Expanding.pipe

finalExpanding.pipe(func,*args,**kwargs)

pandas.api.typing.Expanding.aggregate

Expanding.aggregate(func=None,*args,**kwargs)

pandas.api.typing.Expanding.quantile

Expanding.quantile(q,interpolation='linear',numeric_only=False)

pandas.api.typing.Expanding.sem

Expanding.sem(ddof=1,numeric_only=False)

pandas.api.typing.Expanding.rank

Expanding.rank(method='average',ascending=True,pct=False,numeric_only=False)

pandas.api.typing.Expanding.nunique

Expanding.nunique(numeric_only=False)

pandas.api.typing.ExponentialMovingWindow.mean · ficha p. 76

ExponentialMovingWindow.mean(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.ExponentialMovingWindow.sum

ExponentialMovingWindow.sum(numeric_only=False,engine=None,engine_kwargs=None)

pandas.api.typing.ExponentialMovingWindow.std

ExponentialMovingWindow.std(bias=False,numeric_only=False)

pandas.api.typing.ExponentialMovingWindow.var

ExponentialMovingWindow.var(bias=False,numeric_only=False)

pandas.api.typing.ExponentialMovingWindow.corr

ExponentialMovingWindow.corr(other=None,pairwise=None,numeric_only=False)

pandas.api.typing.ExponentialMovingWindow.cov

ExponentialMovingWindow.cov(other=None,pairwise=None,bias=False,numeric_only=False)

pandas.api.indexers.BaseIndexer

pd.api.indexers.BaseIndexer(index_array=None,window_size=0,**kwargs)

pandas.api.indexers.FixedForwardWindowIndexer · ficha p. 77

pd.api.indexers.FixedForwardWindowIndexer(index_array=None,window_size=0,**kwargs)

pandas.api.indexers.VariableOffsetWindowIndexer

pd.api.indexers.VariableOffsetWindowIndexer(index_array=None,window_size=0,index=None,offset=None,**kwargs)

Remuestreo temporal

28 entradas documentadas.

pandas.api.typing.Resampler.groups

Resampler.groups

pandas.api.typing.Resampler.indices

Resampler.indices

pandas.api.typing.Resampler.get_group

Resampler.get_group(name)

pandas.api.typing.Resampler.apply

Resampler.apply(func=None,*args,**kwargs)

pandas.api.typing.Resampler.aggregate

finalResampler.aggregate(func=None,*args,**kwargs)

pandas.api.typing.Resampler.transform

finalResampler.transform(arg,*args,**kwargs)

pandas.api.typing.Resampler.pipe

finalResampler.pipe(func,*args,**kwargs)

pandas.api.typing.Resampler.ffill · ficha p. 74

finalResampler.ffill(limit=None)

pandas.api.typing.Resampler.bfill

finalResampler.bfill(limit=None)

pandas.api.typing.Resampler.nearest

finalResampler.nearest(limit=None)

pandas.api.typing.Resampler.asfreq

finalResampler.asfreq(fill_value=None)

pandas.api.typing.Resampler.interpolate

finalResampler.interpolate(method='linear',*,axis=0,limit=None,limit_direction='forward',limit_area=None,**kwargs)

pandas.api.typing.Resampler.count

finalResampler.count()

pandas.api.typing.Resampler.nunique

finalResampler.nunique()

pandas.api.typing.Resampler.first

finalResampler.first(numeric_only=False,min_count=0,skipna=True)

pandas.api.typing.Resampler.last

finalResampler.last(numeric_only=False,min_count=0,skipna=True)

pandas.api.typing.Resampler.max

finalResampler.max(numeric_only=False,min_count=0)

pandas.api.typing.Resampler.mean

finalResampler.mean(numeric_only=False)

pandas.api.typing.Resampler.median

finalResampler.median(numeric_only=False)

pandas.api.typing.Resampler.min

finalResampler.min(numeric_only=False,min_count=0)

pandas.api.typing.Resampler.ohlc

finalResampler.ohlc()

pandas.api.typing.Resampler.prod

finalResampler.prod(numeric_only=False,min_count=0)

pandas.api.typing.Resampler.size

finalResampler.size()

pandas.api.typing.Resampler.sem

finalResampler.sem(ddof=1,numeric_only=False)

pandas.api.typing.Resampler.std

finalResampler.std(ddof=1,numeric_only=False)

pandas.api.typing.Resampler.sum

finalResampler.sum(numeric_only=False,min_count=0)

pandas.api.typing.Resampler.var

finalResampler.var(ddof=1,numeric_only=False)

pandas.api.typing.Resampler.quantile

finalResampler.quantile(q=0.5,**kwargs)

Arrays, escalares y tipos

223 entradas documentadas.

pandas.DatetimeTZDtype

pd.DatetimeTZDtype(unit='ns',tz=None)

pandas.Timestamp · ficha p. 67

pd.Timestamp(ts_input=<objectobject>,year=None,month=None,day=None,hour=None,minute=None,second=None,microsecond=None,tzinfo=None,*,nanosecond=None,tz=<objectobject>,unit=None,fold=None)

pandas.Timedelta · ficha p. 67

pd.Timedelta(value=<objectobject>,unit=None,**kwargs)

AVISO DE OBSOLESCENCIA (API o parámetro). Los alias de unidad w, d, MIN, MS, US y NS están obsoletos. Usa W, D, min, ms, us y ns respectivamente.

pandas.PeriodDtype

pd.PeriodDtype(freq)

pandas.Period

pd.Period(value=None,freq=None,ordinal=None,year=None,month=None,quarter=None,day=None,hour=None,minute=None,second=None)

pandas.IntervalDtype

pd.IntervalDtype(subtype=None,closed=None)

pandas.Interval

pd.Interval

pandas.Int64Dtype

pd.Int64Dtype

pandas.Float64Dtype

pd.Float64Dtype

pandas.CategoricalDtype · ficha p. 78

pd.CategoricalDtype(categories=None,ordered=False)

pandas.SparseDtype

pd.SparseDtype(dtype=<class'numpy.float64'>,fill_value=None)

pandas.StringDtype

pd.StringDtype(storage=None,na_value=<NA>)

pandas.BooleanDtype

pd.BooleanDtype

pandas.ArrowDtype

pd.ArrowDtype(pyarrow_dtype)

pandas.NA

pd.NA=<NA>

pandas.array · ficha p. 8

pd.array(data,dtype=None,copy=True)

pandas.Int8Dtype

pd.Int8Dtype

pandas.Int16Dtype

pd.Int16Dtype

pandas.Int32Dtype

pd.Int32Dtype

pandas.UInt8Dtype

pd.UInt8Dtype

pandas.UInt16Dtype

pd.UInt16Dtype

pandas.UInt32Dtype

pd.UInt32Dtype

pandas.UInt64Dtype

pd.UInt64Dtype

pandas.Float32Dtype

pd.Float32Dtype

pandas.arrays.ArrowExtensionArray

pd.arrays.ArrowExtensionArray(values)

pandas.Timestamp.asm8

Timestamp.asm8

pandas.Timestamp.day

Timestamp.day

pandas.Timestamp.dayofweek

Timestamp.dayofweek

pandas.Timestamp.day_of_week

Timestamp.day_of_week

pandas.Timestamp.dayofyear

Timestamp.dayofyear

pandas.Timestamp.day_of_year

Timestamp.day_of_year

pandas.Timestamp.days_in_month

Timestamp.days_in_month

pandas.Timestamp.daysinmonth

Timestamp.daysinmonth

pandas.Timestamp.fold

Timestamp.fold

pandas.Timestamp.hour

Timestamp.hour

pandas.Timestamp.is_leap_year

Timestamp.is_leap_year

pandas.Timestamp.is_month_end

Timestamp.is_month_end

pandas.Timestamp.is_month_start

Timestamp.is_month_start

pandas.Timestamp.is_quarter_end

Timestamp.is_quarter_end

pandas.Timestamp.is_quarter_start

Timestamp.is_quarter_start

pandas.Timestamp.is_year_end

Timestamp.is_year_end

pandas.Timestamp.is_year_start

Timestamp.is_year_start

pandas.Timestamp.max

Timestamp.max=Timestamp('2262-04-1123:47:16.854775807')

pandas.Timestamp.microsecond

Timestamp.microsecond

pandas.Timestamp.min

Timestamp.min=Timestamp('1677-09-2100:12:43.145224193')

pandas.Timestamp.minute

Timestamp.minute

pandas.Timestamp.month

Timestamp.month

pandas.Timestamp.nanosecond

Timestamp.nanosecond

pandas.Timestamp.quarter

Timestamp.quarter

pandas.Timestamp.resolution

Timestamp.resolution=Timedelta('0days00:00:00.000000001')

pandas.Timestamp.second

Timestamp.second

pandas.Timestamp.tz

Timestamp.tz

pandas.Timestamp.tzinfo

Timestamp.tzinfo

pandas.Timestamp.unit

Timestamp.unit

pandas.Timestamp.value

Timestamp.value

pandas.Timestamp.week

Timestamp.week

pandas.Timestamp.weekofyear

Timestamp.weekofyear

pandas.Timestamp.year

Timestamp.year

pandas.Timestamp.as_unit

Timestamp.as_unit(unit,round_ok=True)

pandas.Timestamp.astimezone

Timestamp.astimezone(tz)

pandas.Timestamp.ceil

Timestamp.ceil(freq,ambiguous='raise',nonexistent='raise')

pandas.Timestamp.combine

methodTimestamp.combine(date,time)

pandas.Timestamp.ctime

Timestamp.ctime()

pandas.Timestamp.date

Timestamp.date()

pandas.Timestamp.day_name

Timestamp.day_name(locale=None)

pandas.Timestamp.dst

Timestamp.dst()

pandas.Timestamp.floor

Timestamp.floor(freq,ambiguous='raise',nonexistent='raise')

pandas.Timestamp.fromordinal

methodTimestamp.fromordinal(ordinal,tz=None)

pandas.Timestamp.fromtimestamp

methodTimestamp.fromtimestamp(ts,tz=None)

pandas.Timestamp.isocalendar

Timestamp.isocalendar()

pandas.Timestamp.isoformat

Timestamp.isoformat(sep='T',timespec='auto')

pandas.Timestamp.isoweekday

Timestamp.isoweekday()

pandas.Timestamp.month_name

Timestamp.month_name(locale=None)

pandas.Timestamp.normalize

Timestamp.normalize()

pandas.Timestamp.now

methodTimestamp.now(tz=None)

pandas.Timestamp.replace

Timestamp.replace(year=None,month=None,day=None,hour=None,minute=None,second=None,microsecond=None,nanosecond=None,tzinfo=<class'object'>,fold=None)

pandas.Timestamp.round

Timestamp.round(freq,ambiguous='raise',nonexistent='raise')

pandas.Timestamp.strftime

Timestamp.strftime(format)

pandas.Timestamp.strptime

methodTimestamp.strptime(date_string,format)

pandas.Timestamp.time

Timestamp.time()

pandas.Timestamp.timestamp

Timestamp.timestamp()

pandas.Timestamp.timetuple

Timestamp.timetuple()

pandas.Timestamp.timetz

Timestamp.timetz()

pandas.Timestamp.to_datetime64

Timestamp.to_datetime64()

pandas.Timestamp.to_numpy

Timestamp.to_numpy(dtype=None,copy=False)

pandas.Timestamp.to_julian_date

Timestamp.to_julian_date()

pandas.Timestamp.to_period

Timestamp.to_period(freq=None)

pandas.Timestamp.to_pydatetime

Timestamp.to_pydatetime(warn=True)

pandas.Timestamp.today

methodTimestamp.today(tz=None)

pandas.Timestamp.toordinal

Timestamp.toordinal()

pandas.Timestamp.tz_convert

Timestamp.tz_convert(tz)

pandas.Timestamp.tz_localize

Timestamp.tz_localize(tz,ambiguous='raise',nonexistent='raise')

pandas.Timestamp.tzname

Timestamp.tzname()

pandas.Timestamp.utcfromtimestamp

methodTimestamp.utcfromtimestamp(ts)

pandas.Timestamp.utcnow

methodTimestamp.utcnow()

pandas.Timestamp.utcoffset

Timestamp.utcoffset()

pandas.Timestamp.utctimetuple

Timestamp.utctimetuple()

pandas.Timestamp.weekday

Timestamp.weekday()

pandas.arrays.DatetimeArray

pd.arrays.DatetimeArray(data,dtype=None,freq=None,copy=False)

pandas.Timedelta.asm8

Timedelta.asm8

pandas.Timedelta.components

Timedelta.components

pandas.Timedelta.days

Timedelta.days

pandas.Timedelta.max

Timedelta.max=Timedelta('106751days23:47:16.854775807')

pandas.Timedelta.microseconds

Timedelta.microseconds

pandas.Timedelta.min

Timedelta.min=Timedelta('-106752days+00:12:43.145224193')

pandas.Timedelta.nanoseconds

Timedelta.nanoseconds

pandas.Timedelta.resolution

Timedelta.resolution=Timedelta('0days00:00:00.000000001')

pandas.Timedelta.seconds

Timedelta.seconds

pandas.Timedelta.unit

Timedelta.unit

pandas.Timedelta.value

Timedelta.value

pandas.Timedelta.view

Timedelta.view(dtype)

pandas.Timedelta.as_unit

Timedelta.as_unit(unit,round_ok=True)

pandas.Timedelta.ceil

Timedelta.ceil(freq)

pandas.Timedelta.floor

Timedelta.floor(freq)

pandas.Timedelta.isoformat

Timedelta.isoformat()

pandas.Timedelta.round

Timedelta.round(freq)

pandas.Timedelta.to_pytimedelta

Timedelta.to_pytimedelta()

pandas.Timedelta.to_timedelta64

Timedelta.to_timedelta64()

pandas.Timedelta.to_numpy

Timedelta.to_numpy(dtype=None,copy=False)

pandas.Timedelta.total_seconds

Timedelta.total_seconds()

pandas.arrays.TimedeltaArray

pd.arrays.TimedeltaArray(data,dtype=None,freq=None,copy=False)

pandas.Period.day

Period.day

pandas.Period.dayofweek

Period.dayofweek

pandas.Period.day_of_week

Period.day_of_week

pandas.Period.dayofyear

Period.dayofyear

pandas.Period.day_of_year

Period.day_of_year

pandas.Period.days_in_month

Period.days_in_month

pandas.Period.daysinmonth

Period.daysinmonth

pandas.Period.end_time

Period.end_time

pandas.Period.freq

Period.freq

pandas.Period.freqstr

Period.freqstr

pandas.Period.hour

Period.hour

pandas.Period.is_leap_year

Period.is_leap_year

pandas.Period.minute

Period.minute

pandas.Period.month

Period.month

pandas.Period.ordinal

Period.ordinal

pandas.Period.quarter

Period.quarter

pandas.Period.qyear

Period.qyear

pandas.Period.second

Period.second

pandas.Period.start_time

Period.start_time

pandas.Period.week

Period.week

pandas.Period.weekday

Period.weekday

pandas.Period.weekofyear

Period.weekofyear

pandas.Period.year

Period.year

pandas.Period.asfreq

Period.asfreq(freq,how='E')

pandas.Period.now

methodPeriod.now(freq)

pandas.Period.strftime

Period.strftime(fmt)

pandas.Period.to_timestamp

Period.to_timestamp(freq=None,how='start')

pandas.arrays.PeriodArray

pd.arrays.PeriodArray(values,dtype=None,copy=False)

pandas.Interval.closed

Interval.closed

pandas.Interval.closed_left

Interval.closed_left

pandas.Interval.closed_right

Interval.closed_right

pandas.Interval.is_empty

Interval.is_empty

pandas.Interval.left

Interval.left

pandas.Interval.length

Interval.length

pandas.Interval.mid

Interval.mid

pandas.Interval.open_left

Interval.open_left

pandas.Interval.open_right

Interval.open_right

pandas.Interval.overlaps

Interval.overlaps(other)

pandas.Interval.right

Interval.right

pandas.arrays.IntervalArray

pd.arrays.IntervalArray(data,closed=None,dtype=None,copy=False,verify_integrity=True)

pandas.arrays.IntegerArray

pd.arrays.IntegerArray(values,mask,copy=False)

pandas.arrays.FloatingArray

pd.arrays.FloatingArray(values,mask,copy=False)

pandas.CategoricalDtype.categories

CategoricalDtype.categories

pandas.CategoricalDtype.ordered

CategoricalDtype.ordered

pandas.Categorical.from_codes

methodCategorical.from_codes(codes,categories=None,ordered=None,dtype=None,validate=True)

pandas.Categorical.dtype

Categorical.dtype

pandas.Categorical.categories

Categorical.categories

pandas.Categorical.ordered

Categorical.ordered

pandas.Categorical.codes

Categorical.codes

pandas.Categorical.as_ordered

Categorical.as_ordered()

pandas.Categorical.as_unordered

Categorical.as_unordered()

pandas.Categorical.set_categories

Categorical.set_categories(new_categories,ordered=None,rename=False)

pandas.Categorical.rename_categories

Categorical.rename_categories(new_categories)

pandas.Categorical.reorder_categories

Categorical.reorder_categories(new_categories,ordered=None)

pandas.Categorical.add_categories

Categorical.add_categories(new_categories)

pandas.Categorical.remove_categories

Categorical.remove_categories(removals)

pandas.Categorical.remove_unused_categories

Categorical.remove_unused_categories()

pandas.Categorical.map

Categorical.map(mapper,na_action=None)

pandas.arrays.SparseArray

pd.arrays.SparseArray(data,sparse_index=None,fill_value=None,kind='integer',dtype=None,copy=False)

pandas.arrays.StringArray

pd.arrays.StringArray(values,*,dtype=None,copy=False)

pandas.arrays.ArrowStringArray

pd.arrays.ArrowStringArray(values,*,dtype=None)

pandas.arrays.BooleanArray

pd.arrays.BooleanArray(values,mask,copy=False)

pandas.api.types.union_categoricals

pd.api.types.union_categoricals(to_union,sort_categories=False,ignore_order=False)

pandas.api.types.infer_dtype

pd.api.types.infer_dtype(value,skipna=True)

pandas.api.types.pandas_dtype

pd.api.types.pandas_dtype(dtype)

pandas.api.types.is_any_real_numeric_dtype

pd.api.types.is_any_real_numeric_dtype(arr_or_dtype)

pandas.api.types.is_bool_dtype

pd.api.types.is_bool_dtype(arr_or_dtype)

pandas.api.types.is_categorical_dtype

pd.api.types.is_categorical_dtype(arr_or_dtype)
AVISO DE OBSOLESCENCIA (API o parámetro). Obsoleto desde 2.2.0, 2.2.0. Alternativa indicada por pandas: isinstance(dtype, pd.CategoricalDtype). Consulta el enlace para los detalles.

pandas.api.types.is_complex_dtype

pd.api.types.is_complex_dtype(arr_or_dtype)

pandas.api.types.is_datetime64_any_dtype

pd.api.types.is_datetime64_any_dtype(arr_or_dtype)

pandas.api.types.is_datetime64_dtype

pd.api.types.is_datetime64_dtype(arr_or_dtype)

pandas.api.types.is_datetime64_ns_dtype

pd.api.types.is_datetime64_ns_dtype(arr_or_dtype)

pandas.api.types.is_datetime64tz_dtype

pd.api.types.is_datetime64tz_dtype(arr_or_dtype)
AVISO DE OBSOLESCENCIA (API o parámetro). Obsoleto desde 2.1.0, 2.1.0. Alternativa indicada por pandas: isinstance(dtype, pd.DatetimeTZDtype). Consulta el enlace para los detalles.

pandas.api.types.is_dtype_equal

pd.api.types.is_dtype_equal(source,target)

pandas.api.types.is_extension_array_dtype

pd.api.types.is_extension_array_dtype(arr_or_dtype)

pandas.api.types.is_float_dtype

pd.api.types.is_float_dtype(arr_or_dtype)

pandas.api.types.is_int64_dtype

pd.api.types.is_int64_dtype(arr_or_dtype)
AVISO DE OBSOLESCENCIA (API o parámetro). Obsoleto desde 2.1.0, 2.1.0. Alternativa indicada por pandas: dtype == np.int64. Consulta el enlace para los detalles.

pandas.api.types.is_integer_dtype

pd.api.types.is_integer_dtype(arr_or_dtype)

pandas.api.types.is_interval_dtype

pd.api.types.is_interval_dtype(arr_or_dtype)
AVISO DE OBSOLESCENCIA (API o parámetro). Obsoleto desde 2.2.0, 2.2.0. Alternativa indicada por pandas: isinstance(dtype, pd.IntervalDtype). Consulta el enlace para los detalles.

pandas.api.types.is_numeric_dtype

pd.api.types.is_numeric_dtype(arr_or_dtype)

pandas.api.types.is_object_dtype

pd.api.types.is_object_dtype(arr_or_dtype)

pandas.api.types.is_period_dtype

pd.api.types.is_period_dtype(arr_or_dtype)

AVISO DE OBSOLESCENCIA (API o parámetro). Obsoleto desde 2.2.0, 2.2.0. Alternativa indicada por pandas: isinstance(dtype, pd.PeriodDtype). Consulta el enlace para los detalles.

pandas.api.types.is_signed_integer_dtype

pd.api.types.is_signed_integer_dtype(arr_or_dtype)

pandas.api.types.is_string_dtype

pd.api.types.is_string_dtype(arr_or_dtype)

pandas.api.types.is_timedelta64_dtype

pd.api.types.is_timedelta64_dtype(arr_or_dtype)

pandas.api.types.is_timedelta64_ns_dtype

pd.api.types.is_timedelta64_ns_dtype(arr_or_dtype)

pandas.api.types.is_unsigned_integer_dtype

pd.api.types.is_unsigned_integer_dtype(arr_or_dtype)

pandas.api.types.is_sparse

pd.api.types.is_sparse(arr)

AVISO DE OBSOLESCENCIA (API o parámetro). Obsoleto desde 2.1.0, 2.1.0. Alternativa indicada por pandas: isinstance(dtype, pd.SparseDtype). Consulta el enlace para los detalles.

pandas.api.types.is_dict_like

pd.api.types.is_dict_like(obj)

pandas.api.types.is_file_like

pd.api.types.is_file_like(obj)

pandas.api.types.is_list_like

pd.api.types.is_list_like(obj,allow_sets=True)

pandas.api.types.is_named_tuple

pd.api.types.is_named_tuple(obj)

pandas.api.types.is_iterator

pd.api.types.is_iterator(obj)

pandas.api.types.is_bool

pd.api.types.is_bool(obj)

pandas.api.types.is_complex

pd.api.types.is_complex(obj)

pandas.api.types.is_float

pd.api.types.is_float(obj)

pandas.api.types.is_hashable

pd.api.types.is_hashable(obj,allow_slice=True)

pandas.api.types.is_integer

pd.api.types.is_integer(obj)

pandas.api.types.is_number

pd.api.types.is_number(obj)

pandas.api.types.is_re

pd.api.types.is_re(obj)

pandas.api.types.is_re_compilable

pd.api.types.is_re_compilable(obj)

pandas.api.types.is_scalar

pd.api.types.is_scalar(val)

Desplazamientos de calendario

688 entradas documentadas.

pandas.tseries.offsets.DateOffset

pd.tseries.offsets.DateOffset

pandas.tseries.offsets.DateOffset.freqstr

DateOffset.freqstr

pandas.tseries.offsets.DateOffset.kwds

DateOffset.kwds

pandas.tseries.offsets.DateOffset.name

DateOffset.name

pandas.tseries.offsets.DateOffset.nanos

DateOffset.nanos

pandas.tseries.offsets.DateOffset.normalize

DateOffset.normalize

pandas.tseries.offsets.DateOffset.rule_code

DateOffset.rule_code

pandas.tseries.offsets.DateOffset.n

DateOffset.n

pandas.tseries.offsets.DateOffset.copy

DateOffset.copy()

pandas.tseries.offsets.DateOffset.is_on_offset

DateOffset.is_on_offset(dt)

pandas.tseries.offsets.DateOffset.is_month_start

DateOffset.is_month_start(ts)

pandas.tseries.offsets.DateOffset.is_month_end

DateOffset.is_month_end(ts)

pandas.tseries.offsets.DateOffset.is_quarter_start

DateOffset.is_quarter_start(ts)

pandas.tseries.offsets.DateOffset.is_quarter_end

DateOffset.is_quarter_end(ts)

pandas.tseries.offsets.DateOffset.is_year_start

DateOffset.is_year_start(ts)

pandas.tseries.offsets.DateOffset.is_year_end

DateOffset.is_year_end(ts)

pandas.tseries.offsets.DateOffset.rollback

DateOffset.rollback(dt)

pandas.tseries.offsets.DateOffset.rollforward

DateOffset.rollforward(dt)

pandas.tseries.offsets.BusinessDay

pd.tseries.offsets.BusinessDay

pandas.tseries.offsets.BDay

pd.tseries.offsets.BDay

pandas.tseries.offsets.BusinessDay.freqstr

BusinessDay.freqstr

pandas.tseries.offsets.BusinessDay.kwds

BusinessDay.kwds

pandas.tseries.offsets.BusinessDay.name

BusinessDay.name

pandas.tseries.offsets.BusinessDay.nanos

BusinessDay.nanos

pandas.tseries.offsets.BusinessDay.normalize

BusinessDay.normalize

pandas.tseries.offsets.BusinessDay.rule_code

BusinessDay.rule_code

pandas.tseries.offsets.BusinessDay.n

BusinessDay.n

pandas.tseries.offsets.BusinessDay.weekmask

BusinessDay.weekmask

pandas.tseries.offsets.BusinessDay.holidays

BusinessDay.holidays

pandas.tseries.offsets.BusinessDay.calendar

BusinessDay.calendar

pandas.tseries.offsets.BusinessDay.copy

BusinessDay.copy()

pandas.tseries.offsets.BusinessDay.is_on_offset

BusinessDay.is_on_offset(dt)

pandas.tseries.offsets.BusinessDay.is_month_start

BusinessDay.is_month_start(ts)

pandas.tseries.offsets.BusinessDay.is_month_end

BusinessDay.is_month_end(ts)

pandas.tseries.offsets.BusinessDay.is_quarter_start

BusinessDay.is_quarter_start(ts)

pandas.tseries.offsets.BusinessDay.is_quarter_end

BusinessDay.is_quarter_end(ts)

pandas.tseries.offsets.BusinessDay.is_year_start

BusinessDay.is_year_start(ts)

pandas.tseries.offsets.BusinessDay.is_year_end

BusinessDay.is_year_end(ts)

pandas.tseries.offsets.BusinessHour

pd.tseries.offsets.BusinessHour

pandas.tseries.offsets.BusinessHour.freqstr

BusinessHour.freqstr

pandas.tseries.offsets.BusinessHour.kwds

BusinessHour.kwds

pandas.tseries.offsets.BusinessHour.name

BusinessHour.name

pandas.tseries.offsets.BusinessHour.nanos

BusinessHour.nanos

pandas.tseries.offsets.BusinessHour.normalize

BusinessHour.normalize

pandas.tseries.offsets.BusinessHour.rule_code

BusinessHour.rule_code

pandas.tseries.offsets.BusinessHour.n

BusinessHour.n

pandas.tseries.offsets.BusinessHour.start

BusinessHour.start

pandas.tseries.offsets.BusinessHour.end

BusinessHour.end

pandas.tseries.offsets.BusinessHour.weekmask

BusinessHour.weekmask

pandas.tseries.offsets.BusinessHour.holidays

BusinessHour.holidays

pandas.tseries.offsets.BusinessHour.calendar

BusinessHour.calendar

pandas.tseries.offsets.BusinessHour.copy

BusinessHour.copy()

pandas.tseries.offsets.BusinessHour.is_on_offset

BusinessHour.is_on_offset(dt)

pandas.tseries.offsets.BusinessHour.is_month_start

BusinessHour.is_month_start(ts)

pandas.tseries.offsets.BusinessHour.is_month_end

BusinessHour.is_month_end(ts)

pandas.tseries.offsets.BusinessHour.is_quarter_start

BusinessHour.is_quarter_start(ts)

pandas.tseries.offsets.BusinessHour.is_quarter_end

BusinessHour.is_quarter_end(ts)

pandas.tseries.offsets.BusinessHour.is_year_start

BusinessHour.is_year_start(ts)

pandas.tseries.offsets.BusinessHour.is_year_end

BusinessHour.is_year_end(ts)

pandas.tseries.offsets.CustomBusinessDay

pd.tseries.offsets.CustomBusinessDay

pandas.tseries.offsets.CDay

pd.tseries.offsets.CDay

pandas.tseries.offsets.CustomBusinessDay.freqstr

CustomBusinessDay.freqstr

pandas.tseries.offsets.CustomBusinessDay.kwds

CustomBusinessDay.kwds

pandas.tseries.offsets.CustomBusinessDay.name

CustomBusinessDay.name

pandas.tseries.offsets.CustomBusinessDay.nanos

CustomBusinessDay.nanos

pandas.tseries.offsets.CustomBusinessDay.normalize

CustomBusinessDay.normalize

pandas.tseries.offsets.CustomBusinessDay.rule_code

CustomBusinessDay.rule_code

pandas.tseries.offsets.CustomBusinessDay.n

CustomBusinessDay.n

pandas.tseries.offsets.CustomBusinessDay.weekmask
CustomBusinessDay.weekmask

pandas.tseries.offsets.CustomBusinessDay.calendar
CustomBusinessDay.calendar

pandas.tseries.offsets.CustomBusinessDay.holidays
CustomBusinessDay.holidays

pandas.tseries.offsets.CustomBusinessDay.copy
CustomBusinessDay.copy()

pandas.tseries.offsets.CustomBusinessDay.is_on_offset
CustomBusinessDay.is_on_offset(dt)

pandas.tseries.offsets.CustomBusinessDay.is_month_start
CustomBusinessDay.is_month_start(ts)

pandas.tseries.offsets.CustomBusinessDay.is_month_end
CustomBusinessDay.is_month_end(ts)

pandas.tseries.offsets.CustomBusinessDay.is_quarter_start
CustomBusinessDay.is_quarter_start(ts)

pandas.tseries.offsets.CustomBusinessDay.is_quarter_end
CustomBusinessDay.is_quarter_end(ts)

pandas.tseries.offsets.CustomBusinessDay.is_year_start
CustomBusinessDay.is_year_start(ts)

pandas.tseries.offsets.CustomBusinessDay.is_year_end
CustomBusinessDay.is_year_end(ts)

pandas.tseries.offsets.CustomBusinessHour
pd.tseries.offsets.CustomBusinessHour

pandas.tseries.offsets.CustomBusinessHour.freqstr
CustomBusinessHour.freqstr

pandas.tseries.offsets.CustomBusinessHour.kwds
CustomBusinessHour.kwds

pandas.tseries.offsets.CustomBusinessHour.name
CustomBusinessHour.name

pandas.tseries.offsets.CustomBusinessHour.nanos
CustomBusinessHour.nanos

pandas.tseries.offsets.CustomBusinessHour.normalize
CustomBusinessHour.normalize

pandas.tseries.offsets.CustomBusinessHour.rule_code
CustomBusinessHour.rule_code

pandas.tseries.offsets.CustomBusinessHour.n
CustomBusinessHour.n

pandas.tseries.offsets.CustomBusinessHour.weekmask
CustomBusinessHour.weekmask

pandas.tseries.offsets.CustomBusinessHour.calendar
CustomBusinessHour.calendar

pandas.tseries.offsets.CustomBusinessHour.holidays
CustomBusinessHour.holidays

pandas.tseries.offsets.CustomBusinessHour.start
CustomBusinessHour.start

pandas.tseries.offsets.CustomBusinessHour.end

CustomBusinessHour.end

pandas.tseries.offsets.CustomBusinessHour.copy

CustomBusinessHour.copy()

pandas.tseries.offsets.CustomBusinessHour.is_on_offset

CustomBusinessHour.is_on_offset(dt)

pandas.tseries.offsets.CustomBusinessHour.is_month_start

CustomBusinessHour.is_month_start(ts)

pandas.tseries.offsets.CustomBusinessHour.is_month_end

CustomBusinessHour.is_month_end(ts)

pandas.tseries.offsets.CustomBusinessHour.is_quarter_start

CustomBusinessHour.is_quarter_start(ts)

pandas.tseries.offsets.CustomBusinessHour.is_quarter_end

CustomBusinessHour.is_quarter_end(ts)

pandas.tseries.offsets.CustomBusinessHour.is_year_start

CustomBusinessHour.is_year_start(ts)

pandas.tseries.offsets.CustomBusinessHour.is_year_end

CustomBusinessHour.is_year_end(ts)

pandas.tseries.offsets.MonthEnd

pd.tseries.offsets.MonthEnd

pandas.tseries.offsets.MonthEnd.freqstr

MonthEnd.freqstr

pandas.tseries.offsets.MonthEnd.kwds

MonthEnd.kwds

pandas.tseries.offsets.MonthEnd.name

MonthEnd.name

pandas.tseries.offsets.MonthEnd.nanos

MonthEnd.nanos

pandas.tseries.offsets.MonthEnd.normalize

MonthEnd.normalize

pandas.tseries.offsets.MonthEnd.rule_code

MonthEnd.rule_code

pandas.tseries.offsets.MonthEnd.n

MonthEnd.n

pandas.tseries.offsets.MonthEnd.copy

MonthEnd.copy()

pandas.tseries.offsets.MonthEnd.is_on_offset

MonthEnd.is_on_offset(dt)

pandas.tseries.offsets.MonthEnd.is_month_start

MonthEnd.is_month_start(ts)

pandas.tseries.offsets.MonthEnd.is_month_end

MonthEnd.is_month_end(ts)

pandas.tseries.offsets.MonthEnd.is_quarter_start

MonthEnd.is_quarter_start(ts)

pandas.tseries.offsets.MonthEnd.is_quarter_end

MonthEnd.is_quarter_end(ts)

pandas.tseries.offsets.MonthEnd.is_year_start

MonthEnd.is_year_start(ts)

pandas.tseries.offsets.MonthEnd.is_year_end

MonthEnd.is_year_end(ts)

pandas.tseries.offsets.MonthBegin

pd.tseries.offsets.MonthBegin

pandas.tseries.offsets.MonthBegin.freqstr

MonthBegin.freqstr

pandas.tseries.offsets.MonthBegin.kwds

MonthBegin.kwds

pandas.tseries.offsets.MonthBegin.name

MonthBegin.name

pandas.tseries.offsets.MonthBegin.nanos

MonthBegin.nanos

pandas.tseries.offsets.MonthBegin.normalize

MonthBegin.normalize

pandas.tseries.offsets.MonthBegin.rule_code

MonthBegin.rule_code

pandas.tseries.offsets.MonthBegin.n

MonthBegin.n

pandas.tseries.offsets.MonthBegin.copy

MonthBegin.copy()

pandas.tseries.offsets.MonthBegin.is_on_offset

MonthBegin.is_on_offset(dt)

pandas.tseries.offsets.MonthBegin.is_month_start

MonthBegin.is_month_start(ts)

pandas.tseries.offsets.MonthBegin.is_month_end

MonthBegin.is_month_end(ts)

pandas.tseries.offsets.MonthBegin.is_quarter_start

MonthBegin.is_quarter_start(ts)

pandas.tseries.offsets.MonthBegin.is_quarter_end

MonthBegin.is_quarter_end(ts)

pandas.tseries.offsets.MonthBegin.is_year_start

MonthBegin.is_year_start(ts)

pandas.tseries.offsets.MonthBegin.is_year_end

MonthBegin.is_year_end(ts)

pandas.tseries.offsets.BusinessMonthEnd

pd.tseries.offsets.BusinessMonthEnd

pandas.tseries.offsets.BMonthEnd

pd.tseries.offsets.BMonthEnd

pandas.tseries.offsets.BusinessMonthEnd.freqstr

BusinessMonthEnd.freqstr

pandas.tseries.offsets.BusinessMonthEnd.kwds

BusinessMonthEnd.kwds

pandas.tseries.offsets.BusinessMonthEnd.name

BusinessMonthEnd.name

pandas.tseries.offsets.BusinessMonthEnd.nanos

BusinessMonthEnd.nanos

pandas.tseries.offsets.BusinessMonthEnd.normalize

BusinessMonthEnd.normalize

pandas.tseries.offsets.BusinessMonthEnd.rule_code

BusinessMonthEnd.rule_code

pandas.tseries.offsets.BusinessMonthEnd.n

BusinessMonthEnd.n

pandas.tseries.offsets.BusinessMonthEnd.copy

BusinessMonthEnd.copy()

pandas.tseries.offsets.BusinessMonthEnd.is_on_offset

BusinessMonthEnd.is_on_offset(dt)

pandas.tseries.offsets.BusinessMonthEnd.is_month_start

BusinessMonthEnd.is_month_start(ts)

pandas.tseries.offsets.BusinessMonthEnd.is_month_end

BusinessMonthEnd.is_month_end(ts)

pandas.tseries.offsets.BusinessMonthEnd.is_quarter_start

BusinessMonthEnd.is_quarter_start(ts)

pandas.tseries.offsets.BusinessMonthEnd.is_quarter_end

BusinessMonthEnd.is_quarter_end(ts)

pandas.tseries.offsets.BusinessMonthEnd.is_year_start

BusinessMonthEnd.is_year_start(ts)

pandas.tseries.offsets.BusinessMonthEnd.is_year_end

BusinessMonthEnd.is_year_end(ts)

pandas.tseries.offsets.BusinessMonthBegin

pd.tseries.offsets.BusinessMonthBegin

pandas.tseries.offsets.BMonthBegin

pd.tseries.offsets.BMonthBegin

pandas.tseries.offsets.BusinessMonthBegin.freqstr

BusinessMonthBegin.freqstr

pandas.tseries.offsets.BusinessMonthBegin.kwds

BusinessMonthBegin.kwds

pandas.tseries.offsets.BusinessMonthBegin.name

BusinessMonthBegin.name

pandas.tseries.offsets.BusinessMonthBegin.nanos

BusinessMonthBegin.nanos

pandas.tseries.offsets.BusinessMonthBegin.normalize

BusinessMonthBegin.normalize

pandas.tseries.offsets.BusinessMonthBegin.rule_code

BusinessMonthBegin.rule_code

pandas.tseries.offsets.BusinessMonthBegin.n

BusinessMonthBegin.n

pandas.tseries.offsets.BusinessMonthBegin.copy

BusinessMonthBegin.copy()

pandas.tseries.offsets.BusinessMonthBegin.is_on_offset

BusinessMonthBegin.is_on_offset(dt)

pandas.tseries.offsets.BusinessMonthBegin.is_month_start
BusinessMonthBegin.is_month_start(ts)
pandas.tseries.offsets.BusinessMonthBegin.is_month_end
BusinessMonthBegin.is_month_end(ts)
pandas.tseries.offsets.BusinessMonthBegin.is_quarter_start
BusinessMonthBegin.is_quarter_start(ts)
pandas.tseries.offsets.BusinessMonthBegin.is_quarter_end
BusinessMonthBegin.is_quarter_end(ts)
pandas.tseries.offsets.BusinessMonthBegin.is_year_start
BusinessMonthBegin.is_year_start(ts)
pandas.tseries.offsets.BusinessMonthBegin.is_year_end
BusinessMonthBegin.is_year_end(ts)
pandas.tseries.offsets.CustomBusinessMonthEnd
pd.tseries.offsets.CustomBusinessMonthEnd
pandas.tseries.offsets.CBMonthEnd
pd.tseries.offsets.CBMonthEnd
pandas.tseries.offsets.CustomBusinessMonthEnd.freqstr
CustomBusinessMonthEnd.freqstr
pandas.tseries.offsets.CustomBusinessMonthEnd.kwds
CustomBusinessMonthEnd.kwds
pandas.tseries.offsets.CustomBusinessMonthEnd.m_offset
CustomBusinessMonthEnd.m_offset
pandas.tseries.offsets.CustomBusinessMonthEnd.name
CustomBusinessMonthEnd.name
pandas.tseries.offsets.CustomBusinessMonthEnd.nanos
CustomBusinessMonthEnd.nanos
pandas.tseries.offsets.CustomBusinessMonthEnd.normalize
CustomBusinessMonthEnd.normalize
pandas.tseries.offsets.CustomBusinessMonthEnd.rule_code
CustomBusinessMonthEnd.rule_code
pandas.tseries.offsets.CustomBusinessMonthEnd.n
CustomBusinessMonthEnd.n
pandas.tseries.offsets.CustomBusinessMonthEnd.weekmask
CustomBusinessMonthEnd.weekmask
pandas.tseries.offsets.CustomBusinessMonthEnd.calendar
CustomBusinessMonthEnd.calendar
pandas.tseries.offsets.CustomBusinessMonthEnd.holidays
CustomBusinessMonthEnd.holidays
pandas.tseries.offsets.CustomBusinessMonthEnd.copy
CustomBusinessMonthEnd.copy()
pandas.tseries.offsets.CustomBusinessMonthEnd.is_on_offset
CustomBusinessMonthEnd.is_on_offset(dt)
pandas.tseries.offsets.CustomBusinessMonthEnd.is_month_start
CustomBusinessMonthEnd.is_month_start(ts)
pandas.tseries.offsets.CustomBusinessMonthEnd.is_month_end
CustomBusinessMonthEnd.is_month_end(ts)

pandas.tseries.offsets.CustomBusinessMonthEnd.is_quarter_start
CustomBusinessMonthEnd.is_quarter_start(ts)
pandas.tseries.offsets.CustomBusinessMonthEnd.is_quarter_end
CustomBusinessMonthEnd.is_quarter_end(ts)
pandas.tseries.offsets.CustomBusinessMonthEnd.is_year_start
CustomBusinessMonthEnd.is_year_start(ts)
pandas.tseries.offsets.CustomBusinessMonthEnd.is_year_end
CustomBusinessMonthEnd.is_year_end(ts)
pandas.tseries.offsets.CustomBusinessMonthBegin
pd.tseries.offsets.CustomBusinessMonthBegin
pandas.tseries.offsets.CBMonthBegin
pd.tseries.offsets.CBMonthBegin
pandas.tseries.offsets.CustomBusinessMonthBegin.freqstr
CustomBusinessMonthBegin.freqstr
pandas.tseries.offsets.CustomBusinessMonthBegin.kwds
CustomBusinessMonthBegin.kwds
pandas.tseries.offsets.CustomBusinessMonthBegin.m_offset
CustomBusinessMonthBegin.m_offset
pandas.tseries.offsets.CustomBusinessMonthBegin.name
CustomBusinessMonthBegin.name
pandas.tseries.offsets.CustomBusinessMonthBegin.nanos
CustomBusinessMonthBegin.nanos
pandas.tseries.offsets.CustomBusinessMonthBegin.normalize
CustomBusinessMonthBegin.normalize
pandas.tseries.offsets.CustomBusinessMonthBegin.rule_code
CustomBusinessMonthBegin.rule_code
pandas.tseries.offsets.CustomBusinessMonthBegin.n
CustomBusinessMonthBegin.n
pandas.tseries.offsets.CustomBusinessMonthBegin.weekmask
CustomBusinessMonthBegin.weekmask
pandas.tseries.offsets.CustomBusinessMonthBegin.calendar
CustomBusinessMonthBegin.calendar
pandas.tseries.offsets.CustomBusinessMonthBegin.holidays
CustomBusinessMonthBegin.holidays
pandas.tseries.offsets.CustomBusinessMonthBegin.copy
CustomBusinessMonthBegin.copy()
pandas.tseries.offsets.CustomBusinessMonthBegin.is_on_offset
CustomBusinessMonthBegin.is_on_offset(dt)
pandas.tseries.offsets.CustomBusinessMonthBegin.is_month_start
CustomBusinessMonthBegin.is_month_start(ts)
pandas.tseries.offsets.CustomBusinessMonthBegin.is_month_end
CustomBusinessMonthBegin.is_month_end(ts)
pandas.tseries.offsets.CustomBusinessMonthBegin.is_quarter_start
CustomBusinessMonthBegin.is_quarter_start(ts)
pandas.tseries.offsets.CustomBusinessMonthBegin.is_quarter_end
CustomBusinessMonthBegin.is_quarter_end(ts)

pandas.tseries.offsets.CustomBusinessMonthBegin.is_year_start

CustomBusinessMonthBegin.is_year_start(ts)

pandas.tseries.offsets.CustomBusinessMonthBegin.is_year_end

CustomBusinessMonthBegin.is_year_end(ts)

pandas.tseries.offsets.SemiMonthEnd

pd.tseries.offsets.SemiMonthEnd

pandas.tseries.offsets.SemiMonthEnd.freqstr

SemiMonthEnd.freqstr

pandas.tseries.offsets.SemiMonthEnd.kwds

SemiMonthEnd.kwds

pandas.tseries.offsets.SemiMonthEnd.name

SemiMonthEnd.name

pandas.tseries.offsets.SemiMonthEnd.nanos

SemiMonthEnd.nanos

pandas.tseries.offsets.SemiMonthEnd.normalize

SemiMonthEnd.normalize

pandas.tseries.offsets.SemiMonthEnd.rule_code

SemiMonthEnd.rule_code

pandas.tseries.offsets.SemiMonthEnd.n

SemiMonthEnd.n

pandas.tseries.offsets.SemiMonthEnd.day_of_month

SemiMonthEnd.day_of_month

pandas.tseries.offsets.SemiMonthEnd.copy

SemiMonthEnd.copy()

pandas.tseries.offsets.SemiMonthEnd.is_on_offset

SemiMonthEnd.is_on_offset(dt)

pandas.tseries.offsets.SemiMonthEnd.is_month_start

SemiMonthEnd.is_month_start(ts)

pandas.tseries.offsets.SemiMonthEnd.is_month_end

SemiMonthEnd.is_month_end(ts)

pandas.tseries.offsets.SemiMonthEnd.is_quarter_start

SemiMonthEnd.is_quarter_start(ts)

pandas.tseries.offsets.SemiMonthEnd.is_quarter_end

SemiMonthEnd.is_quarter_end(ts)

pandas.tseries.offsets.SemiMonthEnd.is_year_start

SemiMonthEnd.is_year_start(ts)

pandas.tseries.offsets.SemiMonthEnd.is_year_end

SemiMonthEnd.is_year_end(ts)

pandas.tseries.offsets.SemiMonthBegin

pd.tseries.offsets.SemiMonthBegin

pandas.tseries.offsets.SemiMonthBegin.freqstr

SemiMonthBegin.freqstr

pandas.tseries.offsets.SemiMonthBegin.kwds

SemiMonthBegin.kwds

pandas.tseries.offsets.SemiMonthBegin.name

SemiMonthBegin.name

pandas.tseries.offsets.SemiMonthBegin.nanos

SemiMonthBegin.nanos

pandas.tseries.offsets.SemiMonthBegin.normalize

SemiMonthBegin.normalize

pandas.tseries.offsets.SemiMonthBegin.rule_code

SemiMonthBegin.rule_code

pandas.tseries.offsets.SemiMonthBegin.n

SemiMonthBegin.n

pandas.tseries.offsets.SemiMonthBegin.day_of_month

SemiMonthBegin.day_of_month

pandas.tseries.offsets.SemiMonthBegin.copy

SemiMonthBegin.copy()

pandas.tseries.offsets.SemiMonthBegin.is_on_offset

SemiMonthBegin.is_on_offset(dt)

pandas.tseries.offsets.SemiMonthBegin.is_month_start

SemiMonthBegin.is_month_start(ts)

pandas.tseries.offsets.SemiMonthBegin.is_month_end

SemiMonthBegin.is_month_end(ts)

pandas.tseries.offsets.SemiMonthBegin.is_quarter_start

SemiMonthBegin.is_quarter_start(ts)

pandas.tseries.offsets.SemiMonthBegin.is_quarter_end

SemiMonthBegin.is_quarter_end(ts)

pandas.tseries.offsets.SemiMonthBegin.is_year_start

SemiMonthBegin.is_year_start(ts)

pandas.tseries.offsets.SemiMonthBegin.is_year_end

SemiMonthBegin.is_year_end(ts)

pandas.tseries.offsets.Week

pd.tseries.offsets.Week

pandas.tseries.offsets.Week.freqstr

Week.freqstr

pandas.tseries.offsets.Week.kwds

Week.kwds

pandas.tseries.offsets.Week.name

Week.name

pandas.tseries.offsets.Week.nanos

Week.nanos

pandas.tseries.offsets.Week.normalize

Week.normalize

pandas.tseries.offsets.Week.rule_code

Week.rule_code

pandas.tseries.offsets.Week.n

Week.n

pandas.tseries.offsets.Week.weekday

Week.weekday

pandas.tseries.offsets.Week.copy

Week.copy()

pandas.tseries.offsets.Week.is_on_offset

Week.is_on_offset(dt)

pandas.tseries.offsets.Week.is_month_start

Week.is_month_start(ts)

pandas.tseries.offsets.Week.is_month_end

Week.is_month_end(ts)

pandas.tseries.offsets.Week.is_quarter_start

Week.is_quarter_start(ts)

pandas.tseries.offsets.Week.is_quarter_end

Week.is_quarter_end(ts)

pandas.tseries.offsets.Week.is_year_start

Week.is_year_start(ts)

pandas.tseries.offsets.Week.is_year_end

Week.is_year_end(ts)

pandas.tseries.offsets.WeekOfMonth

pd.tseries.offsets.WeekOfMonth

pandas.tseries.offsets.WeekOfMonth.freqstr

WeekOfMonth.freqstr

pandas.tseries.offsets.WeekOfMonth.kwds

WeekOfMonth.kwds

pandas.tseries.offsets.WeekOfMonth.name

WeekOfMonth.name

pandas.tseries.offsets.WeekOfMonth.nanos

WeekOfMonth.nanos

pandas.tseries.offsets.WeekOfMonth.normalize

WeekOfMonth.normalize

pandas.tseries.offsets.WeekOfMonth.rule_code

WeekOfMonth.rule_code

pandas.tseries.offsets.WeekOfMonth.n

WeekOfMonth.n

pandas.tseries.offsets.WeekOfMonth.week

WeekOfMonth.week

pandas.tseries.offsets.WeekOfMonth.copy

WeekOfMonth.copy()

pandas.tseries.offsets.WeekOfMonth.is_on_offset

WeekOfMonth.is_on_offset(dt)

pandas.tseries.offsets.WeekOfMonth.weekday

WeekOfMonth.weekday

pandas.tseries.offsets.WeekOfMonth.is_month_start

WeekOfMonth.is_month_start(ts)

pandas.tseries.offsets.WeekOfMonth.is_month_end

WeekOfMonth.is_month_end(ts)

pandas.tseries.offsets.WeekOfMonth.is_quarter_start

WeekOfMonth.is_quarter_start(ts)

pandas.tseries.offsets.WeekOfMonth.is_quarter_end

WeekOfMonth.is_quarter_end(ts)

pandas.tseries.offsets.WeekOfMonth.is_year_start

WeekOfMonth.is_year_start(ts)

pandas.tseries.offsets.WeekOfMonth.is_year_end

WeekOfMonth.is_year_end(ts)

pandas.tseries.offsets.LastWeekOfMonth

pd.tseries.offsets.LastWeekOfMonth

pandas.tseries.offsets.LastWeekOfMonth.freqstr

LastWeekOfMonth.freqstr

pandas.tseries.offsets.LastWeekOfMonth.kwds

LastWeekOfMonth.kwds

pandas.tseries.offsets.LastWeekOfMonth.name

LastWeekOfMonth.name

pandas.tseries.offsets.LastWeekOfMonth.nanos

LastWeekOfMonth.nanos

pandas.tseries.offsets.LastWeekOfMonth.normalize

LastWeekOfMonth.normalize

pandas.tseries.offsets.LastWeekOfMonth.rule_code

LastWeekOfMonth.rule_code

pandas.tseries.offsets.LastWeekOfMonth.n

LastWeekOfMonth.n

pandas.tseries.offsets.LastWeekOfMonth.weekday

LastWeekOfMonth.weekday

pandas.tseries.offsets.LastWeekOfMonth.week

LastWeekOfMonth.week

pandas.tseries.offsets.LastWeekOfMonth.copy

LastWeekOfMonth.copy()

pandas.tseries.offsets.LastWeekOfMonth.is_on_offset

LastWeekOfMonth.is_on_offset(dt)

pandas.tseries.offsets.LastWeekOfMonth.is_month_start

LastWeekOfMonth.is_month_start(ts)

pandas.tseries.offsets.LastWeekOfMonth.is_month_end

LastWeekOfMonth.is_month_end(ts)

pandas.tseries.offsets.LastWeekOfMonth.is_quarter_start

LastWeekOfMonth.is_quarter_start(ts)

pandas.tseries.offsets.LastWeekOfMonth.is_quarter_end

LastWeekOfMonth.is_quarter_end(ts)

pandas.tseries.offsets.LastWeekOfMonth.is_year_start

LastWeekOfMonth.is_year_start(ts)

pandas.tseries.offsets.LastWeekOfMonth.is_year_end

LastWeekOfMonth.is_year_end(ts)

pandas.tseries.offsets.BQuarterEnd

pd.tseries.offsets.BQuarterEnd

pandas.tseries.offsets.BQuarterEnd.freqstr

BQuarterEnd.freqstr

pandas.tseries.offsets.BQuarterEnd.kwds

BQuarterEnd.kwds

pandas.tseries.offsets.BQuarterEnd.name

BQuarterEnd.name

pandas.tseries.offsets.BQuarterEnd.nanos

BQuarterEnd.nanos

pandas.tseries.offsets.BQuarterEnd.normalize

BQuarterEnd.normalize

pandas.tseries.offsets.BQuarterEnd.rule_code

BQuarterEnd.rule_code

pandas.tseries.offsets.BQuarterEnd.n

BQuarterEnd.n

pandas.tseries.offsets.BQuarterEnd.startingMonth

BQuarterEnd.startingMonth

pandas.tseries.offsets.BQuarterEnd.copy

BQuarterEnd.copy()

pandas.tseries.offsets.BQuarterEnd.is_on_offset

BQuarterEnd.is_on_offset(dt)

pandas.tseries.offsets.BQuarterEnd.is_month_start

BQuarterEnd.is_month_start(ts)

pandas.tseries.offsets.BQuarterEnd.is_month_end

BQuarterEnd.is_month_end(ts)

pandas.tseries.offsets.BQuarterEnd.is_quarter_start

BQuarterEnd.is_quarter_start(ts)

pandas.tseries.offsets.BQuarterEnd.is_quarter_end

BQuarterEnd.is_quarter_end(ts)

pandas.tseries.offsets.BQuarterEnd.is_year_start

BQuarterEnd.is_year_start(ts)

pandas.tseries.offsets.BQuarterEnd.is_year_end

BQuarterEnd.is_year_end(ts)

pandas.tseries.offsets.BQuarterBegin

pd.tseries.offsets.BQuarterBegin

pandas.tseries.offsets.BQuarterBegin.freqstr

BQuarterBegin.freqstr

pandas.tseries.offsets.BQuarterBegin.kwds

BQuarterBegin.kwds

pandas.tseries.offsets.BQuarterBegin.name

BQuarterBegin.name

pandas.tseries.offsets.BQuarterBegin.nanos

BQuarterBegin.nanos

pandas.tseries.offsets.BQuarterBegin.normalize

BQuarterBegin.normalize

pandas.tseries.offsets.BQuarterBegin.rule_code

BQuarterBegin.rule_code

pandas.tseries.offsets.BQuarterBegin.n

BQuarterBegin.n

pandas.tseries.offsets.BQuarterBegin.startingMonth

BQuarterBegin.startingMonth

pandas.tseries.offsets.BQuarterBegin.copy

BQuarterBegin.copy()

pandas.tseries.offsets.BQuarterBegin.is_on_offset

BQuarterBegin.is_on_offset(dt)

pandas.tseries.offsets.BQuarterBegin.is_month_start

BQuarterBegin.is_month_start(ts)

pandas.tseries.offsets.BQuarterBegin.is_month_end

BQuarterBegin.is_month_end(ts)

pandas.tseries.offsets.BQuarterBegin.is_quarter_start

BQuarterBegin.is_quarter_start(ts)

pandas.tseries.offsets.BQuarterBegin.is_quarter_end

BQuarterBegin.is_quarter_end(ts)

pandas.tseries.offsets.BQuarterBegin.is_year_start

BQuarterBegin.is_year_start(ts)

pandas.tseries.offsets.BQuarterBegin.is_year_end

BQuarterBegin.is_year_end(ts)

pandas.tseries.offsets.QuarterEnd

pd.tseries.offsets.QuarterEnd

pandas.tseries.offsets.QuarterEnd.freqstr

QuarterEnd.freqstr

pandas.tseries.offsets.QuarterEnd.kwds

QuarterEnd.kwds

pandas.tseries.offsets.QuarterEnd.name

QuarterEnd.name

pandas.tseries.offsets.QuarterEnd.nanos

QuarterEnd.nanos

pandas.tseries.offsets.QuarterEnd.normalize

QuarterEnd.normalize

pandas.tseries.offsets.QuarterEnd.rule_code

QuarterEnd.rule_code

pandas.tseries.offsets.QuarterEnd.n

QuarterEnd.n

pandas.tseries.offsets.QuarterEnd.startingMonth

QuarterEnd.startingMonth

pandas.tseries.offsets.QuarterEnd.copy

QuarterEnd.copy()

pandas.tseries.offsets.QuarterEnd.is_on_offset

QuarterEnd.is_on_offset(dt)

pandas.tseries.offsets.QuarterEnd.is_month_start

QuarterEnd.is_month_start(ts)

pandas.tseries.offsets.QuarterEnd.is_month_end

QuarterEnd.is_month_end(ts)

pandas.tseries.offsets.QuarterEnd.is_quarter_start

QuarterEnd.is_quarter_start(ts)

pandas.tseries.offsets.QuarterEnd.is_quarter_end

QuarterEnd.is_quarter_end(ts)

pandas.tseries.offsets.QuarterEnd.is_year_start
QuarterEnd.is_year_start(ts)
pandas.tseries.offsets.QuarterEnd.is_year_end
QuarterEnd.is_year_end(ts)
pandas.tseries.offsets.QuarterBegin
pd.tseries.offsets.QuarterBegin
pandas.tseries.offsets.QuarterBegin.freqstr
QuarterBegin.freqstr
pandas.tseries.offsets.QuarterBegin.kwds
QuarterBegin.kwds
pandas.tseries.offsets.QuarterBegin.name
QuarterBegin.name
pandas.tseries.offsets.QuarterBegin.nanos
QuarterBegin.nanos
pandas.tseries.offsets.QuarterBegin.normalize
QuarterBegin.normalize
pandas.tseries.offsets.QuarterBegin.rule_code
QuarterBegin.rule_code
pandas.tseries.offsets.QuarterBegin.n
QuarterBegin.n
pandas.tseries.offsets.QuarterBegin.startingMonth
QuarterBegin.startingMonth
pandas.tseries.offsets.QuarterBegin.copy
QuarterBegin.copy()
pandas.tseries.offsets.QuarterBegin.is_on_offset
QuarterBegin.is_on_offset(dt)
pandas.tseries.offsets.QuarterBegin.is_month_start
QuarterBegin.is_month_start(ts)
pandas.tseries.offsets.QuarterBegin.is_month_end
QuarterBegin.is_month_end(ts)
pandas.tseries.offsets.QuarterBegin.is_quarter_start
QuarterBegin.is_quarter_start(ts)
pandas.tseries.offsets.QuarterBegin.is_quarter_end
QuarterBegin.is_quarter_end(ts)
pandas.tseries.offsets.QuarterBegin.is_year_start
QuarterBegin.is_year_start(ts)
pandas.tseries.offsets.QuarterBegin.is_year_end
QuarterBegin.is_year_end(ts)
pandas.tseries.offsets.BHalfYearEnd
pd.tseries.offsets.BHalfYearEnd
pandas.tseries.offsets.BHalfYearEnd.freqstr
BHalfYearEnd.freqstr
pandas.tseries.offsets.BHalfYearEnd.kwds
BHalfYearEnd.kwds
pandas.tseries.offsets.BHalfYearEnd.name
BHalfYearEnd.name

pandas.tseries.offsets.BHalfYearEnd.nanos

BHalfYearEnd.nanos

pandas.tseries.offsets.BHalfYearEnd.normalize

BHalfYearEnd.normalize

pandas.tseries.offsets.BHalfYearEnd.rule_code

BHalfYearEnd.rule_code

pandas.tseries.offsets.BHalfYearEnd.n

BHalfYearEnd.n

pandas.tseries.offsets.BHalfYearEnd.startingMonth

BHalfYearEnd.startingMonth

pandas.tseries.offsets.BHalfYearEnd.copy

BHalfYearEnd.copy()

pandas.tseries.offsets.BHalfYearEnd.is_on_offset

BHalfYearEnd.is_on_offset(dt)

pandas.tseries.offsets.BHalfYearEnd.is_month_start

BHalfYearEnd.is_month_start(ts)

pandas.tseries.offsets.BHalfYearEnd.is_month_end

BHalfYearEnd.is_month_end(ts)

pandas.tseries.offsets.BHalfYearEnd.is_quarter_start

BHalfYearEnd.is_quarter_start(ts)

pandas.tseries.offsets.BHalfYearEnd.is_quarter_end

BHalfYearEnd.is_quarter_end(ts)

pandas.tseries.offsets.BHalfYearEnd.is_year_start

BHalfYearEnd.is_year_start(ts)

pandas.tseries.offsets.BHalfYearEnd.is_year_end

BHalfYearEnd.is_year_end(ts)

pandas.tseries.offsets.BHalfYearBegin

pd.tseries.offsets.BHalfYearBegin

pandas.tseries.offsets.BHalfYearBegin.freqstr

BHalfYearBegin.freqstr

pandas.tseries.offsets.BHalfYearBegin.kwds

BHalfYearBegin.kwds

pandas.tseries.offsets.BHalfYearBegin.name

BHalfYearBegin.name

pandas.tseries.offsets.BHalfYearBegin.nanos

BHalfYearBegin.nanos

pandas.tseries.offsets.BHalfYearBegin.normalize

BHalfYearBegin.normalize

pandas.tseries.offsets.BHalfYearBegin.rule_code

BHalfYearBegin.rule_code

pandas.tseries.offsets.BHalfYearBegin.n

BHalfYearBegin.n

pandas.tseries.offsets.BHalfYearBegin.startingMonth

BHalfYearBegin.startingMonth

pandas.tseries.offsets.BHalfYearBegin.copy

BHalfYearBegin.copy()

pandas.tseries.offsets.BHalfYearBegin.is_on_offset
BHalfYearBegin.is_on_offset(dt)
pandas.tseries.offsets.BHalfYearBegin.is_month_start
BHalfYearBegin.is_month_start(ts)
pandas.tseries.offsets.BHalfYearBegin.is_month_end
BHalfYearBegin.is_month_end(ts)
pandas.tseries.offsets.BHalfYearBegin.is_quarter_start
BHalfYearBegin.is_quarter_start(ts)
pandas.tseries.offsets.BHalfYearBegin.is_quarter_end
BHalfYearBegin.is_quarter_end(ts)
pandas.tseries.offsets.BHalfYearBegin.is_year_start
BHalfYearBegin.is_year_start(ts)
pandas.tseries.offsets.BHalfYearBegin.is_year_end
BHalfYearBegin.is_year_end(ts)
pandas.tseries.offsets.HalfYearEnd
pd.tseries.offsets.HalfYearEnd
pandas.tseries.offsets.HalfYearEnd.freqstr
HalfYearEnd.freqstr
pandas.tseries.offsets.HalfYearEnd.kwds
HalfYearEnd.kwds
pandas.tseries.offsets.HalfYearEnd.name
HalfYearEnd.name
pandas.tseries.offsets.HalfYearEnd.nanos
HalfYearEnd.nanos
pandas.tseries.offsets.HalfYearEnd.normalize
HalfYearEnd.normalize
pandas.tseries.offsets.HalfYearEnd.rule_code
HalfYearEnd.rule_code
pandas.tseries.offsets.HalfYearEnd.n
HalfYearEnd.n
pandas.tseries.offsets.HalfYearEnd.startingMonth
HalfYearEnd.startingMonth
pandas.tseries.offsets.HalfYearEnd.copy
HalfYearEnd.copy()
pandas.tseries.offsets.HalfYearEnd.is_on_offset
HalfYearEnd.is_on_offset(dt)
pandas.tseries.offsets.HalfYearEnd.is_month_start
HalfYearEnd.is_month_start(ts)
pandas.tseries.offsets.HalfYearEnd.is_month_end
HalfYearEnd.is_month_end(ts)
pandas.tseries.offsets.HalfYearEnd.is_quarter_start
HalfYearEnd.is_quarter_start(ts)
pandas.tseries.offsets.HalfYearEnd.is_quarter_end
HalfYearEnd.is_quarter_end(ts)
pandas.tseries.offsets.HalfYearEnd.is_year_start
HalfYearEnd.is_year_start(ts)

pandas.tseries.offsets.HalfYearEnd.is_year_end

HalfYearEnd.is_year_end(ts)

pandas.tseries.offsets.HalfYearBegin

pd.tseries.offsets.HalfYearBegin

pandas.tseries.offsets.HalfYearBegin.freqstr

HalfYearBegin.freqstr

pandas.tseries.offsets.HalfYearBegin.kwds

HalfYearBegin.kwds

pandas.tseries.offsets.HalfYearBegin.name

HalfYearBegin.name

pandas.tseries.offsets.HalfYearBegin.nanos

HalfYearBegin.nanos

pandas.tseries.offsets.HalfYearBegin.normalize

HalfYearBegin.normalize

pandas.tseries.offsets.HalfYearBegin.rule_code

HalfYearBegin.rule_code

pandas.tseries.offsets.HalfYearBegin.n

HalfYearBegin.n

pandas.tseries.offsets.HalfYearBegin.startingMonth

HalfYearBegin.startingMonth

pandas.tseries.offsets.HalfYearBegin.copy

HalfYearBegin.copy()

pandas.tseries.offsets.HalfYearBegin.is_on_offset

HalfYearBegin.is_on_offset(dt)

pandas.tseries.offsets.HalfYearBegin.is_month_start

HalfYearBegin.is_month_start(ts)

pandas.tseries.offsets.HalfYearBegin.is_month_end

HalfYearBegin.is_month_end(ts)

pandas.tseries.offsets.HalfYearBegin.is_quarter_start

HalfYearBegin.is_quarter_start(ts)

pandas.tseries.offsets.HalfYearBegin.is_quarter_end

HalfYearBegin.is_quarter_end(ts)

pandas.tseries.offsets.HalfYearBegin.is_year_start

HalfYearBegin.is_year_start(ts)

pandas.tseries.offsets.HalfYearBegin.is_year_end

HalfYearBegin.is_year_end(ts)

pandas.tseries.offsets.BYearEnd

pd.tseries.offsets.BYearEnd

pandas.tseries.offsets.BYearEnd.freqstr

BYearEnd.freqstr

pandas.tseries.offsets.BYearEnd.kwds

BYearEnd.kwds

pandas.tseries.offsets.BYearEnd.name

BYearEnd.name

pandas.tseries.offsets.BYearEnd.nanos

BYearEnd.nanos

pandas.tseries.offsets.BYearEnd.normalize

BYearEnd.normalize

pandas.tseries.offsets.BYearEnd.rule_code

BYearEnd.rule_code

pandas.tseries.offsets.BYearEnd.n

BYearEnd.n

pandas.tseries.offsets.BYearEnd.month

BYearEnd.month

pandas.tseries.offsets.BYearEnd.copy

BYearEnd.copy()

pandas.tseries.offsets.BYearEnd.is_on_offset

BYearEnd.is_on_offset(dt)

pandas.tseries.offsets.BYearEnd.is_month_start

BYearEnd.is_month_start(ts)

pandas.tseries.offsets.BYearEnd.is_month_end

BYearEnd.is_month_end(ts)

pandas.tseries.offsets.BYearEnd.is_quarter_start

BYearEnd.is_quarter_start(ts)

pandas.tseries.offsets.BYearEnd.is_quarter_end

BYearEnd.is_quarter_end(ts)

pandas.tseries.offsets.BYearEnd.is_year_start

BYearEnd.is_year_start(ts)

pandas.tseries.offsets.BYearEnd.is_year_end

BYearEnd.is_year_end(ts)

pandas.tseries.offsets.BYearBegin

pd.tseries.offsets.BYearBegin

pandas.tseries.offsets.BYearBegin.freqstr

BYearBegin.freqstr

pandas.tseries.offsets.BYearBegin.kwds

BYearBegin.kwds

pandas.tseries.offsets.BYearBegin.name

BYearBegin.name

pandas.tseries.offsets.BYearBegin.nanos

BYearBegin.nanos

pandas.tseries.offsets.BYearBegin.normalize

BYearBegin.normalize

pandas.tseries.offsets.BYearBegin.rule_code

BYearBegin.rule_code

pandas.tseries.offsets.BYearBegin.n

BYearBegin.n

pandas.tseries.offsets.BYearBegin.month

BYearBegin.month

pandas.tseries.offsets.BYearBegin.copy

BYearBegin.copy()

pandas.tseries.offsets.BYearBegin.is_on_offset

BYearBegin.is_on_offset(dt)

pandas.tseries.offsets.BYearBegin.is_month_start

BYearBegin.is_month_start(ts)

pandas.tseries.offsets.BYearBegin.is_month_end

BYearBegin.is_month_end(ts)

pandas.tseries.offsets.BYearBegin.is_quarter_start

BYearBegin.is_quarter_start(ts)

pandas.tseries.offsets.BYearBegin.is_quarter_end

BYearBegin.is_quarter_end(ts)

pandas.tseries.offsets.BYearBegin.is_year_start

BYearBegin.is_year_start(ts)

pandas.tseries.offsets.BYearBegin.is_year_end

BYearBegin.is_year_end(ts)

pandas.tseries.offsets.YearEnd

pd.tseries.offsets.YearEnd(n=1,normalize=False,month=None)

pandas.tseries.offsets.YearEnd.freqstr

YearEnd.freqstr

pandas.tseries.offsets.YearEnd.kwds

YearEnd.kwds

pandas.tseries.offsets.YearEnd.name

YearEnd.name

pandas.tseries.offsets.YearEnd.nanos

YearEnd.nanos

pandas.tseries.offsets.YearEnd.normalize

YearEnd.normalize

pandas.tseries.offsets.YearEnd.rule_code

YearEnd.rule_code

pandas.tseries.offsets.YearEnd.n

YearEnd.n

pandas.tseries.offsets.YearEnd.month

YearEnd.month

pandas.tseries.offsets.YearEnd.copy

YearEnd.copy()

pandas.tseries.offsets.YearEnd.is_on_offset

YearEnd.is_on_offset(dt)

pandas.tseries.offsets.YearEnd.is_month_start

YearEnd.is_month_start(ts)

pandas.tseries.offsets.YearEnd.is_month_end

YearEnd.is_month_end(ts)

pandas.tseries.offsets.YearEnd.is_quarter_start

YearEnd.is_quarter_start(ts)

pandas.tseries.offsets.YearEnd.is_quarter_end

YearEnd.is_quarter_end(ts)

pandas.tseries.offsets.YearEnd.is_year_start

YearEnd.is_year_start(ts)

pandas.tseries.offsets.YearEnd.is_year_end

YearEnd.is_year_end(ts)

pandas.tseries.offsets.YearBegin

pd.tseries.offsets.YearBegin

pandas.tseries.offsets.YearBegin.freqstr

YearBegin.freqstr

pandas.tseries.offsets.YearBegin.kwds

YearBegin.kwds

pandas.tseries.offsets.YearBegin.name

YearBegin.name

pandas.tseries.offsets.YearBegin.nanos

YearBegin.nanos

pandas.tseries.offsets.YearBegin.normalize

YearBegin.normalize

pandas.tseries.offsets.YearBegin.rule_code

YearBegin.rule_code

pandas.tseries.offsets.YearBegin.n

YearBegin.n

pandas.tseries.offsets.YearBegin.month

YearBegin.month

pandas.tseries.offsets.YearBegin.copy

YearBegin.copy()

pandas.tseries.offsets.YearBegin.is_on_offset

YearBegin.is_on_offset(dt)

pandas.tseries.offsets.YearBegin.is_month_start

YearBegin.is_month_start(ts)

pandas.tseries.offsets.YearBegin.is_month_end

YearBegin.is_month_end(ts)

pandas.tseries.offsets.YearBegin.is_quarter_start

YearBegin.is_quarter_start(ts)

pandas.tseries.offsets.YearBegin.is_quarter_end

YearBegin.is_quarter_end(ts)

pandas.tseries.offsets.YearBegin.is_year_start

YearBegin.is_year_start(ts)

pandas.tseries.offsets.YearBegin.is_year_end

YearBegin.is_year_end(ts)

pandas.tseries.offsets.FY5253

pd.tseries.offsets.FY5253

pandas.tseries.offsets.FY5253.freqstr

FY5253.freqstr

pandas.tseries.offsets.FY5253.kwds

FY5253.kwds

pandas.tseries.offsets.FY5253.name

FY5253.name

pandas.tseries.offsets.FY5253.nanos

FY5253.nanos

pandas.tseries.offsets.FY5253.normalize

FY5253.normalize

pandas.tseries.offsets.FY5253.rule_code

FY5253.rule_code

pandas.tseries.offsets.FY5253.n

FY5253.n

pandas.tseries.offsets.FY5253.startingMonth

FY5253.startingMonth

pandas.tseries.offsets.FY5253.variation

FY5253.variation

pandas.tseries.offsets.FY5253.weekday

FY5253.weekday

pandas.tseries.offsets.FY5253.copy

FY5253.copy()

pandas.tseries.offsets.FY5253.get_rule_code_suffix

FY5253.get_rule_code_suffix()

pandas.tseries.offsets.FY5253.get_year_end

FY5253.get_year_end(dt)

pandas.tseries.offsets.FY5253.is_on_offset

FY5253.is_on_offset(dt)

pandas.tseries.offsets.FY5253.is_month_start

FY5253.is_month_start(ts)

pandas.tseries.offsets.FY5253.is_month_end

FY5253.is_month_end(ts)

pandas.tseries.offsets.FY5253.is_quarter_start

FY5253.is_quarter_start(ts)

pandas.tseries.offsets.FY5253.is_quarter_end

FY5253.is_quarter_end(ts)

pandas.tseries.offsets.FY5253.is_year_start

FY5253.is_year_start(ts)

pandas.tseries.offsets.FY5253.is_year_end

FY5253.is_year_end(ts)

pandas.tseries.offsets.FY5253Quarter

pd.tseries.offsets.FY5253Quarter

pandas.tseries.offsets.FY5253Quarter.freqstr

FY5253Quarter.freqstr

pandas.tseries.offsets.FY5253Quarter.kwds

FY5253Quarter.kwds

pandas.tseries.offsets.FY5253Quarter.name

FY5253Quarter.name

pandas.tseries.offsets.FY5253Quarter.nanos

FY5253Quarter.nanos

pandas.tseries.offsets.FY5253Quarter.normalize

FY5253Quarter.normalize

pandas.tseries.offsets.FY5253Quarter.rule_code

FY5253Quarter.rule_code

pandas.tseries.offsets.FY5253Quarter.n

FY5253Quarter.n

pandas.tseries.offsets.FY5253Quarter.qtr_with_extra_week

FY5253Quarter.qtr_with_extra_week

pandas.tseries.offsets.FY5253Quarter.startingMonth

FY5253Quarter.startingMonth

pandas.tseries.offsets.FY5253Quarter.variation

FY5253Quarter.variation

pandas.tseries.offsets.FY5253Quarter.weekday

FY5253Quarter.weekday

pandas.tseries.offsets.FY5253Quarter.copy

FY5253Quarter.copy()

pandas.tseries.offsets.FY5253Quarter.get_rule_code_suffix

FY5253Quarter.get_rule_code_suffix()

pandas.tseries.offsets.FY5253Quarter.get_weeks

FY5253Quarter.get_weeks(dt)

pandas.tseries.offsets.FY5253Quarter.is_on_offset

FY5253Quarter.is_on_offset(dt)

pandas.tseries.offsets.FY5253Quarter.year_has_extra_week

FY5253Quarter.year_has_extra_week(dt)

pandas.tseries.offsets.FY5253Quarter.is_month_start

FY5253Quarter.is_month_start(ts)

pandas.tseries.offsets.FY5253Quarter.is_month_end

FY5253Quarter.is_month_end(ts)

pandas.tseries.offsets.FY5253Quarter.is_quarter_start

FY5253Quarter.is_quarter_start(ts)

pandas.tseries.offsets.FY5253Quarter.is_quarter_end

FY5253Quarter.is_quarter_end(ts)

pandas.tseries.offsets.FY5253Quarter.is_year_start

FY5253Quarter.is_year_start(ts)

pandas.tseries.offsets.FY5253Quarter.is_year_end

FY5253Quarter.is_year_end(ts)

pandas.tseries.offsets.Easter

pd.tseries.offsets.Easter

pandas.tseries.offsets.Easter.freqstr

Easter.freqstr

pandas.tseries.offsets.Easter.kwds

Easter.kwds

pandas.tseries.offsets.Easter.name

Easter.name

pandas.tseries.offsets.Easter.nanos

Easter.nanos

pandas.tseries.offsets.Easter.normalize

Easter.normalize

pandas.tseries.offsets.Easter.rule_code

Easter.rule_code

pandas.tseries.offsets.Easter.n

Easter.n

pandas.tseries.offsets.Easter.copy

Easter.copy()

pandas.tseries.offsets.Easter.is_on_offset

Easter.is_on_offset(dt)

pandas.tseries.offsets.Easter.is_month_start

Easter.is_month_start(ts)

pandas.tseries.offsets.Easter.is_month_end

Easter.is_month_end(ts)

pandas.tseries.offsets.Easter.is_quarter_start

Easter.is_quarter_start(ts)

pandas.tseries.offsets.Easter.is_quarter_end

Easter.is_quarter_end(ts)

pandas.tseries.offsets.Easter.is_year_start

Easter.is_year_start(ts)

pandas.tseries.offsets.Easter.is_year_end

Easter.is_year_end(ts)

pandas.tseries.offsets.Tick

pd.tseries.offsets.Tick

pandas.tseries.offsets.Tick.freqstr

Tick.freqstr

pandas.tseries.offsets.Tick.kwds

Tick.kwds

pandas.tseries.offsets.Tick.name

Tick.name

pandas.tseries.offsets.Tick.nanos

Tick.nanos

pandas.tseries.offsets.Tick.normalize

Tick.normalize

pandas.tseries.offsets.Tick.rule_code

Tick.rule_code

pandas.tseries.offsets.Tick.n

Tick.n

pandas.tseries.offsets.Tick.copy

Tick.copy()

pandas.tseries.offsets.Tick.is_on_offset

Tick.is_on_offset(dt)

pandas.tseries.offsets.Tick.is_month_start

Tick.is_month_start(ts)

pandas.tseries.offsets.Tick.is_month_end

Tick.is_month_end(ts)

pandas.tseries.offsets.Tick.is_quarter_start

Tick.is_quarter_start(ts)

pandas.tseries.offsets.Tick.is_quarter_end

Tick.is_quarter_end(ts)

pandas.tseries.offsets.Tick.is_year_start

Tick.is_year_start(ts)

pandas.tseries.offsets.Tick.is_year_end

Tick.is_year_end(ts)

pandas.tseries.offsets.Day

pd.tseries.offsets.Day

pandas.tseries.offsets.Day.freqstr

Day.freqstr

pandas.tseries.offsets.Day.kwds

Day.kwds

pandas.tseries.offsets.Day.name

Day.name

pandas.tseries.offsets.Day.nanos

Day.nanos

pandas.tseries.offsets.Day.normalize

Day.normalize

pandas.tseries.offsets.Day.rule_code

Day.rule_code

pandas.tseries.offsets.Day.n

Day.n

pandas.tseries.offsets.Day.copy

Day.copy()

pandas.tseries.offsets.Day.is_on_offset

Day.is_on_offset(dt)

pandas.tseries.offsets.Day.is_month_start

Day.is_month_start(ts)

pandas.tseries.offsets.Day.is_month_end

Day.is_month_end(ts)

pandas.tseries.offsets.Day.is_quarter_start

Day.is_quarter_start(ts)

pandas.tseries.offsets.Day.is_quarter_end

Day.is_quarter_end(ts)

pandas.tseries.offsets.Day.is_year_start

Day.is_year_start(ts)

pandas.tseries.offsets.Day.is_year_end

Day.is_year_end(ts)

pandas.tseries.offsets.Hour

pd.tseries.offsets.Hour

pandas.tseries.offsets.Hour.freqstr

Hour.freqstr

pandas.tseries.offsets.Hour.kwds

Hour.kwds

pandas.tseries.offsets.Hour.name

Hour.name

pandas.tseries.offsets.Hour.nanos

Hour.nanos

pandas.tseries.offsets.Hour.normalize

Hour.normalize

pandas.tseries.offsets.Hour.rule_code

Hour.rule_code

pandas.tseries.offsets.Hour.n

Hour.n

pandas.tseries.offsets.Hour.copy

Hour.copy()

pandas.tseries.offsets.Hour.is_on_offset

Hour.is_on_offset(dt)

pandas.tseries.offsets.Hour.is_month_start

Hour.is_month_start(ts)

pandas.tseries.offsets.Hour.is_month_end

Hour.is_month_end(ts)

pandas.tseries.offsets.Hour.is_quarter_start

Hour.is_quarter_start(ts)

pandas.tseries.offsets.Hour.is_quarter_end

Hour.is_quarter_end(ts)

pandas.tseries.offsets.Hour.is_year_start

Hour.is_year_start(ts)

pandas.tseries.offsets.Hour.is_year_end

Hour.is_year_end(ts)

pandas.tseries.offsets.Minute

pd.tseries.offsets.Minute

pandas.tseries.offsets.Minute.freqstr

Minute.freqstr

pandas.tseries.offsets.Minute.kwds

Minute.kwds

pandas.tseries.offsets.Minute.name

Minute.name

pandas.tseries.offsets.Minute.nanos

Minute.nanos

pandas.tseries.offsets.Minute.normalize

Minute.normalize

pandas.tseries.offsets.Minute.rule_code

Minute.rule_code

pandas.tseries.offsets.Minute.n

Minute.n

pandas.tseries.offsets.Minute.copy

Minute.copy()

pandas.tseries.offsets.Minute.is_on_offset

Minute.is_on_offset(dt)

pandas.tseries.offsets.Minute.is_month_start

Minute.is_month_start(ts)

pandas.tseries.offsets.Minute.is_month_end

Minute.is_month_end(ts)

pandas.tseries.offsets.Minute.is_quarter_start

Minute.is_quarter_start(ts)

pandas.tseries.offsets.Minute.is_quarter_end

Minute.is_quarter_end(ts)

pandas.tseries.offsets.Minute.is_year_start

Minute.is_year_start(ts)

pandas.tseries.offsets.Minute.is_year_end

Minute.is_year_end(ts)

pandas.tseries.offsets.Second

pd.tseries.offsets.Second

pandas.tseries.offsets.Second.freqstr

Second.freqstr

pandas.tseries.offsets.Second.kwds

Second.kwds

pandas.tseries.offsets.Second.name

Second.name

pandas.tseries.offsets.Second.nanos

Second.nanos

pandas.tseries.offsets.Second.normalize

Second.normalize

pandas.tseries.offsets.Second.rule_code

Second.rule_code

pandas.tseries.offsets.Second.n

Second.n

pandas.tseries.offsets.Second.copy

Second.copy()

pandas.tseries.offsets.Second.is_on_offset

Second.is_on_offset(dt)

pandas.tseries.offsets.Second.is_month_start

Second.is_month_start(ts)

pandas.tseries.offsets.Second.is_month_end

Second.is_month_end(ts)

pandas.tseries.offsets.Second.is_quarter_start

Second.is_quarter_start(ts)

pandas.tseries.offsets.Second.is_quarter_end

Second.is_quarter_end(ts)

pandas.tseries.offsets.Second.is_year_start

Second.is_year_start(ts)

pandas.tseries.offsets.Second.is_year_end

Second.is_year_end(ts)

pandas.tseries.offsets.Milli

pd.tseries.offsets.Milli

pandas.tseries.offsets.Milli.freqstr

Milli.freqstr

pandas.tseries.offsets.Milli.kwds

Milli.kwds

pandas.tseries.offsets.Milli.name

Milli.name

pandas.tseries.offsets.Milli.nanos

Milli.nanos

pandas.tseries.offsets.Milli.normalize

Milli.normalize

pandas.tseries.offsets.Milli.rule_code

Milli.rule_code

pandas.tseries.offsets.Milli.n

Milli.n

pandas.tseries.offsets.Milli.copy

Milli.copy()

pandas.tseries.offsets.Milli.is_on_offset

Milli.is_on_offset(dt)

pandas.tseries.offsets.Milli.is_month_start

Milli.is_month_start(ts)

pandas.tseries.offsets.Milli.is_month_end

Milli.is_month_end(ts)

pandas.tseries.offsets.Milli.is_quarter_start

Milli.is_quarter_start(ts)

pandas.tseries.offsets.Milli.is_quarter_end

Milli.is_quarter_end(ts)

pandas.tseries.offsets.Milli.is_year_start

Milli.is_year_start(ts)

pandas.tseries.offsets.Milli.is_year_end

Milli.is_year_end(ts)

pandas.tseries.offsets.Micro

pd.tseries.offsets.Micro

pandas.tseries.offsets.Micro.freqstr

Micro.freqstr

pandas.tseries.offsets.Micro.kwds

Micro.kwds

pandas.tseries.offsets.Micro.name

Micro.name

pandas.tseries.offsets.Micro.nanos

Micro.nanos

pandas.tseries.offsets.Micro.normalize

Micro.normalize

pandas.tseries.offsets.Micro.rule_code

Micro.rule_code

pandas.tseries.offsets.Micro.n

Micro.n

pandas.tseries.offsets.Micro.copy

Micro.copy()

pandas.tseries.offsets.Micro.is_on_offset

Micro.is_on_offset(dt)

pandas.tseries.offsets.Micro.is_month_start

Micro.is_month_start(ts)

pandas.tseries.offsets.Micro.is_month_end

Micro.is_month_end(ts)

pandas.tseries.offsets.Micro.is_quarter_start

Micro.is_quarter_start(ts)

pandas.tseries.offsets.Micro.is_quarter_end

Micro.is_quarter_end(ts)

pandas.tseries.offsets.Micro.is_year_start

Micro.is_year_start(ts)

pandas.tseries.offsets.Micro.is_year_end

Micro.is_year_end(ts)

pandas.tseries.offsets.Nano

pd.tseries.offsets.Nano

pandas.tseries.offsets.Nano.freqstr

Nano.freqstr

pandas.tseries.offsets.Nano.kwds

Nano.kwds

pandas.tseries.offsets.Nano.name

Nano.name

pandas.tseries.offsets.Nano.nanos

Nano.nanos

pandas.tseries.offsets.Nano.normalize

Nano.normalize

pandas.tseries.offsets.Nano.rule_code

Nano.rule_code

pandas.tseries.offsets.Nano.n

Nano.n

pandas.tseries.offsets.Nano.copy

Nano.copy()

pandas.tseries.offsets.Nano.is_on_offset

Nano.is_on_offset(dt)

pandas.tseries.offsets.Nano.is_month_start

Nano.is_month_start(ts)

pandas.tseries.offsets.Nano.is_month_end

Nano.is_month_end(ts)

pandas.tseries.offsets.Nano.is_quarter_start

Nano.is_quarter_start(ts)

pandas.tseries.offsets.Nano.is_quarter_end

Nano.is_quarter_end(ts)

pandas.tseries.offsets.Nano.is_year_start

Nano.is_year_start(ts)

pandas.tseries.offsets.Nano.is_year_end

Nano.is_year_end(ts)

pandas.tseries.frequencies.to_offset

pd.tseries.frequencies.to_offset(freq,is_period=False)

Estilos de presentación

42 entradas documentadas.

pandas.io.formats.style.Styler

pd.io.formats.style.Styler(data,precision=None,table_styles=None,uuid=None,caption=None,table_attributes=None,cell_ids=True,na_rep=None,uuid_len=5,decimal=None,thousands=None,escape=None,formatter=None)

pandas.io.formats.style.Styler.from_custom_template

methodStyler.from_custom_template(searchpath,html_table=None,html_style=None)

pandas.io.formats.style.Styler.env

Styler.env=<jinja2.environment.Environmentobject>

pandas.io.formats.style.Styler.template_html

Styler.template_html=<Template'html.tpl'>

pandas.io.formats.style.Styler.template_html_style

Styler.template_html_style=<Template'html_style.tpl'>

pandas.io.formats.style.Styler.template_html_table

Styler.template_html_table=<Template'html_table.tpl'>

pandas.io.formats.style.Styler.template_latex

Styler.template_latex=<Template'latex.tpl'>

pandas.io.formats.style.Styler.template_typst

Styler.template_typst=<Template'typst.tpl'>

pandas.io.formats.style.Styler.template_string

Styler.template_string=<Template'string.tpl'>

pandas.io.formats.style.Styler.loader

Styler.loader=<jinja2.loaders.FileSystemLoaderobject>

pandas.io.formats.style.Styler.apply

Styler.apply(func,axis=0,subset=None,**kwargs)

pandas.io.formats.style.Styler.map

Styler.map(func,subset=None,**kwargs)

pandas.io.formats.style.Styler.apply_index

Styler.apply_index(func,axis=0,level=None,**kwargs)

pandas.io.formats.style.Styler.map_index

Styler.map_index(func,axis=0,level=None,**kwargs)

pandas.io.formats.style.Styler.format

Styler.format(formatter=None,subset=None,na_rep=None,precision=None,decimal='.',thousands=None,escape=None,hyperlinks=None)

pandas.io.formats.style.Styler.format_index

Styler.format_index(formatter=None,axis=0,level=None,na_rep=None,precision=None,decimal='.',thousands=None,escape=None,hyperlinks=None)

pandas.io.formats.style.Styler.format_index_names

Styler.format_index_names(formatter=None,axis=0,level=None,na_rep=None,precision=None,decimal='.',thousands=None,escape=None,hyperlinks=None)

pandas.io.formats.style.Styler.relabel_index

Styler.relabel_index(labels,axis=0,level=None)

pandas.io.formats.style.Styler.hide

Styler.hide(subset=None,axis=0,level=None,names=False)

pandas.io.formats.style.Styler.concat

Styler.concat(other)

pandas.io.formats.style.Styler.set_td_classes

Styler.set_td_classes(classes)

pandas.io.formats.style.Styler.set_table_styles

Styler.set_table_styles(table_styles=None,axis=0,overwrite=True,css_class_names=None)

pandas.io.formats.style.Styler.set_table_attributes

Styler.set_table_attributes(attributes)

pandas.io.formats.style.Styler.set_tooltips

Styler.set_tooltips(ttips,props=None,css_class=None,as_title_attribute=False)

pandas.io.formats.style.Styler.set_caption

Styler.set_caption(caption)

pandas.io.formats.style.Styler.set_sticky

Styler.set_sticky(axis=0,pixel_size=None,levels=None)

pandas.io.formats.style.Styler.set_properties

Styler.set_properties(subset=None,**kwargs)

pandas.io.formats.style.Styler.set_uuid

Styler.set_uuid(uuid)

pandas.io.formats.style.Styler.clear

Styler.clear()

pandas.io.formats.style.Styler.pipe

Styler.pipe(func,*args,**kwargs)

pandas.io.formats.style.Styler.highlight_null

Styler.highlight_null(color='red',subset=None,props=None)

pandas.io.formats.style.Styler.highlight_max

Styler.highlight_max(subset=None,color='yellow',axis=0,props=None)

pandas.io.formats.style.Styler.highlight_min

Styler.highlight_min(subset=None,color='yellow',axis=0,props=None)

pandas.io.formats.style.Styler.highlight_between

Styler.highlight_between(subset=None,color='yellow',axis=0,left=None,right=None,inclusive='both',props=None)

pandas.io.formats.style.Styler.highlight_quantile

Styler.highlight_quantile(subset=None,color='yellow',axis=0,q_left=0.0,q_right=1.0,interpolation='linear',inclusive='both',props=None)

pandas.io.formats.style.Styler.background_gradient

Styler.background_gradient(cmap='PuBu',low=0,high=0,axis=0,subset=None,text_color_threshold=0.408,vmin=None,vmax=None,gmap=None)

pandas.io.formats.style.Styler.text_gradient

Styler.text_gradient(cmap='PuBu',low=0,high=0,axis=0,subset=None,vmin=None,vmax=None,gmap=None)

pandas.io.formats.style.Styler.bar

Styler.bar(subset=None,axis=0,*,color=None,cmap=None,width=100,height=100,align='mid',vmin=None,vmax=None,props='width:10em;')

pandas.io.formats.style.Styler.to_typst

Styler.to_typst(buf=None,*,encoding=None,sparse_index=None,sparse_columns=None,max_rows=None,max_columns=None)

pandas.io.formats.style.Styler.to_string

Styler.to_string(buf=None,*,encoding=None,sparse_index=None,sparse_columns=None,max_rows=None,max_columns=None,delimiter=')

pandas.io.formats.style.Styler.export

Styler.export()

pandas.io.formats.style.Styler.use

Styler.use(styles)

Visualización

12 entradas documentadas.

pandas.plotting.andrews_curves

pd.plotting.andrews_curves(frame,class_column,ax=None,samples=200,color=None,colormap=None,**kwargs)

pandas.plotting.autocorrelation_plot

pd.plotting.autocorrelation_plot(series,ax=None,**kwargs)

pandas.plotting.bootstrap_plot

pd.plotting.bootstrap_plot(series,fig=None,size=50,samples=500,**kwds)

pandas.plotting.boxplot

pd.plotting.boxplot(data,column=None,by=None,ax=None,fontsize=None,rot=0,grid=True,figsize=None,layout=None,return_type=None,**kwargs)

pandas.plotting.deregister_matplotlib_converters

pd.plotting.deregister_matplotlib_converters()

pandas.plotting.lag_plot

pd.plotting.lag_plot(series,lag=1,ax=None,**kwds)

pandas.plotting.parallel_coordinates

pd.plotting.parallel_coordinates(frame,class_column,cols=None,ax=None,color=None,use_columns=False,xticks=None,colormap=None,axvlines=True,axvlines_kwds=None,sort_labels=False,**kwargs)

pandas.plotting.plot_params

pd.plotting.plot_params={'xaxis.compat':False}

pandas.plotting.radviz

pd.plotting.radviz(frame,class_column,ax=None,color=None,colormap=None,**kwds)

pandas.plotting.register_matplotlib_converters

pd.plotting.register_matplotlib_converters()

pandas.plotting.scatter_matrix · ficha p. 92

pd.plotting.scatter_matrix(frame,alpha=0.5,figsize=None,ax=None,grid=False,diagonal='hist',marker='.',density_kwds=None,hist_kwds=None,range_padding=0.05,**kwargs)

pandas.plotting.table

pd.plotting.table(ax,data,**kwargs)

Opciones

6 entradas documentadas.

pandas.describe_option

pd.describe_option(pat='',_print_desc=True)

pandas.reset_option · ficha p. 93

pd.reset_option(pat)

pandas.get_option · ficha p. 93

pd.get_option(pat)

pandas.set_option · ficha p. 93

pd.set_option(*args)

pandas.option_context · ficha p. 93

pd.option_context(*args)

pandas.set_eng_float_format

pd.set_eng_float_format(accuracy=3,use_eng_prefix=False)

Extensiones públicas

8 entradas documentadas.

pandas.api.extensions.register_extension_dtype

pd.api.extensions.register_extension_dtype(cls)

pandas.api.extensions.register_dataframe_accessor

pd.api.extensions.register_dataframe_accessor(name)

pandas.api.extensions.register_series_accessor

pd.api.extensions.register_series_accessor(name)

pandas.api.extensions.register_index_accessor

pd.api.extensions.register_index_accessor(name)

pandas.api.extensions.ExtensionDtype

pd.api.extensions.ExtensionDtype

pandas.api.extensions.ExtensionArray

pd.api.extensions.ExtensionArray

pandas.arrays.NumpyExtensionArray

pd.arrays.NumpyExtensionArray(values,copy=False)

pandas.api.indexers.check_array_indexer

pd.api.indexers.check_array_indexer(array,indexer)

Pruebas, errores y avisos

52 entradas documentadas.

pandas.testing.assert_frame_equal · ficha p. 94

pd.testing.assert_frame_equal(left,right,check_dtype=True,check_index_type='equiv',check_column_type='equiv',check_frame_type=True,check_names=True,by_blocks=False,check_exact=<no_default>,check_datetimelike_compat=False,check_categorical=True,check_like=False,check_freq=True,check_flags=True,rtol=<no_default>,atol=<no_default>,obj='DataFrame')

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro check_datetimelike_compat está obsoleto. Convierte explícitamente ambos objetos al mismo tipo temporal y compáralos con check_dtype; la función sigue vigente.

pandas.testing.assert_series_equal

pd.testing.assert_series_equal(left,right,check_dtype=True,check_index_type='equiv',check_series_type=True,check_names=True,check_exact=<no_default>,check_datetimelike_compat=False,check_categorical=True,check_category_order=True,check_freq=True,check_flags=True,rtol=<no_default>,atol=<no_default>,obj='Series',*,check_index=True,check_like=False)

AVISO DE OBSOLESCENCIA (API o parámetro). El parámetro check_datetimelike_compat está obsoleto. Convierte explícitamente ambos objetos al mismo tipo temporal y compáralos con check_dtype; la función sigue vigente.

pandas.testing.assert_index_equal

pd.testing.assert_index_equal(left,right,exact='equiv',check_names=True,check_exact=True,check_categorical=True,check_order=True,rtol=1e-05,atol=1e-08,obj=None)

pandas.testing.assert_extension_array_equal

pd.testing.assert_extension_array_equal(left,right,check_dtype=True,index_values=None,check_exact=<no_default>,rtol=<no_default>,atol=<no_default>,obj='ExtensionArray')

pandas.errors.AbstractMethodError

exceptionpd.errors.AbstractMethodError(class_instance,methodtype='method')

pandas.errors.AttributeConflictWarning

exceptionpd.errors.AttributeConflictWarning

pandas.errors.CategoricalConversionWarning

exceptionpd.errors.CategoricalConversionWarning

pandas.errors.ChainedAssignmentError

exceptionpd.errors.ChainedAssignmentError

pandas.errors.ClosedFileError

exceptionpd.errors.ClosedFileError

pandas.errors.CSSWarning

exceptionpd.errors.CSSWarning

pandas.errors.DatabaseError

exceptionpd.errors.DatabaseError

pandas.errors.DataError

exceptionpd.errors.DataError

pandas.errors.DtypeWarning

exceptionpd.errors.DtypeWarning

pandas.errors.DuplicateLabelError

exceptionpd.errors.DuplicateLabelError

pandas.errors.EmptyDataError

exceptionpd.errors.EmptyDataError

pandas.errors.IncompatibilityWarning

exceptionpd.errors.IncompatibilityWarning

pandas.errors.IncompatibleFrequency

exceptionpd.errors.IncompatibleFrequency

pandas.errors.IndexingError

exceptionpd.errors.IndexingError

pandas.errors.InvalidColumnName

exceptionpd.errors.InvalidColumnName

pandas.errors.InvalidComparison

exceptionpd.errors.InvalidComparison

pandas.errors.InvalidIndexError

exceptionpd.errors.InvalidIndexError

pandas.errors.InvalidVersion

exceptionpd.errors.InvalidVersion

pandas.errors.IntCastingNaNError

exceptionpd.errors.IntCastingNaNError

pandas.errors.LossySetitemError

exceptionpd.errors.LossySetitemError

pandas.errors.MergeError

exceptionpd.errors.MergeError

pandas.errors.NoBufferPresent

exceptionpd.errors.NoBufferPresent

pandas.errors.NullFrequencyError

exceptionpd.errors.NullFrequencyError

pandas.errors.NumbaUtilError

exceptionpd.errors.NumbaUtilError

pandas.errors.NumExprClobberingError

exceptionpd.errors.NumExprClobberingError

pandas.errors.OptionError

exceptionpd.errors.OptionError

pandas.errors.OutOfBoundsDatetime

exceptionpd.errors.OutOfBoundsDatetime

pandas.errors.OutOfBoundsTimedelta

exceptionpd.errors.OutOfBoundsTimedelta

pandas.errors.PandasChangeWarning

exceptionpd.errors.PandasChangeWarning

pandas.errors.Pandas4Warning

exceptionpd.errors.Pandas4Warning

pandas.errors.Pandas5Warning

exceptionpd.errors.Pandas5Warning

pandas.errors.PandasPendingDeprecationWarning

exceptionpd.errors.PandasPendingDeprecationWarning

pandas.errors.PandasDeprecationWarning

exceptionpd.errors.PandasDeprecationWarning

pandas.errors.PandasFutureWarning

exceptionpd.errors.PandasFutureWarning

pandas.errors.ParserError

exceptionpd.errors.ParserError

pandas.errors.ParserWarning

exceptionpd.errors.ParserWarning

pandas.errors.PerformanceWarning

exceptionpd.errors.PerformanceWarning

pandas.errors.PossibleDataLossError

exceptionpd.errors.PossibleDataLossError

pandas.errors.PossiblePrecisionLoss

exceptionpd.errors.PossiblePrecisionLoss

pandas.errors.PyperclipException

exceptionpd.errors.PyperclipException

pandas.errors.PyperclipWindowsException

exceptionpd.errors.PyperclipWindowsException(message)

pandas.errors.SpecificationError

exceptionpd.errors.SpecificationError

pandas.errors.UndefinedVariableError

exceptionpd.errors.UndefinedVariableError(name,is_local=None)

pandas.errors.UnsortedIndexError

exceptionpd.errors.UnsortedIndexError

pandas.errors.UnsupportedFunctionCall

exceptionpd.errors.UnsupportedFunctionCall

pandas.errors.ValueLabelTypeMismatch

exceptionpd.errors.ValueLabelTypeMismatch

pandas.show_versions

pd.show_versions(as_json=False)

pandas.test

pd.test(extra_args=None,run_doctests=False)

Valores ausentes

1 entradas documentadas.

pandas.NaT

pd.NaT=NaT

Índice alfabético

Nombre completo · página. Los nombres son enlaces internos. Se prioriza la ficha práctica; si no existe, la página corresponde al catálogo del anexo. Orden alfabético por identificador completo; busca también con Ctrl+F.

pd.api.extensions.ExtensionArray	195
pd.api.extensions.ExtensionDtype	195
pd.api.extensions.register_dataframe_accessor	195
pd.api.extensions.register_extension_dtype	195
pd.api.extensions.register_index_accessor	195
pd.api.extensions.register_series_accessor	195
pd.api.indexers.BaseIndexer	148
pd.api.indexers.check_array_indexer	195
pd.api.indexers.FixedForwardWindowIndexer	77
pd.api.indexers.VariableOffsetWindowIndexer	148
pd.api.interchange.from_dataframe	102
pd.api.types.infer_dtype	159
pd.api.types.is_any_real_numeric_dtype	159
pd.api.types.is_bool	160
pd.api.types.is_bool_dtype	159
pd.api.types.is_categorical_dtype	159
pd.api.types.is_complex	160
pd.api.types.is_complex_dtype	159
pd.api.types.is_datetime64_any_dtype	159
pd.api.types.is_datetime64_dtype	159
pd.api.types.is_datetime64_ns_dtype	159
pd.api.types.is_datetime64tz_dtype	159
pd.api.types.is_dict_like	160
pd.api.types.is_dtype_equal	159
pd.api.types.is_extension_array_dtype	159
pd.api.types.is_file_like	160
pd.api.types.is_float	160
pd.api.types.is_float_dtype	159
pd.api.types.is_hashable	160
pd.api.types.is_int64_dtype	159
pd.api.types.is_integer	160
pd.api.types.is_integer_dtype	159
pd.api.types.is_interval_dtype	159
pd.api.types.is_iterator	160
pd.api.types.is_list_like	160
pd.api.types.is_named_tuple	160
pd.api.types.is_number	160
pd.api.types.is_numeric_dtype	159
pd.api.types.is_object_dtype	159
pd.api.types.is_period_dtype	160
pd.api.types.is_re	160
pd.api.types.is_re_compilable	160
pd.api.types.is_scalar	160
pd.api.types.is_signed_integer_dtype	160

pd.api.types.is_sparse	160
pd.api.types.is_string_dtype	160
pd.api.types.is_timedelta64_dtype	160
pd.api.types.is_timedelta64_ns_dtype	160
pd.api.types.is_unsigned_integer_dtype	160
pd.api.types.pandas_dtype	159
pd.api.types.union_categoricals	159
DataFrameGroupBy.agg	48
DataFrameGroupBy.aggregate	140
DataFrameGroupBy.all	140
DataFrameGroupBy.any	140
DataFrameGroupBy.apply	49
DataFrameGroupBy.bfill	141
DataFrameGroupBy.boxplot	145
DataFrameGroupBy.corr	141
DataFrameGroupBy.corrwith	141
DataFrameGroupBy.count	49
DataFrameGroupBy.cov	141
DataFrameGroupBy.cumcount	50
DataFrameGroupBy.cummax	141
DataFrameGroupBy.cummin	141
DataFrameGroupBy.cumprod	141
DataFrameGroupBy.cumsum	141
DataFrameGroupBy.describe	141
DataFrameGroupBy.diff	141
DataFrameGroupBy.ewm	141
DataFrameGroupBy.expanding	141
DataFrameGroupBy.ffill	141
DataFrameGroupBy.filter	49
DataFrameGroupBy.first	52
DataFrameGroupBy.get_group	50
DataFrameGroupBy.groups	51
DataFrameGroupBy.head	51
DataFrameGroupBy.hist	145
DataFrameGroupBy.idxmax	141
DataFrameGroupBy.idxmin	141
DataFrameGroupBy.indices	53
DataFrameGroupBy.kurt	142
DataFrameGroupBy.last	52
DataFrameGroupBy.max	141
DataFrameGroupBy.mean	141
DataFrameGroupBy.median	142
DataFrameGroupBy.min	142
DataFrameGroupBy.ngroup	51
DataFrameGroupBy.ngroups	53
DataFrameGroupBy.nth	52
DataFrameGroupBy.nunique	52
DataFrameGroupBy.ohlc	142

DataFrameGroupBy.pct_change	142
DataFrameGroupBy.pipe	140
DataFrameGroupBy.plot	145
DataFrameGroupBy.prod	142
DataFrameGroupBy.quantile	142
DataFrameGroupBy.rank	142
DataFrameGroupBy.resample	142
DataFrameGroupBy.rolling	142
DataFrameGroupBy.sample	142
DataFrameGroupBy.sem	142
DataFrameGroupBy.shift	142
DataFrameGroupBy.size	49
DataFrameGroupBy.skew	142
DataFrameGroupBy.std	142
DataFrameGroupBy.sum	142
DataFrameGroupBy.tail	142
DataFrameGroupBy.take	142
DataFrameGroupBy.transform	140
DataFrameGroupBy.value_counts	143
DataFrameGroupBy.var	142
Expanding.aggregate	147
Expanding.apply	147
Expanding.corr	147
Expanding.count	147
Expanding.cov	147
Expanding.first	147
Expanding.kurt	147
Expanding.last	147
Expanding.max	147
Expanding.mean	147
Expanding.median	147
Expanding.min	147
Expanding.nunique	148
Expanding.pipe	147
Expanding.quantile	147
Expanding.rank	147
Expanding.sem	147
Expanding.skew	147
Expanding.std	147
Expanding.sum	147
Expanding.var	147
ExponentialMovingWindow.corr	148
ExponentialMovingWindow.cov	148
ExponentialMovingWindow.mean	76
ExponentialMovingWindow.std	148
ExponentialMovingWindow.sum	148
ExponentialMovingWindow.var	148
Resampler.agg	73

Resampler.aggregate	149
Resampler.apply	149
Resampler.asfreq	149
Resampler.bfill	149
Resampler.count	149
Resampler.ffill	74
Resampler.first	149
Resampler.get_group	149
Resampler.groups	149
Resampler.indices	149
Resampler.interpolate	149
Resampler.last	149
Resampler.max	149
Resampler.mean	149
Resampler.median	149
Resampler.min	149
Resampler.nearest	149
Resampler.nunique	149
Resampler.ohlc	149
Resampler.pipe	149
Resampler.prod	149
Resampler.quantile	150
Resampler.sem	150
Resampler.size	150
Resampler.std	150
Resampler.sum	150
Resampler.transform	149
Resampler.var	150
Rolling.aggregate	146
Rolling.apply	76
Rolling.corr	146
Rolling.count	146
Rolling.cov	146
Rolling.first	146
Rolling.kurt	146
Rolling.last	146
Rolling.max	146
Rolling.mean	75
Rolling.median	146
Rolling.min	146
Rolling.nunique	146
Rolling.pipe	146
Rolling.quantile	146
Rolling.rank	146
Rolling.sem	146
Rolling.skew	146
Rolling.std	146
Rolling.sum	75

Rolling.var	146
SeriesGroupBy.agg	140
SeriesGroupBy.aggregate	140
SeriesGroupBy.all	143
SeriesGroupBy.any	143
SeriesGroupBy.apply	140
SeriesGroupBy.bfill	143
SeriesGroupBy.corr	143
SeriesGroupBy.count	143
SeriesGroupBy.cov	143
SeriesGroupBy.cumcount	143
SeriesGroupBy.cummax	143
SeriesGroupBy.cummin	143
SeriesGroupBy.cumprod	143
SeriesGroupBy.cumsum	143
SeriesGroupBy.describe	143
SeriesGroupBy.diff	143
SeriesGroupBy.ewm	143
SeriesGroupBy.expanding	143
SeriesGroupBy.ffill	143
SeriesGroupBy.filter	140
SeriesGroupBy.first	143
SeriesGroupBy.get_group	140
SeriesGroupBy.groups	140
SeriesGroupBy.head	143
SeriesGroupBy.hist	145
SeriesGroupBy.idxmax	143
SeriesGroupBy.idxmin	143
SeriesGroupBy.indices	140
SeriesGroupBy.is_monotonic_decreasing	144
SeriesGroupBy.is_monotonic_increasing	143
SeriesGroupBy.kurt	145
SeriesGroupBy.last	143
SeriesGroupBy.max	144
SeriesGroupBy.mean	50
SeriesGroupBy.median	144
SeriesGroupBy.min	144
SeriesGroupBy.ngroup	144
SeriesGroupBy.nlargest	144
SeriesGroupBy.nsmallest	144
SeriesGroupBy.nth	144
SeriesGroupBy.nunique	144
SeriesGroupBy.ohlc	144
SeriesGroupBy.pct_change	144
SeriesGroupBy.pipe	140
SeriesGroupBy.plot	145
SeriesGroupBy.prod	144
SeriesGroupBy.quantile	144

SeriesGroupBy.rank	144
SeriesGroupBy.resample	144
SeriesGroupBy.rolling	53
SeriesGroupBy.sample	144
SeriesGroupBy.sem	144
SeriesGroupBy.shift	51
SeriesGroupBy.size	144
SeriesGroupBy.skew	144
SeriesGroupBy.std	145
SeriesGroupBy.sum	50
SeriesGroupBy.tail	145
SeriesGroupBy.take	145
SeriesGroupBy.transform	48
SeriesGroupBy.unique	144
SeriesGroupBy.value_counts	145
SeriesGroupBy.var	145
Window.mean	77
Window.std	147
Window.sum	147
Window.var	147
pd.array	8
pd.arrays.ArrowExtensionArray	152
pd.arrays.ArrowStringArray	158
pd.arrays.BooleanArray	159
pd.arrays.DatetimeArray	155
pd.arrays.FloatingArray	158
pd.arrays.IntegerArray	158
pd.arrays.IntervalArray	158
pd.arrays.NumpyExtensionArray	195
pd.arrays.PeriodArray	157
pd.arrays.SparseArray	158
pd.arrays.StringArray	158
pd.arrays.TimedeltaArray	156
pd.ArrowDtype	151
pd.bdate_range	66
pd.BooleanDtype	151
pd.Categorical	9
pd.Categorical.add_categories	158
pd.Categorical.as_ordered	158
pd.Categorical.as_unordered	158
pd.Categorical.categories	158
pd.Categorical.codes	158
pd.Categorical.dtype	158
pd.Categorical.from_codes	158
pd.Categorical.map	158
pd.Categorical.ordered	158
pd.Categorical.remove_categories	158
pd.Categorical.remove_unused_categories	158

pd.Categorical.rename_categories	158
pd.Categorical.reorder_categories	158
pd.Categorical.set_categories	158
pd.CategoricalDtype	78
pd.CategoricalDtype.categories	158
pd.CategoricalDtype.ordered	158
pd.CategoricalIndex	132
pd.CategoricalIndex.add_categories	133
pd.CategoricalIndex.append	132
pd.CategoricalIndex.as_ordered	133
pd.CategoricalIndex.as_unordered	133
pd.CategoricalIndex.categories	132
pd.CategoricalIndex.codes	132
pd.CategoricalIndex.equals	133
pd.CategoricalIndex.map	133
pd.CategoricalIndex.ordered	133
pd.CategoricalIndex.remove_categories	133
pd.CategoricalIndex.remove_unused_categories	133
pd.CategoricalIndex.rename_categories	133
pd.CategoricalIndex.reorder_categories	133
pd.CategoricalIndex.set_categories	133
pd.col	102
pd.concat	55
pd.crosstab	59
pd.cut	36
pd.DataFrame	8
pd.DataFrame.abs	106
pd.DataFrame.add	104
pd.DataFrame.add_prefix	107
pd.DataFrame.add_suffix	107
pd.DataFrame.agg	89
pd.DataFrame.aggregate	106
pd.DataFrame.align	107
pd.DataFrame.all	106
pd.DataFrame.any	106
pd.DataFrame.apply	88
pd.DataFrame.asfreq	110
pd.DataFrame.asof	110
pd.DataFrame.assign	26
pd.DataFrame.astype	29
pd.DataFrame.at	22
pd.DataFrame.at_time	107
pd.DataFrame.attrs	20
pd.DataFrame.axes	103
pd.DataFrame.between_time	71
pd.DataFrame.bfill	33
pd.DataFrame.boxplot	92
pd.DataFrame.clip	106

pd.DataFrame.columns	17
pd.DataFrame.combine	105
pd.DataFrame.combine_first	56
pd.DataFrame.compare	110
pd.DataFrame.convert_dtypes	29
pd.DataFrame.copy	28
pd.DataFrame.corr	42
pd.DataFrame.corrwith	106
pd.DataFrame.count	106
pd.DataFrame.cov	42
pd.DataFrame.cummax	106
pd.DataFrame.cummin	106
pd.DataFrame.cumprod	106
pd.DataFrame.cumsum	106
pd.DataFrame.describe	16
pd.DataFrame.diff	106
pd.DataFrame.div	104
pd.DataFrame.dot	105
pd.DataFrame.drop	26
pd.DataFrame.drop_duplicates	34
pd.DataFrame.droplevel	109
pd.DataFrame.dropna	33
pd.DataFrame.dtypes	18
pd.DataFrame.duplicated	34
pd.DataFrame.empty	18
pd.DataFrame.eq	105
pd.DataFrame.equals	90
pd.DataFrame.eval	28
pd.DataFrame.ewm	106
pd.DataFrame.expanding	106
pd.DataFrame.explode	58
pd.DataFrame.ffill	33
pd.DataFrame.fillna	33
pd.DataFrame.filter	23
pd.DataFrame.first_valid_index	110
pd.DataFrame.floordiv	104
pd.DataFrame.from_arrow	111
pd.DataFrame.from_dict	9
pd.DataFrame.from_records	9
pd.DataFrame.ge	105
pd.DataFrame.get	25
pd.DataFrame.groupby	48
pd.DataFrame.gt	105
pd.DataFrame.head	15
pd.DataFrame.hist	91
pd.DataFrame.iat	22
pd.DataFrame.idxmax	108
pd.DataFrame.idxmin	108

pd.DataFrame.iloc	22
pd.DataFrame.index	17
pd.DataFrame.infer_objects	29
pd.DataFrame.info	16
pd.DataFrame.insert	26
pd.DataFrame.interpolate	109
pd.DataFrame.isetitem	104
pd.DataFrame.isin	104
pd.DataFrame.isna	32
pd.DataFrame.isnull	109
pd.DataFrame.items	90
pd.DataFrame.iterrows	90
pd.DataFrame.itertuples	90
pd.DataFrame.join	54
pd.DataFrame.keys	21
pd.DataFrame.kurt	106
pd.DataFrame.kurtosis	106
pd.DataFrame.last_valid_index	110
pd.DataFrame.le	105
pd.DataFrame.loc	22
pd.DataFrame.lt	105
pd.DataFrame.map	88
pd.DataFrame.mask	24
pd.DataFrame.max	107
pd.DataFrame.mean	107
pd.DataFrame.median	107
pd.DataFrame.melt	57
pd.DataFrame.memory_usage	18
pd.DataFrame.merge	54
pd.DataFrame.min	107
pd.DataFrame.mod	105
pd.DataFrame.mode	107
pd.DataFrame.mul	104
pd.DataFrame.ndim	17
pd.DataFrame.ne	105
pd.DataFrame.nlargest	35
pd.DataFrame.notna	109
pd.DataFrame.notnull	109
pd.DataFrame.nsmallest	36
pd.DataFrame.nunique	107
pd.DataFrame.pct_change	107
pd.DataFrame.pipe	89
pd.DataFrame.pivot	57
pd.DataFrame.pivot_table	57
pd.DataFrame.plot	91
pd.DataFrame.plot.area	111
pd.DataFrame.plot.bar	91
pd.DataFrame.plot.barh	111

pd.DataFrame.plot.box	111
pd.DataFrame.plot.density	111
pd.DataFrame.plot.hexbin	111
pd.DataFrame.plot.hist	111
pd.DataFrame.plot.kde	111
pd.DataFrame.plot.line	111
pd.DataFrame.plot.pie	111
pd.DataFrame.plot.scatter	91
pd.DataFrame.pop	27
pd.DataFrame.pow	105
pd.DataFrame.prod	107
pd.DataFrame.product	107
pd.DataFrame.quantile	107
pd.DataFrame.query	23
pd.DataFrame.radd	105
pd.DataFrame.rank	107
pd.DataFrame.rdiv	105
pd.DataFrame.reindex	28
pd.DataFrame.reindex_like	108
pd.DataFrame.rename	27
pd.DataFrame.rename_axis	108
pd.DataFrame.reorder_levels	85
pd.DataFrame.replace	109
pd.DataFrame.resample	110
pd.DataFrame.reset_index	82
pd.DataFrame.rfloordiv	105
pd.DataFrame.rmod	105
pd.DataFrame.rmul	105
pd.DataFrame.rolling	106
pd.DataFrame.round	107
pd.DataFrame.rpow	105
pd.DataFrame.rsub	105
pd.DataFrame.rtruediv	105
pd.DataFrame.sample	15
pd.DataFrame.select_dtypes	21
pd.DataFrame.sem	107
pd.DataFrame.set_axis	27
pd.DataFrame.set_flags	103
pd.DataFrame.set_index	82
pd.DataFrame.shape	16
pd.DataFrame.shift	110
pd.DataFrame.size	17
pd.DataFrame.skew	107
pd.DataFrame.sort_index	35
pd.DataFrame.sort_values	35
pd.DataFrame.sparse	121
pd.DataFrame.sparse.density	111
pd.DataFrame.sparse.from_spmatrix	111

pd.DataFrame.sparse.to_coo	111
pd.DataFrame.sparse.to_dense	111
pd.DataFrame.squeeze	110
pd.DataFrame.stack	58
pd.DataFrame.std	107
pd.DataFrame.style	112
pd.DataFrame.sub	104
pd.DataFrame.sum	107
pd.DataFrame.swaplevel	85
pd.DataFrame.T	20
pd.DataFrame.tail	15
pd.DataFrame.take	24
pd.DataFrame.to_clipboard	97
pd.DataFrame.to_csv	10
pd.DataFrame.to_dict	14
pd.DataFrame.to_excel	10
pd.DataFrame.to_feather	99
pd.DataFrame.to_hdf	112
pd.DataFrame.to_html	13
pd.DataFrame.to_iceberg	99
pd.DataFrame.to_json	11
pd.DataFrame.to_latex	98
pd.DataFrame.to_markdown	112
pd.DataFrame.to_numpy	30
pd.DataFrame.to_orc	99
pd.DataFrame.to_parquet	11
pd.DataFrame.to_period	110
pd.DataFrame.to_pickle	14
pd.DataFrame.to_records	14
pd.DataFrame.to_sql	12
pd.DataFrame.to_stata	100
pd.DataFrame.to_string	112
pd.DataFrame.to_timestamp	110
pd.DataFrame.to_xarray	110
pd.DataFrame.to_xml	13
pd.DataFrame.transform	89
pd.DataFrame.transpose	110
pd.DataFrame.truediv	104
pd.DataFrame.truncate	108
pd.DataFrame.tz_convert	110
pd.DataFrame.tz_localize	111
pd.DataFrame.unstack	58
pd.DataFrame.update	27
pd.DataFrame.value_counts	107
pd.DataFrame.values	103
pd.DataFrame.var	107
pd.DataFrame.where	24
pd.DataFrame.xs	24

pd.date_range	66
pd.DateOffset	67
pd.DatetimeIndex	135
pd.DatetimeIndex.as_unit	137
pd.DatetimeIndex.ceil	137
pd.DatetimeIndex.date	135
pd.DatetimeIndex.day	135
pd.DatetimeIndex.day_name	137
pd.DatetimeIndex.day_of_week	136
pd.DatetimeIndex.day_of_year	136
pd.DatetimeIndex.dayofweek	136
pd.DatetimeIndex.dayofyear	136
pd.DatetimeIndex.floor	137
pd.DatetimeIndex.freq	136
pd.DatetimeIndex.freqstr	136
pd.DatetimeIndex.hour	135
pd.DatetimeIndex.indexer_at_time	136
pd.DatetimeIndex.indexer_between_time	136
pd.DatetimeIndex.inferred_freq	136
pd.DatetimeIndex.is_leap_year	136
pd.DatetimeIndex.is_month_end	136
pd.DatetimeIndex.is_month_start	136
pd.DatetimeIndex.is_quarter_end	136
pd.DatetimeIndex.is_quarter_start	136
pd.DatetimeIndex.is_year_end	136
pd.DatetimeIndex.is_year_start	136
pd.DatetimeIndex.mean	137
pd.DatetimeIndex.microsecond	135
pd.DatetimeIndex.minute	135
pd.DatetimeIndex.month	135
pd.DatetimeIndex.month_name	137
pd.DatetimeIndex.nanosecond	135
pd.DatetimeIndex.normalize	136
pd.DatetimeIndex.quarter	136
pd.DatetimeIndex.round	137
pd.DatetimeIndex.second	135
pd.DatetimeIndex.snap	137
pd.DatetimeIndex.std	137
pd.DatetimeIndex.strftime	136
pd.DatetimeIndex.time	136
pd.DatetimeIndex.timetz	136
pd.DatetimeIndex.to_frame	137
pd.DatetimeIndex.to_julian_date	137
pd.DatetimeIndex.to_period	137
pd.DatetimeIndex.to_pydatetime	137
pd.DatetimeIndex.to_series	137
pd.DatetimeIndex.tz	136
pd.DatetimeIndex.tz_convert	137

pd.DatetimeIndex.tz_localize	137
pd.DatetimeIndex.weekday	136
pd.DatetimeIndex.year	135
pd.DatetimeTZDtype	151
pd.describe_option	194
pd.errors.AbstractMethodError	196
pd.errors.AttributeConflictWarning	196
pd.errors.CategoricalConversionWarning	196
pd.errors.ChainedAssignmentError	196
pd.errors.ClosedFileError	196
pd.errors.CSSWarning	196
pd.errors.DatabaseError	196
pd.errors.DataError	196
pd.errors.DtypeWarning	196
pd.errors.DuplicateLabelError	196
pd.errors.EmptyDataError	196
pd.errors.IncompatibilityWarning	196
pd.errors.IncompatibleFrequency	196
pd.errors.IndexingError	196
pd.errors.IntCastingNaNError	197
pd.errors.InvalidColumnName	196
pd.errors.InvalidComparison	197
pd.errors.InvalidIndexError	197
pd.errors.InvalidVersion	197
pd.errors.LossySetitemError	197
pd.errors.MergeError	197
pd.errors.NoBufferPresent	197
pd.errors.NullFrequencyError	197
pd.errors.NumbaUtilError	197
pd.errors.NumExprClobberingError	197
pd.errors.OptionError	197
pd.errors.OutOfBoundsDatetime	197
pd.errors.OutOfBoundsTimedelta	197
pd.errors.Pandas4Warning	197
pd.errors.Pandas5Warning	197
pd.errors.PandasChangeWarning	197
pd.errors.PandasDeprecationWarning	197
pd.errors.PandasFutureWarning	197
pd.errors.PandasPendingDeprecationWarning	197
pd.errors.ParserError	197
pd.errors.ParserWarning	197
pd.errors.PerformanceWarning	197
pd.errors.PossibleDataLossError	197
pd.errors.PossiblePrecisionLoss	198
pd.errors.PyperclipException	198
pd.errors.PyperclipWindowsException	198
pd.errors.SpecificationError	198
pd.errors.UndefinedVariableError	198

pd.errors.UnsortedIndexError	198
pd.errors.UnsupportedFunctionCall	198
pd.errors.ValueLabelTypeMismatch	198
pd.eval	102
pd.ExcelFile	97
pd.ExcelFile.book	97
pd.ExcelFile.parse	98
pd.ExcelFile.sheet_names	97
pd.ExcelWriter	98
pd.factorize	90
pd.Flags	111
pd.Float32Dtype	152
pd.Float64Dtype	151
pd.from_dummies	59
pd.get_dummies	59
pd.get_option	93
pd.Grouper	140
pd.HDFStore.append	99
pd.HDFStore.get	99
pd.HDFStore.groups	99
pd.HDFStore.info	99
pd.HDFStore.keys	99
pd.HDFStore.put	99
pd.HDFStore.select	99
pd.HDFStore.walk	99
pd.Index	8
pd.Index.all	129
pd.Index.any	129
pd.Index.append	131
pd.Index.argmax	130
pd.Index.argmin	129
pd.Index.argsort	131
pd.Index.array	129
pd.Index.asof	132
pd.Index.asof_locs	132
pd.Index.astype	131
pd.Index.copy	130
pd.Index.delete	130
pd.Index.difference	84
pd.Index.drop	130
pd.Index.drop_duplicates	86
pd.Index.droplevel	131
pd.Index.dropna	131
pd.Index.dtype	129
pd.Index.duplicated	86
pd.Index.empty	129
pd.Index.equals	130
pd.Index.factorize	130

pd.Index.fillna	131
pd.Index.get_indexer	83
pd.Index.get_indexer_for	132
pd.Index.get_indexer_non_unique	132
pd.Index.get_level_values	132
pd.Index.get_loc	83
pd.Index.get_slice_bound	132
pd.Index.has_duplicates	129
pd.Index.hasnans	129
pd.Index.identical	130
pd.Index.infer_objects	131
pd.Index.inferred_type	129
pd.Index.insert	130
pd.Index.intersection	83
pd.Index.is_	130
pd.Index.is_monotonic_decreasing	129
pd.Index.is_monotonic_increasing	86
pd.Index.is_unique	82
pd.Index.isin	84
pd.Index.isna	131
pd.Index.item	131
pd.Index.join	131
pd.Index.map	131
pd.Index.max	130
pd.Index.memory_usage	129
pd.Index.min	130
pd.Index.name	82
pd.Index.names	83
pd.Index.nbytes	129
pd.Index.ndim	129
pd.Index.notna	131
pd.Index.nunique	130
pd.Index.putmask	130
pd.Index.ravel	131
pd.Index.reindex	130
pd.Index.rename	84
pd.Index.repeat	130
pd.Index.searchsorted	131
pd.Index.set_names	130
pd.Index.shape	129
pd.Index.shift	131
pd.Index.size	129
pd.Index.slice_indexer	132
pd.Index.slice_locs	132
pd.Index.sort_values	131
pd.Index.str	121
pd.Index.symmetric_difference	132
pd.Index.T	129

pd.Index.take	130
pd.Index.to_frame	86
pd.Index.to_list	131
pd.Index.to_numpy	131
pd.Index.to_series	131
pd.Index.union	83
pd.Index.unique	130
pd.Index.value_counts	130
pd.Index.values	129
pd.Index.view	131
pd.Index.where	130
pd.IndexSlice	135
pd.infer_freq	102
pd.Int16Dtype	151
pd.Int32Dtype	151
pd.Int64Dtype	151
pd.Int8Dtype	151
pd.Interval	151
pd.Interval.closed	157
pd.Interval.closed_left	157
pd.Interval.closed_right	157
pd.Interval.is_empty	157
pd.Interval.left	157
pd.Interval.length	157
pd.Interval.mid	157
pd.Interval.open_left	157
pd.Interval.open_right	157
pd.Interval.overlaps	157
pd.Interval.right	158
pd.interval_range	102
pd.IntervalDtype	151
pd.IntervalIndex	133
pd.IntervalIndex.closed	133
pd.IntervalIndex.contains	134
pd.IntervalIndex.from_arrays	133
pd.IntervalIndex.from_breaks	133
pd.IntervalIndex.from_tuples	133
pd.IntervalIndex.get_indexer	134
pd.IntervalIndex.get_loc	134
pd.IntervalIndex.is_empty	133
pd.IntervalIndex.is_non_overlapping_monotonic	133
pd.IntervalIndex.is_overlapping	134
pd.IntervalIndex.left	133
pd.IntervalIndex.length	133
pd.IntervalIndex.mid	133
pd.IntervalIndex.overlaps	134
pd.IntervalIndex.right	133
pd.IntervalIndex.set_closed	134

pd.IntervalIndex.to_tuples	134
pd.IntervalIndex.values	133
pd.io.formats.style.Styler	191
pd.io.formats.style.Styler.apply	191
pd.io.formats.style.Styler.apply_index	191
pd.io.formats.style.Styler.background_gradient	192
pd.io.formats.style.Styler.bar	192
pd.io.formats.style.Styler.clear	192
pd.io.formats.style.Styler.concat	191
pd.io.formats.style.Styler.env	191
pd.io.formats.style.Styler.export	192
pd.io.formats.style.Styler.format	191
pd.io.formats.style.Styler.format_index	191
pd.io.formats.style.Styler.format_index_names	191
pd.io.formats.style.Styler.from_custom_template	191
pd.io.formats.style.Styler.hide	191
pd.io.formats.style.Styler.highlight_between	192
pd.io.formats.style.Styler.highlight_max	192
pd.io.formats.style.Styler.highlight_min	192
pd.io.formats.style.Styler.highlight_null	192
pd.io.formats.style.Styler.highlight_quantile	192
pd.io.formats.style.Styler.loader	191
pd.io.formats.style.Styler.map	191
pd.io.formats.style.Styler.map_index	191
pd.io.formats.style.Styler.pipe	192
pd.io.formats.style.Styler.relabel_index	191
pd.io.formats.style.Styler.set_caption	192
pd.io.formats.style.Styler.set_properties	192
pd.io.formats.style.Styler.set_sticky	192
pd.io.formats.style.Styler.set_table_attributes	192
pd.io.formats.style.Styler.set_table_styles	192
pd.io.formats.style.Styler.set_td_classes	191
pd.io.formats.style.Styler.set_tooltips	192
pd.io.formats.style.Styler.set_uuid	192
pd.io.formats.style.Styler.template_html	191
pd.io.formats.style.Styler.template_html_style	191
pd.io.formats.style.Styler.template_html_table	191
pd.io.formats.style.Styler.template_latex	191
pd.io.formats.style.Styler.template_string	191
pd.io.formats.style.Styler.template_typst	191
pd.io.formats.style.Styler.text_gradient	192
pd.io.formats.style.Styler.to_excel	98
pd.io.formats.style.Styler.to_html	98
pd.io.formats.style.Styler.to_latex	98
pd.io.formats.style.Styler.to_string	192
pd.io.formats.style.Styler.to_typst	192
pd.io.formats.style.Styler.use	192
pd.io.json.build_table_schema	98

pd.io.stata.StataReader.data_label	100
pd.io.stata.StataReader.value_labels	100
pd.io.stata.StataReader.variable_labels	100
pd.io.stata.StataWriter.write_file	100
pd.isna	32
pd.isnull	101
pd.json_normalize	98
pd.lreshape	101
pd.melt	101
pd.merge	54
pd.merge_asof	55
pd.merge_ordered	55
pd.MultiIndex	109
pd.MultiIndex.append	135
pd.MultiIndex.codes	87
pd.MultiIndex.copy	135
pd.MultiIndex.drop	135
pd.MultiIndex.droplevel	86
pd.MultiIndex.dtypes	134
pd.MultiIndex.from_arrays	134
pd.MultiIndex.from_frame	134
pd.MultiIndex.from_product	84
pd.MultiIndex.from_tuples	85
pd.MultiIndex.get_indexer	135
pd.MultiIndex.get_level_values	85
pd.MultiIndex.get_loc	135
pd.MultiIndex.get_loc_level	135
pd.MultiIndex.get_locs	135
pd.MultiIndex.levels	87
pd.MultiIndex.levshape	134
pd.MultiIndex.names	134
pd.MultiIndex.nlevels	134
pd.MultiIndex.remove_unused_levels	87
pd.MultiIndex.reorder_levels	135
pd.MultiIndex.set_codes	134
pd.MultiIndex.set_levels	134
pd.MultiIndex.sortlevel	134
pd.MultiIndex.swaplevel	135
pd.MultiIndex.to_flat_index	134
pd.MultiIndex.to_frame	134
pd.MultiIndex.truncate	135
pd.NA	151
pd.NamedAgg	140
pd.NaT	199
pd.notna	32
pd.notnull	102
pd.option_context	93
pd.Period	151

pd.Period.asfreq	157
pd.Period.day	156
pd.Period.day_of_week	156
pd.Period.day_of_year	156
pd.Period.dayofweek	156
pd.Period.dayofyear	156
pd.Period.days_in_month	156
pd.Period.daysinmonth	156
pd.Period.end_time	156
pd.Period.freq	156
pd.Period.freqstr	156
pd.Period.hour	156
pd.Period.is_leap_year	156
pd.Period.minute	156
pd.Period.month	156
pd.Period.now	157
pd.Period.ordinal	156
pd.Period.quarter	157
pd.Period.qyear	157
pd.Period.second	157
pd.Period.start_time	157
pd.Period.strftime	157
pd.Period.to_timestamp	157
pd.Period.week	157
pd.Period.weekday	157
pd.Period.weekofyear	157
pd.Period.year	157
pd.period_range	66
pd.PeriodDtype	151
pd.PeriodIndex	138
pd.PeriodIndex.asfreq	139
pd.PeriodIndex.day	138
pd.PeriodIndex.day_of_week	138
pd.PeriodIndex.day_of_year	138
pd.PeriodIndex.dayofweek	138
pd.PeriodIndex.dayofyear	138
pd.PeriodIndex.days_in_month	138
pd.PeriodIndex.daysinmonth	138
pd.PeriodIndex.end_time	138
pd.PeriodIndex.freq	138
pd.PeriodIndex.freqstr	138
pd.PeriodIndex.from_fields	139
pd.PeriodIndex.from_ordinals	139
pd.PeriodIndex.hour	138
pd.PeriodIndex.is_leap_year	138
pd.PeriodIndex.minute	138
pd.PeriodIndex.month	138
pd.PeriodIndex.quarter	139

pd.PeriodIndex.qyear	139
pd.PeriodIndex.second	139
pd.PeriodIndex.start_time	139
pd.PeriodIndex.strftime	139
pd.PeriodIndex.to_timestamp	139
pd.PeriodIndex.week	139
pd.PeriodIndex.weekday	139
pd.PeriodIndex.weekofyear	139
pd.PeriodIndex.year	139
pd.pivot	101
pd.pivot_table	101
pd.plotting.andrews_curves	193
pd.plotting.autocorrelation_plot	193
pd.plotting.bootstrap_plot	193
pd.plotting.boxplot	193
pd.plotting.deregister_matplotlib_converters	193
pd.plotting.lag_plot	193
pd.plotting.parallel_coordinates	193
pd.plotting.plot_params	193
pd.plotting.radviz	193
pd.plotting.register_matplotlib_converters	193
pd.plotting.scatter_matrix	92
pd.plotting.table	193
pd.qcut	36
pd.RangeIndex	132
pd.RangeIndex.from_range	132
pd.RangeIndex.start	132
pd.RangeIndex.step	132
pd.RangeIndex.stop	132
pd.read_clipboard	97
pd.read_csv	10
pd.read_excel	10
pd.read_feather	99
pd.read_fwf	97
pd.read_hdf	99
pd.read_html	12
pd.read_iceberg	99
pd.read_json	11
pd.read_orc	99
pd.read_parquet	11
pd.read_pickle	13
pd.read_sas	99
pd.read_spss	99
pd.read_sql	12
pd.read_sql_query	12
pd.read_sql_table	99
pd.read_stata	100
pd.read_table	97

pd.read_xml	13
pd.reset_option	93
pd.Series	8
pd.Series.abs	41
pd.Series.add	41
pd.Series.add_prefix	119
pd.Series.add_suffix	119
pd.Series.agg	116
pd.Series.aggregate	116
pd.Series.align	118
pd.Series.all	42
pd.Series.any	42
pd.Series.apply	89
pd.Series.argmax	47
pd.Series.argmin	47
pd.Series.argsort	119
pd.Series.array	31
pd.Series.asfreq	71
pd.Series.asof	120
pd.Series.astype	113
pd.Series.at	114
pd.Series.at_time	120
pd.Series.attrs	126
pd.Series.autocorr	116
pd.Series.between	23
pd.Series.between_time	120
pd.Series.bfill	119
pd.Series.case_when	118
pd.Series.cat	121
pd.Series.cat.add_categories	79
pd.Series.cat.as_ordered	80
pd.Series.cat.as_unordered	81
pd.Series.cat.categories	78
pd.Series.cat.codes	78
pd.Series.cat.ordered	78
pd.Series.cat.remove_categories	81
pd.Series.cat.remove_unused_categories	79
pd.Series.cat.rename_categories	79
pd.Series.cat.reorder_categories	80
pd.Series.cat.set_categories	80
pd.Series.clip	41
pd.Series.combine	115
pd.Series.combine_first	115
pd.Series.compare	120
pd.Series.convert_dtypes	113
pd.Series.copy	114
pd.Series.corr	116
pd.Series.count	38

pd.Series.cov	116
pd.Series.cummax	40
pd.Series.cummin	44
pd.Series.cumprod	39
pd.Series.cumsum	39
pd.Series.describe	116
pd.Series.diff	40
pd.Series.div	41
pd.Series.dot	115
pd.Series.drop	118
pd.Series.drop_duplicates	118
pd.Series.droplevel	118
pd.Series.dropna	119
pd.Series.dt	121
pd.Series.dt.as_unit	123
pd.Series.dt.ceil	73
pd.Series.dt.components	73
pd.Series.dt.date	121
pd.Series.dt.day	68
pd.Series.dt.day_name	72
pd.Series.dt.day_of_week	121
pd.Series.dt.day_of_year	121
pd.Series.dt.dayofweek	68
pd.Series.dt.dayofyear	121
pd.Series.dt.days	123
pd.Series.dt.days_in_month	72
pd.Series.dt.daysinmonth	122
pd.Series.dt.end_time	123
pd.Series.dt.floor	69
pd.Series.dt.freq	122
pd.Series.dt.hour	121
pd.Series.dt.is_leap_year	122
pd.Series.dt.is_month_end	72
pd.Series.dt.is_month_start	122
pd.Series.dt.is_quarter_end	122
pd.Series.dt.is_quarter_start	122
pd.Series.dt.is_year_end	122
pd.Series.dt.is_year_start	122
pd.Series.dt.isocalendar	70
pd.Series.dt.microsecond	121
pd.Series.dt.microseconds	123
pd.Series.dt.minute	121
pd.Series.dt.month	68
pd.Series.dt.month_name	122
pd.Series.dt.nanosecond	121
pd.Series.dt.nanoseconds	123
pd.Series.dt.normalize	69
pd.Series.dt.quarter	72

pd.Series.dt.qyear	123
pd.Series.dt.round	73
pd.Series.dt.second	121
pd.Series.dt.seconds	123
pd.Series.dt.start_time	123
pd.Series.dt.strftime	68
pd.Series.dt.time	121
pd.Series.dt.timetz	121
pd.Series.dt.to_period	69
pd.Series.dt.to_pydatetime	122
pd.Series.dt.to_pytimedelta	123
pd.Series.dt.total_seconds	70
pd.Series.dt.tz	122
pd.Series.dt.tz_convert	70
pd.Series.dt.tz_localize	69
pd.Series.dt.unit	122
pd.Series.dt.weekday	121
pd.Series.dt.year	67
pd.Series.dtype	18
pd.Series.dtypes	113
pd.Series.duplicated	118
pd.Series.empty	113
pd.Series.eq	45
pd.Series.equals	118
pd.Series.ewm	76
pd.Series.expanding	76
pd.Series.explode	120
pd.Series.factorize	117
pd.Series.ffill	119
pd.Series.fillna	119
pd.Series.filter	119
pd.Series.first_valid_index	120
pd.Series.flags	113
pd.Series.floordiv	44
pd.Series.from_arrow	127
pd.Series.ge	46
pd.Series.get	114
pd.Series.groupby	116
pd.Series.gt	46
pd.Series.hasnans	34
pd.Series.head	118
pd.Series.hist	127
pd.Series.iat	114
pd.Series.idxmax	39
pd.Series.idxmin	39
pd.Series.iloc	114
pd.Series.index	20
pd.Series.infer_objects	114

pd.Series.info	113
pd.Series.interpolate	34
pd.Series.is_monotonic_decreasing	118
pd.Series.is_monotonic_increasing	118
pd.Series.is_unique	117
pd.Series.isin	23
pd.Series.isna	119
pd.Series.isnull	119
pd.Series.item	31
pd.Series.items	114
pd.Series.keys	114
pd.Series.kurt	43
pd.Series.kurtosis	117
pd.Series.last_valid_index	120
pd.Series.le	46
pd.Series.list.flatten	126
pd.Series.list.len	126
pd.Series.loc	114
pd.Series.lt	46
pd.Series.map	88
pd.Series.mask	119
pd.Series.max	38
pd.Series.mean	37
pd.Series.median	37
pd.Series.memory_usage	113
pd.Series.min	37
pd.Series.mod	45
pd.Series.mode	43
pd.Series.mul	44
pd.Series.name	19
pd.Series.nbytes	113
pd.Series.ndim	113
pd.Series.ne	45
pd.Series.nlargest	117
pd.Series.notna	119
pd.Series.notnull	119
pd.Series.nsmallest	117
pd.Series.nunique	19
pd.Series.pct_change	40
pd.Series.pipe	116
pd.Series.plot	127
pd.Series.plot.area	127
pd.Series.plot.bar	127
pd.Series.plot.barh	127
pd.Series.plot.box	127
pd.Series.plot.density	127
pd.Series.plot.hist	127
pd.Series.plot.kde	127

pd.Series.plot.line	127
pd.Series.plot.pie	127
pd.Series.pop	114
pd.Series.pow	45
pd.Series.prod	42
pd.Series.product	115
pd.Series.quantile	38
pd.Series.radd	115
pd.Series.rank	36
pd.Series.rdiv	115
pd.Series.reindex	118
pd.Series.reindex_like	118
pd.Series.rename	118
pd.Series.rename_axis	118
pd.Series.reorder_levels	120
pd.Series.repeat	120
pd.Series.replace	119
pd.Series.resample	70
pd.Series.reset_index	118
pd.Series.rfloordiv	115
pd.Series.rmod	115
pd.Series.rmul	115
pd.Series.rolling	75
pd.Series.round	40
pd.Series.rpow	115
pd.Series.rsub	115
pd.Series.rtruediv	115
pd.Series.sample	118
pd.Series.searchsorted	47
pd.Series.sem	43
pd.Series.set_axis	119
pd.Series.set_flags	113
pd.Series.shape	20
pd.Series.shift	71
pd.Series.size	113
pd.Series.skew	43
pd.Series.sort_index	120
pd.Series.sort_values	120
pd.Series.sparse	121
pd.Series.sparse.density	126
pd.Series.sparse.fill_value	126
pd.Series.sparse.from_coo	126
pd.Series.sparse.npoints	126
pd.Series.sparse.sp_values	126
pd.Series.sparse.to_coo	126
pd.Series.squeeze	120
pd.Series.std	38
pd.Series.str	121

pd.Series.str.capitalize	123
pd.Series.str.casefold	64
pd.Series.str.cat	63
pd.Series.str.center	123
pd.Series.str.contains	61
pd.Series.str.count	64
pd.Series.str.decode	123
pd.Series.str.encode	123
pd.Series.str.endswith	61
pd.Series.str.extract	62
pd.Series.str.extractall	62
pd.Series.str.find	64
pd.Series.str.findall	124
pd.Series.str.fullmatch	63
pd.Series.str.get	65
pd.Series.str.get_dummies	65
pd.Series.str.index	124
pd.Series.str.isalnum	125
pd.Series.str.isalpha	125
pd.Series.str.isascii	124
pd.Series.str.isdecimal	125
pd.Series.str.isdigit	125
pd.Series.str.islower	125
pd.Series.str.isnumeric	125
pd.Series.str.isspace	125
pd.Series.str.istitle	125
pd.Series.str.isupper	125
pd.Series.str.join	65
pd.Series.str.len	62
pd.Series.str.ljust	124
pd.Series.str.lower	60
pd.Series.str.lstrip	124
pd.Series.str.match	124
pd.Series.str.normalize	63
pd.Series.str.pad	124
pd.Series.str.partition	124
pd.Series.str.removeprefix	124
pd.Series.str.removesuffix	124
pd.Series.str.repeat	124
pd.Series.str.replace	61
pd.Series.str.rfind	124
pd.Series.str.rindex	124
pd.Series.str.rjust	124
pd.Series.str.rpartition	124
pd.Series.str.rsplit	125
pd.Series.str.rstrip	125
pd.Series.str.slice	63
pd.Series.str.slice_replace	125

pd.Series.str.split	62
pd.Series.str.startswith	61
pd.Series.str.strip	60
pd.Series.str.swapcase	125
pd.Series.str.title	64
pd.Series.str.translate	125
pd.Series.str.upper	60
pd.Series.str.wrap	125
pd.Series.str.zfill	65
pd.Series.struct.dtypes	126
pd.Series.struct.explode	126
pd.Series.struct.field	126
pd.Series.sub	44
pd.Series.sum	37
pd.Series.swaplevel	120
pd.Series.T	113
pd.Series.tail	119
pd.Series.take	119
pd.Series.to_clipboard	128
pd.Series.to_csv	127
pd.Series.to_dict	127
pd.Series.to_excel	127
pd.Series.to_frame	127
pd.Series.to_hdf	127
pd.Series.to_json	128
pd.Series.to_latex	128
pd.Series.to_list	114
pd.Series.to_markdown	128
pd.Series.to_numpy	114
pd.Series.to_period	114
pd.Series.to_pickle	127
pd.Series.to_sql	127
pd.Series.to_string	128
pd.Series.to_timestamp	114
pd.Series.to_xarray	127
pd.Series.tolist	31
pd.Series.transform	116
pd.Series.truediv	114
pd.Series.truncate	119
pd.Series.tz_convert	120
pd.Series.tz_localize	120
pd.Series.unique	19
pd.Series.unstack	120
pd.Series.update	120
pd.Series.value_counts	19
pd.Series.values	113
pd.Series.var	38
pd.Series.where	119

pd.Series.xs	114
pd.set_eng_float_format	194
pd.set_option	93
pd.show_versions	198
pd.SparseDtype	151
pd.StringDtype	151
pd.test	198
pd.testing.assert_extension_array_equal	196
pd.testing.assert_frame_equal	94
pd.testing.assert_index_equal	196
pd.testing.assert_series_equal	196
pd.Timedelta	67
pd.Timedelta.as_unit	155
pd.Timedelta.asm8	155
pd.Timedelta.ceil	155
pd.Timedelta.components	155
pd.Timedelta.days	155
pd.Timedelta.floor	156
pd.Timedelta.isoformat	156
pd.Timedelta.max	155
pd.Timedelta.microseconds	155
pd.Timedelta.min	155
pd.Timedelta.nanoseconds	155
pd.Timedelta.resolution	155
pd.Timedelta.round	156
pd.Timedelta.seconds	155
pd.Timedelta.to_numpy	156
pd.Timedelta.to_pytimedelta	156
pd.Timedelta.to_timedelta64	156
pd.Timedelta.total_seconds	156
pd.Timedelta.unit	155
pd.Timedelta.value	155
pd.Timedelta.view	155
pd.timedelta_range	66
pd.TimedeltaIndex	137
pd.TimedeltaIndex.as_unit	138
pd.TimedeltaIndex.ceil	138
pd.TimedeltaIndex.components	137
pd.TimedeltaIndex.days	137
pd.TimedeltaIndex.floor	138
pd.TimedeltaIndex.inferred_freq	137
pd.TimedeltaIndex.mean	138
pd.TimedeltaIndex.microseconds	137
pd.TimedeltaIndex.nanoseconds	137
pd.TimedeltaIndex.round	138
pd.TimedeltaIndex.seconds	137
pd.TimedeltaIndex.to_frame	138
pd.TimedeltaIndex.to_pytimedelta	138

pd.TimedeltaIndex.to_series	138
pd.Timestamp	67
pd.Timestamp.as_unit	153
pd.Timestamp.asm8	152
pd.Timestamp.astimezone	153
pd.Timestamp.ceil	153
pd.Timestamp.combine	153
pd.Timestamp.ctime	153
pd.Timestamp.date	153
pd.Timestamp.day	152
pd.Timestamp.day_name	153
pd.Timestamp.day_of_week	152
pd.Timestamp.day_of_year	152
pd.Timestamp.dayofweek	152
pd.Timestamp.dayofyear	152
pd.Timestamp.days_in_month	152
pd.Timestamp.daysinmonth	152
pd.Timestamp.dst	153
pd.Timestamp.floor	153
pd.Timestamp.fold	152
pd.Timestamp.fromordinal	154
pd.Timestamp.fromtimestamp	154
pd.Timestamp.hour	152
pd.Timestamp.is_leap_year	152
pd.Timestamp.is_month_end	152
pd.Timestamp.is_month_start	152
pd.Timestamp.is_quarter_end	152
pd.Timestamp.is_quarter_start	152
pd.Timestamp.is_year_end	152
pd.Timestamp.is_year_start	152
pd.Timestamp.isocalendar	154
pd.Timestamp.isoformat	154
pd.Timestamp.isoweekday	154
pd.Timestamp.max	152
pd.Timestamp.microsecond	152
pd.Timestamp.min	153
pd.Timestamp.minute	153
pd.Timestamp.month	153
pd.Timestamp.month_name	154
pd.Timestamp.nanosecond	153
pd.Timestamp.normalize	154
pd.Timestamp.now	154
pd.Timestamp.quarter	153
pd.Timestamp.replace	154
pd.Timestamp.resolution	153
pd.Timestamp.round	154
pd.Timestamp.second	153
pd.Timestamp.strftime	154

pd.Timestamp.strptime	154
pd.Timestamp.time	154
pd.Timestamp.timestamp	154
pd.Timestamp.timetuple	154
pd.Timestamp.timetz	154
pd.Timestamp.to_datetime64	154
pd.Timestamp.to_julian_date	154
pd.Timestamp.to_numpy	154
pd.Timestamp.to_period	154
pd.Timestamp.to_pydatetime	154
pd.Timestamp.today	154
pd.Timestamp.toordinal	154
pd.Timestamp.tz	153
pd.Timestamp.tz_convert	155
pd.Timestamp.tz_localize	155
pd.Timestamp.tzinfo	153
pd.Timestamp.tzname	155
pd.Timestamp.unit	153
pd.Timestamp.utcfromtimestamp	155
pd.Timestamp.utcnow	155
pd.Timestamp.utcoffset	155
pd.Timestamp.utctimetuple	155
pd.Timestamp.value	153
pd.Timestamp.week	153
pd.Timestamp.weekday	155
pd.Timestamp.weekofyear	153
pd.Timestamp.year	153
pd.to_datetime	30
pd.to_numeric	30
pd.to_timedelta	30
pd.tseries.api.guess_datetime_format	102
pd.tseries.frequencies.to_offset	190
pd.tseries.offsets.BDay	161
pd.tseries.offsets.BHalfYearBegin	177
pd.tseries.offsets.BHalfYearBegin.copy	177
pd.tseries.offsets.BHalfYearBegin.freqstr	177
pd.tseries.offsets.BHalfYearBegin.is_month_end	178
pd.tseries.offsets.BHalfYearBegin.is_month_start	178
pd.tseries.offsets.BHalfYearBegin.is_on_offset	178
pd.tseries.offsets.BHalfYearBegin.is_quarter_end	178
pd.tseries.offsets.BHalfYearBegin.is_quarter_start	178
pd.tseries.offsets.BHalfYearBegin.is_year_end	178
pd.tseries.offsets.BHalfYearBegin.is_year_start	178
pd.tseries.offsets.BHalfYearBegin.kwds	177
pd.tseries.offsets.BHalfYearBegin.n	177
pd.tseries.offsets.BHalfYearBegin.name	177
pd.tseries.offsets.BHalfYearBegin.nanos	177
pd.tseries.offsets.BHalfYearBegin.normalize	177

pd.tseries.offsets.BHalfYearBegin.rule_code	177
pd.tseries.offsets.BHalfYearBegin.startingMonth	177
pd.tseries.offsets.BHalfYearEnd	176
pd.tseries.offsets.BHalfYearEnd.copy	177
pd.tseries.offsets.BHalfYearEnd.freqstr	176
pd.tseries.offsets.BHalfYearEnd.is_month_end	177
pd.tseries.offsets.BHalfYearEnd.is_month_start	177
pd.tseries.offsets.BHalfYearEnd.is_on_offset	177
pd.tseries.offsets.BHalfYearEnd.is_quarter_end	177
pd.tseries.offsets.BHalfYearEnd.is_quarter_start	177
pd.tseries.offsets.BHalfYearEnd.is_year_end	177
pd.tseries.offsets.BHalfYearEnd.is_year_start	177
pd.tseries.offsets.BHalfYearEnd.kwds	176
pd.tseries.offsets.BHalfYearEnd.n	177
pd.tseries.offsets.BHalfYearEnd.name	176
pd.tseries.offsets.BHalfYearEnd.nanos	177
pd.tseries.offsets.BHalfYearEnd.normalize	177
pd.tseries.offsets.BHalfYearEnd.rule_code	177
pd.tseries.offsets.BHalfYearEnd.startingMonth	177
pd.tseries.offsets.BMonthBegin	167
pd.tseries.offsets.BMonthEnd	166
pd.tseries.offsets.BQuarterBegin	174
pd.tseries.offsets.BQuarterBegin.copy	175
pd.tseries.offsets.BQuarterBegin.freqstr	174
pd.tseries.offsets.BQuarterBegin.is_month_end	175
pd.tseries.offsets.BQuarterBegin.is_month_start	175
pd.tseries.offsets.BQuarterBegin.is_on_offset	175
pd.tseries.offsets.BQuarterBegin.is_quarter_end	175
pd.tseries.offsets.BQuarterBegin.is_quarter_start	175
pd.tseries.offsets.BQuarterBegin.is_year_end	175
pd.tseries.offsets.BQuarterBegin.is_year_start	175
pd.tseries.offsets.BQuarterBegin.kwds	174
pd.tseries.offsets.BQuarterBegin.n	174
pd.tseries.offsets.BQuarterBegin.name	174
pd.tseries.offsets.BQuarterBegin.nanos	174
pd.tseries.offsets.BQuarterBegin.normalize	174
pd.tseries.offsets.BQuarterBegin.rule_code	174
pd.tseries.offsets.BQuarterBegin.startingMonth	174
pd.tseries.offsets.BQuarterEnd	173
pd.tseries.offsets.BQuarterEnd.copy	174
pd.tseries.offsets.BQuarterEnd.freqstr	173
pd.tseries.offsets.BQuarterEnd.is_month_end	174
pd.tseries.offsets.BQuarterEnd.is_month_start	174
pd.tseries.offsets.BQuarterEnd.is_on_offset	174
pd.tseries.offsets.BQuarterEnd.is_quarter_end	174
pd.tseries.offsets.BQuarterEnd.is_quarter_start	174
pd.tseries.offsets.BQuarterEnd.is_year_end	174
pd.tseries.offsets.BQuarterEnd.is_year_start	174

pd.tseries.offsets.BQuarterEnd.kwds	173
pd.tseries.offsets.BQuarterEnd.n	174
pd.tseries.offsets.BQuarterEnd.name	174
pd.tseries.offsets.BQuarterEnd.nanos	174
pd.tseries.offsets.BQuarterEnd.normalize	174
pd.tseries.offsets.BQuarterEnd.rule_code	174
pd.tseries.offsets.BQuarterEnd.startingMonth	174
pd.tseries.offsets.BusinessDay	161
pd.tseries.offsets.BusinessDay.calendar	162
pd.tseries.offsets.BusinessDay.copy	162
pd.tseries.offsets.BusinessDay.freqstr	161
pd.tseries.offsets.BusinessDay.holidays	162
pd.tseries.offsets.BusinessDay.is_month_end	162
pd.tseries.offsets.BusinessDay.is_month_start	162
pd.tseries.offsets.BusinessDay.is_on_offset	162
pd.tseries.offsets.BusinessDay.is_quarter_end	162
pd.tseries.offsets.BusinessDay.is_quarter_start	162
pd.tseries.offsets.BusinessDay.is_year_end	162
pd.tseries.offsets.BusinessDay.is_year_start	162
pd.tseries.offsets.BusinessDay.kwds	161
pd.tseries.offsets.BusinessDay.n	162
pd.tseries.offsets.BusinessDay.name	162
pd.tseries.offsets.BusinessDay.nanos	162
pd.tseries.offsets.BusinessDay.normalize	162
pd.tseries.offsets.BusinessDay.rule_code	162
pd.tseries.offsets.BusinessDay.weekmask	162
pd.tseries.offsets.BusinessHour	162
pd.tseries.offsets.BusinessHour.calendar	163
pd.tseries.offsets.BusinessHour.copy	163
pd.tseries.offsets.BusinessHour.end	163
pd.tseries.offsets.BusinessHour.freqstr	162
pd.tseries.offsets.BusinessHour.holidays	163
pd.tseries.offsets.BusinessHour.is_month_end	163
pd.tseries.offsets.BusinessHour.is_month_start	163
pd.tseries.offsets.BusinessHour.is_on_offset	163
pd.tseries.offsets.BusinessHour.is_quarter_end	163
pd.tseries.offsets.BusinessHour.is_quarter_start	163
pd.tseries.offsets.BusinessHour.is_year_end	163
pd.tseries.offsets.BusinessHour.is_year_start	163
pd.tseries.offsets.BusinessHour.kwds	162
pd.tseries.offsets.BusinessHour.n	163
pd.tseries.offsets.BusinessHour.name	162
pd.tseries.offsets.BusinessHour.nanos	162
pd.tseries.offsets.BusinessHour.normalize	162
pd.tseries.offsets.BusinessHour.rule_code	162
pd.tseries.offsets.BusinessHour.start	163
pd.tseries.offsets.BusinessHour.weekmask	163
pd.tseries.offsets.BusinessMonthBegin	167

pd.tseries.offsets.BusinessMonthBegin.copy	167
pd.tseries.offsets.BusinessMonthBegin.freqstr	167
pd.tseries.offsets.BusinessMonthBegin.is_month_end	168
pd.tseries.offsets.BusinessMonthBegin.is_month_start	168
pd.tseries.offsets.BusinessMonthBegin.is_on_offset	167
pd.tseries.offsets.BusinessMonthBegin.is_quarter_end	168
pd.tseries.offsets.BusinessMonthBegin.is_quarter_start	168
pd.tseries.offsets.BusinessMonthBegin.is_year_end	168
pd.tseries.offsets.BusinessMonthBegin.is_year_start	168
pd.tseries.offsets.BusinessMonthBegin.kwds	167
pd.tseries.offsets.BusinessMonthBegin.n	167
pd.tseries.offsets.BusinessMonthBegin.name	167
pd.tseries.offsets.BusinessMonthBegin.nanos	167
pd.tseries.offsets.BusinessMonthBegin.normalize	167
pd.tseries.offsets.BusinessMonthBegin.rule_code	167
pd.tseries.offsets.BusinessMonthEnd	166
pd.tseries.offsets.BusinessMonthEnd.copy	167
pd.tseries.offsets.BusinessMonthEnd.freqstr	166
pd.tseries.offsets.BusinessMonthEnd.is_month_end	167
pd.tseries.offsets.BusinessMonthEnd.is_month_start	167
pd.tseries.offsets.BusinessMonthEnd.is_on_offset	167
pd.tseries.offsets.BusinessMonthEnd.is_quarter_end	167
pd.tseries.offsets.BusinessMonthEnd.is_quarter_start	167
pd.tseries.offsets.BusinessMonthEnd.is_year_end	167
pd.tseries.offsets.BusinessMonthEnd.is_year_start	167
pd.tseries.offsets.BusinessMonthEnd.kwds	166
pd.tseries.offsets.BusinessMonthEnd.n	167
pd.tseries.offsets.BusinessMonthEnd.name	166
pd.tseries.offsets.BusinessMonthEnd.nanos	167
pd.tseries.offsets.BusinessMonthEnd.normalize	167
pd.tseries.offsets.BusinessMonthEnd.rule_code	167
pd.tseries.offsets.BYearBegin	180
pd.tseries.offsets.BYearBegin.copy	180
pd.tseries.offsets.BYearBegin.freqstr	180
pd.tseries.offsets.BYearBegin.is_month_end	181
pd.tseries.offsets.BYearBegin.is_month_start	181
pd.tseries.offsets.BYearBegin.is_on_offset	180
pd.tseries.offsets.BYearBegin.is_quarter_end	181
pd.tseries.offsets.BYearBegin.is_quarter_start	181
pd.tseries.offsets.BYearBegin.is_year_end	181
pd.tseries.offsets.BYearBegin.is_year_start	181
pd.tseries.offsets.BYearBegin.kwds	180
pd.tseries.offsets.BYearBegin.month	180
pd.tseries.offsets.BYearBegin.n	180
pd.tseries.offsets.BYearBegin.name	180
pd.tseries.offsets.BYearBegin.nanos	180
pd.tseries.offsets.BYearBegin.normalize	180
pd.tseries.offsets.BYearBegin.rule_code	180

pd.tseries.offsets.BYearEnd	179
pd.tseries.offsets.BYearEnd.copy	180
pd.tseries.offsets.BYearEnd.freqstr	179
pd.tseries.offsets.BYearEnd.is_month_end	180
pd.tseries.offsets.BYearEnd.is_month_start	180
pd.tseries.offsets.BYearEnd.is_on_offset	180
pd.tseries.offsets.BYearEnd.is_quarter_end	180
pd.tseries.offsets.BYearEnd.is_quarter_start	180
pd.tseries.offsets.BYearEnd.is_year_end	180
pd.tseries.offsets.BYearEnd.is_year_start	180
pd.tseries.offsets.BYearEnd.kwds	179
pd.tseries.offsets.BYearEnd.month	180
pd.tseries.offsets.BYearEnd.n	180
pd.tseries.offsets.BYearEnd.name	179
pd.tseries.offsets.BYearEnd.nanos	179
pd.tseries.offsets.BYearEnd.normalize	180
pd.tseries.offsets.BYearEnd.rule_code	180
pd.tseries.offsets.CBMonthBegin	169
pd.tseries.offsets.CBMonthEnd	168
pd.tseries.offsets.CDay	163
pd.tseries.offsets.CustomBusinessDay	163
pd.tseries.offsets.CustomBusinessDay.calendar	164
pd.tseries.offsets.CustomBusinessDay.copy	164
pd.tseries.offsets.CustomBusinessDay.freqstr	163
pd.tseries.offsets.CustomBusinessDay.holidays	164
pd.tseries.offsets.CustomBusinessDay.is_month_end	164
pd.tseries.offsets.CustomBusinessDay.is_month_start	164
pd.tseries.offsets.CustomBusinessDay.is_on_offset	164
pd.tseries.offsets.CustomBusinessDay.is_quarter_end	164
pd.tseries.offsets.CustomBusinessDay.is_quarter_start	164
pd.tseries.offsets.CustomBusinessDay.is_year_end	164
pd.tseries.offsets.CustomBusinessDay.is_year_start	164
pd.tseries.offsets.CustomBusinessDay.kwds	163
pd.tseries.offsets.CustomBusinessDay.n	163
pd.tseries.offsets.CustomBusinessDay.name	163
pd.tseries.offsets.CustomBusinessDay.nanos	163
pd.tseries.offsets.CustomBusinessDay.normalize	163
pd.tseries.offsets.CustomBusinessDay.rule_code	163
pd.tseries.offsets.CustomBusinessDay.weekmask	164
pd.tseries.offsets.CustomBusinessHour	164
pd.tseries.offsets.CustomBusinessHour.calendar	164
pd.tseries.offsets.CustomBusinessHour.copy	165
pd.tseries.offsets.CustomBusinessHour.end	165
pd.tseries.offsets.CustomBusinessHour.freqstr	164
pd.tseries.offsets.CustomBusinessHour.holidays	164
pd.tseries.offsets.CustomBusinessHour.is_month_end	165
pd.tseries.offsets.CustomBusinessHour.is_month_start	165
pd.tseries.offsets.CustomBusinessHour.is_on_offset	165

pd.tseries.offsets.CustomBusinessHour.is_quarter_end	165
pd.tseries.offsets.CustomBusinessHour.is_quarter_start	165
pd.tseries.offsets.CustomBusinessHour.is_year_end	165
pd.tseries.offsets.CustomBusinessHour.is_year_start	165
pd.tseries.offsets.CustomBusinessHour.kwds	164
pd.tseries.offsets.CustomBusinessHour.n	164
pd.tseries.offsets.CustomBusinessHour.name	164
pd.tseries.offsets.CustomBusinessHour.nanos	164
pd.tseries.offsets.CustomBusinessHour.normalize	164
pd.tseries.offsets.CustomBusinessHour.rule_code	164
pd.tseries.offsets.CustomBusinessHour.start	164
pd.tseries.offsets.CustomBusinessHour.weekmask	164
pd.tseries.offsets.CustomBusinessMonthBegin	169
pd.tseries.offsets.CustomBusinessMonthBegin.calendar	169
pd.tseries.offsets.CustomBusinessMonthBegin.copy	169
pd.tseries.offsets.CustomBusinessMonthBegin.freqstr	169
pd.tseries.offsets.CustomBusinessMonthBegin.holidays	169
pd.tseries.offsets.CustomBusinessMonthBegin.is_month_end	169
pd.tseries.offsets.CustomBusinessMonthBegin.is_month_start	169
pd.tseries.offsets.CustomBusinessMonthBegin.is_on_offset	169
pd.tseries.offsets.CustomBusinessMonthBegin.is_quarter_end	169
pd.tseries.offsets.CustomBusinessMonthBegin.is_quarter_start	169
pd.tseries.offsets.CustomBusinessMonthBegin.is_year_end	170
pd.tseries.offsets.CustomBusinessMonthBegin.is_year_start	170
pd.tseries.offsets.CustomBusinessMonthBegin.kwds	169
pd.tseries.offsets.CustomBusinessMonthBegin.m_offset	169
pd.tseries.offsets.CustomBusinessMonthBegin.n	169
pd.tseries.offsets.CustomBusinessMonthBegin.name	169
pd.tseries.offsets.CustomBusinessMonthBegin.nanos	169
pd.tseries.offsets.CustomBusinessMonthBegin.normalize	169
pd.tseries.offsets.CustomBusinessMonthBegin.rule_code	169
pd.tseries.offsets.CustomBusinessMonthBegin.weekmask	169
pd.tseries.offsets.CustomBusinessMonthEnd	168
pd.tseries.offsets.CustomBusinessMonthEnd.calendar	168
pd.tseries.offsets.CustomBusinessMonthEnd.copy	168
pd.tseries.offsets.CustomBusinessMonthEnd.freqstr	168
pd.tseries.offsets.CustomBusinessMonthEnd.holidays	168
pd.tseries.offsets.CustomBusinessMonthEnd.is_month_end	168
pd.tseries.offsets.CustomBusinessMonthEnd.is_month_start	168
pd.tseries.offsets.CustomBusinessMonthEnd.is_on_offset	168
pd.tseries.offsets.CustomBusinessMonthEnd.is_quarter_end	169
pd.tseries.offsets.CustomBusinessMonthEnd.is_quarter_start	169
pd.tseries.offsets.CustomBusinessMonthEnd.is_year_end	169
pd.tseries.offsets.CustomBusinessMonthEnd.is_year_start	169
pd.tseries.offsets.CustomBusinessMonthEnd.kwds	168
pd.tseries.offsets.CustomBusinessMonthEnd.m_offset	168
pd.tseries.offsets.CustomBusinessMonthEnd.n	168
pd.tseries.offsets.CustomBusinessMonthEnd.name	168

pd.tseries.offsets.CustomBusinessMonthEnd.nanos	168
pd.tseries.offsets.CustomBusinessMonthEnd.normalize	168
pd.tseries.offsets.CustomBusinessMonthEnd.rule_code	168
pd.tseries.offsets.CustomBusinessMonthEnd.weekmask	168
pd.tseries.offsets.DateOffset	161
pd.tseries.offsets.DateOffset.copy	161
pd.tseries.offsets.DateOffset.freqstr	161
pd.tseries.offsets.DateOffset.is_month_end	161
pd.tseries.offsets.DateOffset.is_month_start	161
pd.tseries.offsets.DateOffset.is_on_offset	161
pd.tseries.offsets.DateOffset.is_quarter_end	161
pd.tseries.offsets.DateOffset.is_quarter_start	161
pd.tseries.offsets.DateOffset.is_year_end	161
pd.tseries.offsets.DateOffset.is_year_start	161
pd.tseries.offsets.DateOffset.kwds	161
pd.tseries.offsets.DateOffset.n	161
pd.tseries.offsets.DateOffset.name	161
pd.tseries.offsets.DateOffset.nanos	161
pd.tseries.offsets.DateOffset.normalize	161
pd.tseries.offsets.DateOffset.rollback	161
pd.tseries.offsets.DateOffset.rollforward	161
pd.tseries.offsets.DateOffset.rule_code	161
pd.tseries.offsets.Day	186
pd.tseries.offsets.Day.copy	186
pd.tseries.offsets.Day.freqstr	186
pd.tseries.offsets.Day.is_month_end	186
pd.tseries.offsets.Day.is_month_start	186
pd.tseries.offsets.Day.is_on_offset	186
pd.tseries.offsets.Day.is_quarter_end	186
pd.tseries.offsets.Day.is_quarter_start	186
pd.tseries.offsets.Day.is_year_end	186
pd.tseries.offsets.Day.is_year_start	186
pd.tseries.offsets.Day.kwds	186
pd.tseries.offsets.Day.n	186
pd.tseries.offsets.Day.name	186
pd.tseries.offsets.Day.nanos	186
pd.tseries.offsets.Day.normalize	186
pd.tseries.offsets.Day.rule_code	186
pd.tseries.offsets.Easter	184
pd.tseries.offsets.Easter.copy	185
pd.tseries.offsets.Easter.freqstr	184
pd.tseries.offsets.Easter.is_month_end	185
pd.tseries.offsets.Easter.is_month_start	185
pd.tseries.offsets.Easter.is_on_offset	185
pd.tseries.offsets.Easter.is_quarter_end	185
pd.tseries.offsets.Easter.is_quarter_start	185
pd.tseries.offsets.Easter.is_year_end	185
pd.tseries.offsets.Easter.is_year_start	185

pd.tseries.offsets.Easter.kwds	184
pd.tseries.offsets.Easter.n	184
pd.tseries.offsets.Easter.name	184
pd.tseries.offsets.Easter.nanos	184
pd.tseries.offsets.Easter.normalize	184
pd.tseries.offsets.Easter.rule_code	184
pd.tseries.offsets.FY5253	182
pd.tseries.offsets.FY5253.copy	183
pd.tseries.offsets.FY5253.freqstr	182
pd.tseries.offsets.FY5253.get_rule_code_suffix	183
pd.tseries.offsets.FY5253.get_year_end	183
pd.tseries.offsets.FY5253.is_month_end	183
pd.tseries.offsets.FY5253.is_month_start	183
pd.tseries.offsets.FY5253.is_on_offset	183
pd.tseries.offsets.FY5253.is_quarter_end	183
pd.tseries.offsets.FY5253.is_quarter_start	183
pd.tseries.offsets.FY5253.is_year_end	183
pd.tseries.offsets.FY5253.is_year_start	183
pd.tseries.offsets.FY5253.kwds	182
pd.tseries.offsets.FY5253.n	183
pd.tseries.offsets.FY5253.name	182
pd.tseries.offsets.FY5253.nanos	182
pd.tseries.offsets.FY5253.normalize	182
pd.tseries.offsets.FY5253.rule_code	183
pd.tseries.offsets.FY5253.startingMonth	183
pd.tseries.offsets.FY5253.variation	183
pd.tseries.offsets.FY5253.weekday	183
pd.tseries.offsets.FY5253Quarter	183
pd.tseries.offsets.FY5253Quarter.copy	184
pd.tseries.offsets.FY5253Quarter.freqstr	183
pd.tseries.offsets.FY5253Quarter.get_rule_code_suffix	184
pd.tseries.offsets.FY5253Quarter.get_weeks	184
pd.tseries.offsets.FY5253Quarter.is_month_end	184
pd.tseries.offsets.FY5253Quarter.is_month_start	184
pd.tseries.offsets.FY5253Quarter.is_on_offset	184
pd.tseries.offsets.FY5253Quarter.is_quarter_end	184
pd.tseries.offsets.FY5253Quarter.is_quarter_start	184
pd.tseries.offsets.FY5253Quarter.is_year_end	184
pd.tseries.offsets.FY5253Quarter.is_year_start	184
pd.tseries.offsets.FY5253Quarter.kwds	183
pd.tseries.offsets.FY5253Quarter.n	183
pd.tseries.offsets.FY5253Quarter.name	183
pd.tseries.offsets.FY5253Quarter.nanos	183
pd.tseries.offsets.FY5253Quarter.normalize	183
pd.tseries.offsets.FY5253Quarter.qtr_with_extra_week	184
pd.tseries.offsets.FY5253Quarter.rule_code	183
pd.tseries.offsets.FY5253Quarter.startingMonth	184
pd.tseries.offsets.FY5253Quarter.variation	184

pd.tseries.offsets.FY5253Quarter.weekday	184
pd.tseries.offsets.FY5253Quarter.year_has_extra_week	184
pd.tseries.offsets.HalfYearBegin	179
pd.tseries.offsets.HalfYearBegin.copy	179
pd.tseries.offsets.HalfYearBegin.freqstr	179
pd.tseries.offsets.HalfYearBegin.is_month_end	179
pd.tseries.offsets.HalfYearBegin.is_month_start	179
pd.tseries.offsets.HalfYearBegin.is_on_offset	179
pd.tseries.offsets.HalfYearBegin.is_quarter_end	179
pd.tseries.offsets.HalfYearBegin.is_quarter_start	179
pd.tseries.offsets.HalfYearBegin.is_year_end	179
pd.tseries.offsets.HalfYearBegin.is_year_start	179
pd.tseries.offsets.HalfYearBegin.kwds	179
pd.tseries.offsets.HalfYearBegin.n	179
pd.tseries.offsets.HalfYearBegin.name	179
pd.tseries.offsets.HalfYearBegin.nanos	179
pd.tseries.offsets.HalfYearBegin.normalize	179
pd.tseries.offsets.HalfYearBegin.rule_code	179
pd.tseries.offsets.HalfYearBegin.startingMonth	179
pd.tseries.offsets.HalfYearEnd	178
pd.tseries.offsets.HalfYearEnd.copy	178
pd.tseries.offsets.HalfYearEnd.freqstr	178
pd.tseries.offsets.HalfYearEnd.is_month_end	178
pd.tseries.offsets.HalfYearEnd.is_month_start	178
pd.tseries.offsets.HalfYearEnd.is_on_offset	178
pd.tseries.offsets.HalfYearEnd.is_quarter_end	178
pd.tseries.offsets.HalfYearEnd.is_quarter_start	178
pd.tseries.offsets.HalfYearEnd.is_year_end	179
pd.tseries.offsets.HalfYearEnd.is_year_start	178
pd.tseries.offsets.HalfYearEnd.kwds	178
pd.tseries.offsets.HalfYearEnd.n	178
pd.tseries.offsets.HalfYearEnd.name	178
pd.tseries.offsets.HalfYearEnd.nanos	178
pd.tseries.offsets.HalfYearEnd.normalize	178
pd.tseries.offsets.HalfYearEnd.rule_code	178
pd.tseries.offsets.HalfYearEnd.startingMonth	178
pd.tseries.offsets.Hour	186
pd.tseries.offsets.Hour.copy	187
pd.tseries.offsets.Hour.freqstr	186
pd.tseries.offsets.Hour.is_month_end	187
pd.tseries.offsets.Hour.is_month_start	187
pd.tseries.offsets.Hour.is_on_offset	187
pd.tseries.offsets.Hour.is_quarter_end	187
pd.tseries.offsets.Hour.is_quarter_start	187
pd.tseries.offsets.Hour.is_year_end	187
pd.tseries.offsets.Hour.is_year_start	187
pd.tseries.offsets.Hour.kwds	186
pd.tseries.offsets.Hour.n	187

pd.tseries.offsets.Hour.name	186
pd.tseries.offsets.Hour.nanos	186
pd.tseries.offsets.Hour.normalize	186
pd.tseries.offsets.Hour.rule_code	187
pd.tseries.offsets.LastWeekOfMonth	173
pd.tseries.offsets.LastWeekOfMonth.copy	173
pd.tseries.offsets.LastWeekOfMonth.freqstr	173
pd.tseries.offsets.LastWeekOfMonth.is_month_end	173
pd.tseries.offsets.LastWeekOfMonth.is_month_start	173
pd.tseries.offsets.LastWeekOfMonth.is_on_offset	173
pd.tseries.offsets.LastWeekOfMonth.is_quarter_end	173
pd.tseries.offsets.LastWeekOfMonth.is_quarter_start	173
pd.tseries.offsets.LastWeekOfMonth.is_year_end	173
pd.tseries.offsets.LastWeekOfMonth.is_year_start	173
pd.tseries.offsets.LastWeekOfMonth.kwds	173
pd.tseries.offsets.LastWeekOfMonth.n	173
pd.tseries.offsets.LastWeekOfMonth.name	173
pd.tseries.offsets.LastWeekOfMonth.nanos	173
pd.tseries.offsets.LastWeekOfMonth.normalize	173
pd.tseries.offsets.LastWeekOfMonth.rule_code	173
pd.tseries.offsets.LastWeekOfMonth.week	173
pd.tseries.offsets.LastWeekOfMonth.weekday	173
pd.tseries.offsets.Micro	189
pd.tseries.offsets.Micro.copy	189
pd.tseries.offsets.Micro.freqstr	189
pd.tseries.offsets.Micro.is_month_end	190
pd.tseries.offsets.Micro.is_month_start	189
pd.tseries.offsets.Micro.is_on_offset	189
pd.tseries.offsets.Micro.is_quarter_end	190
pd.tseries.offsets.Micro.is_quarter_start	190
pd.tseries.offsets.Micro.is_year_end	190
pd.tseries.offsets.Micro.is_year_start	190
pd.tseries.offsets.Micro.kwds	189
pd.tseries.offsets.Micro.n	189
pd.tseries.offsets.Micro.name	189
pd.tseries.offsets.Micro.nanos	189
pd.tseries.offsets.Micro.normalize	189
pd.tseries.offsets.Micro.rule_code	189
pd.tseries.offsets.Milli	188
pd.tseries.offsets.Milli.copy	189
pd.tseries.offsets.Milli.freqstr	188
pd.tseries.offsets.Milli.is_month_end	189
pd.tseries.offsets.Milli.is_month_start	189
pd.tseries.offsets.Milli.is_on_offset	189
pd.tseries.offsets.Milli.is_quarter_end	189
pd.tseries.offsets.Milli.is_quarter_start	189
pd.tseries.offsets.Milli.is_year_end	189
pd.tseries.offsets.Milli.is_year_start	189

pd.tseries.offsets.Milli.kwds	188
pd.tseries.offsets.Milli.n	189
pd.tseries.offsets.Milli.name	188
pd.tseries.offsets.Milli.nanos	189
pd.tseries.offsets.Milli.normalize	189
pd.tseries.offsets.Milli.rule_code	189
pd.tseries.offsets.Minute	187
pd.tseries.offsets.Minute.copy	187
pd.tseries.offsets.Minute.freqstr	187
pd.tseries.offsets.Minute.is_month_end	187
pd.tseries.offsets.Minute.is_month_start	187
pd.tseries.offsets.Minute.is_on_offset	187
pd.tseries.offsets.Minute.is_quarter_end	188
pd.tseries.offsets.Minute.is_quarter_start	187
pd.tseries.offsets.Minute.is_year_end	188
pd.tseries.offsets.Minute.is_year_start	188
pd.tseries.offsets.Minute.kwds	187
pd.tseries.offsets.Minute.n	187
pd.tseries.offsets.Minute.name	187
pd.tseries.offsets.Minute.nanos	187
pd.tseries.offsets.Minute.normalize	187
pd.tseries.offsets.Minute.rule_code	187
pd.tseries.offsets.MonthBegin	166
pd.tseries.offsets.MonthBegin.copy	166
pd.tseries.offsets.MonthBegin.freqstr	166
pd.tseries.offsets.MonthBegin.is_month_end	166
pd.tseries.offsets.MonthBegin.is_month_start	166
pd.tseries.offsets.MonthBegin.is_on_offset	166
pd.tseries.offsets.MonthBegin.is_quarter_end	166
pd.tseries.offsets.MonthBegin.is_quarter_start	166
pd.tseries.offsets.MonthBegin.is_year_end	166
pd.tseries.offsets.MonthBegin.is_year_start	166
pd.tseries.offsets.MonthBegin.kwds	166
pd.tseries.offsets.MonthBegin.n	166
pd.tseries.offsets.MonthBegin.name	166
pd.tseries.offsets.MonthBegin.nanos	166
pd.tseries.offsets.MonthBegin.normalize	166
pd.tseries.offsets.MonthBegin.rule_code	166
pd.tseries.offsets.MonthEnd	165
pd.tseries.offsets.MonthEnd.copy	165
pd.tseries.offsets.MonthEnd.freqstr	165
pd.tseries.offsets.MonthEnd.is_month_end	165
pd.tseries.offsets.MonthEnd.is_month_start	165
pd.tseries.offsets.MonthEnd.is_on_offset	165
pd.tseries.offsets.MonthEnd.is_quarter_end	165
pd.tseries.offsets.MonthEnd.is_quarter_start	165
pd.tseries.offsets.MonthEnd.is_year_end	166
pd.tseries.offsets.MonthEnd.is_year_start	166

pd.tseries.offsets.MonthEnd.kwds	165
pd.tseries.offsets.MonthEnd.n	165
pd.tseries.offsets.MonthEnd.name	165
pd.tseries.offsets.MonthEnd.nanos	165
pd.tseries.offsets.MonthEnd.normalize	165
pd.tseries.offsets.MonthEnd.rule_code	165
pd.tseries.offsets.Nano	190
pd.tseries.offsets.Nano.copy	190
pd.tseries.offsets.Nano.freqstr	190
pd.tseries.offsets.Nano.is_month_end	190
pd.tseries.offsets.Nano.is_month_start	190
pd.tseries.offsets.Nano.is_on_offset	190
pd.tseries.offsets.Nano.is_quarter_end	190
pd.tseries.offsets.Nano.is_quarter_start	190
pd.tseries.offsets.Nano.is_year_end	190
pd.tseries.offsets.Nano.is_year_start	190
pd.tseries.offsets.Nano.kwds	190
pd.tseries.offsets.Nano.n	190
pd.tseries.offsets.Nano.name	190
pd.tseries.offsets.Nano.nanos	190
pd.tseries.offsets.Nano.normalize	190
pd.tseries.offsets.Nano.rule_code	190
pd.tseries.offsets.QuarterBegin	176
pd.tseries.offsets.QuarterBegin.copy	176
pd.tseries.offsets.QuarterBegin.freqstr	176
pd.tseries.offsets.QuarterBegin.is_month_end	176
pd.tseries.offsets.QuarterBegin.is_month_start	176
pd.tseries.offsets.QuarterBegin.is_on_offset	176
pd.tseries.offsets.QuarterBegin.is_quarter_end	176
pd.tseries.offsets.QuarterBegin.is_quarter_start	176
pd.tseries.offsets.QuarterBegin.is_year_end	176
pd.tseries.offsets.QuarterBegin.is_year_start	176
pd.tseries.offsets.QuarterBegin.kwds	176
pd.tseries.offsets.QuarterBegin.n	176
pd.tseries.offsets.QuarterBegin.name	176
pd.tseries.offsets.QuarterBegin.nanos	176
pd.tseries.offsets.QuarterBegin.normalize	176
pd.tseries.offsets.QuarterBegin.rule_code	176
pd.tseries.offsets.QuarterBegin.startingMonth	176
pd.tseries.offsets.QuarterEnd	175
pd.tseries.offsets.QuarterEnd.copy	175
pd.tseries.offsets.QuarterEnd.freqstr	175
pd.tseries.offsets.QuarterEnd.is_month_end	175
pd.tseries.offsets.QuarterEnd.is_month_start	175
pd.tseries.offsets.QuarterEnd.is_on_offset	175
pd.tseries.offsets.QuarterEnd.is_quarter_end	175
pd.tseries.offsets.QuarterEnd.is_quarter_start	175
pd.tseries.offsets.QuarterEnd.is_year_end	176

pd.tseries.offsets.QuarterEnd.is_year_start	176
pd.tseries.offsets.QuarterEnd.kwds	175
pd.tseries.offsets.QuarterEnd.n	175
pd.tseries.offsets.QuarterEnd.name	175
pd.tseries.offsets.QuarterEnd.nanos	175
pd.tseries.offsets.QuarterEnd.normalize	175
pd.tseries.offsets.QuarterEnd.rule_code	175
pd.tseries.offsets.QuarterEnd.startingMonth	175
pd.tseries.offsets.Second	188
pd.tseries.offsets.Second.copy	188
pd.tseries.offsets.Second.freqstr	188
pd.tseries.offsets.Second.is_month_end	188
pd.tseries.offsets.Second.is_month_start	188
pd.tseries.offsets.Second.is_on_offset	188
pd.tseries.offsets.Second.is_quarter_end	188
pd.tseries.offsets.Second.is_quarter_start	188
pd.tseries.offsets.Second.is_year_end	188
pd.tseries.offsets.Second.is_year_start	188
pd.tseries.offsets.Second.kwds	188
pd.tseries.offsets.Second.n	188
pd.tseries.offsets.Second.name	188
pd.tseries.offsets.Second.nanos	188
pd.tseries.offsets.Second.normalize	188
pd.tseries.offsets.Second.rule_code	188
pd.tseries.offsets.SemiMonthBegin	170
pd.tseries.offsets.SemiMonthBegin.copy	171
pd.tseries.offsets.SemiMonthBegin.day_of_month	171
pd.tseries.offsets.SemiMonthBegin.freqstr	170
pd.tseries.offsets.SemiMonthBegin.is_month_end	171
pd.tseries.offsets.SemiMonthBegin.is_month_start	171
pd.tseries.offsets.SemiMonthBegin.is_on_offset	171
pd.tseries.offsets.SemiMonthBegin.is_quarter_end	171
pd.tseries.offsets.SemiMonthBegin.is_quarter_start	171
pd.tseries.offsets.SemiMonthBegin.is_year_end	171
pd.tseries.offsets.SemiMonthBegin.is_year_start	171
pd.tseries.offsets.SemiMonthBegin.kwds	170
pd.tseries.offsets.SemiMonthBegin.n	171
pd.tseries.offsets.SemiMonthBegin.name	170
pd.tseries.offsets.SemiMonthBegin.nanos	171
pd.tseries.offsets.SemiMonthBegin.normalize	171
pd.tseries.offsets.SemiMonthBegin.rule_code	171
pd.tseries.offsets.SemiMonthEnd	170
pd.tseries.offsets.SemiMonthEnd.copy	170
pd.tseries.offsets.SemiMonthEnd.day_of_month	170
pd.tseries.offsets.SemiMonthEnd.freqstr	170
pd.tseries.offsets.SemiMonthEnd.is_month_end	170
pd.tseries.offsets.SemiMonthEnd.is_month_start	170
pd.tseries.offsets.SemiMonthEnd.is_on_offset	170

pd.tseries.offsets.SemiMonthEnd.is_quarter_end	170
pd.tseries.offsets.SemiMonthEnd.is_quarter_start	170
pd.tseries.offsets.SemiMonthEnd.is_year_end	170
pd.tseries.offsets.SemiMonthEnd.is_year_start	170
pd.tseries.offsets.SemiMonthEnd.kwds	170
pd.tseries.offsets.SemiMonthEnd.n	170
pd.tseries.offsets.SemiMonthEnd.name	170
pd.tseries.offsets.SemiMonthEnd.nanos	170
pd.tseries.offsets.SemiMonthEnd.normalize	170
pd.tseries.offsets.SemiMonthEnd.rule_code	170
pd.tseries.offsets.Tick	185
pd.tseries.offsets.Tick.copy	185
pd.tseries.offsets.Tick.freqstr	185
pd.tseries.offsets.Tick.is_month_end	185
pd.tseries.offsets.Tick.is_month_start	185
pd.tseries.offsets.Tick.is_on_offset	185
pd.tseries.offsets.Tick.is_quarter_end	185
pd.tseries.offsets.Tick.is_quarter_start	185
pd.tseries.offsets.Tick.is_year_end	186
pd.tseries.offsets.Tick.is_year_start	185
pd.tseries.offsets.Tick.kwds	185
pd.tseries.offsets.Tick.n	185
pd.tseries.offsets.Tick.name	185
pd.tseries.offsets.Tick.nanos	185
pd.tseries.offsets.Tick.normalize	185
pd.tseries.offsets.Tick.rule_code	185
pd.tseries.offsets.Week	171
pd.tseries.offsets.Week.copy	171
pd.tseries.offsets.Week.freqstr	171
pd.tseries.offsets.Week.is_month_end	172
pd.tseries.offsets.Week.is_month_start	172
pd.tseries.offsets.Week.is_on_offset	172
pd.tseries.offsets.Week.is_quarter_end	172
pd.tseries.offsets.Week.is_quarter_start	172
pd.tseries.offsets.Week.is_year_end	172
pd.tseries.offsets.Week.is_year_start	172
pd.tseries.offsets.Week.kwds	171
pd.tseries.offsets.Week.n	171
pd.tseries.offsets.Week.name	171
pd.tseries.offsets.Week.nanos	171
pd.tseries.offsets.Week.normalize	171
pd.tseries.offsets.Week.rule_code	171
pd.tseries.offsets.Week.weekday	171
pd.tseries.offsets.WeekOfMonth	172
pd.tseries.offsets.WeekOfMonth.copy	172
pd.tseries.offsets.WeekOfMonth.freqstr	172
pd.tseries.offsets.WeekOfMonth.is_month_end	172
pd.tseries.offsets.WeekOfMonth.is_month_start	172

pd.tseries.offsets.WeekOfMonth.is_on_offset	172
pd.tseries.offsets.WeekOfMonth.is_quarter_end	172
pd.tseries.offsets.WeekOfMonth.is_quarter_start	172
pd.tseries.offsets.WeekOfMonth.is_year_end	173
pd.tseries.offsets.WeekOfMonth.is_year_start	173
pd.tseries.offsets.WeekOfMonth.kwds	172
pd.tseries.offsets.WeekOfMonth.n	172
pd.tseries.offsets.WeekOfMonth.name	172
pd.tseries.offsets.WeekOfMonth.nanos	172
pd.tseries.offsets.WeekOfMonth.normalize	172
pd.tseries.offsets.WeekOfMonth.rule_code	172
pd.tseries.offsets.WeekOfMonth.week	172
pd.tseries.offsets.WeekOfMonth.weekday	172
pd.tseries.offsets.YearBegin	182
pd.tseries.offsets.YearBegin.copy	182
pd.tseries.offsets.YearBegin.freqstr	182
pd.tseries.offsets.YearBegin.is_month_end	182
pd.tseries.offsets.YearBegin.is_month_start	182
pd.tseries.offsets.YearBegin.is_on_offset	182
pd.tseries.offsets.YearBegin.is_quarter_end	182
pd.tseries.offsets.YearBegin.is_quarter_start	182
pd.tseries.offsets.YearBegin.is_year_end	182
pd.tseries.offsets.YearBegin.is_year_start	182
pd.tseries.offsets.YearBegin.kwds	182
pd.tseries.offsets.YearBegin.month	182
pd.tseries.offsets.YearBegin.n	182
pd.tseries.offsets.YearBegin.name	182
pd.tseries.offsets.YearBegin.nanos	182
pd.tseries.offsets.YearBegin.normalize	182
pd.tseries.offsets.YearBegin.rule_code	182
pd.tseries.offsets.YearEnd	181
pd.tseries.offsets.YearEnd.copy	181
pd.tseries.offsets.YearEnd.freqstr	181
pd.tseries.offsets.YearEnd.is_month_end	181
pd.tseries.offsets.YearEnd.is_month_start	181
pd.tseries.offsets.YearEnd.is_on_offset	181
pd.tseries.offsets.YearEnd.is_quarter_end	181
pd.tseries.offsets.YearEnd.is_quarter_start	181
pd.tseries.offsets.YearEnd.is_year_end	181
pd.tseries.offsets.YearEnd.is_year_start	181
pd.tseries.offsets.YearEnd.kwds	181
pd.tseries.offsets.YearEnd.month	181
pd.tseries.offsets.YearEnd.n	181
pd.tseries.offsets.YearEnd.name	181
pd.tseries.offsets.YearEnd.nanos	181
pd.tseries.offsets.YearEnd.normalize	181
pd.tseries.offsets.YearEnd.rule_code	181
pd.UInt16Dtype	151

pd.UInt32Dtype	152
pd.UInt64Dtype	152
pd.UInt8Dtype	151
pd.unique	101
pd.util.hash_array	102
pd.util.hash_pandas_object	94
pd.wide_to_long	59